

SplatNav 学弟单文档总手册：从入门到接手项目

这本手册是给“第一次真正接触这个项目的人”写的。默认读者不是已经会导航算法、会 Linux 工程、会三维重建的人，而是一个代码基础比较弱、跨专业进入、愿意认真学、也愿意把事情做成的学弟。整本书只做一件事：把 `/home/howard/Splat-Nav` 这套工作区讲成一条能走的路，让后来者别一上来就被目录、术语、脚本、论文和结果一起压住。

你可以把它当成正式交接材料来看。它不追求“面面俱到地覆盖所有相关领域”，而追求“把当前阶段最该懂、最该做、最该看的东西讲顺”。前面先帮你站到门口，知道项目是什么、当前边界在哪里；中间带你做第一周最该做的动作，学会看一次运行、看 `run` 目录、看结果；后面再把项目全景、阶段目标和术语附录收起来。这样安排的目的很简单，不是为了排版好看，而是为了让新人读着读着能慢慢建立方向感。

全书结构

1. 第一编：先把门口看清
2. 第二编：第一周照着做
3. 第三编：把一次运行真正看懂
4. 第四编：把项目全景和代码主线看透
5. 第五编：把长期节奏放稳
6. 第六编：术语附录与查词索引

章节树

1. 第一编 先把门口看清 1.1 先用一句话把项目说清楚 1.2 先知道这本书在解决什么 1.3 先知道第一次阅读该怎么走 1.4 先知道当前阶段什么算过关
2. 第二编 第一周照着做 2.1 把 SSH、Cursor、GitHub 和远程工作区站稳 2.2 按顺序做最小检查、最小 smoke 和第一次结果复盘 2.3 建立 `run` 目录意识、留痕习惯和慢复盘习惯
3. 第三编 把一次运行真正看懂 3.1 讲清输入是什么 3.2 讲清 smoke、batch、overnight 的差别 3.3 讲清输出、验收和假成功
4. 第四编 把项目全景和代码主线看透 4.1 讲清论文主张和当前工作区边界 4.2 讲清目录地图、wrapper 层、上游源码层和数据流 4.3 讲清哪些文件值得优先看、为什么值得看
5. 第五编 把长期节奏放稳 5.1 讲清一周、一个月、三个月分别该长出什么能力 5.2 讲清阶段证据、假进展和保底路线
6. 第六编 术语附录与查词索引 6.1 把系统词、环境词、项目词、实验词和研究词放回真实语境 6.2 让你在读代码、看结果、做汇报时随时回查

使用建议

第一次读，不要试图把所有细节一次记住。更舒服也更有效的办法，是先顺着正文建立主线：先把第一编看透，再跟着第二编动手；第三编和第四编负责让你把运行和结构讲清楚；第五编帮你校准长期节奏；第六编

则像一本查词词典，遇到不熟悉的词，随时回来翻。这样读起来会更像“跟着人接项目”，而不是“在背一摞考试资料”。

第一编 先把门口看清

这一编的任务只有一个：先把门口站稳。很多新人真正吃亏的地方，不是学不会，而是一开始连自己站在哪里、眼前这套材料在解决什么、第一阶段边界在哪都没弄清楚，于是每打开一个目录都像在拆盲盒。只要门口站稳，后面很多焦虑其实会自己掉下去。

如果你是第一次接触 Linux、SSH、Conda、tmux、3DGS、规划、可视化、批处理这些词，请不要紧张。你现在面对的本来就是一个同时涉及系统、工程、三维表示、规划、实验和汇报的项目。刚上手时觉得“每个词都认识一点，但整段话就是串不起来”，这非常正常。第一编就是要先帮你把这些混乱拆开，告诉你哪些是环境问题，哪些是代码问题，哪些是论文问题，哪些是结果验收问题，以及它们在这本手册里大概各自放在哪。

一、先知道这本书在解决什么

对一个第一次进项目的人来说，先知道“我现在到底在学什么、为什么要这么排顺序”，往往比一上来就钻某个脚本细节更重要。这里先不急着想做动作，而是先把门牌、边界和主线说清楚。

1、先用一句话把项目说清楚

Splat-Nav 是一个建立在 Gaussian Splatting 场景表示上的导航系统。它的重点不是“把场景看起来做得更漂亮”，而是让机器人在这种三维场景表示里安全地规划路径、在执行过程中不断重规划，并在需要的时候只靠 RGB 图像估计自己的位姿。论文把系统拆成两个模块：Splat-Plan 负责“怎么安全地从当前点到目标点”，Splat-Loc 负责“只看图像时，怎么估计机器人现在在哪”。

放回当前这台机器上的工作区，第一阶段的目标其实更聚焦。我们现在不是要一口气把真机、ROS、整套闭环系统全部打穿，而是先把 Splat-Plan 相关的离线/仿真主链路跑明白，能解释输入输出，能读懂 runs 目录里的结果，能说清 wrapper 脚本层和官方源码层分别在做什么。只要这条主线站稳，后面不论做汇报、带学弟还是继续往深处读源码，都会顺很多。

2、这本手册的边界

这本手册只服务一个非常具体的目标：让你围绕 /home/howard/Splat-Nav 这套工作区，把作者给出的离线/仿真主链路真正走通。它不会从零教你搭整个机器人系统，也不会把 ROS2、Gazebo、Nav2、视觉 SLAM、无人机控制一股脑全展开。那些东西以后当然可能会接上，但眼下如果什么都讲，反而最容易把新人带散。

你现在只要牢牢记住一句话就够了：现阶段的主任务不是“懂所有相关领域”，而是“围绕当前工作区，把能落地的最短路径走通”。整本书的结构，也是围绕这个判断排出来的。先理解什么，后做什么，再看什么结果，最后怎么判断自己有没有真的在前进，这些都要服务这个目标。

3、这本手册的六编分别在做什么

3.1 第一编：总入口、总边界、总路线

你现在读的这一编，就是整本书的入口。它负责回答三个问题：第一，项目一句话到底怎么说；第二，这本书应该怎么用；第三，当前做到什么程度才算第一阶段过关。只要这三个问题先稳住，后面你读脚本、跑命令、看 run 目录时就不太容易一头雾水。

3.2 第二编：把第一周的动作顺序排出来

第二编不是资料堆，而是一条路线。它会把 Cursor、SSH、GitHub、ChatGPT、Codex 插件这些工具链为什么要先准备讲清楚，再把“第一天先干什么、为什么先 smoke、跑完以后先看什么”这些动作排成顺序。它的价值不在于命令多，而在于顺序清楚。

3.3 第三编：把“一次运行”这件事讲明白

很多人会跑命令，但说不清“输入是什么”“输出是什么”“为什么这次没报错却还不能算真正成功”。第三编专门处理这件事。它会把 smoke、batch、overnight 三种规模拆开，讲清配置、模型、场景、脚本参数、日志、JSON、图表、CSV 和 runs 目录之间是什么关系。

3.4 第四编：把论文、目录、脚本和源码放回一张地图

第四编是整本书里最像“项目大地图”的部分。它会从论文出发，把 $\text{Splat-Nav} = \text{Splat-Plan} + \text{Splat-Loc}$ 说清楚，再把当前工作区里的三层结构讲清楚：本地 wrapper 工程层、上游官方源码层、结果与数据沉淀层。你会在那里真正理解 `/home/howard/Splat-Nav/scripts` 为什么存在，`/home/howard/Splat-Nav/splatnav-official` 为什么重要，哪些文件属于“论文实现本体”，哪些文件属于“为了复现稳定而加的工程壳”。

3.5 第五编：把一周、一个月、三个月的节奏放稳

新人最怕的不是难，而是不知道自己到底有没有在前进。第五编不是来给你排死板时间表的，而是来告诉你：一周做到什么样算入门过关，一个月做到什么样算能接项目，三个月做到什么样才算开始具备独立汇报和研究判断能力。它还会提醒你哪些进展只是看起来热闹，实际上并没有形成证据和沉淀。

3.6 第六编：把高频术语放回真实语境里

很多新人不是不努力，而是一开始就被术语堵住了。比如 SSH、Conda、config.yml、GSplatLoader、feasible、polytope、batch、LD_LIBRARY_PATH 这些词，如果只是听过名字、却没真正对应到项目里的路径、日志和脚本，很容易出现“每句话都认识一点，但整段意思抓不住”的状态。第六编就是为了解决这个问题。它不要求你一次背完，但你遇到卡词时，应该随时回去查。

4、第一次阅读该怎么走

如果你是从零开始，我建议第一次阅读就顺着这本书现在的排法往下走，不要再自己到处跳。先把第一编读完，让门口站稳；接着进入第二编，知道第一周最小动作顺序；然后读第三编，开始理解一次运行的输入、输出和验收；再读第四编，把项目骨架、目录和代码主线真正装进脑子里；第五编负责帮你校准阶段目标；第六编则是全程陪跑的查词附录，哪里不熟就回哪里。

这本书虽然是单文档，但不是那种“一口气从第一页背到最后一页”的教科书。更像真实情况的说法是：它有一条主线，但也允许你带着问题回跳。比如你在第二编做 smoke 时碰到不熟悉的词，可以回第六编；你在第四编看见 GSplatLoader、SFC、Bezier 这些词时，也可以回第六编查；你在第三编看 run 目录和

JSON 结构看得发虚，就先回前面读懂第二编里的动作顺序。真正有效的阅读，不是硬冲到底，而是围绕同一条主线不断来回焊接理解。

5、当前工作区到底是什么

你现在所在的根目录是 `/home/howard/Splat-Nav`。它不是论文作者的原始源码树直接裸放在这里，而是一套在本机上做过工程化整理的工作区。它的价值主要有三点。第一，它把环境、日志、长任务、批跑、结果归档这些复现里最容易乱掉的部分收敛到了统一脚本和 `runs` 规范里。第二，它通过本地引入上游官方仓库，让你既能保留官方实现原貌，又能在本地用统一入口去读代码、跑任务。第三，它把文档、脚本、结果落盘规范放在一起，减少“脚本在这里、数据在那里、结果又在另一个地方”的碎片感。

这件事非常重要。很多新人一开始看到 `scripts/`，会误以为这里就是论文算法本体；再看到 `splatnav-official/`，又怀疑是不是有两套互相独立的代码。其实不是。简单说，`scripts/` 主要是 `wrapper` 层，负责环境、路径、`tmux`、`smoke`、`batch`、`overnight`、汇总和图表；`splatnav-official/` 才是论文核心实现的主要承载位置。你读代码时需要同时知道这两层，但绝不能把它们混成一层。

6、当前阶段什么算过关

对一个零基础学弟来说，第一阶段不要求你立刻提出创新点，不要求你马上改算法，也不要求你一周内读透所有数学推导。真正的过关标准，是你能拿出可信证据证明三件事：第一，你知道项目的主链路是什么；第二，你知道怎么在这台机器上把已有场景的离线流程走起来；第三，你知道跑完以后该去哪里看结果，怎么用大白话把结果讲给别人听。

所以，现阶段的“成功”不是“背下所有术语”，也不是“把目录逛过一遍”，而是你能把项目讲清楚、把最小流程走一遍、把结果文件解释清楚。哪怕你现在还不会改源码，只要这三件事做到了，你就已经从“看不懂项目”进入“能接手项目”的状态了。

7、拆分旧稿怎么看

`docs/Codex` 目录里还保留着两份更早的拆分稿：

1. `SplatNav_主手册_训练营与教科书双轨_样章v0.1_2026-03-30.md`
2. `SplatNav_主手册_训练营与教科书双轨_完整版v0.2_2026-03-30.md`

它们保留，是为了留下写作演变的痕迹，不是为了让新人继续把它们当主线。你以后带学弟时，主线就发这一份总手册就行。旧稿如果要看，也只适合拿来查历史表达，不适合拿来做正式交接。

二、这本手册该怎么用

很多手册的问题不在于信息少，而在于读者不知道该怎么用。这里不把它当成一份静态资料，而把它当成围绕当前工作区服务的导航系统来看。

1、重点不是“读完”，而是“顺着它推进”

很多人会问，既然已经合成成一本书，是不是要先从头到尾全部读完，再开始碰机器。答案是否定的。更有办法，是先把第一编读透，知道边界和路线；随后跟着第二编做最小动作；在真正遇到 `run` 目录、JSON、场景配置和 `batch` 结果时，再用第三编和第四编把事情讲明白；遇到生词就回第六编，阶段焦虑起来时就回第五编。这种读法和真实接项目更接近。

你也要慢慢养成一个习惯：任何一段命令、任何一个目录、任何一个结果文件，都不要只问“它怎么用”，还要问“它在整条链路里解决了什么问题”。如果你只学“命令怎么敲”，很快就会陷入记忆负担；但如果你知道“为什么现在运行它、它生成什么证据、出了问题先去哪排”，命令反而会变得更容易记。

2、一条很朴素但非常重要的主线

把第一阶段压成最朴素的五步，会比一上来给自己扣上“我要完全掌握这个项目”更有效。第一步，把工具链打通，也就是 SSH、Cursor、GitHub、ChatGPT/Codex 这些前置条件先准备好。第二步，知道工作区三层结构是什么，别把 wrapper 和上游算法混了。第三步，围绕 smoke 路线理解最小输入、最小输出和最小验收。第四步，读懂 runs 目录里的证据，知道一轮 batch 或 overnight 的结果该怎么解释。第五步，在这个基础上再进入更深入的源码阅读、可视化理解和阶段性汇报。

如果你未来要写专利、做创新、投会议，那些事情当然重要，但都得建立在这五步真的做扎实的基础上。一个对输入输出和运行证据都说不清楚的人，太早谈创新，往往只会越来越虚。反过来，一个能把现有链路讲明白、跑明白、记录明白的人，即便现在还没有新点子，也已经走在很稳的路上了。

3、你现在最应该做的事

如果你是第一次打开这本手册，不要继续在目录里左右横跳了。先把第一编读完，然后立即进入第二编，把第一周最小动作顺序装进脑子里。遇到不熟悉的词，就翻第六编；想把项目骨架和目录关系看得更清楚，就去第四编；跑出第一轮结果以后，再回第三编把输入输出和验收彻底讲顺。这样读，你会发现这个项目不是“天书”，而是一条可以被拆开、被走通、被复盘的链路。

4、读完这一编后，你应该答得出的三个问题

第一，你应该能用自己的话说出：这个项目现阶段主要在复现什么，为什么不能一上来把目标喊成“全懂、全会、全改”。第二，你应该能说出：当前工作区是 wrapper 工程层加上游官方源码层的组合，而不是一坨混在一起的脚本。第三，你应该知道：自己下一步该直接进入第二编开始动手，而不是继续在一堆历史稿和零散资料里打转。

如果这三个问题你已经能答上来，说明第一编已经开始起作用了。下一步，最自然的动作就是进入第二编。

三、把它当成一张导航图来用

到了这里，重点已经不再是“书里写了什么”，而是“你现在处在哪种状态、下一步该往哪去”。这一节就是要把六编之间的跳转关系真正搭起来。

1、真正的阅读入口，不是按页码硬翻，而是按自己当前状态进入

虽然我们已经把内容合成成一本书，但它仍然不是那种必须从第一页机械翻到最后一页的线性课本。更贴近真实情况的说法是：它是一张带主线的导航图。导航图的价值，不在于逼着所有人按完全一样的速度走，而在于你在不同状态下，都能被送到最该去的那一段。一个完全没接触过远程机器和 Linux 的学弟，和一个已经能跑 smoke、正准备看 batch 结果的人，本来就不应该用完全一样的阅读入口。

所以你以后带学弟时，不妨少问一句“你读到第几页了”，多问一句“你现在最困惑的是哪一层”。如果他连 SSH、工作区路径、scripts/ 和 splatnav-official/ 的关系都还没站稳，那么他的入口应该是第一编配合第二编开头，再辅以后六编查词。如果他已能进工作区，但 run 目录、JSON 和日志看不明白，那

入口就该更靠近第三编。如果他知道项目是什么，却迟迟不知道自己第一步该干嘛，那最该去的是第二编。如果他已经做了一些动作，却老在焦虑“我现在算不算在正路上”，那就该回到第五编。

2、六编之间的接力关系，应该怎么理解

如果用更像接力棒的方式来理解，这六编的关系其实很清楚。第一编把人领进门，告诉你这本手册到底怎么用；第二编把动作排成路线，让你真的迈出第一步、第二步和第三步；第三编把“一次运行”拆成可解释的链条；第四编把项目骨架、论文主线、目录和代码主线真正搭起来；第五编告诉你不同时间尺度下什么叫真的前进；第六编则像一本随时回查的词典，负责在你卡词、卡概念、卡表达时托你一下。

这条接力关系里有一个很重要的特点，就是前面的内容不会“过期”。你已经进入第二编真正开始做 smoke 了，也依然会经常回第六编查词、回第四编确认结构、回第三编核对输入输出。你已经进入第五编做阶段定位了，也仍然需要回第二编看最小动作顺序。所以整本书不是一次性通关的楼梯，更像一张网。只是这张网有一根很清楚的主绳，不会让你掉下去。

3、如果你今天只剩 30 分钟，该怎么用

一套好的手册不应该只适合“大块时间系统阅读”，也应该适合碎片化推进。比如你今天只有半小时，完全可以先回看第一编，把边界和主线重新校准一遍；然后翻到第六编，查最常碰到的环境、路径、日志、run 目录这些词；如果你已经开始动手了，那这半小时更适合拿来配合第三编，看一眼已有 run 目录，重新解释一遍输入输出。

半小时当然不可能让你立刻变成项目专家，但它足够让你第二天不再把工作区看成一整团黑。很多真正的进步，就是在这种“短时间但方向对”的小步里慢慢长出来的。

4、如果你今天有半天时间，最推荐的推进方式是什么

半天其实是一个非常舒服的长度。它既不短到只能扫几段文字，也不长到必须安排复杂计划。对零基础学弟来说，一个比较顺的半天通常是这样：先用四十分钟左右读完第一编，把门口站稳；再用一个小时左右开始第二编，做环境检查或已有 run 目录复盘；然后休息一下，再拿一小时读第四编开头，把项目骨架装进脑子；最后留二三十分钟，回第三编，把刚才做过的动作重新用输入、处理、输出、验收四段讲一遍。

这个节奏最大的好处，就是不会把“读”和“做”撕开。你不是先读几天再碰机器，也不是一上来只敲命令不看材料。你是在同一个半天里，让框架、动作和结果慢慢互相对上。对新人来说，这种体验特别重要，因为很多真正的信心，就是在第一次“我开始能看懂一点了”里长出来的。

5、如果你已经能跑 smoke，但还不会讲项目，最该回哪几编

这是一个非常常见、也非常值得单独讲清楚的状态。很多人到了这里会有点迷惑：命令其实能跑，run 目录也看到了，为什么一被问到“这个项目到底在干什么”，自己还是会发虚？原因往往不是你做得不够，而是你缺少一条从动作回到结构的回路。

这时候最有效的路线，通常不是继续多跑几次同样的 smoke，而是回到第四编和第三编。你应该先用第四编把项目总骨架重新过一遍，尤其是工作区三层结构、论文主线和 wrapper 与上游的关系；然后用第三编把自己刚刚跑过的那轮 smoke 拆成输入、处理、输出、验收四段。做完这两步以后，再回第二编看动作链条，你会明显感觉“我不是只会跑了，我开始知道自己跑的到底是什么了”。

6、如果你已经会讲项目，但还不会稳定排错，最该回哪几编

还有一种状态也很典型，就是学弟口头上已经能把项目讲得八九不离十，可一旦真的碰到环境异常、路径不对、JSON 字段异常或者 run 目录不完整，还是会明显慌掉。这说明他的结构感已经开始有了，但还没真正把结构感落实到排错动作里。

这时候最好的跳转方式，通常是先回第五编，再回第三编和第二编。为什么先看第五编？因为很多排错焦虑，其实不是技术问题先起，而是阶段判断先乱了。你要先知道，当前目标是不是本来就只要求最小 smoke，还是已经进入 batch 稳定化阶段。目标一旦清楚，再回第三编用输入、处理、输出、验收四段去拆问题，再回第二编看自己现在最该做的最小动作。这样排错就会从“我是不是哪都不行”重新落回“我现在只是某一层卡住了”。

7、一份真正可交接的导航系统，至少要回答五个问题

如果我们真想把第一编写成一张能长期交接的导航台，它至少应该稳定回答五个问题。第一，我现在到底在什么阶段。第二，我这一阶段最该读哪几编。第三，我读完以后下一步是去做动作，还是去回查术语、结构、输入输出。第四，如果我卡住，应该优先回哪一编而不是到处乱翻。第五，如果我要带别人，应该先把别人送到哪一编，而不是把一整个目录丢给他自己摸。

你现在可以对照一下，前面的第一编已经把这些问题回答出一个基础框架了，但更重要的不是“知道这里有导航”，而是真能顺着自己的当前状态，找到最合适的起点、下一跳和回查点。

8、三种最常见的阅读入口，我替你先分好

为了让导航更实用，我们不妨把最常见的三种阅读入口直接分出来。第一种入口，是“我几乎什么都不会，甚至不知道项目到底长什么样”。这种情况下，最推荐的顺序是：先把第一编读稳，再带着第六编一起用，然后尽快进入第二编开头。第二种入口，是“我已经能连进机器，也愿意开始动手，但我不知道第一步该落在哪”。这种情况下，最推荐的顺序是：第一编打底，直接进入第二编建立动作感觉，再回第三编补输入输出解释。第三种入口，是“我已经跑过一些东西，但讲不明白，也不知道自己接下来是不是在正路上”。这种情况下，最推荐的顺序是：先回第五编做阶段定位，再回第四编补结构，最后回第三编补结果解释。

这三种入口不是把人固定死，而是帮你少走弯路。新人最怕的一件事，其实不是材料少，而是材料很多却没有入口。只要入口对了，后面的努力基本都能积起来；入口错了，越努力越容易绕远。

9、带教时，师兄最值得发给学弟的不是一串路径，而是一条阅读动作链

如果你以后真要把这套材料交给学弟，最不建议的方式就是只发一句“你先看看 docs”。这种说法太宽了，新人拿到以后很容易继续靠本能乱点。更好的交接方式，是发一条很短但很清楚的阅读动作链。比如你完全可以说：先把第一编读完，知道整本书怎么用；再翻第六编前半，把最常碰到的术语扫一遍；然后回到第二编做第一次最小动作；做完以后拿着结果去第三编对照输入输出；等基本不发懵了，再认真啃第四编的大地图；最后用第五编给自己定位，看现在属于一周目标还是一月目标。

这条交接链的好处，在于它既没有压得太死，又不会把新人放进完全开放的树林里。它像一根牵引绳，牵着他先沿主线走。等主线走熟了，再慢慢允许他自由穿梭于不同文档之间。

10、对师兄来说，第一编还有一个隐藏用途：它是阶段沟通时的对齐文本

除了给学弟用，第一编对带教的人也很有价值。因为带学弟最累的一个时刻，不是他不会，而是你们对“现在到底该做到什么程度”没有对齐。你觉得他现在应该先把 smoke 路线站稳，他却以为自己应该马上想创新点；你觉得他现在主要缺 run 目录阅读能力，他却拼命在啃论文里的某个高级细节。这个错位一旦出现，双方都会累。

所以第一编还可以作为一个对齐文本来用。你们在阶段沟通时，完全可以回到这里重新校准：当前主线解决的是什么，第一阶段的边界是什么，现在最重要的是哪一层。这种校准会非常省力。因为很多时候，人不是做得不认真，而是起点和目标没对齐。

11、为什么我一直强调“主线文档”和“历史草稿”要分开

这件事表面看像文档管理问题，实际上和学习效率直接相关。一个新人在刚开始时，最需要的是稳定、可依赖、顺着走不会太痛苦的主线。如果主线和历史草稿混在一起，他就会不停地在“这个看起来也像手册”“那个也好像有内容”之间来回切，最后很难建立对任何一份材料的信任感。

所以把这本总手册明确成主线，把旧的 v0.1 和 v0.2 明确成历史草稿，并不是在否定旧稿，而是在保护新人的注意力。你以后如果要带学弟，这一点一定要坚持。真正好的交接，不是给更多材料，而是给更清楚的主线。

12、如果把这套导航系统再压成一句最朴素的话

如果一定要把整本书的导航关系再压成一句最朴素的话，我会愿意这么说：先知道自己面对的是什么，再知道自己该从哪里开始；先知道动作顺序，再知道一次运行到底发生了什么；先知道结构，再判断自己现在处在哪个阶段；哪里卡住了，就回对应那一编。这个说法听起来很平常，但它几乎就是这本手册想替学弟修出来的那条路。

只要这句话开始在脑子里立住，复杂项目就不会再那么像一团密不透风的墙。它会慢慢变成一条虽然长、但有方向、有路标、有扶手的路。

13、最后给你一份跳转索引，方便你真把这本书当导航台来用

如果你现在最想做的是搞清项目是什么，就先读第四编；如果你现在最想做的是开始动手，就直接进第二编；如果你已经跑出过东西，但不确定算不算真正成功，就去第三编；如果你最焦虑的是自己到底有没有在正路上，就回第五编；如果你读着读着总被术语卡住，就把第六编放在手边当词典。

如果你是带学弟的人，那么最建议的发法也很简单：直接把这本总手册发给他，先让他把第一编读完；然后根据他当下的状态，把他送去第二编、第三编、第四编或第五编。这样这套材料才会真的像一个导航系统，而不是一个资料堆。

第二编 第一周照着做

这一编不追求面面俱到，而是强调动作顺序。师弟如果只想知道“我今天先干什么、做完以后去哪看”，就从这里开始。

这一份文档不追求“把所有知识一次讲完”，而追求“让你知道现在先做什么、为什么先做它、做完以后去哪看结果”。对零基础新人来说，这种顺序感比知识总量更重要。因为项目真正让人崩溃的，常常不是哪一部分绝对难，而是每件事看起来都要学，于是完全不知道第一步应该落在哪。

所以这里不会按“领域知识章节”展开，而是按动作顺序展开。你可以把它当作一条带你从站在门外，到真正推开门、走进屋子、看见桌子上放着什么、最后能自己做一个最小实验的路线。它不是死板的日程表，也不是那种背完就会的秘籍，而是一条适合当前工作区和当前机器条件的最短上手路径。

一、正式开始之前，先把入口和工具链站稳

快速上手的第一步不是跑算法，而是把入口打通。SSH、Cursor、GitHub、账号和远程工作区这些看起来琐碎的东西，实际上决定了后面一切动作有没有真实上下文。

1、第一天开始之前，先把工具链打通

很多同学一上来就想看论文、跑代码，这种冲劲是好的，但如果工具链没打通，你很快会在最基础的问题上反复消耗。对于当前项目，第一批必须具备的工具不是 fancy，而是非常朴素：SSH、Cursor 远程连接、GitHub 账户、ChatGPT 账户、Codex 插件、一个能稳定接收注册邮件的邮箱。你不需要把这些工具研究得很深，但至少要进入“能用”的状态。

1.1 Cursor + SSH

你首先要能从自己的电脑稳定进入这台工控机。这里的关键不是炫技，而是建立“我现在所有阅读、运行和结果查看，都是在真正的项目机器上发生”的意识。连接之前，你需要向师兄索要账户、IP、密码或密钥信息。这个动作看起来很琐碎，但它是后面一切工作的前提。没有 SSH，你就没有真正的工作区上下文。

真正开始连接时，建议你先在本地终端验证 SSH 本身是否能工作，再让 Cursor 接入远程工作区。原因很简单：终端里的 SSH 成功，说明网络和账户层没问题；再在 Cursor 里接入，会更容易判断问题是 SSH 本身还是编辑器插件层。

下面这条命令的目的不是开始跑项目，而只是确认“这扇门能不能打开”。你应该把它当成网络与账户的最小验证。

```
ssh howard@<服务器IP>
```

如果这一步成功，你应该会看到远程 shell 提示符，而不是继续停留在本地终端。如果失败，先不要急着怀疑项目，先检查 IP、账户名、密码/密钥、网络连接和防火墙。你只有在“能稳定进远端”这个事实上站稳了，后面的路径和环境问题才值得谈。

1.2 GitHub 账户与基本配置

GitHub 在当前阶段不是让你立刻发 PR 或参与开源协作，而是为了建立最基本的工程协作能力。你需要知道仓库是什么、提交是什么、远程仓库是什么，以及将来如何保存自己的变更历史。对于学弟来说，只要先完成注册、登录和最基础的身份配置就够了。

下面这组命令的目的，是在远程机器上留下最基本的 Git 身份信息。它不是项目算法命令，但以后你做任何文档整理、脚本修改和代码提交时，都会用到。

```
git config --global user.name "你的名字"
git config --global user.email "你的邮箱"
```

跑完以后，不会有很热闹的输出。它更像是在给未来的自己铺路。如果你后面真正需要提交代码，再由师兄决定是否配置 SSH key、是否接入具体仓库远程。

1.3 ChatGPT、Codex 插件与 Gmail

这一层工具链的作用，不是替你思考，而是降低你求助、查阅、生成总结和理解代码的门槛。你现在面对的是一个跨系统、跨工程、跨论文的项目，完全靠自己一点点摸会非常慢。一个稳定的 ChatGPT 账号、Cursor 里的 Codex 插件、一个能接收验证邮件的 Gmail 邮箱，会显著改善你的学习效率。

这里要有一个态度边界：这些工具是辅助，不是外包。你不能只会把一段报错贴出去等别人救你，也不能让工具替你作所有判断。但你完全可以借助它们来理解术语、解释日志、整理运行步骤和归纳代码结构。对零基础新人来说，这是一种很现实的放大器。

二、第一天先建立位置感和主线感

这一部分的目标很朴素，就是先让你知道自己站在哪、该先读什么、为什么不建议一开始就在目录里乱逛。位置站稳了，后面动作才会顺。

1、真正进入项目后，不要急着乱逛目录

SSH 和工具链打通以后，很多人会立刻在整个工作区里到处点。这个动作很自然，但效率不高。当前项目最好的开局方式，不是先逛所有目录，而是先确认自己确实来到了正确的工作区，然后再按主线看文档。

下面这两条命令的目的，是确认你当前所处的位置，以及这个目录下都有哪些顶层入口。它们是最基础也最值得保留的起点命令。

```
pwd
ls
```

如果一切正常，你应该看到当前目录是 `/home/howard/Splat-Nav`，并看到 `docs`、`scripts`、`splatnav-official`、`runs`、`README.md` 这些顶层内容。如果不是这个路径，不要急着继续。先把路径站稳，再做后续动作。很多后续错误，其实都来自第一步就站错了地方。

2、先读什么，不要自己猜

工具链就绪以后，第一件真正有价值的事不是马上跑命令，而是建立阅读顺序。因为项目已经不是一个只有单个 `main.py` 的小作业，而是一套带上游、带 wrapper、带运行规范的工作区。如果没有阅读顺序，你很容易一会儿看 `archive`，一会儿看旧稿，一会儿看脚本，一会儿看上游代码，最后什么都像看过，脑子里却没有主线。

当前最推荐的顺序非常明确。先读本书开头的第一编，知道整套材料怎么用。然后读第六编前半部分，把 Linux、路径、环境、日志、`run` 目录这些高频词清一清。接着读第四编，建立整体地图。做到这里以后，你再来读第二编，就会轻松很多。

下面这组命令的目的，是快速进入新主线文档目录，而不是继续在历史文档里漂。

```
cd /home/howard/Splat-Nav/docs/Codex
ls
```

你应该能看到 这本总手册以及旁边保留的拆分版文稿。如果你还看到了 `v0.1`、`v0.2` 旧稿，不要再把它们当主线。它们保留只是为了历史记录，不再是正式交接路径。

3、第一波最小检查：不要先跑算法，先看环境是否健康

很多新人最想做的是“跑个什么看看”。但如果环境本身不健康，这一步往往会浪费很多时间。更稳的动作是先做环境健康检查。这样做的价值，不是拖延，而是用最小成本先排除最基础的系统问题。

下面这条命令的目的，是通过统一脚本确认 Conda、Python、GPU、Torch CUDA 和关键依赖是否在可用状态。它不是算法命令，但它会决定后面的算法命令有没有意义。

```
bash /home/howard/Splat-Nav/scripts/env_health_check.sh splatnav
```

跑完以后，你应该重点看几件事：是否成功激活了 `splatnav` 环境；`torch.cuda.is_available()` 是否为 `True`；`nerfstudio`、`open3d`、`cv2`、`clarabel` 等关键依赖的 `import` 是否通过。如果这里已经报错，就不要跳过它直接跑 `smoke`。先把环境这层站稳，后面会省很多时间。

4、第一天真正开始读代码之前，先确认工作区三层结构

到了这里，你已经可以开始看一些目录了，但仍然不建议一股脑乱读。你最先要确认的，是工作区的三层结构：`scripts/` 是 wrapper 工程层，`splatnav-official/` 是上游论文源码层，`runs/` 是结果沉淀层。只要这张图没有进脑子，后面读什么都会绕。

下面这组三条命令的目的，是把这三层最关键的位置肉眼确认一遍，尤其是上游源码软链接到底指向哪。

```
ls /home/howard/Splat-Nav
ls /home/howard/Splat-Nav/scripts
readlink -f /home/howard/Splat-Nav/splatnav-official
```

跑完以后，你应该能说出三件事：第一，wrapper 脚本都在 `scripts/`；第二，官方源码入口在 `splatnav-official/`；第三，这个入口最终指向数据盘上的真实仓库，而不是 `home` 目录里另一份独立拷贝。如果这三件事你能讲明白，说明你对项目地图已经开始成形了。

三、第一次真正动手，要先把最小链路走通

从这里开始，手册才真正进入动作层。重点不是跑很多，而是先把一条最小而可靠的执行链做成闭环。

1、第一次真正动手：为什么先 `smoke`，而不是直接 `batch`

到了这一步，才轮到真正跑规划。这里最容易犯的错，就是一上来想跑正式大实验，甚至 `overnight`。这样做的问题不是“勇气可嘉”，而是变量太多。一旦失败，你很难判断是环境、路径、模型、方法、图表还是汇总出了问题。`smoke` 的价值就在于它把变量压到最低。

当前最适合第一次动手的动作，是先找一个已有的 `config.yml`，然后用 `run_splatplan_smoke.sh` 对某个场景、某种方法做一次最小规划测试。这个动作的目标不是跑出论文级结果，而是让你第一次真实看到：输入怎么进，`run` 目录怎么建，JSON 怎么产出，日志在哪看。

下面这条命令的目的，是用一个已有场景的配置做一次最小 smoke。这里的 `CONFIG` 路径需要替换成当前机器上已经存在、你和师兄确认过的那个配置文件。

```
bash /home/howard/Splat-Nav/scripts/run_splatplan_smoke.sh old_union /path/to/config.yml sfc-1
```

跑完以后，你不应该第一反应去看终端最后一行，而应该先记住脚本打印出的 `run_dir`。真正的证据都在那里。然后你去看那个目录里的 `stdout.log`、`cmd.txt` 和 `splatplan_smoke_output.json`。如果 JSON 结构完整、关键信息可读、`feasible` 与耗时能解释，那就说明你第一次最小链路已经碰到实体了。

2、为什么建议第二步看 run 目录，而不是继续敲下一条命令

很多新人第一次跑完一个命令，马上就想接着跑第二个。这种推进感很自然，但容易把最宝贵的学习机会浪费掉。因为你真正开始理解项目，往往不是敲命令那一刻，而是回头看刚刚那个 `run` 目录那一刻。命令只是启动，`run` 目录才是你真正获得证据和理解的地方。

你第一次 smoke 之后，建议优先做这几件事。先打开 `stdout.log` 看结尾有没有异常退出或关键提示。再打开 `cmd.txt`，确认这次 `run` 和你记忆里的命令一致。然后打开 JSON，看 `meta.scene_name`、`meta.method`、`plan_time_sec`、`output.feasible`、`output.num_polytopes` 这些字段。你只有在这里真的看明白了，下一步进入 batch 才有意义。

下面这组命令的目的，是围绕刚刚那次 smoke 的 `run` 目录做最小复盘。这里的 `RUN_DIR` 请替换成刚才脚本打印出来的目录。

```
cd <RUN_DIR>
ls
tail -n 40 stdout.log
```

如果最后几十行显示规划正常完成，而且目录里能看到 JSON，那么你已经迈过了从“只会敲命令”到“会看结果证据”的第一步。

3、第二次升级：什么时候进入 batch

只有当你已经能说清一次 smoke 的输入和输出以后，才建议进入 batch。batch 的意义不是“把命令变复杂”，而是让你第一次看到方法间对比和样本统计。也就是说，它会逼着你开始关心可行率、平均耗时和轨迹分布，而不再只盯着某一条单独路径。

下面这条命令的目的，是围绕一个场景，跑一个小规模 batch。这里故意不建议你一上来就把 `trials` 设得很大，因为学弟第一次做 batch，重点是理解结构，而不是堆数量。


```
python /home/howard/Splat-Nav/scripts/run_official_style_batch.py \  
--config /path/to/config.yml \  
--scene old_union \  
--methods splatplan,sfc-1,sfc-2 \  
--trials 12 \  
--seed 1 \  
--keep_traj_limit 3 \  
--output /home/howard/Splat-Nav/runs/manual_batch_demo/batch_results.json
```

跑完以后，你应该去看 `batch_results.json`，重点关注每个方法条目的 `feasible_rate`、`avg_plan_time_sec` 和保存下来的 `trial_records`。如果这一步你能看懂，就说明你已经从“单样例思维”进入“实验统计思维”了。

4、什么时候用 tmux，为什么要用 tmux

tmux 不是高深技巧，而是长任务场景下的基本生存工具。你和远程机器之间的连接可能会断，Cursor 可能会重载，网络可能会波动。如果所有长任务都直接绑在当前终端上，一旦连接断了，任务就可能一起死掉。tmux 的作用，就是把任务留在远程机器自己的会话里继续跑。

当前工作区已经把 tmux 使用方式包装进了 `run_in_tmux.sh`。所以你不需要一开始就手动管理很多 tmux 命令，只要先学会“什么时候该把任务送进 tmux”。原则非常简单：只要预计任务运行时间长于你愿意盯着终端等待的时间，就优先考虑 tmux。

下面这条命令的目的，是把一个环境检查或简单任务送进统一 tmux 会话，让你感受一下 run 目录、tmux window 和日志是如何绑定的。

```
bash /home/howard/Splat-Nav/scripts/run_in_tmux.sh env_check env_health "bash /home/howard/Splat-Nav/scripts/  
env_health_check.sh splatnav"
```

跑完以后，脚本会打印 session、window 和 `run_dir`。这时候你应该意识到：tmux 不是一个额外麻烦，而是让长任务“有家可回”的工具。

5、第一次批跑之后，优先打开哪些文件

一轮 batch 跑完，最容易被先打开的是图。但如果你正在学习阶段，更推荐你先看结构化结果，再看图。因为图是压缩后的解释层，JSON 才是事实层。一个更稳的顺序是：先看 `stdout.log` 有没有异常；再看 `batch_results.json` 的总体统计；然后看图；最后自己试着用一句话总结每种方法的大概差异。

如果有可视化目录，比如 `figs/`，你可以先看 `summary.md`，因为它比图更适合作为“快读总览”。当你已经习惯这种顺序后，未来做汇报会自然很多，因为你不再是“先找一张最好看的图”，而是“先抓事实，再选图来解释事实”。

四、第一周里最该培养的工作习惯

学弟真正能不能接住项目，很多时候取决于习惯，而不是一时的冲劲。这一部分集中讲边界意识、留痕习惯、慢复盘和表达能力。

1、`sudo`、`rm` 和输入密码的命令为什么要格外谨慎

你们共用这台机器，而且当前账户和环境边界已经明确划好了。对新人来说，最重要的习惯之一就是：不要轻易运行需要 `sudo` 的命令，不要随手执行不确定目标的 `rm`，任何需要输入密码的动作都要先想清楚它究竟在改哪里、影响谁。因为这些命令的风险不在于它们“长得吓人”，而在于一旦作用范围判断错了，后果往往超出你当前的控制能力。

在当前工作区，绝大多数日常复现和阅读动作根本不需要 `sudo`。如果你发现自己频繁需要管理员权限，那往往意味着方向可能已经偏了，或者你正在试图修复一个不该由你这个层级修复的问题。这个时候最好的动作不是硬着头皮继续，而是停下来确认。

2、什么时候算第一周真正入门

第一周真正的入门，不是你看完了多少页文档，也不是你能把多少命令背下来，而是你已经形成一个基本闭环：能通过 SSH 和 Cursor 进入工作区；知道新文档主线在哪；知道 wrapper 和上游源码的区别；能完成最小环境检查；能跑一次 smoke；能找到 run 目录并读出关键信息；能用自己的话解释当前跑的是什么、看到了什么、还不懂什么。

如果这些事情你做到了，就已经非常好了。因为你已经从“项目看起来像天书”进入“项目可以被拆开理解”的状态了。后面的 batch、overnight、源码深读、周报和创新思考，都会建立在这个基础上。

3、接下来你该怎么继续

如果你刚刚按这一部分走完了一轮最小路线，下一步建议你回到 第三编，把刚才亲手跑过的那次 smoke 或 batch 再对照着解释一遍。然后再去读 第五编，把自己定位到当前阶段。

真正能把事情做成的人，往往不是最开始懂得最多的人，而是最早形成“动作有顺序、结果有证据、卡点会复盘”这种工作方式的人。希望你从第一天起，就把这个习惯建立起来。

4、不要把“快速上手”理解成“急着多跑”，而要理解成“尽快建立稳定节奏”

很多新人一看到“快速上手”这四个字，就会下意识地以为这里讲的是速度，仿佛越快敲出更多命令、越快点开更多文件，就越接近上手。其实在复杂项目里，真正值钱的从来不是表面动作的速度，而是你能不能尽快形成一种稳定节奏。也就是说，你知道什么时候该先确认环境，什么时候该先看文档，什么时候该先做最小验证，什么时候该停下来解释自己刚刚到底做了什么。

我特别想把这一层讲透，是因为零基础学弟最容易被一种假勤奋拖住。一天看了很多文件，跑了很多命令，窗口开了一堆，感觉自己很努力；但晚上回头一想，却说不清今天真正推进了什么。真正的快速上手，不是这种到处出汗的忙，而是开始知道每一步为什么排在这里、做完之后应该看到什么。只要节奏一稳，后面自然会快；如果节奏不稳，再多时间也会被空转吞掉。

5、第一天最重要的成果，不是跑通算法，而是把“我站在哪里”这件事讲清楚

很多同学会对第一天有一种很高的期待，仿佛只要自己够努力，第一天最好就要把算法跑起来。这个心情我完全理解，但对当前项目来说，第一天最有价值的成果其实不是一张图或者一个 JSON，而是你开始能把自己的位置讲清楚。换句话说，你要能回答：我现在用的是哪台机器、哪个账户、哪个工作区、哪一套环境、哪一套文档主线。

你别小看这个目标。它听上去不“酷”，但实际上是复杂项目里特别硬的一层底座。因为所有后续动作，都要建立在“我知道自己站在哪”这件事上。你第一天如果只是着急去碰算法，而没有把位置站稳，后面几乎一定会反复出现那种很挫败的情况：明明自己做了不少事，却总有一种脚下是虚的感觉。

所以我会更愿意把第一天的过关标准说成这样：你能稳定 SSH 进去，能在 Cursor 里连到远程工作区，能确认 `/home/howard/Splat-Nav` 是当前主工作区，能说清 `scripts/` 是 wrapper 层、`splatnav-official/` 是上游算法层、`runs/` 是结果层，能打开 `docs/Codex` 并知道新主线从第一编开始。只要这些站稳了，第一天就是成功的。

6、第一天真正该怎么读文档：不是从头背，而是读到“能开始动手”为止

很多零基础同学一碰到文档就容易走两个极端。一个极端是完全不看，想着先跑起来再说；另一个极端是想把所有文档先完整看完，再开始动手。前者会导致动作没有地图，后者会导致迟迟不敢开始。当前项目更适合的节奏，其实是读到“能开始动手”为止。

这意味着你第一天读文档时，不需要强迫自己把每一章都吃透。更现实的目标是：先从第一编里知道整个体系怎么组织；再从第六编里把眼下会高频出现的词，比如 SSH、Conda、tmux、run 目录、日志、JSON、wrapper、上游源码这些词先读顺；然后到第四编里把项目总地图看一遍。到这一步，如果你已经知道项目分几层、脚本和源码谁管什么、运行结果会落在哪里，其实就已经可以开始做最小动手了。

也就是说，文档在第一天不应该成为一道门槛，而应该成为你进入项目的扶手。你是扶着它往前走，不是抱着它原地不动。

7、第一次打开 `scripts/` 时，不要想着“这些我都得会”，而要先学会分组

`/home/howard/Splat-Nav/scripts` 这个目录，对新人来说很容易带来一种压迫感。因为一眼看过去，既有 `.sh`，也有 `.py`，既有环境类脚本，也有批跑类脚本，还有汇总和可视化脚本。如果你此时的心态是“这些我是不是都得先搞懂”，那压力会很大。更好的做法，是先把它们分组，而不是试图一次掌握全部。

一种很实用的分组方式是这样的。第一组是环境和边界类脚本，比如 `bootstrap_env.sh`、`env_health_check.sh`、`fix_torch_cuda.sh`，它们解决的是“机器和 Python 世界今天能不能正常说话”。第二组是运行启动类脚本，比如 `run_splatplan_smoke.sh`、`run_splatplan_smoke_tmux.sh`、`run_official_style_batch.py`、`run_overnight_12h.sh`，它们解决的是“我要如何把一次实验真正跑起来”。第三组是结果整理类脚本，比如 `visualize_batch_results.py`、`aggregate_batch_runs.py`、`post_overnight_report.sh`，它们解决的是“跑完之后结果怎么整理成可读资产”。第四组才是少量数据准备或补洞脚本，比如下载、链接和缺文件补齐相关的那几份脚本。

只要这四组先分开，脚本目录就不再像一锅粥，而会变成几类有职责的工具箱。新手不需要第一天把工具箱里的每把扳手都研究明白，但必须知道哪类问题该翻哪一层工具。

8、第二天开始，真正该练的不是“多跑几轮”，而是“每做一步都留痕”

一旦第一天的位置感站稳，第二天开始最值得刻意练的，其实是留痕。所谓留痕，不是把每次敲命令都截图发群里，而是学会让自己的工作天然留下可回查的痕迹。当前项目已经很贴心地把这个习惯嵌入 wrapper 层了，比如 run 目录、日志、`cmd.txt`、`env.txt` 和 `notes.md`。学弟真正需要练的，是不要跳过这些东西，而是把它们当成工作的一部分。

这件事对零基础同学尤其重要。因为初学者最常见的挫败之一，就是前天好像做出过一点东西，今天却完全接不上。不是因为你变笨了，而是因为昨天的痕迹没留下来。只要留痕习惯开始长出来，你就会发现自己的进度开始变得可回溯，信心也会明显稳很多。

所以第二天你再做环境检查、再进工作区、再跑一次 smoke 时，不妨强迫自己多做一个动作：每次做完以后，都到对应的 run 目录里看一眼，顺手在 notes.md 里补上一两句。哪怕只是写“这轮链路通了，但 config 细节还没完全搞懂”，它都比什么不留要强得多。

9、第一次 smoke 前，最该准备好的不是命令本身，而是“跑完以后我去哪看”的意识

很多新人第一次 smoke 的焦虑，表面上来自不会写命令，实际上更深一层来自不知道跑完以后该怎么办。所以他们会反复盯着命令本身，生怕少一个参数、拼错一个路径。其实更有效的准备方式是反过来：在敲命令之前，先把“跑完以后我去哪看”这件事想好。

比如你应该先知道，这轮 smoke 跑起来以后，脚本会创建一个新的 run 目录；这个目录里至少会有 cmd.txt、stdout.log 和结果 JSON；你最先应该看的是日志结尾、命令记录和 JSON 关键信息，而不是着急找一张漂亮的图。如果这套预期提前有了，真正敲命令时你心里会稳很多。因为你知道命令不是终点，它只是把你送进一个自己已经大致知道该怎么阅读的结果空间。

这类预期意识其实非常像做实验前先知道观测指标。它不会让你一步不出错，但会显著降低那种“跑完以后整个人发懵”的感觉。

10、第一次 smoke 跑完以后，别急着升级到 batch，先做一次“慢复盘”

我很想替很多新人把这个坑提前填上。因为第一次 smoke 跑完时，人通常会很兴奋，特别容易顺势就想继续跑 batch，甚至再看看 overnight。这个冲动可以理解，但如果你不先做一次慢复盘，后面的提升常常会变成表面的。所谓慢复盘，就是你故意不追求立刻进入下一轮，而是回头用十到二十分钟把刚刚这一轮的输入、脚本、run 目录、日志和 JSON 串起来。

这个过程会有点慢，但非常值。因为它能让你第一次把“动手”和“解释”接到一起。你会开始意识到，自己不仅是在运行脚本，也是在和一个复杂系统建立关系。很多真正的理解，都是在这种慢复盘里长出来的，而不是在连续多跑几轮的热闹里突然降临的。

所以如果学弟第一次 smoke 已经跑通，我会非常建议他做一件事：拿着 run 目录，照着第三编里讲的输入输出视角，试着用自己的话把这一轮说一遍。哪怕说得磕磕绊绊，也比立刻继续狂跑更值。

五、从 smoke 升级到 batch，再慢慢走向更稳的节奏

当最小闭环开始形成，下一步才是顺着规模自然升级。这一部分讲的不是猛冲，而是怎样在不失控的前提下把动作做大、把判断做稳。

1、第一次 batch 的目标，不是“接近论文数字”，而是“第一次学会看总体统计”

一进入 batch，很多人心里就会突然冒出一一种很重的比较心理，仿佛自己应该马上开始对照论文里的结果、担心数字是不是不够漂亮。对入门阶段来说，这样想太早了。第一次 batch 的真正目标，其实是让你第一次学会看总体统计，而不是单条路径。也就是说，你要开始知道什么叫 feasible rate，什么叫平均耗时，什么叫 P95，以及为什么这些东西值得被同时摆出来。

这一层一旦开始理解，你的视角就会自然升级。你不再只是盯着“某条轨迹成功没成功”，而开始关心“某种方法在一批样本上的整体稳定性怎样”。这件事和论文结果当然是连着的，但它比“数字像不像论文”更基础，也更适合当前阶段。因为只有你先学会看统计，后面才谈得上真正做比较、讲结果、想创新。

2、tmux 最容易学会的方式，不是背命令，而是把它看成“远程任务的保险箱”

很多零基础同学一听到 tmux 就会紧张，觉得这是不是某种高手工具。其实对当前项目来说，tmux 最值得记住的不是一串命令，而是一个很简单的比喻：它像远程任务的保险箱。你把你一个长任务放进去，即便本地终端断了、Cursor 重连了、网络抖了一下，任务也不会跟着轻易消失。

一旦把 tmux 看成保险箱，很多事情就立刻简单了。你不再纠结“tmux 到底有多少命令我还会不会”，而是会先问一个更实用的问题：这件事值不值得放进保险箱。对于当前项目，环境检查这种很短的动作不一定必须进 tmux，但长一点的 batch、overnight，或者你不想一直盯着的任务，就非常适合放进去。

而且当前工作区并没有要求学弟一开始就手打所有 tmux 操作，而是通过 `run_in_tmux.sh` 做了一层包装。这个设计很友好。因为它允许新手先建立“该不该用 tmux”的判断，再慢慢熟悉“tmux 具体怎么管”。这就是一种很典型的工程友好设计：先降低门槛，再逐步深化。

3、如果某一步卡住了，先判断自己卡的是“哪一层”，再决定查哪一类东西

快速上手里最怕的，不是卡住，而是卡住以后方向乱掉。很多新人一出问题就开始同时怀疑环境、脚本、论文、数据、路径和自己，结果越查越散。更稳的习惯是，先判断自己卡的是哪一层，再去对应层找线索。

如果你连 SSH 都进不去，显然是机器接入层的问题；如果能进机器但环境检查不过，显然更像环境层；如果环境没问题但 smoke 命令一跑就说找不到文件，那更像路径或资产层；如果文件都能读但 JSON 字段异常，那就更像运行结果解释层；如果结果有了但你说不清它说明什么，那就是理解与表达层。只要先把层次分对，排错效率会高很多。

这件事你现在就值得练。因为以后无论项目变复杂多少，先分层再处理的能力都会一直值钱。复杂项目真正压人的，往往不是复杂本身，而是人一着急就把所有复杂性搅在一起。

4、第一周里最值得刻意培养的一种能力，是“把卡点说清楚”

很多同学会以为第一周最重要的能力是“学会很多内容”。这当然没错，但还有一个更隐蔽、也更值得早练的能力，就是把卡点说清楚。因为你不可能第一周就什么都懂，可你完全可以第一周就开始练会清楚地表达“我现在卡在哪”。这件事对合作尤其重要。

比如更有用的表达不会是“我这里不太行”“这个项目太难了”，而会是“我现在 SSH 和工作区已经通了，环境检查也能过，但我在 smoke 这一步还不能稳定解释 `config.yml` 背后的模型入口，所以结果虽然出来了，我心里还不够稳”。你看，这种表达一出来，别人几乎立刻就知道怎么帮你。因为你的卡点不是情绪化的一团雾，而是已经开始有边界了。

学弟越早学会这样表达，和你、和师兄、和我协作都会更轻松。因为别人不是怕你不会，而是怕你卡得很深却说不清自己到底卡在哪里。

5、到了第三到第五天，真正值得做的是“把最小闭环从自己手里走顺”

如果前两天已经把工具链、工作区、文档主线和第一次 smoke 摸到了，那么第三到第五天最值得追求的，就不是继续横向扩很多内容，而是把最小闭环在自己手里再走顺一遍。所谓最小闭环，就是从登录远程机

器、进入工作区、做环境确认、找到合适文档、跑一次最小 smoke、阅读 run 目录，到最后能留下一小段自己的结论。只要这一圈能基本顺下来，你就已经真正迈入门状态了。

这个阶段非常不建议心态太浮。很多人一看自己已经能做一点事，就急着把目标抬得特别高，仿佛立刻就该深入所有上游源码、论文细节和创新点。其实更成熟的推进方式，是先让最小闭环在自己手里变得越来越顺。因为任何更大的事情，最终也还是建立在许多这种闭环之上。

6、到了第六到第七天，才适合开始把“我会做”往“我会讲”推一步

第一周后段最自然的升级，不是突然去碰所有高级内容，而是从“我会做一点”往“我会讲一点”过渡。也就是说，你可以开始尝试做两件简单但很值钱的事。第一件事，是把自己已经跑过的一轮 smoke 或 batch，按输入、过程、输出、边界的顺序讲一遍。第二件事，是试着向师兄说明当前工作区三层结构和 wrapper 脚本的大致分工。

为什么这一步很重要？因为真正的理解一旦开始能讲出来，就不容易再滑回“只是跟着做”。而且“会讲”这件事，恰恰是后来带学弟、做周报、做专利材料和做论文表达时最重要的底子之一。你第一周末不必要讲得很成熟，但非常值得开始试。

7、如果一周结束时你还没碰 batch，也不代表你慢；关键要看你手里有没有形成真实闭环

我特别想把这个焦虑提前拆掉。因为很多新人会给自己一种无形压力，觉得别人一周都在跑 batch、看图、聊结果，自己如果一周结束还在 smoke 附近打转，是不是就太慢了。其实对当前项目来说，这个判断标准并不公平。因为真正重要的不是你表面碰到了多少关键词，而是你手里有没有形成真实闭环。

如果你一周结束时，已经能独立 SSH 进入、知道去哪里看主线文档、知道 wrapper 和上游的关系、能跑一次稳定 smoke、能找到并解释 run 目录里的核心文件、能把自己的卡点说清楚，那这就是非常扎实的一周。相反，如果有人表面上已经碰了很多 batch 或图表，但问到输入输出就说不清，那种快反而是虚的。

所以第一周结束时，最好的自我评估不是“我碰到哪一步了”，而是“我手里的闭环是不是开始成形了”。这个判断会更真实，也更能保护你的节奏。

8、跑完第一轮 batch 以后，最值得做的不是继续堆更多 trial，而是学会写一段像样的结论

当学弟终于跑完第一轮 batch，往往会有一种非常强的冲动，想把 trial 再加大一点、方法再加多一点、图再多出几张。这种冲动当然能理解，但从成长效率看，更值钱的一步往往是先学会写一段像样的结论。也就是说，你要开始训练自己把这一轮 batch 的总体印象说清楚：哪种方法总体更稳，哪个指标最值得注意，哪一点还不能贸然下结论。

这个动作之所以重要，是因为它会逼着你从“信息接收”走向“判断组织”。图表和 JSON 是外部材料，而结论是你自己的结构。只要你开始能写出哪怕三五句话的像样结论，你和项目之间的关系就会明显变深。你不再只是看见结果，而是开始处理结果。

如果不知道怎么写，完全可以先用一个非常朴素的模板：这一轮在哪个场景、跑了哪些方法、总体可行率大致怎样、耗时层面有什么直观差别、当前还缺什么信息。这个模板不华丽，但很实用。

六、从第一周走向长期可持续的工作方式

最后这部分把短期动作重新接回长期工作方式。目标是让这份手册不只服务“第一次上手”，也服务后面持续几周、几个月的推进。

1、什么时候可以开始读更多上游源码？答案不是“越早越好”，而是“当你开始知道自己想从源码里找什么”

很多同学看到 `splatnav-official/` 里那么多目录，会自然地产生一种冲动：我是不是应该尽快冲进去，把所有源码都看了。阅读源码当然非常重要，但更舒服、更有效的节点，并不是越早越好，而是你开始知道自己想从源码里找什么的时候。否则你很容易出现“看了很多文件，但不知道自己到底在看什么”的疲劳感。

一个比较好的时机通常是这样的：你已经有过一两次真实运行，知道 `wrapper` 启动器和上游实现的大致边界，也已经知道自己最关心的是哪条链路。比如你现在最想搞清楚 `planner` 到底从哪里实例化，或者 `GSplatLoader` 到底把 `config.yml` 背后的东西加载成了什么形态，这时再进上游源码，就会特别有抓手。因为你不是在盲看，而是在带着问题看。

所以不要把“暂时还没系统精读上游源码”误解成自己懒。更重要的是先把问题意识长出来。一旦问题变清楚，源码阅读会顺很多，而且更不容易陷入只见树木不见森林。

2、第二编真正想给学弟的，不是时间表，而是一种能持续推进的工作方式

写到这里，我其实特别想再把这一部分的目标说透一次。它真正想给学弟的，不是一张严丝合缝的时间表，也不是一种“按我说的每天几点做什么就一定成功”的幻觉。更真实的目标，是帮助他形成一种能持续推进的工作方式：先站稳位置，再建立主线；先做最小验证，再读结果证据；先分层判断卡点，再决定求助方向；先形成闭环，再自然升级规模。

这种工作方式听起来不华丽，却是复杂项目里最不容易过时的东西。因为你以后换项目、换代码库、换导师要求，表面内容会变，但这套推进方式依然能帮你稳住自己。对一个零基础但愿意投入的人来说，这反而是最值得早早长出来的能力。

3、读完整个“快速上手”部分以后，你下一步最应该做的三件事

如果你已经把第二编读到这里，下一步其实非常清楚。第一，回到你自己的机器和账户上下文，重新确认 SSH、工作区路径和 `docs/Codex` 主线没有偏。第二，结合第三编再走一次最小 smoke，把 `run` 目录真正当档案袋来读，而不是只盯着终端。第三，给自己留一小段总结，哪怕只有几句话，也把这几天最稳的一步、最卡的一步和下一步想做的事写下来。

这三件事都不花哨，但非常有效。因为它们会把文档里的阅读感，重新接回真实动作里。只有读和做重新咬合起来，这本手册体系才算真正开始发挥作用。

4、最后送给学弟的一句话：不要怕自己现在懂得慢，怕的是一直不把事情讲清楚

最后我想留一句特别简单的话，送给后来真的会照着这一部分走的人。复杂项目里，懂得慢一点并不可怕。零基础、跨专业、第一次接触 Linux、第一次用远程机器，这些都很正常。真正可惜的，往往不是慢，而是一直不把事情讲清楚。今天不清楚自己站在哪，明天不清楚自己卡在哪，后天不清楚自己刚刚到底跑出了什么，这样时间很容易被稀释掉。

所以你完全可以允许自己慢一点，但一定要尽量让自己越来越清楚。清楚路径，清楚边界，清楚输入输出，清楚结果证据，清楚自己不懂什么。只要这种“越来越清楚”的趋势开始出现，这个项目就已经真正开始被你接住了。那时，所谓快速上手，就不再是一句口号，而会变成你脚下正在发生的现实。

第三编 把一次运行真正看懂

等最小动作已经能做出来以后，下一步就是把一次运行解释清楚。这里重点解决输入、输出、验收、日志和 run 目录这些最容易混掉的地方。

这一份文档专门解决一个很实际的问题：我在这个项目里到底往程序里送了什么，它又吐出了什么，怎样才算真正跑成功。这个问题看起来朴素，其实特别重要。因为很多新人会把“能启动脚本”和“理解一次运行”混在一起。前者只需要照抄命令，后者则要求你知道输入是什么、输出是什么、为什么现在看这个文件、如果结果不对要先查哪一层。

如果这一关不过，后面很容易出现一种假象：命令我会敲，目录我也见过，图我也能打开，但一旦别人追问“这次输入用了什么场景、哪个脚本起了作用、图里这个数字是怎么算出来的、为什么这次虽然没报错却不能算成功”，你就会立刻卡住。所以这一部分不是教你背命令，而是教你把一次运行拆成一个个可解释的部件。

一、先建立“一次运行”的整体观念

这一部分先不急着想钻进脚本，而是先把“一次运行”这件事从概念上讲顺。只有整体观站稳了，后面 smoke、batch、overnight 才不会被看成三堆互相无关的命令。

1、先把“输入”这个词讲清楚

很多人一说输入，就只想到命令行参数。其实在 Splat-Nav 当前工作区里，一次运行的输入至少有四层。第一层是场景与模型输入，也就是某个 config.yml 所指向的 GSplat 场景模型。第二层是实验设定输入，包括场景名、机器人半径、速度、加速度、体素边界、分辨率、方法名这些信息。第三层是运行环境输入，包括你是否进入了正确的 Conda 环境、当前 Python 是否可用、GPU 和依赖状态是否健康。第四层才是命令行输入，也就是你在 shell 或 tmux 里真正敲下去的那一串命令参数。

只要把这四层分开，你对“输入”的理解就会立刻提升一个层次。因为你会发现，很多失败并不是参数本身输错了，而是前面三层有一层不成立。比如 --scene old_union 没问题，但 --config 指错了；或者 --config 对了，但当前环境里的 torch.cuda.is_available() 是 False；又或者环境没问题，但 run 的工作目录不对，导致相对路径失效。真正成熟的输入意识，不是死记参数，而是知道输入是一整条链。

2、一次运行最常见的三种规模

在当前工作区里，你最常接触的运行规模有三种：smoke、batch 和 overnight。它们不是三个完全不同的项目，而是同一条主线上的三个层次。

smoke 是最小冒烟测试。它只做一件事：用最小代价确认主链路能不能通。对于学弟来说，这是最适合第一次动手的层次，因为它引入的变量最少，一旦出错，也最容易定位。

batch 是批量实验层。它不再满足于一条路径跑通，而是要在多个起终点样本上连续运行，并统计可行率、平均耗时、P95 耗时等指标。它比 smoke 更接近论文实验和真正的结果对比。

overnight 则是把 batch 在更长时间窗口里持续跑，自动一轮轮归档、可视化和汇总。它的意义在于充分利用夜间机器空闲时间，积累更稳的统计结果，而不是为了把命令写得 longer。

理解这三个层次非常重要。因为你如果一上来就混着看，会觉得当前工作区脚本很多、关系很乱。实际上，它们只是围绕同一条主链路，对应不同规模和管理需求的不同包装。

二、先把最小 smoke 链路讲透

smoke 是整个项目最适合新人建立感觉的入口。这部分把 smoke 的输入、处理中间链和最低成功标准完整串起来。

1、smoke 的输入到底是什么

先从最小的 smoke 说起，因为这是整个项目里最容易看懂、也最适合作为“输入输出样板”的一条链路。当前最小规划测试的核心 Python 入口是 `scripts/smoke_splatplan.py`。它要求你提供四个显式参数：`--config`、`--scene`、`--method`、`--output`。

这里面最关键的是 `--config`。它指向某个具体的 `config.yml`，而这个文件背后关联着场景模型和数据解析信息。简单说，你不是直接把一堆 Gaussian 参数手工塞给规划器，而是告诉脚本：去这个位置读取某个已经训练好的 GSplat 场景。当前项目里，很多“为什么同一个脚本在不同场景上行为不同”的根源，最终都要回到 `config.yml`。

`--scene` 是实验场景名，比如 `old_union`、`flight`、`stonehenge`、`statues`。它看起来只是一个短字符串，但实际作用非常大。因为脚本内部会根据这个名字去选取一组预设，包括机器人半径、速度上限、加速度上限、体素下边界、上边界和分辨率。这意味着场景名不是纯标签，而是决定实验几何范围和机器人配置的一把钥匙。

`--method` 决定你要跑的是 `splatplan` 主方法，还是 `sfc-1` 到 `sfc-4` 这些 baseline。它的作用不是切换一个显示名称，而是决定到底构建哪个 `planner` 类。`splatplan` 走 `SplatPlan`，而 `sfc-*` 走 `SafeFlightCorridor` 的不同模式。

`--output` 则决定这次 smoke 的结构化结果写到哪里。注意，它不是说“程序自己随便找个地方存一下”，而是你明确指定一个 JSON 输出位置。通常这个位置会被安排在某个 `run` 目录里，这样日志、命令、环境摘要和结果 JSON 才能放在一起。

2、smoke 的中间处理过程在做什么

当 `smoke_splatplan.py` 接收到这些输入后，它先做的不是直接规划，而是构造上下文。第一步是根据 `--scene` 把预设字典里对应的机器人和体素信息取出来。第二步是通过 `GSplatLoader` 读取 `config.yml` 指向的 GSplat 模型，把场景变成后续规划可用的张量属性。第三步是根据 `--method` 决定要实例化 `SplatPlan` 还是 `SafeFlightCorridor`。第四步才是选取一组固定起终点，调用 `generate_path` 执行真正的规划。

这个顺序一定要理解。很多新人会把 `generate_path` 想成一条神秘黑箱命令，输入起点终点，输出轨迹。实际上，起点终点之前还有场景装载、机器人参数设定、体素边界建立和 `planner` 构建这些步骤。你越早理解这些“前置处理”，后面读 `batch` 和 `overnight` 脚本时就越不容易迷路。

3、smoke 的输出长什么样

一次规范的 smoke 运行，至少会产生两类输出。一类是结构化结果，也就是你通过 `--output` 指定写出的 JSON 文件。另一类是工程化证据，也就是 `run` 目录里的 `cmd.txt`、`stdout.log`、`env.txt`、`notes.md`。

先说 JSON。它通常会包含 `meta`、`device`、`plan_time_sec`、`start`、`goal` 和 `output` 这些字段。`meta` 里会记录场景名、方法名、加载时间、机器人参数、体素参数等；`device` 会告诉你这次是在 `cuda` 还是 `cpu`；`plan_time_sec` 记录的是整体规划耗时；`start` 和 `goal` 是这一次实际使用的起终点；而 `output` 里则是更贴近算法结果本身的字段，比如 `feasible`、`path`、`traj`、`num_polytopes` 和若干阶段耗时。

再说工程化证据。它们经常被新人低估，但其实非常重要。`cmd.txt` 说明这次到底跑了什么命令，`stdout.log` 说明运行过程中说了什么、报了什么，`env.txt` 说明当时的机器和环境上下文是什么，`notes.md` 则预留给人工补充结论、问题和后续动作。你以后做复盘、排错、周报和交接，很多价值不一定来自 JSON 本身，而是来自这些配套证据。

4、一次 smoke 运行最低怎样才算成功

这个问题非常关键。当前项目里，一次 smoke 不能因为“终端没报错”就直接算成功。最基本的成功标准至少有四条。第一，run 目录确实创建出来了，而且里面有 `cmd.txt`、`stdout.log` 和结果 JSON。第二，结果 JSON 结构完整，关键字段能读。第三，这次运行使用的场景、方法、配置路径和输出路径彼此一致，没有明显串场。第四，`output.feasible` 和耗时等字段至少在语义上能解释，不是空值、乱值或者明显异常。

需要特别提醒的是，`feasible = false` 并不总是等于“程序坏了”，但在入门阶段，如果你连这一点都解释不清楚，就还不能说自己真正理解了一次运行。真正成熟的说法应该是：程序链路跑通了，但在当前约束下这次规划不可行，或者程序链路没通，结果字段根本不完整。这两类情况是完全不同的。

三、从 batch 到 overnight，规模放大以后发生了什么

当单次运行变成成组实验，再变成长时段批跑，输入输出的意义都会升级。这一部分专门处理这种规模变化带来的理解变化。

1、batch 的输入比 smoke 多了什么

当你从 smoke 进入 batch，变化最大的不是“参数更多了”，而是实验对象从单次规划变成了成组样本。`scripts/run_official_style_batch.py` 在输入上仍然保留 `config`、`scene`、`methods`、`trials`、`seed`、`keep_traj_limit`、`output` 这些参数，但它内部会生成一整组起终点对，而不是只跑一次固定起终点。

这里的 `methods` 往往是一个列表，比如 `sfc-1,sfc-2,sfc-3,sfc-4,splatplan`。这意味着 batch 本身就是为横向比较而设计的。`trials` 决定每种方法要跑多少个样本，`seed` 决定样本顺序或随机过程的稳定性，`keep_traj_limit` 决定保留多少条详细轨迹记录，以避免 JSON 过于庞大。对新人来说，理解这些输入的重点不是参数名，而是它们分别在控制什么层次：方法集合、样本数量、随机性、结果保存粒度。

2、batch 的中间处理在做什么

batch 的核心流程可以理解成“在单次 smoke 的基础上，加上样本生成、方法循环和统计汇总”。它先根据场景预设生成一组围绕场景中心的起终点对，再按方法名构建 planner。随后，对每种方法，它会在多个 trial 上循环调用 `generate_path`，记录每次调用的可行性、耗时和部分轨迹结果，最后再把这些 trial 记录汇总成这一方法的整体统计。

这就是为什么 batch 输出里会有 `feasible_rate`、`avg_plan_time_sec`、`p95_plan_time_sec` 这些字段。它们不是某一条轨迹的属性，而是“一种方法在一批样本上的总体表现”。理解这一点后，你再看到图表和 CSV，就知道自己在看的是“总体行为”，不是单次偶然现象。

3、batch 的输出具体看什么

一次 batch 运行的主结构化输出通常是一个 `batch_results.json`。它里面至少会包含 `meta` 和 `results` 两层。`meta` 负责记录这次 batch 的整体背景，比如场景、方法列表、样本数量等；`results` 则通常是按方法

组织的列表，每个方法条目里会有 `feasible_rate`、平均耗时、P95 耗时、保存轨迹数和若干 `trial_records`。

你第一次看这个 JSON 时，不要试图把所有 trial 细节全啃掉。最有效的方法，是先从每种方法的总体统计开始看。也就是说，先问：`splatplan` 的可行率多少，`sfc-1` 到 `sfc-4` 各是多少，平均耗时差多少。等这些总览数字有了感觉，再回头抽几条 `trial_records` 看具体轨迹细节。

如果可视化脚本也一起运行，那么你通常还会得到一组 PNG 图和一个简单的 Markdown 摘要。比如 `feasible_rate.png`、`plan_time_boxplot.png`、`traj_xy_overlay.png` 这些文件，就是为了把 batch 的主要统计用图形形式表达出来。它们不是替代 JSON，而是帮助你更快看懂 JSON。

4、overnight 比 batch 又多了什么

从 batch 到 overnight，真正新增的不是算法复杂度，而是“多轮任务管理”。`scripts/run_overnight_12h.sh` 会在指定总时长内反复创建 run 目录、调用 batch 脚本、保存结果并生成图表。也就是说，overnight 是“多轮 batch 的流水线”。

这件事对新人很重要，因为它提醒你：当实验从单轮走向长时段时，项目管理需求会迅速上升。你不仅要关心一轮跑得对不对，还要关心每轮的命名是否规范、日志是否完整、图是否生成、汇总是否顺利。这正是 wrapper 层存在的价值。

5、overnight 的输出应该如何阅读

一轮 overnight 跑完后，你往往会面对很多个 `formal_batch_<scene>_iterN` 目录。如果没有整理思路，很容易看花眼。更稳的阅读顺序是这样的。先看 `post-report` 阶段生成的总报告目录，通常里面会有 `overnight_summary.csv`、`overnight_summary.md` 和趋势图。因为这一层能先帮你回答“总体上哪种方法更好、哪几轮结果最值得回看”。

然后再回到具体某一轮的 run 目录，去看该轮的 `batch_results.json`、`figs/summary.md` 和各类 PNG。如果某一轮特别好或特别怪，再进一步打开它的 `stdout.log` 看中间过程。这个顺序非常重要。因为总报告帮你先筛重点，具体 run 目录再负责细看细节。

四、结果阅读、验收证据与假成功识别

很多人会跑，却不会验。这里集中讲结果该怎么看、证据该怎么留、假成功该怎么识别，以及 run 目录为什么要被当成档案袋。

1、跑完后优先打开哪些文件

这是当前项目里非常实用的一条经验。无论是 smoke、batch 还是 overnight，跑完以后都不要急着先看图，更不要急着凭终端最后一行文字下结论。最稳的顺序通常是：先看 `stdout.log` 末尾，确认任务到底是正常结束、异常退出，还是虽然结束但中途有关键警告；再看 `cmd.txt`，确认当前看的 run 与你记忆中的命令一致；然后看结果 JSON 的关键字段，确认 `scene`、`method`、`feasible`、耗时和输出结构合理；最后才去看图和 Markdown 摘要。

对 batch 和 overnight 来说，图很有用，但图是解释层，不是事实源头。真正的事实源头仍然是日志和结构化结果。如果这个阅读顺序建立起来了，你以后对结果的判断会稳很多。

2、runs 命名规范为什么重要

当前工作区统一采用 `runs/YYYY-MM-DD_HHMM_<topic>/` 的命名规则。表面看，这只是一个文件夹名字；但它其实承载了三层信息。第一，它告诉你任务发生在什么时候。第二，它告诉你这次任务是什么主题，比如 smoke、formal batch、overnight report。第三，它天然适合按时间排序，便于你后来回溯。

命名规范不是形式主义。实验一旦多起来，如果目录名里没有时间和主题，你会非常难找。更要命的是，未来做汇报时，你会拿不准自己说的是哪一轮结果。规范命名，本质上是在为未来的自己和未来的学弟减少混乱。

3、一次运行里“哪个脚本做了哪个事”

很多新人最想知道的其实就是这一点，所以这里直接说透。若是一次最小 smoke，通常是 `run_splatplan_smoke.sh` 负责创建 run 目录和准备环境，`smoke_splatplan.py` 负责真正读取模型、构建 planner 并执行单次规划。若是一次 batch，`run_official_style_batch.py` 是真正执行主循环和生成统计 JSON 的核心。若是 overnight，`run_overnight_12h.sh` 是大调度器，负责在时间窗口内多次调用 batch；`visualize_batch_results.py` 负责某一轮 batch 的图；`aggregate_batch_runs.py` 和 `post_overnight_report.sh` 负责事后汇总。若想让任务脱离当前终端长期运行，则 `run_in_tmux.sh` 负责把命令送进 tmux 并和 run 目录绑定。

你把这一层关系记住了，就不会再面对脚本列表时一脸茫然。你会知道：有些脚本是启动器，有些是主执行器，有些是汇总器，有些是看护器。

4、几种最常见的假成功场景

第一种假成功，是终端退出了，但 run 目录里只有日志，没有结构完整的 JSON 或图表。说明流程可能在中途就断了。第二种假成功，是 JSON 存在，但关键字段为空、缺失或语义明显异常。第三种假成功，是图已经生成，但图里的数字和 JSON 汇总对不上。第四种假成功，是 `feasible` 全是 `false`，但你却因为“程序没崩”就宣布方法跑通。第五种假成功，是看的 run 目录根本不是你刚才那轮命令对应的目录，自己把结果串场了。

学会识别假成功，是成长非常快的一步。因为它会逼着你从“看终端最后一行”升级到“核对日志、命令、JSON 和图之间的一致性”。

5、一次真正过关的运行，应该留下哪些证据

最小过关证据至少包括：一个命名规范的 run 目录；能说明运行背景的 `env.txt`；能复原操作的 `cmd.txt`；能追过程的 `stdout.log`；至少一个结构完整的结果 JSON；如果是 batch 或 overnight，最好再有图表和汇总 Markdown。再往上走一层，还应该有你自己在 `notes.md` 里留下的结论、问题和下一步。

证据这个词很重要。因为项目不是只做给当下这一分钟看的，而是要能交接、能复盘、能汇报。没有证据的“我昨天好像跑出来过”，在合作和科研里几乎没有价值。

6、读完这一部分后，你应该具备的能力

如果这一份文档起作用了，你现在应该已经能把一次运行拆成输入、处理和输出三段。你应该知道 `config.yml`、场景名、方法名、环境状态和命令行参数共同组成输入；知道 smoke、batch、overnight 只

是同一主链路的不同规模；知道 run 目录、JSON、日志、图和汇总之间各自承担什么职责；也知道“没报错”不等于“真正成功”。

接下来最自然的下一步，就是进入 第二编。因为现在你已经知道一轮运行是什么样子的了，下一步就该开始按阶段去做真正的动作了。

五、把“一次运行”真正讲明白

前面解决的是事实层，最后这部分解决的是解释层。目标是让你不仅知道一次运行发生了什么，还能把它复述出来、汇报出来、带着失败案例一起讲清楚。

1、如果把“一次运行”讲成人话，它其实更像一条流水线，而不是一条命令

很多新人最容易被一个错觉带偏，那就是以为“一次运行”就是自己在终端里敲下去的那条命令。这个理解不能说完全错，但确实太窄了。更贴近当前项目实际的说法是：一次运行是一条流水线。你在终端里敲的那条命令，只是把这条流水线启动起来；真正的运行，还包括它之前已经准备好的场景、模型、环境和路径边界，也包括它之后沉淀下来的日志、JSON、图表和汇总。只有把这些前后都看进去，你才算真的理解了“一次运行”。

我之所以反复强调这个角度，是因为它会直接改变你的学习方式。你如果把运行理解成一条命令，就会自然把注意力过度集中在“命令怎么写”；一旦命令敲完、终端滚了几行字，你就会误以为事情结束了。可如果你把它理解成一条流水线，你就会更自然地去问：这个命令前面准备了什么，这个命令中间调用了谁，这个命令后面留下了什么。问题一换，理解就会深很多。

对学弟来说，这个思路尤其重要。因为零基础同学最怕的并不是多看几页文档，而是明明跟着做了几个动作，却始终不知道这些动作属于哪一层。只要“一次运行是一条流水线”这个概念立住了，后面很多输入输出问题都会开始自己归位。

2、一次运行开始之前，你其实已经默认依赖了很多前置事实

有时候我们会说“我现在准备跑一轮 smoke”或者“我准备开一轮 batch”。这句话表面上像是在描述一个即将发生的动作，实际上它已经偷偷依赖了不少前提。比如说，这台机器现在是你自己的账户，不是你师兄的账户；你当前确实在 `/home/howard/Splat-Nav` 这个工作区，而不是某个同名但不是这个项目的目录；`splatnav` 这个 Conda 环境已经存在，里面的大部分关键依赖能 import；你手上的 `config.yml` 确实指向某个已经能读取的场景模型；你打算把输出写到的路径具备写权限；你现在看的脚本版本与当前工作区是一致的。

这些前提平时不会全都写在命令里，所以新人很容易忽略它们。可一旦哪一项不成立，问题就会出现，而且常常不是很显眼的那种。例如，你脚本命令看着全对，但 `config.yml` 背后的目标路径其实不存在，于是 loader 在很后面才炸；或者你当前 Python 没切到 `splatnav` 环境，前面甚至能跑几行，到了某个 import 才突然中断。你如果没有前置事实意识，就会很容易把这类错误误认成“算法问题”或者“代码问题”。

所以我希望学弟以后每次准备启动一轮运行时，脑子里先有一个更稳的自问：我这条命令不是从真空中长出来的，它依赖的前提今天都还成立吗？这个习惯看似慢一点，其实会极大降低后续返工的概率。

3、把 smoke 真正拆开来看：从你敲下命令那一刻，到 JSON 落盘，中间到底发生了什么

我们不妨把一次最典型的 smoke 运行完整讲一遍。因为只要这条最小链路你真的看顺了，后面的 batch、overnight 都会轻松很多。一次标准的 smoke，通常会从 `scripts/run_splatplan_smoke.sh` 这层 wrapper 开始，而不是直接从 Python 入口开始。这里有个非常重要的工程含义：项目没有要求你每次都手工创建结果目录、手工记录命令、手工保存日志，而是先由 shell 启动器帮你把“跑一次”该有的工程骨架搭出来。

这意味着，当你运行 `run_splatplan_smoke.sh` 时，脚本做的第一件事不是规划，而是先调用 `create_run_dir.sh` 去生成一个命名规范的 `run` 目录。这个目录里会先生成 `env.txt`、`notes.md`、`cmd.txt`、`stdout.log` 这些骨架文件。你可以把这一步理解成“先铺桌布，再上菜”。如果没有这一步，后面哪怕算法跑出来了，实验也还是零散的。

接下来，这个 shell 脚本会把你此次运行的命令写进 `cmd.txt`，然后激活 `splatnav` 环境，清掉运行时不希望强绑的 `LD_LIBRARY_PATH`，最后真正调用 `scripts/smoke_splatplan.py`。从这一刻开始，算法链才真正启动。也就是说，在 smoke 里，命令、环境、run 目录、日志挂接和算法入口并不是混成一团的，它们是分层完成的。学弟只要把这层顺序看清，就会很容易明白为什么当前工作区看起来文件很多，但其实是分工的。

而到了 Python 入口 `smoke_splatplan.py` 内部，它首先不是直接算轨迹，而是先解析参数，再根据当前工作区的软链接/目录策略找到上游仓库根目录，然后把这个上游路径插入 `sys.path`。这一步解决的是“算法真正实现代码到底从哪里来”的问题。只有上游路径解析稳了，后面导入 `GSplatLoader`、`SplatPlan`、`SafeFlightCorridor` 这些核心类才有意义。

再往后，它会根据 `--scene` 去取对应场景的预设，包括机器人半径、加速度上限、速度上限、体素边界、分辨率以及和某些场景相关的均值配置。然后再通过 `GSplatLoader` 去读取 `config.yml` 背后的模型。这一步特别像“把场景请进来”。如果场景没有真正进来，后面的 planner 再聪明也无从谈起。也正因为如此，学弟以后碰到 smoke 出错时，不要只盯着 planner，往往 loader 这一层才是前几步最值得先查的地方。

最后，脚本才会根据 `--method` 决定实例化哪个 planner，并对一组固定的起终点调用 `generate_path`。这里为什么用固定起终点？就是为了降低第一次理解的难度。因为只要起终点也随机化，新手就又多了一层变量，很多比较就不稳定了。固定起终点的好处，是你第一次看 smoke 时，注意力可以集中在“链路有没有跑通、输出长什么样、可行性和耗时怎么解释”这些更基础的问题上。

4、如果你第一次 smoke 跑完了，最应该问自己的不是“成功了吗”，而是“我现在能把它复述出来吗”

项目学习里有一个非常关键但经常被忽略的节点，就是“能不能复述”。很多人会把“我亲手跑了一次”误以为“我已经理解了这次运行”。其实这两者差得很远。真正能说明你开始理解的，不是你终端里敲过那条命令，而是你能不能把这轮 smoke 的来龙去脉讲给别人听。

比如更成熟的复述应该大致像这样：这次我是通过 `run_splatplan_smoke.sh` 触发了一次最小规划测试；它先给我创建了一个 `run` 目录，把环境和命令挂进去；然后调用 `smoke_splatplan.py`；Python 脚本根据 `scene` 加载预设，根据 `config.yml` 用 `GSplatLoader` 读取场景，再按 `method` 构造 planner，最后对固定起终点执行规划并把结果写到 JSON。你看，这里面并没有高深的新概念，但它已经说明你不是只会敲命令了。

如果你现在还做不到这种复述，也完全没关系，但你要知道那意味着什么：说明你下一步最应该做的，不是再跑三次同样的命令，而是回头看一次刚才那轮 run 目录，结合脚本入口把流程顺一遍。很多时候，少跑一轮，多复述一遍，反而更能长理解力。

5、batch 为什么会让很多人第一次真正意识到“结果”这个词的分量

在 smoke 阶段，你面对的是一轮最小链路，它的价值是让你看见最基础的输入和输出。但一旦进入 batch，项目的味道会明显变掉。因为 batch 不再只要求“这条路径能不能出结果”，它开始要求“多个方法在一批样本上整体表现如何”。也就是说，结果不再是单个点，而开始变成一个统计意义上的面。

这一步对新人是很大的认知升级。因为在 batch 之前，你很容易把运行理解成“某个命令 + 某个输出 JSON”。而到了 batch，你会第一次被迫思考更像实验的问题：为什么要固定 seed，为什么要设 trials，为什么要把多种 method 放进同一次批跑，为什么要限制详细轨迹保存数量，为什么会有 feasible rate 和 p95 这种整体指标。这些问题一旦进脑子，你对项目的理解就已经从“脚本使用者”开始转向“实验参与者”了。

当前工作区里，run_official_style_batch.py 就承担了这个角色。它在工程上仍然是一个脚本，但在认知上它更像一张实验框架。学弟以后读它，不要只盯着循环里具体怎么写，而要先看到它的实验结构：先拿到场景和配置，再决定方法集合，再生成多组起终点样本，再对每种方法逐个 trial 跑，最后把 trial 结果压成统计摘要。这条结构感一旦建立起来，后面看 JSON、看图、看汇总都会顺很多。

6、batch_results.json 真正难的地方，不是字段多，而是你要知道每一层字段在回答什么问题

很多新人第一次打开 batch_results.json，会有一种“内容好多，但不知道从哪下嘴”的感觉。这个感觉很正常。因为 batch 输出的难点不在于 JSON 语法，而在于它把好几个层次的信息装在了一起。你如果没有先分层，就很容易把一条记录里不同层次的字段混着看，越看越乱。

一个比较稳的读法，是先把 batch 输出分成三层。第一层是“这一整轮 batch 的背景”，通常放在 meta 里，回答的是这轮实验在哪个场景、用了哪些方法、计划跑多少 trial、保存策略怎样。第二层是“每种方法的总体表现”，也就是 results 列表里每个方法条目顶部的 feasible_rate、平均时间、P95 时间这些字段，它们回答的是“这类方法总体上站得住吗”。第三层才是具体 trial_records，回答的是“某一次样本里到底发生了什么”。

你只要记住这三层，第一次看 batch JSON 就会轻松很多。正确顺序几乎总是：先看 meta，再看每个方法的总览统计，最后才抽几条 trial 细看。不要一上来就钻 trial 细节，那样非常容易在局部里打转，看半天却没抓到主结论。

7、图表不是用来替代 JSON 的，而是用来帮你更快发现应该回到 JSON 的哪一块

很多同学天然更喜欢图，因为图看起来直观。这个偏好没问题，但如果在学习阶段把图当成主要事实来源，会有两个风险。第一，图会帮你压缩信息，但也会顺便隐藏不少上下文；第二，你很容易先被某张“看起来不错”的图吸走注意力，却没意识到那张图到底是从什么数据压出来的。

当前工作区里的 visualize_batch_results.py 就是一个很典型的辅助层脚本。它做的事并不是替你判定方法好坏，而是把 batch_results.json 里最值得先看的几类统计压成图，比如 feasible rate 柱状图、plan time 分布图、轨迹投影图和一个 summary.md。它的正确用法不是“我只看图就够了”，而是“我先看图找到异常或重点，再回到 JSON 确认这个判断站不站得住”。

你以后越早建立这个顺序，结果判断就越稳。图是望远镜，JSON 是地面。只拿望远镜不落地，很容易飘；只盯 JSON 不抬头，也容易看不出整体形状。两者配合才是最舒服的状态。

8、overnight 的真正难点，不是“跑得久”，而是“多轮结果还能不能保持可读”

到了 overnight，很多新人会以为这是一个纯粹拼机器时间的阶段。其实机器时间当然重要，但更关键的挑战是：当 batch 变成多轮 batch 之后，结果还能不能保持可读、可归档、可比较。因为一旦 overnight 开始循环跑，事情就不再只是“某一轮跑得对不对”，而开始变成“第 17 轮和第 46 轮放在一起时，你还能不能一眼看出谁是谁、哪轮更值得关注、哪些图和哪些 JSON 是一一对应的”。

这正是为什么当前工作区用了 `run_overnight_12h.sh`、`aggregate_batch_runs.py`、`post_overnight_report.sh` 这套链。它们不是为了让命令显得高级，而是在解决一个很实际的问题：多轮实验如果没有统一归档和总报告层，第二天早上你面对的就不是成果，而是一地散文件。学弟越早理解这层价值，以后就越不会把工程化包装误认为“附加工作”。很多科研项目真正把人拖垮的，不是算法本身，而是结果一多就开始失去可读性。

9、run 目录为什么应该被当成“这一轮运行的档案袋”

如果一定要给 run 目录找一个很适合新人的比喻，我会愿意叫它“档案袋”。档案袋的价值不在于它看起来整齐，而在于与同一件事相关的证据都被放到了一个能回查、能搬运、能交接的地方。你以后只要看到某个 `runs/YYYY-MM-DD_HHMM_<topic>/`，脑子里最好立刻浮现的不是“一个输出目录”，而是“这是一轮运行的完整档案”。

这种理解会直接改变你看文件的方式。你不会再把 `env.txt` 视作无聊文本，因为它在记录这轮运行当时的机器和环境上下文；你不会再把 `cmd.txt` 当成重复内容，因为它在保存“我到底跑了什么”；你不会再把 `stdout.log` 看成杂乱输出，因为它其实是在保留过程证词；你也不会再把 `notes.md` 看成可有可无，因为那是人类判断、问题归纳和下一步动作最自然的落点。只要你把 run 目录当档案袋，对“证据”这件事的敏感度就会明显提高。

10、把 run 目录里的关键文件一个一个讲成人话

为了让这种档案袋意识更扎实，我们不妨把 run 目录里最常见的几个文件逐个讲成人话。`env.txt` 的作用，其实就是给未来的你和未来的别人留一句话：这轮运行当时站在什么环境上。它记录了时间、用户、主机、当前目录、GPU、Python 版本，有时还会附带上游仓库 commit。你可以把它看成“实验现场背景板”。

`cmd.txt` 更像操作记录单。很多同学刚开始会觉得，“我又不是不记得自己刚敲了什么，为什么还要写下来？”这个想法在命令很短、事情很少的时候似乎能成立，但实验一多、时间一长，人对自己做过什么的记忆会迅速模糊。`cmd.txt` 的价值，就在于把主观记忆换成客观记录。后面你如果需要复现、排错或汇报，它会非常值钱。

`stdout.log` 则更像过程录音。它不是只在出错时才有用。哪怕任务正常完成，日志也能帮你确认各阶段顺序、加载耗时、警告信息和结束位置。很多“好像成功了，但总感觉哪里不对”的情况，最后往往都是通过日志看出来的。

结果 JSON 则是算法与实验层的正式答卷。它不像日志那样是滚动文本，而是更结构化地保存了关键背景字段和结果字段。也正因为如此，它特别适合被后续脚本继续处理、画图、做汇总、拼成 Markdown。你以后只要理解了 JSON 是正式答卷，日志是过程录音，很多文件之间的关系就会自然得多。

最后是 `notes.md`。这个文件特别值得鼓励学弟去用。因为任何自动化脚本都擅长保存机器能自己知道的东西，但它保存不了人的判断。比如“这轮 `smoke` 其实链路通了，但场景配置我还没完全理解”“这轮 `batch` 看起来 `sfc-3` 比预期稳，需要回头对照论文解释”这种东西，只有你自己写下才会留下来。一个项目是不是越来越可交接，很多时候就差在这类人工摘要有没有被认真留下。

11、如果师兄问你“这轮运行输入了什么”，你最好不要只回答一个文件路径

这是一个很容易暴露理解层次的提问。很多新人被问“输入是什么”时，第一反应是回答 `config.yml` 路径，或者回答“我跑的是 `old_union`”。这些回答不算错，但都太单薄。更像样的回答，应该至少能同时覆盖几层：场景配置输入、方法选择输入、环境输入和命令级输入。

比如你完全可以这样回答：这轮运行用的是 `old_union` 场景，对应的模型入口是某个 `config.yml`；方法上我跑的是 `splatplan` 和几种 `sfc-*` baseline；运行环境是在 `splatnav` Conda 环境下，用当前工作区 `wrapper` 启动；命令层则是通过某个 shell 脚本或 Python 入口触发。你看，这样一说，别人立刻就能感觉到你不是只记了一个路径，而是知道输入是怎么分层的。

这类回答训练得越早，你后面做组会、做周报、写阶段汇报就越不会虚。因为复杂项目里，真正成熟的表达从来不只是“我输入了某个文件”，而是“我知道这轮运行站在怎样的一组前提与选择上”。

12、如果师兄问你“这轮运行输出了什么”，你最好也不要只回答“一张图”

和输入一样，输出也很容易被说窄。很多人第一次出图以后很兴奋，于是别人一问“输出是什么”，脑子里就只剩那张最显眼的图。可实际上，图只是输出的一种，而且往往不是最基础的那一种。对当前项目来说，输出至少要分成结构化结果输出、日志输出、归档输出和可视化输出。

更成熟的回答应该像这样：这轮运行留下了一个规范命名的 `run` 目录；里面有 `cmd.txt`、`env.txt`、`stdout.log` 这些工程化证据；有一个 `smoke` JSON 或 `batch` JSON 作为结构化结果；如果是 `batch` 以上规模，还会有图表和一个简单摘要。这种回答的好处是，它能一下子把“我知道什么叫真正的输出”表现出来。因为你已经不再把输出理解成一张图或一个终端文本，而是理解成一整套结果资产。

13、看起来最像“懂了”的时候，反而最容易忽略一致性检查

学习过程中有一个挺有意思的现象：最危险的时候，往往不是完全看不懂，而是已经看懂了一些，于是开始放松一致性检查。比如你一看到 JSON 里有 `feasible_rate`，就急着下结论；或者一看到某张图里蓝色柱子最高，就立刻说某方法最好；又或者一看到日志最后写着 `completed`，就默认这轮全对。这些都很像“已经开始会看结果”，但如果你不再核对命令、场景、方法、JSON、图和 `run` 目录之间是不是一致，就很容易掉进假成功。

所以我特别建议学弟在学习阶段，把“一致性”当成一个很值得练的词。一个结果值不值得信，很多时候不是看它长得漂不漂亮，而是看它上下文是不是闭合。命令和结果对得上吗，场景和图对得上吗，方法名和汇总项对得上吗，`run` 目录和你记忆中的那轮任务对得上吗。只要这层习惯开始长出来，项目判断力就会稳很多。

14、第一次自己解释一轮结果时，不妨按“背景、过程、结果、边界”这四句话来讲

很多新人面对结果时容易散，不是因为不知道点什么，而是因为没有表达骨架。我很推荐一个特别适合入门阶段的小骨架，就是四句话：背景、过程、结果、边界。第一句讲这轮运行是在什么场景、什么方法、什么

规模下做的。第二句讲它通过哪条脚本链路跑起来。第三句讲它最后沉淀了哪些关键结果，比如可行率、耗时、图表或 JSON。第四句讲当前还不能过度解读什么，或者还有哪些地方需要继续核实。

这个骨架的好处，是它既不会把你逼成背稿机器，也不会让你讲着讲着失去主线。你以后无论是在师兄旁边口头说一下，还是在 `notes.md` 里留一句总结，甚至未来做 PPT，这四句话都很够用。对零基础学弟来说，先学会把结果讲成四句话，比一上来堆很多术语有用得多。

15、把“失败”也纳入输入输出视角，你对项目的掌控感会强很多

最后我特别想补一层：失败并不是输入输出视角里的例外，恰恰相反，它也是输入输出的一部分。也就是说，就算一轮运行没有得到你理想中的可行路径、没有生成完整图表、甚至在中途报错，它仍然留下了某些输入、某些过程痕迹和某些输出残片。你如果能用同样的分层方法去看失败，就会发现失败其实没那么可怕。

比如这轮任务报错了，那输入是不是还可以复原？可以，因为 `cmd.txt` 和 `env.txt` 还在。处理中断了，那能不能知道断在哪一步？可以，因为 `stdout.log` 还在。输出不完整，那能不能分辨是结构化结果没生成，还是图表环节没跑完？通常也可以。只要你开始这样看，失败就不再是“黑箱突然坏掉”，而会重新变成一条可以被拆解、被定位、被汇报的问题链。

这就是为什么我一直希望学弟早点建立输入、处理、输出这个三段式视角。因为它不只适用于“成功的一轮”，它同样适用于“没做成的一轮”。一旦这个视角稳了，项目就真的开始变成你能接得住的东西，而不再只是看上去会动、但自己心里发虚的一堆脚本。

16、读完“输入输出与一次运行验收”部分以后，最值得带走的不是更多字段名，而是一种“运行可解释”的底气

如果这一部分真正起作用了，那么你读到这里时，脑子里最该留下来的，不应该只是又多认了几个 JSON 字段或脚本名。更重要的，是你开始长出一种底气：哪怕面对一轮以前没见过的运行，你也知道该从哪里拆、从哪里看、从哪里讲。你知道输入不只是参数，输出也不只是一张图；你知道 `run` 目录不是垃圾桶，而是档案袋；你知道 `smoke`、`batch`、`overnight` 不是三套互不相关的东西，而是一条主线在不同规模下的展开。

这种底气对新人极其宝贵。因为复杂项目最磨人的，不是学习量本身，而是“我好像在做，但我解释不清”。一旦你开始能解释，项目对你的压迫感就会明显下降。它仍然复杂，但不再模糊。它仍然大，但不再一团黑。很多真正的入门，往往就发生在这一刻。

接下来最自然的动作，就是回到 第二编，把这里刚刚建立起来的输入输出意识，重新放回真正的动手路径里。因为理解到这一步以后，下一阶段就不该只是“知道”，而是要开始更稳地“做”。

第四编 把项目全景和代码主线看透

真正接项目，最后一定要回到结构。这里会把论文主张、工作区目录、wrapper 层、upstream 层和数据流放到一张图里，让师弟知道自己到底在读什么。

这一份文档是整个体系里最关键的一份。前面的术语词典解决的是“词不认识怎么办”，而这里要解决的是另一种更深的困惑：词我好像都见过了，可是项目整体到底是怎么串起来的？为什么同一个工作区里会同时有 `scripts/`、`runs/`、`docs/`、`splatnav-official/`、`third_party/`？为什么有些脚本看起来像算法，有些

看起来像环境工具？论文里讲的是 `Splat-Plan` 和 `Splat-Loc`，可我眼前看到的又是一堆 `wrapper` 脚本和 `run` 目录，这些东西到底谁是主角，谁是配角？

如果这类问题不讲明白，新人会非常痛苦。因为你会一会儿钻进环境脚本，以为那就是算法本体；一会儿又钻进上游仓库，看到一堆模块后完全失去方向；最后看似文件很多、努力很多，但脑子里没有一张稳定地图。我们写这一份文档，就是为了先把地图画出来。地图一旦有了，后面读代码、跑脚本、看结果，都会省很多力。

一、先把论文主张和当前边界说清楚

这部分先解决“项目到底是什么”和“当前阶段到底做到哪里”这两个总问题。只有这两个问题稳住了，后面看目录、看脚本、看上游源码才不会一上来就混层。

1、先把论文讲的到底是什么说清楚

1.1 一句话版本

`Splat-Nav` 是一套建立在 `Gaussian Splatting` 场景表示上的机器人导航系统。它不满足于“把场景重建出来”，而是进一步利用这种场景表示，让机器人在场景里做安全规划和视觉定位。

1.2 30 秒版本

论文把系统拆成两个模块。`Splat-Plan` 负责从当前位置到目标位置，构造安全走廊并生成平滑轨迹；`Splat-Loc` 负责只根据机载 RGB 图像估计机器人当前位姿。两个模块合起来，才能支持机器人在复杂场景里一边估计自己在哪，一边持续规划一条安全路径。

1.3 3 分钟版本

如果用更通俗的话来讲，这篇论文是在回答两个连续问题。第一个问题是：如果我们已经有了一个用 `Gaussian Splatting` 表示出来的三维场景，那么能不能不把它简单退化成传统地图，而是直接在这种表示上做安全规划？第二个问题是：如果机器人在执行过程中只有摄像头图像，能不能也利用这个 `GSplat` 场景来做位姿估计，从而形成可重规划的导航闭环？

论文的回答是肯定的。它把 `Splat-Plan` 设计成“先建立安全多面体走廊，再在走廊中生成平滑轨迹”的规划模块；把 `Splat-Loc` 设计成“利用 `GSplat` 场景和 RGB 观测做实时位姿递推”的定位模块。项目页和官方 `README` 都明确强调，这样的组合能在仿真和硬件实验中实现实时重规划与视觉定位，并且速度明显快于一类基于 `NeRF` 的导航方法。

对我们当前工作区来说，最重要的不是把论文里的每个实验数字背下来，而是抓住结构：论文总系统有两大块，而我们当前第一阶段主要在复现和工程化整理 `Splat-Plan` 的离线/仿真链路。这个边界一定要非常清楚。否则你会以为自己一上来就必须把整套机器人系统全部打穿，压力会非常大，也不现实。

2、把当前工作区看成三层结构

很多混乱，其实都来自把不同层次的东西放在一个平面里看。当前 `/home/howard/Splat-Nav` 这个工作区，最稳妥的看法是“三层结构”。

第一层是论文与算法层，也就是上游官方仓库真正实现 Splat-Plan、Splat-Loc、GSplat 读取、几何处理、样条优化的地方。你看到的入口是 `/home/howard/Splat-Nav/splatnav-official`，但它其实通过软链接最终指向 `/data/howard/repos/splatnav-official`。

第二层是本地 wrapper 工程层，也就是 `/home/howard/Splat-Nav/scripts` 这一层。它不去重写论文算法，而是负责把环境、tmux、run 目录、smoke、batch、overnight、结果汇总和可视化这些工程动作标准化。这一层非常像大厂项目里常见的“实验操作系统”。它不替代算法，但决定你能不能稳定、可追踪地复现算法。

第三层是文档与结果层，也就是 `/home/howard/Splat-Nav/docs` 和 `/home/howard/Splat-Nav/runs`。前者负责把知识、规范和交接路线沉淀下来，后者负责把每一次任务的证据、日志、配置和图表沉淀下来。你可以把这一层看成“项目的记忆系统”。

只要把这三层分开，很多问题就立刻清晰了。比如你要问“论文的核心算法在哪”，答案应该落在第一层。你要问“如何让长任务稳定跑在 tmux 里并自动归档结果”，答案在第二层。你要问“我应该读什么、怎么知道这次实验算成功”，答案在第三层。很多新人之所以累，就是因为把这三类问题全部丢给同一份代码去回答。

二、工作区地图与目录结构

这一部分把 `/home/howard/Splat-Nav` 这棵树真正讲开。目标不是让你背目录，而是让你看见每个目录各自承担什么职责、彼此为什么这样放。

1、当前工作区根目录里到底有什么

1.1 `/home/howard/Splat-Nav`

这是主工作区根目录，也是你未来绝大多数操作的逻辑起点。你可以把它想成“整个项目在本机上的主门面”。它本身不是论文原仓库，也不是纯粹的数据目录，而是把上游源码、wrapper、文档、结果入口组织在一起的工作区。

1.2 `configs/`

这个目录目前主要放工作区级的路径和环境约定，比如 `configs/project_paths.env`。它的意义在于：某些和本机部署相关、但又不适合散落在脚本里的路径信息，可以在这里集中管理。对新人来说，它不是第一优先级的阅读对象，但你要知道它代表的是“工作区的工程配置”，不是论文数学参数。

1.3 `docs/`

这是文档总目录，用来沉淀项目说明、交接资料、排错、规范和阶段进展。里面又分成三类。`archive/` 是历史材料和合并版草稿，信息覆盖广，但不再作为新人主线。`guidang/` 更像工程执行过程中的规范和归档文档。`Codex/` 则是现在重新搭建的主文档体系，专门面向零基础学弟和当前工作区的最短路径阅读。

1.4 `runs/`

`runs/` 是结果落盘入口。当前根目录下这个路径本身是一个软链接，实际指向数据盘上的结果目录。这样设计的目的很现实：系统盘空间紧张，但实验日志、JSON、图表和批跑结果会越来越多，所以要尽量把重资产沉淀到 `/data`。你以后看到的 `YYYY-MM-DD_HHMM_topic` 目录，几乎都会从这里进入。

1.5 scripts/

这里是 wrapper 工程层最密集的地方。它不是论文主算法本体，但非常重要，因为你真正日常执行的大多数命令，都要通过这里进入。这里面既有环境相关脚本，也有数据下载、smoke、batch、overnight、watchdog、可视化和汇总脚本。后面我们会逐个讲。

1.6 splatnav-official/

这是上游官方源码的推荐阅读入口。它是一个软链接，指向 `third_party/splatnav-official`，后者再指向真正的上游仓库 `/data/howard/repos/splatnav-official`。对新人来说，这个设计有两个好处：第一，工作区内部可以直接看到官方源码；第二，官方源码保留在数据盘位置，避免 home 目录和数据盘之间出现两套真正独立的副本。

1.7 third_party/

这个目录体现的是工程化协作里常见的第三方依赖管理思路。当前它最重要的内容是 `third_party/splatnav-official` 这个软链接入口。它在概念上提醒你：上游官方仓库是被当作第三方依赖接入当前工作区的，而不是和 wrapper 混为一谈。

1.8 README.md

根 README 是这套工作区的总入口说明之一。它强调三件事：这是一个 wrapper workspace；复现范围聚焦离线/仿真，不碰别人的环境和系统 CUDA；`scripts/` 负责工程化，核心论文算法仍在上游源码树。对于学弟来说，这份 README 很适合当第一眼总览，但它不能替代你现在读的这一份文档，因为它更像总览摘要，而不是完整地图。

1.9 Makefile

`Makefile` 提供的是若干快捷入口，比如环境安装、环境检查、数据下载、数据链接、smoke 这类常见动作。它的功能更偏“常用命令别每次手打太长”，而不是实现复杂业务逻辑。对新人来说，你要知道它是便捷入口，不是算法核心。

三、wrapper 工程层是怎么把事情组织起来的

对当前工作区来说，wrapper 层是理解成本最高、但也是最值得先看懂的一层。这里会把环境、tmux、run 目录、批跑、可视化和汇总脚本系统地归到一起。

1、wrapper 脚本层到底承担什么职责

这里我们不按文件名机械罗列，而是先按职责分组，再在组内解释每个脚本。这样你更容易建立“我现在该找哪类脚本”的直觉。

1.1 环境与健康检查脚本

1.1.1 scripts/bootstrap_env.sh

这是当前工作区环境搭建的主入口之一。它会连接 `/data/howard/miniconda3` 这套 Miniconda 安装，在其中创建或复用 `splatnav` 环境，然后安装项目需要的关键 Python 依赖，包括 `nerfstudio`、`clarabel`、`cvxpy`、`open3d`、`viser`、`LightGlue` 等。这个脚本的重要性不在于“装很多包”，而在于它把环境搭建策略固定下来了：优先把环境、缓存和临时目录放在数据盘，不去动系统 CUDA，不去污染其他人的环境。

如果你将来带学弟，这个脚本就是最值得你先讲清楚的环境脚本之一。因为它直接体现当前项目的边界意识：只在自己的 Conda 环境里解决问题。

1.1.2 scripts/env_health_check.sh

环境搭好以后，不能凭感觉宣布成功，这就是 `env_health_check.sh` 的价值。它会检查系统信息、GPU、Python 版本、Torch CUDA 状态以及一系列关键依赖的 import 能否通过。对新人来说，这个脚本的意义非常大，因为它把“环境能不能用”变成了可见证据，而不是主观判断。

你以后看到某次 smoke 失败，不要直接怀疑算法，先回到这个脚本确认环境是否健康，这是非常稳的排查起点。

1.1.3 scripts/fix_torch_cuda.sh

这个脚本是针对 Torch 与 CUDA 运行时兼容问题的定向修复脚本。它的核心思想不是升级系统 CUDA，也不是升级显卡驱动，而是在当前 `splatnav` 环境里重装一个与现有驱动兼容的 Torch 版本。对于共用机器、共用显卡、又明确不能乱动系统环境的场景，这个脚本非常符合边界要求。

你可以把它理解成“只在自己的小屋里修灯泡，不去动整栋楼总闸”。这个比喻很土，但很好记。

1.2 run 目录、tmux 与长任务调度脚本

1.2.1 scripts/create_run_dir.sh

这是 `run` 目录规范的核心脚本。它负责创建 `runs/YYYY-MM-DD_HHMM_topic/` 这样的目录，并自动生成 `env.txt`、`notes.md`、空的 `cmd.txt` 和 `stdout.log`。对工程化复现来说，这个脚本的价值非常大，因为它把“每次任务都应留下什么证据”写成了默认动作。

你后面无论是 smoke、tmux 任务还是 overnight，都能看到它的身影。对新人来说，理解这个脚本，就相当于理解“项目为什么强调每次任务都要留下上下文”。

1.2.2 scripts/run_in_tmux.sh

这是当前工作区里最重要的长任务入口之一。它会自动准备 `run` 目录、生成 `job` 脚本、创建或复用 `splatnav_jobs` 这个 `tmux` 会话，并把你的命令放到新的 `tmux window` 里执行，同时把输出写入 `stdout.log`。它不是算法脚本，但在工程实践里价值极高，因为它解决的是“任务如何在断开终端后继续跑、如何把命令和日志绑定保存”的问题。

如果学弟以后只记住一个 `wrapper` 级核心脚本，我认为就是这个。因为它体现了当前工作区的一个重要原则：长任务尽量进入 `tmux`，并与 `run` 目录规范绑定。

1.2.3 scripts/watch_overnight_job.sh

这个脚本扮演的是 `watchdog` 角色。它会监测目标 `overnight` 进程是否还活着，如果发现任务消失，就尝试重新拉起。对新人来说，`watchdog` 这个概念本身就很值得学：它说明工程上的问题不仅是“怎么启动一次”，还包括“任务中途挂了怎么办，能不能自动恢复”。

当然，这个脚本也提醒你一件事：自动恢复是补救，不是替代排错。真正长期稳定，还是要回到环境、输入、脚本链路本身。

1.3 数据与资源准备脚本

1.3.1 scripts/download_official_data.sh

这是一个官方数据下载入口，用 `gdown --folder` 从论文 README 给出的 Google Drive 地址拉取官方资产。它的意义不是“下载就是结束”，而是把数据来源和下载动作固定在一个可追踪的脚本里。对于新人来说，这能减少随手点网页、下载到奇怪目录、最后自己也不知道文件从哪来的问题。

1.3.2 scripts/link_official_assets.sh

这个脚本负责把下载好的 `training/data`、`training/outputs`、`traj` 等内容，通过软链接接到上游仓库期望的位置。它之所以重要，是因为上游论文代码往往默认某种目录结构，而我们本机又希望把重数据统一放在数据盘。软链接正好解决这个矛盾：既不改上游逻辑，也能把真实资产放在更合适的位置。

1.3.3 scripts/retry_gdown_folder.sh

网络下载尤其是 Google Drive 批量下载，很容易出现中途中断、缺文件、文件数不足的问题。这个脚本的作用，就是围绕一个文件夹 URL 做重试下载，并用帧数、`transforms.json`、`sparse_pc.ply` 这些条件判断是否真正下载完整。它代表了一种很重要的工程思想：下载成功不是“命令退出码看起来没事”，而是“目标文件结构达到最低验收条件”。

1.3.4 scripts/fill_missing_scene_files.sh

这个脚本进一步处理“日志里显示下载过，但本地还缺文件”的场景。它会从 `gdown` 日志中解析文件 ID，再对缺失的图片、`transforms.json` 和 `sparse_pc.ply` 做定向补齐。对新人来说，这个脚本很有启发：很多真实工程问题不是完全重来，而是“如何补最后那几块缺失拼图”。

1.3.5 scripts/pull_images2_full.sh

这个脚本专门围绕 `images2` 这样的场景资源做多次尝试拉取，并按帧数是否达到最低标准来判断完成度。它本质上和前两个下载补齐脚本属于同一类：处理网络资源不稳定、需要重试和最低数量验收的下载问题。

1.4 smoke、batch、overnight 主执行链脚本

1.4.1 scripts/smoke_splatplan.py

这是最小规划冒烟测试的 Python 入口。它负责做几件非常具体的事：解析 `--config`、`--scene`、`--method`、`--output` 参数；根据场景预设构建机器人与体素配置；通过 `GSplatLoader` 加载模型；根据方法名构建 `SplatPlan` 或 `SafeFlightCorridor`；然后只对一组固定的起终点做一次规划，并把结果写成 JSON。这个脚本的重要性不在于它短，而在于它是“最小可跑链路”的缩影。

对新人来说，读这个脚本非常合适，因为它不会像大批量脚本那样一下子引入太多循环和统计逻辑。你可以在这里先看懂：输入是什么，核心 planner 是怎么被构建出来的，输出 JSON 大概长什么样。

1.4.2 scripts/run_splatplan_smoke.sh

这个 shell 脚本是 `smoke` Python 入口的直接包装。它会创建 `run` 目录、记录命令、激活 Conda 环境、清理 `LD_LIBRARY_PATH`，然后调用 `smoke_splatplan.py`，并把日志写进当前 `run` 目录。它的角色很像一个“规范化启动器”：不是替代 Python 主逻辑，而是保证每次 `smoke` 都按统一工程规矩执行。

1.4.3 scripts/run_splatplan_smoke_tmux.sh

这是 smoke 的 tmux 版本入口。它不会直接自己跑规划，而是组装好命令后交给 run_in_tmux.sh。这个设计很好理解：如果 smoke 任务也想脱离当前终端独立运行，就复用统一 tmux 调度入口，而不是每个脚本自己再造一套后台运行逻辑。

1.4.4 scripts/run_official_style_batch.py

这是当前 wrapper 层最重要的批量实验脚本。它会在多个起终点对上运行 splatplan 或 sfc-1..4 等方法，记录每次 trial 的结果，统计可行率、平均耗时、P95 耗时，并把部分轨迹和详细记录打包成一个 batch_results.json。你可以把它理解成“从单次 smoke 走向正式实验统计”的主桥梁。

它和上游 run_splatplan.py 的关系值得你认真体会。上游脚本更像论文作者自己组织实验的原始入口，而 wrapper 这份 batch 脚本则在保留算法调用逻辑的同时，把输入参数、输出结构、保存格式、轨迹抽样、统计摘要都包装得更适合本机复现和结果沉淀。这就是工程化封装的典型样子。

1.4.5 scripts/run_overnight_12h.sh

这个脚本会在给定的时长内，循环调用 batch 脚本，每一轮都建立新的 run 目录、保存 JSON，并在成功后生成可视化图。它的本质不是另一种算法，而是把一轮又一轮正式 batch 组织成可长期运行、可分轮归档的实验流水线。对于当前机器夜间空闲、GPU 可持续使用的场景，这个脚本非常实用。

1.4.6 scripts/post_overnight_report.sh

overnight 跑完之后，如果只留下很多 run 目录而没有总览，第二天会很难看。这个脚本的作用就是在目标 overnight 任务结束后，自动运行汇总脚本，产出 CSV、Markdown 报告和最新报告的稳定软链接。它体现的是“结果不仅要产生，还要能被第二天的人快速消费”。

1.5 汇总与可视化脚本

1.5.1 scripts/visualize_batch_results.py

这个脚本负责把 batch JSON 结果进一步做成更容易阅读的图，包括可行率柱状图、规划耗时箱线图、示例轨迹 XY 投影叠图，并生成一个简要的 Markdown 摘要。它属于“从原始统计到汇报素材”的一层。你以后做组会、周报，很可能第一眼看的就是它生成的图，而不是原始 JSON。

1.5.2 scripts/aggregate_batch_runs.py

这个脚本的目标是把多轮 batch 的结果横向汇总起来，形成 overnight_summary.csv、overnight_summary.md 和迭代曲线图。它是“跨 run 目录”的汇总层。与 visualize_batch_results.py 不同，后者更关注某一次 batch 的内部对比，这个脚本则更关注多轮正式实验之间的整体趋势。

1.6 同步与维护脚本

1.6.1 scripts/sync_upstream.sh

这个脚本用于查看和同步上游官方仓库的分支与提交状态。它不会替你解决所有合并问题，但会明确告诉你当前上游提交和分支情况。对新人来说，它的意义是建立一个概念：上游不是死的，wrapper 也不是闭门造车，二者之间要有版本感知。

四、上游官方源码层到底在做什么

当你已经知道 wrapper 在做什么之后，才更适合进入上游源码。这里重点处理的是：哪些文件是主轴，哪些目录是支撑层，哪些内容现在先知道位置就够。

1、上游官方源码层到底怎么看

很多人一看到上游源码目录多，就立刻想逐文件读。这个想法听起来勤奋，但对新人来说通常效率很低。更稳的方式，是抓住主链路入口，再顺着调用关系往里走。对当前项目，最值得你优先读懂的入口就是以下几类。

1.1 run_splatplan.py

这是上游最直接的 Splat-Plan 仿真运行入口。你从这份脚本里能一眼看到论文实现者自己是怎么组织实验的：先设定场景名、方法名、机器人参数、体素边界、分辨率和 GSplat 配置路径；再加载 GSplatLoader；再根据方法构建 SplatPlan 或 SafeFlightCorridor；然后在一圈起终点样本上循环调用 generate_path，最终把结果写进 trajns/ 目录。

这份脚本非常像“原始实验台面”。它没有 wrapper 那么讲究 run 目录和日志结构，但恰恰因为直接，它很适合你理解：上游论文作者心中的实验主流程到底是什么。

1.2 run_splatloc.py

这是上游的 Splat-Loc 入口。它主要围绕 GSplat 场景、相机内参、图像数据集和位姿估计流程展开。对当前阶段来说，它不是最先要啃透的文件，但你至少应该知道它在总系统里代表“定位链路”，从而和 run_splatplan.py 形成清晰对照。

如果学弟读到这里觉得压力大，很正常。第一阶段只要先知道它存在、知道它负责什么，就够了。

1.3 splat/splat_utils.py

这份文件里最关键的类是 GSplatLoader。它的作用是把 GSplat 模型从指定位置加载出来，并整理成后续规划所需的张量属性，比如均值、旋转、尺度、颜色、不透明度以及相关协方差表示。对新人来说，这份文件很重要，因为它回答了一个非常基础但关键的问题：规划模块到底是怎么接触到“场景”的。

你如果把 Splat-Plan 想象成一个只吃起点终点的黑盒，会很难理解为什么 config.yml 这么重要。看过 GSplatLoader 以后，你会知道规划器不是凭空工作，而是建立在一个具体的 GSplat 场景表示之上。

1.4 SFC/corridor_utils.py

这份文件承载的是 SafeFlightCorridor 及相关走廊生成逻辑。你会在这里看到多种 mode 对应 sfc-1 到 sfc-4 这些 baseline 的差异：有的在点云网格上初始化，有的在 GSplat 网格上初始化，有的走廊收缩更保守。对新人来说，这个文件很值钱，因为它不仅实现一个方法，还通过 mode 让你看到“对比方法的家族差异”。

你读它的时候，不要试图一次记住所有几何细节。先抓住结构：初始化路径、获取碰撞集合、构造多面体、最终生成轨迹。只要这个骨架稳住，细节可以后面再补。

1.5 splatplan/splatplan.py

这份文件是 SplatPlan 核心实现。它大致做四件事：利用体素网格生成初始路径；逐段计算碰撞集合；根据高斯相关几何关系构造安全多面体；再调用样条优化器得到平滑轨迹。这个文件很适合作为你理解论文核心主张的代码落点，因为它把项目页和论文中那句“safe-by-construction polytope corridor + Bézier trajectory”真正在代码里展开了。

你以后如果要向别人解释 Splat-Plan，这份文件是最值得你反复读的核心落点。

1.6 splatplan/spline_utils.py

这份文件更多承载“给定走廊以后，如何把轨迹优化出来”的逻辑。里面会出现样条基、导数、约束矩阵、cvxpy、clarabel 这些内容。很多新人读到这里会被吓住，但你只要记住：这不是另一套无关算法，而是 Splat-Plan 最后一段“从安全走廊到平滑轨迹”的求解器实现。

五、从数据流和调用链把整套系统串起来

目录、脚本和源码真正有意义，是因为它们共同组成了一条流。这里开始把输入资产、调用顺序、结果落盘和阅读顺序整体串成链。

1、文件夹之间的数据流到底怎么走

讲清项目全景，最终一定要落到数据流上。因为只有数据流清楚了，你才知道“输入从哪里进，输出往哪里出，中间谁做了什么”。

最上游的输入，是一个已经训练好的场景模型及其配置，通常表现为某个 config.yml。这个配置文件背后关联着 Nerfstudio 风格的训练输出目录、数据解析信息和场景模型参数。wrapper 或上游入口脚本接收到这个配置路径后，会调用 GSplatLoader 把模型读进来。

读进来之后，规划链路会根据场景名选择一组机器人和体素预设，包括机器人半径、最大速度、最大加速度、体素边界和分辨率。这一步不是数据下载意义上的输入，而是实验设定意义上的输入。接着，脚本根据方法名决定构建 SplatPlan 还是 SafeFlightCorridor。此时，场景表示已经和规划器绑定起来了。

随后，单次 smoke 会选择一组固定起终点并调用 generate_path；batch 会生成一批起终点对并循环调用；overnight 则会在更长时间窗口里多次调用 batch。每一次调用都会得到规划结果，里面包含是否可行、耗时、多面体数量、路径和轨迹等信息。wrapper 脚本会把这些原始结果整理成 JSON，并额外生成图表、Markdown 摘要、CSV 汇总等更方便阅读的产物。

最终，这些结果都会沉淀到 runs/ 下某个具体 run 目录里。也就是说，当前工作区真正的核心数据流，可以简单概括成：config.yml 与场景预设 -> 规划器构建 -> 单次或批量 generate_path -> JSON 与日志 -> 图表与汇总 -> runs 目录沉淀。把这条线背住，你再去看单个文件，就不会那么容易迷路。

2、这个项目现在能做什么，不能做什么

当前这套工作区和文档体系，已经很适合做以下几类事情。第一，围绕作者场景做离线和仿真级别的 SplatPlan 复现。第二，使用统一 wrapper 脚本做 smoke、batch、overnight 实验，并把结果按 run 目录规范保存。第三，基于 runs 里的 JSON、图和汇总文件，完成周报、组会和阶段性汇报。第四，在不破坏上游原样的前提下，逐步做外层工程整理、注释补充和有限的重构思考。

但它暂时不等于一套完整真机部署系统。当前阶段不碰实车，不展开 ROS 分支的节点部署，不把 Splat-Loc 和硬件飞行闭环作为新人第一周的刚性任务，也不要求一开始就进入论文创新点实现。你可以把这些方向放在后续一月、三月阶段里逐步展开，但不能混入第一阶段目标。

这个边界说清楚，其实是在保护项目推进节奏。很多人不是能力不够，而是一开始给自己塞了过多目标，最后所有目标都推进得很浅。

六、读源码、带学弟和做汇报时该怎样建立地图感

这部分更偏读者视角。它不只解释系统本身，也解释学弟第一次读到这里后脑子里应该形成什么结构，以及带教时该怎样用这份地图。

1、学弟第一次读到这里后应该形成什么地图

如果这一份文档起作用了，你现在脑子里应该已经有一张比较稳定的大地图。你知道论文层面有 Splat-Plan 和 Splat-Loc 两大块；你知道当前工作区可以分成上游算法层、wrapper 工程层和文档/结果层；你知道 scripts/ 不是论文创新本体，而是工程化执行入口；你知道 splatnav-official/ 是上游源码阅读主入口；你也知道真实的数据流是从 config.yml 和场景预设进入规划器，再落到 runs 目录里的各种结果文件。

接下来，你最该做的不是继续泛泛看目录，而是进入 第三编。因为地图已经有了，下一步就该把“一次实际运行”拆开，看看输入到底是什么，输出到底长什么样，什么才算真正跑成功。

2、这一部分的事实锚点来自哪里

这一份文档不是凭印象写的，而是明确建立在几类一手材料之上。论文层面的系统描述，主要对齐 Splat-Nav: Safe Real-Time Robot Navigation in Gaussian Splatting Maps 的 arXiv 主论文和项目页。仓库层面的目录、依赖、运行入口和模块名称，主要对齐上游官方仓库的 README.md 与当前工作区可见源码。工作区层面的路径、脚本、run 目录规范和工程边界，则对齐本机 /home/howard/Splat-Nav 中实际存在的 wrapper 脚本、软链接结构和文档目录。

如果你以后在阅读中发现“论文说法”和“本机工作区说法”看起来不完全一样，不要慌。优先做的不是猜谁对谁错，而是先分清语境：论文讲的是系统设计与实验主张，上游仓库讲的是作者实现，当前工作区讲的是本机复现与工程化包装。把这个三层语境先分开，你会少很多无谓困惑。

3、把论文语言翻译成你每天会遇到的工作语言

前面那几节，其实只是先帮你把大地图钉在墙上。接下来我们要做的，是把地图上的抽象文字翻译成你真会遇到的工作语言。因为很多新人真正崩溃的地方，不是“论文看不懂”，而是“论文里那些看起来很高级的话，跟我眼前的文件、脚本、日志，到底有什么关系”。只要这层翻译没做，项目就会一直悬在空中。你知道它厉害，也知道它大概做导航，但你不知道自己今天下午打开哪个脚本，和论文里哪句主张其实是同一件事。

所以我建议你换一个很接地气的读法。以后每次看到论文里的一个关键词，不要急着去背定义，先问三个问题。第一，它在本机工作区里落到哪个目录、哪个脚本、哪个类上。第二，它输入了什么、输出了什么。第三，它如果坏了，我会在哪一层看到异常。只要这三个问题能回答出来，这个概念就真正开始归你用了。

3.1 论文说“Gaussian Splatting map”，你在工作区里该想到什么

论文里的 Gaussian Splatting map，如果只停留在“一个很新潮的三维表示”，那它对新人来说帮助不大。你真正应该想到的，是它在当前项目里不是一张普通图片，也不是一个传统的二维栅格图，而是一套通过 Nerfstudio 训练出来、最终由 config.yml 作为入口的场景模型。也就是说，当你在 wrapper 脚本里写 `--config ../../config.yml` 时，你不是在给脚本随便塞个配置文件，而是在告诉它“你现在要读哪一个 GSplat 场景”。

一旦这么想，很多事情会突然顺起来。你会明白为什么本地文档老是反复提 `config.yml`，明白为什么 `GSplatLoader` 是核心入口之一，也明白为什么数据下载、资产软链接、`outputs/` 目录结构这些事情会和规划算法扯上关系。因为规划器本身并不是无中生有，它是站在一个已经训练好的 GSplat 场景表示之上工作的。

3.2 论文说“safe-by-construction polytope corridor”，你在代码里该想到什么

这句话看起来很学术，但如果你把它翻译成代码语言，其实可以变得很具体。所谓 `safe-by-construction polytope corridor`，不是一句口号，而是会落到 `SplatPlan.generate_path()` 里那条很明确的链路：先用体素网格做 A* 找一条初始化路径，再按段构造碰撞集合，再逐段生成多面体约束，最后把这些多面体送进样条优化器。也就是说，安全不是最后画图时顺手检查一下，而是在规划过程中被逐段写进约束里的。

你后面读 `splatnav-official/splatplan/splatplan.py` 时，别一上来盯着每个张量的维度。先抓住这个骨架：A* 初始化 -> collision set -> polytope -> spline optimization。只要骨架稳住，后面的矩阵、半空间、约束拼接，就都只是往骨架上长肌肉，而不是完全陌生的天书。

3.3 论文说“recursive re-planning”，你在当前阶段该怎么理解

很多新人一看到 `re-planning`，脑海里就会直接跳到真机飞行、闭环控制、相机在动、机器人在房间里躲障碍。这种联想不完全错，但对当前阶段来说，太早了。你在本机工作区第一阶段更合适的理解，是“同一套规划器被多次调用，在不同起终点、不同 trial、不同 batch 里重复运行，并把结果做成结构化统计”。

也就是说，当前阶段你暂时不用把 `re-planning` 想得那么像机器人操作系统里的闭环回路。你可以先把它理解成“对规划器的重复、可追踪、可比较调用”。`smoke` 是一次很小的调用，`batch` 是一批系统化调用，`overnight` 是很多轮调用的组织方式。等这三层都真正摸熟以后，你再回头看论文里的在线重规划，会发现自己已经有了很强的地基。

3.4 论文说“Splat-Loc”，你在当前工作区里应该保持什么态度

`Splat-Loc` 既不能被你忽略，也不能被你第一周就当成必须打透的主线。更稳妥的态度是：知道它在总系统里很关键，知道它负责的是视觉定位链路，知道 `run_splatloc.py`、`pose_estimator/utils.py`、`ns_utils/nerfstudio_utils.py` 这些地方和它强相关，但当前离线复现主任务主要先把 `Splat-Plan` 跑顺、讲清、固化下来。

这样安排不是在回避难点，而是在做任务排序。因为如果学弟连工作区三层结构都没稳住，就直接扎进 `Splat-Loc` 的特征匹配、PnP-RANSAC、ICP、UKF 式状态估计细节里，阅读压力会非常大。相反，如果先把 `Splat-Plan` 的输入、输出、结果目录、规划链路看稳，再回来看定位链路，就像先学会走一条路，再学怎么在路上定位自己。顺序会合理很多。

3.5 论文说“real-time”，你在共用工控机上该怎么落地这个词

real-time 在论文里当然有算法和系统层面的含义，但落到你现在这台机器上，它还多了一层很实际的工程含义：任务要稳、结果要能看、日志要能留、环境不能乱、资源不能伤及别人。也就是说，实时性不只关乎算法快不快，也关乎你能不能用一套可靠的 wrapper 和 run 规范，把实验长期稳定地组织起来。

这也是为什么当前工作区里 `scripts/run_in_tmux.sh`、`create_run_dir.sh`、`aggregate_batch_runs.py` 这些文件虽然不是论文创新，却同样重要。它们保证的不是“数学上更优”，而是“工程上更稳”。如果你以后去大厂做算法工程，会越来越感受到这一点。真正能把工作推进下去的人，往往不是最会讲概念的人，而是既懂概念，又能把概念压进工程流程的人。

4、如果把这个项目当成一天的工作流，它会是什么样子

新手经常有一种错觉，以为项目结构图只是给汇报用的。其实不是。最好的结构图，是你每天工作时真的能用上的图。为了让这件事更具体，我们不妨把当前项目想象成一个完整工作日。你早上连上机器，白天读文档、读代码、跑小实验，晚上可能发起长任务，第二天再回来看结果。只要你能把一天的动作和目录结构一一对应起来，整个项目就会变得非常立体。

4.1 早上：先确认“我站在哪里，今天要动哪一层”

你早上第一件事，不应该是直接打开一个复杂脚本，而是先确认自己站稳了没有。你会 SSH 登录到机器，进入 `/home/howard/Splat-Nav`，确认自己看到的是这套 wrapper 工作区，而不是别的路径。这个动作听上去平淡，但它相当于把你带回项目主门口。只要主门口站稳，后面无论你是去读文档、看脚本、开 tmux 还是查 run 目录，都有了统一起点。

如果你愿意把这个动作做成习惯，下面这几行命令非常值得记住。它们不复杂，但信息密度很高，能帮助你快速确认“我到底在什么工作区、结果目录和上游源码入口又实际落到哪里”。

```
cd /home/howard/Splat-Nav
pwd
ls
readlink -f runs
readlink -f splatnav-official
```

这组命令的目的，不是让你秀命令行，而是让你在开工前快速完成一次“地图复位”。你会看到主工作区根目录、`runs` 的真实目标路径，以及 `splatnav-official` 最终指向的数据盘仓库。只要这一步稳了，你脑子里那张图就不会飘。

4.2 上午：先读文档，再带着问题去看代码

很多新人最容易犯的错，是一上来就直扑代码，认为这样最踏实。事实上，对当前项目这种“论文算法 + 上游源码 + wrapper 工程 + 本机运行边界”四股信息缠在一起的工作区来说，先读主文档反而更省时间。不是因为文档比代码高级，而是因为文档能先帮你划边界、划层次、划优先级。

所以对学弟来说，上午最理想的动作不是到处点文件，而是先在 `docs/Codex` 里把主线走一遍。先看 第一编，知道自己应该从哪进；再回看 第六编，把眼下会撞到的词压一压；最后来到这一份 第四编，把项目地图定下来。这样做之后，你再打开 `scripts/smoke_splatplan.py` 或 `splatnav-official/splatplan/splatplan.py`，会轻松很多。

4.3 下午：开始做小规模验证，把“看懂一点”变成“我真跑过”

到了下午，你通常已经不满足于只看文字了。这个时候最自然的动作，就是先做一条最小验证链。也就是说，不追求大规模 batch，不追求炫图，而是先找一个可信的 `config.yml`，用 smoke 跑一条最短路径，让自己第一次亲眼看到 `run_dir`、`stdout.log`、输出 JSON 和里面的 `feasible`、`path`、`traj` 等字段。

为什么这一步重要？因为很多理解只有在你真的跑过一次以后才会变得牢。比如你以前知道 `GSplatLoader` 是读场景的，可直到你发现 smoke 入口确实在先构建 planner、再传入 `config.yml`，这个概念才真正落地。再比如你知道 `create_run_dir.sh` 会创建 `run` 目录，可直到你亲手看到 `cmd.txt`、`env.txt`、`stdout.log` 被生成出来，这个规范才不再只是文档里的要求。

4.4 傍晚：开始看结构化结果，而不是只盯着终端输出

很多人跑出第一条结果以后，会下意识只盯着终端里最后几行输出，仿佛那里藏着所有答案。其实不是。真正成熟一点的工作方式，是尽快把注意力从终端滚动文本转移到结构化结果上。你会开始打开 JSON，开始对照 `meta`、`device`、`plan_time_sec`、`output.feasible`，开始看图、看 Markdown 摘要、看 CSV 汇总。

这个转变非常重要。因为只看终端输出，很容易让你一直停留在“程序有没有炸”。而看结构化结果，会把你带向另一种更高级的问题：这次规划到底表现怎样，和上次有什么差异，失败是链路故障还是问题本身不可行。科研和工程真正的成长，很多时候就是从这里开始的。

4.5 晚上：长任务交给 tmux 和 run 规范，自己不要做系统的人肉守夜员

当前工作区里，晚上最典型的动作不是继续手盯终端，而是把需要长时间运行的任务规范地交给 `tmux`、交给 `run` 目录、交给 `watchdog` 和后处理脚本。这背后其实是一种很重要的工程态度：人应该负责设计和判断，不应该长期充当“人肉终端保持器”。

所以如果你晚上要跑正式 batch 或 overnight，最值钱的不是再多盯两小时，而是提前把命令、结果目录、监控和汇总链路安排好。这样第二天早上回来，你看到的就不是一团混乱输出，而是一套已经归档到 `runs/` 的实验记录。学弟越早理解这种“把任务交给系统规范”的思路，越能减少被重复劳动吞掉的精力。

5、把根目录真正讲细：每个文件夹在这套工程里都扮演什么角色

前面我们已经大概走过一遍根目录，但那更像总览。这里我们把它讲得更细一点。不是为了追求百科全书式覆盖，而是因为学弟第一次看一个工程，最容易被目录名骗。很多目录看起来都很普通，真正的区别藏在它们对项目节奏的影响上。你如果只看名字，不看角色，就会一直把工具层、算法层、结果层混在一起。

5.1 .git/：它平时很安静，但它是这套工作区的时间轴

很多初学者一看到 `.git/` 会下意识略过，觉得那是版本控制内部实现，不重要。对做项目的人来说，这个判断并不完全对。你当然不需要天天钻进 `.git/objects` 去看对象存储，但你必须知道 `.git` 的存在意味着什么。它意味着当前这个 `/home/howard/Splat-Nav` 工作区是一个被版本控制管理的工程，而不是一堆随手扔在磁盘上的文件。

这件事的重要性，在你未来开始整理 wrapper、补注释、做结构优化时会越来越明显。因为一旦文档、脚本和外层工程开始稳定演化，版本历史就会成为你的第二套记忆系统。你会知道哪些修改是你做的，哪些是历史遗留，哪些适合继续推进，哪些只是一次性草稿。也就是说，`.git` 不只是开发者的工具，它其实是项目成长的时间轴。

5.2 configs/：它不吵闹，但它在帮工作区收拢机器相关约定

`configs/` 在很多工程里都会被新人低估，因为它不像 `scripts/` 那样“会跑”，也不像 `docs/` 那样“能读”。但它的作用其实很稳：把一些和当前机器部署、路径约定、环境变量相关的信息放在一个单独位置，不让这些内容散落到几十个脚本里。当前工作区里它最典型的例子是 `configs/project_paths.env` 这类文件。

你可以把这个目录理解成“工程习惯的中控台”。它不直接产生论文结果，但它决定很多脚本会不会越来越乱。对于带新人来说，这个目录还有一个隐含价值：它提醒大家，本机部署相关的东西和论文算法参数不是一回事。别把这两类配置混着讲。

5.3 docs/：它不是陪衬，而是这个项目真正的交接面

对学生项目来说，`docs/` 经常被当成“最后想起来再补一补”的东西。可在当前这套工程里，`docs/` 实际上是很关键的主层之一。因为你们现在做的事情，不只是自己把实验跑通，还包括把这个项目整理成可以交接、可以带人、可以汇报、可以让后来者接着干的形态。只要有这个目标，文档就不再是附属品，而是项目本体的一部分。

而且你会注意到，`docs/` 在当前工作区里不是单薄的一层。它内部又自然分成三个性格不同的区域：历史档案、归档规范和当前主线。理解这三者的差别，对学弟很重要。

5.4 docs/archive/：像旧讲义，覆盖广，但不适合直接拿来带零基础同学

`archive/` 最大的优点是信息覆盖广。你在里面能看到很多历史版本的合并稿、全景稿、基础知识稿。这些材料对你后面做深挖很有价值，因为它们常常保留了很多最初搜集到的知识点、路径和上下文。但它的缺点也很明显：内容容易混杂、叙事层级不稳定、对零基础读者来说负担偏大。

这就是为什么当前 `docs/Codex` 要重建主线，而不是直接把 `archive` 扔给学弟。不是因为 `archive` 没价值，而是因为它更像资料库，不像真正的陪跑教材。等学弟第一轮稳定入门以后，你当然可以再带他回去挖里面那些更全、更散、更原始的材料。

5.5 docs/guidang/：更偏执行与规范，是项目管理感比较强的一层

如果说 `archive` 像资料库，那么 `guidang` 更像工程过程中的制度层。这里面的东西往往更偏执行约束、归档口径、阶段说明。它不一定是学弟第一天最适合啃的内容，但当你开始把实验做成周期任务、需要做验收口径和结果归档时，这一层会非常有价值。

你以后如果开始真正带人，就会发现 `guidang` 这种目录非常必要。因为很多问题不是“不知道怎么做”，而是“不知道做到什么程度算完成”。而规范目录存在的意义，就是减少这种模糊地带。

5.6 docs/Codex/：现在这套主线文档真正希望你从这里开始

`docs/Codex/` 是当前重新搭建后的主阅读入口。它的设计目标非常明确：不是信息最全，而是对现在这台机器、现在这套 `wrapper`、现在这个复现边界最友好。这里既放着这本总手册，也保留着几份拆分版文稿，方便后续维护和回查。但对第一次进项目的人来说，最重要的仍然是先抓住这本总手册的主线，把原本散在 `archive`、`README`、脚本注释和口头交接里的知识，重新走成一条能往前推进的路。

所以学弟第一次进工作区，真正应该优先看的就是这一层。它不会替代上游源码，也不会替代论文，但它会把“先看什么、后看什么、现在做到哪一步算过关”讲清楚。对于入门阶段，这种作用非常大。

5.7 runs/：这是项目最有证据感的目录，不是普通输出箱

你以后会越来越发现，一个项目做得稳不稳，不是看它能不能偶尔跑通，而是看它能不能把每一次运行都留下可靠证据。从这个角度看，`runs/` 是当前工作区里极其关键的一层。它不是简单的“输出目录”，更像实验事实的沉淀区。每一个带时间戳的子目录，都对一次具体任务、一个明确命令、一个环境快照和一组结果文件。

更重要的是，当前 `runs/` 本身还是一个软链接，最终落到 `/data/howard/splatnav/runs`。这个细节看似只是磁盘管理，实际上含义很强：重资产和长历史要尽量往数据盘沉，系统盘要克制；结果入口依然放在工作区根目录，保证查找和讲解方便。这就是一种很成熟的工程取舍。

5.8 scripts/：它像中控台，不是论文算法本体，但你每天都离不开它

很多人第一次看到 `scripts/` 会很容易出现两个极端。一个极端是完全忽略它，觉得真正的东西都在上游源码。另一个极端是把它当成全部项目，以为论文算法都写在这里。两种看法都不对。更准确的理解是：

`scripts/` 是当前工作区的中控台。它负责把环境、路径、tmux、smoke、batch、overnight、汇总和可视化这些工程动作规范化。

为什么说它不是论文算法本体？因为真正的规划器类和定位逻辑还在 `splatnav-official/`。为什么又说你每天都离不开它？因为你绝大多数日常动作，不会直接从上游源码敲起，而是要通过这里的脚本进入。你可以把它理解成“让上游论文实现更适合本机复现”的外层操作系统。

5.9 scripts/__pycache__/：它不值得读，但它提醒你 Python 在这里被实际执行过

像 `__pycache__` 这种目录，新手往往会疑惑它是不是很重要。结论很简单：它不值得你花时间读，也通常不该被当成项目主结构的一部分。但它至少提醒你，这里对应的 Python 文件已经在本机被解释器加载和执行过，系统因此留下了缓存。

对学弟来说，最有价值的不是研究它内部，而是形成一种“什么是源码、什么是缓存、什么是结果、什么是依赖入口”的基本分层意识。很多混乱，本质上就是把这些不同性质的东西都看成“目录而已”。

5.10 splatnav-official/：它是你阅读官方源码最顺手的门

这一层我们前面已经提过很多次，但这里仍然值得再压实一下。`splatnav-official/` 在当前工作区里的价值，不只是“能打开官方代码”，而是“给所有文档和讨论提供统一入口”。如果没有这个入口，你每次跟学弟讲代码都要绕去数据盘真实路径，叙事会变得很重。现在有了这个入口，大家都能从同一个门进去，再通过 `readlink -f` 追到真实仓库位置。

这在教学和协作里非常有帮助。因为好的入口本身就会降低理解成本。你以后让学弟打开 `[splatplan/splatplan.py](/home/howard/Splat-Nav/splatnav-official/splatplan/splatplan.py)`，就比让他背数据盘长路径要轻松得多。

5.11 third_party/：它像一个“关系声明”，提醒你这份官方仓库是被接入进来的

很多工程里都会有 `third_party/`，但新人第一次看到时常常不明白它为什么非要存在。当前工作区里，它最主要的意义其实是一种结构声明：官方 Splat-Nav 源码在概念上是作为第三方依赖被接入当前 wrapper 工作区的。你没有把它和自己的工程层糊成一团，而是明确说“这是上游，这是外层包装”。

这种分层非常值得珍惜。因为一旦这层边界清楚，后面你做同步、做注释、做封装、做局部优化时，心里就会一直知道“哪些是上游原样保留，哪些是我们自己加的”。对于共享机器和多人协作环境，这种边界意识会保护项目很多次。

5.12 README.md：它是门口告示牌，不是整本教材

根 README 很重要，但你要用对它。它最适合承担的是“门口告示牌”的角色，也就是告诉后来者：这里是什么工作区，边界是什么，重资产放哪，核心路径怎么找，典型命令从哪进。它不适合直接承载所有教学任务，因为 README 必须相对短，必须适合第一次进门的人快速扫一眼。

所以如果学弟问“是不是把 README 看懂就够了”，更准确的回答应该是：README 是起点，不是终点。它负责把你带进门，而现在这些 docs/Codex 文档负责带你真正走路。

5.13 Makefile：它更像快捷拨号盘，不是秘密控制中心

有些新人看到 Makefile 会莫名紧张，觉得里面一定藏着整个项目的核心逻辑。其实对当前工作区来说，Makefile 更像快捷拨号盘。它把一些常用动作，比如环境初始化、环境检查、数据下载、数据链接、smoke 入口，整理成更短的命令，方便你少打字、少记长命令。

这种设计当然很实用，但也要防止另一个误区：不要把 make xxx 当成魔法。你最好始终知道它最终调的还是哪些脚本。这样以后出问题，你就不会被隐藏的调用关系蒙住。

5.14 根目录这张图，最终要在你脑子里变成什么样

如果把上面这些目录都压成一句很朴素的话，那么当前根目录其实在做三件事：给你一个稳定门面，给你一个干净边界，给你一个容易带人的讲法。门面体现在你总能从 /home/howard/Splat-Nav 进入项目；边界体现在 wrapper、上游、文档、结果、第三方之间没有硬糊在一起；带人的讲法体现在你可以很自然地根目录出发，往 docs、scripts、runs、splatnav-official 四个方向展开。

到了这里，你应该已经能感觉到，项目结构不是为了好看，而是为了降低未来每一次解释、每一次排错、每一次交接的成本。真正好的结构，第一天可能不觉得华丽，但一个月以后你会开始非常感谢它。

6、把 wrapper 层真正讲透：这些脚本不是散兵游勇，而是一条工程主线

如果你已经接受了“scripts/ 不是论文算法本体，但又是你每天最常接触的一层”，那么下一步就不能只停留在“我知道有这个目录”了。你要继续往前走，真正把 wrapper 层看成一条有主次、有入口、有顺序的工程主线。很多同学之所以会觉得 scripts/ 零碎，是因为他是按文件名一个个偶遇这些脚本，而不是按项目任务流去理解它们。

更顺的视角是：这些脚本并不是各自为政，它们是在回答六类固定问题。第一，环境怎么搭和怎么查。第二，数据和资产怎么准备。第三，最小 smoke 怎么起。第四，正式 batch 和 overnight 怎么组织。第五，结果怎么汇总和可视化。第六，上游源码和当前 wrapper 的关系怎么保持清楚。你只要按这个视角看，脚本层就不再像工具箱，而更像一条有序的生产线。

6.1 先讲一个总原则：wrapper 脚本最重要的价值，是把“重复但容易出错”的事情固定下来

为什么工程里总喜欢写 wrapper？因为真正消耗人的，往往不是那些你只做大一次的大难题，而是那些你每周都要重复做、但每次稍不注意就会出问题的动作。比如环境激活到底是不是同一个，LD_LIBRARY_PATH 要不

要清、run 目录有没有按规范创建、命令有没有被落到 `cmd.txt`、日志有没有被保存在 `stdout.log`、批跑结果有没有顺手生成图和摘要。这些事单独看都不难，但一旦要重复做十次、二十次，就非常容易出错。

wrapper 脚本的价值，就是把这些动作写成默认行为。这样你以后不是靠记忆去完成这些步骤，而是靠脚本去托底。这一点对新人尤其关键。因为新人最怕的不是复杂，而是每一步都不确定自己有没有漏掉什么。wrapper 的存在，恰恰是在把这种不确定性降下来。

6.2 环境组脚本：它们负责回答“这台机器上到底怎么开始”

当前工作区里，最先应该被学弟认识的一组脚本，是环境相关脚本。不是因为环境比算法高级，而是因为它们决定你后面所有动作有没有地基。对机械转代码背景的同学来说，这一组脚本越早讲清，心里越稳。

6.2.1 scripts/bootstrap_env.sh：为什么它不只是“装包脚本”

如果你只从名字看，`bootstrap_env.sh` 很容易被理解成“一个把依赖装上去的脚本”。这样理解只说对了一半。它真正的重要性在于，它把当前项目在这台共享工控机上的环境边界写成了默认策略。脚本里明确把 Conda 根放在 `/data/howard/miniconda3`，把临时目录和 pip 缓存也放到 `/data/howard/splatnav/...`，然后在这个边界内创建或复用 `splatnav` 环境。它安装的依赖里既有 `Nerfstudio`，也有 `clarabel`、`cvxpy`、`dijkstra3d`、`open3d`、`viser`、`LightGlue` 这些当前上游代码会直接或间接用到的包。

换句话说，这个脚本不是在粗暴地“把一堆包塞进系统里”，而是在明确地说：我们只在自己的 Conda 小环境里解决问题，不碰系统 CUDA，不碰别人账户的环境，不把缓存乱堆到系统盘。这种克制本身就是工程能力的一部分。学弟如果能从第一天就建立这种边界感，后面会少走很多弯路。

6.2.2 scripts/env_health_check.sh：为什么“环境搭好”不能靠感觉宣布

很多人一旦看到 `pip install` 结束，就会忍不住说“环境应该差不多了”。这是最常见的幻觉之一。真正稳妥的做法，是让环境自己出示证据。`env_health_check.sh` 做的正是这件事。它会打印系统信息、GPU 信息、Python 版本、Torch CUDA 状态，并进一步尝试 `import` 关键依赖模块。这样你就能分清楚：是安装阶段出了问题，还是运行时链接有问题，还是某个包虽然装了但不能正常 `import`。

这对新人极其重要。因为一旦没有这一步，你后面所有 `smoke` 失败都可能被误判成算法问题。可实际上，很多“算法没跑起来”的事故，根本还停留在环境层。这个脚本的价值，就是帮你把这一层先排干净。

6.2.3 scripts/fix_torch_cuda.sh：为什么它体现的是“只在自己边界内修复”

这个脚本从名字看也很直接，但它背后的工程态度值得学弟记一记。它不是去升级整机 CUDA，更不是碰显卡驱动，而是在当前 Conda 环境内重装一个和现有驱动兼容的 PyTorch CUDA 运行时版本。也就是说，它修的是“我的小环境里 Torch 跟当前机器的关系”，不是“整个机器的全局软件栈”。

对于共享机器来说，这种做法非常成熟。它承认自己要解决的问题是局部问题，就只在局部边界里动手。很多新手一遇到 CUDA 相关报错就很容易慌，脑子里只剩“是不是要升级 CUDA”。你越早知道并不是所有 CUDA 问题都要动系统层，就越不容易做出过度破坏性的决定。

6.3 数据准备组脚本：它们在处理“论文给了数据，但真实下载没你想得那么顺”

对于第一次复现论文项目的同学来说，最容易低估的一件事，就是数据准备。很多人会想：作者不是给了 Google Drive 吗，那我下下来不就行了。现实往往没这么丝滑。网络波动、文件夹下载不完整、某些帧缺

失、`transforms.json` 或 `sparse_pc.ply` 漏掉、训练输出和数据目录之间还要重新挂接，这些都是真实问题。当前 `wrapper` 里的这一组脚本，就是在替你处理这些“下载不是一句话”的现实。

6.3.1 `scripts/download_official_data.sh`：它像统一下载入口，帮你把来源和落点说清

这份脚本的价值不在于它用了 `gdown`，而在于它把“从哪里下载、下载到哪里、在什么环境里执行”这三件事固定了下来。你以后做复盘时，也更容易说清：数据是按官方给出的 Google Drive 来源下载的，落在数据盘指定目录，而不是散落在 `home` 目录、浏览器默认下载目录或某个临时文件夹里。

这种可追踪性对新人非常重要。因为很多人第一次复现时最大的混乱，不是代码，而是“我到底从哪下的、下到了哪、这个目录是不是完整的”。统一入口恰恰就是在减少这种混乱。

6.3.2 `scripts/link_official_assets.sh`：为什么软链接比复制更像成熟工程

这个脚本做的事情特别值得讲，因为它把“上游默认目录结构”和“本机真实资产存放策略”优雅地接起来了。它会把数据盘里的 `training/data`、`training/outputs`、`traj` 通过软链接接到上游仓库期望的位置。这样你既不需要真的复制出两大份重资产，也不用为了一次复现去魔改上游源码里写死的相对路径逻辑。

这其实就是工程里非常经典的一类操作：尽量不改上游假设，而是在外围把环境整理成它能接受的样子。学弟如果能理解这一点，后面读更大的工程也会更顺。因为很多时候，高质量工程不是“把别人代码改得面目全非”，而是“在不破坏上游的前提下，把自己的接入层做漂亮”。

6.3.3 `scripts/retry_gdown_folder.sh`、`fill_missing_scene_files.sh`、`pull_images2_full.sh`：为什么这些脚本听起来琐碎，却非常像真实项目

这三个脚本放在一起看，能很好地帮助学弟建立一种现实感。真实世界里的数据准备，往往不是一次命令就收工，而是要面对“不稳定但可补救”的状态。也就是说，下载过程不是全成全败，而是可能出现少数帧缺、关键文件缺、文件夹结构看着像到了但其实不完整的情况。

`retry_gdown_folder.sh` 更像“多次重试并按最低验收条件判断完整度”；`fill_missing_scene_files.sh` 更像“根据已有日志精确补洞”；`pull_images2_full.sh` 更像“针对已知的图片文件夹做多轮尝试直到达到最低数量门槛”。你不必把这几个脚本全背下来，但你应该通过它们学会一个成熟观念：很多工程问题不是重新来一遍，而是识别缺口、局部修补、用条件判断是否真正完整。

6.4 run 管理与 `tmux` 组脚本：它们在回答“长任务怎么跑得像个工程，而不是像赌运气”

很多人初学时，脑海里只有一种运行方式：打开终端，敲命令，盯着输出。这样当然也能工作，但一旦任务需要几小时，或者你需要把日志和环境都留住，或者中途网络断了，或者你第二天还要复盘，就会很痛苦。当前工作区在这方面做得比较成熟，因为它把 `run` 目录规范和 `tmux` 绑定在了一起。

6.4.1 `scripts/create_run_dir.sh`：为什么它是当前工程层的“证据生成器”

学弟一定要把这份脚本看得重一点。不是因为它长，而是因为它定义了“每次任务应该留下什么痕迹”。它会按 `runs/YYYY-MM-DD_HHMM_topic` 的命名规范建目录，同时自动生成 `env.txt`、`notes.md`、`cmd.txt` 和 `stdout.log`。你可以把它理解成一个实验记录起点。任何一次任务，只要先经过它，就不会是“跑过就算了”。

这背后其实是一种非常值得培养的职业习惯。科研也好，工程也好，真正能复盘的人，一定会让每次运行有最小证据集。以后你回看某次失败实验，不是只能靠模糊记忆想“当时好像跑的是那个命令”，而是能直接打开 `cmd.txt` 和 `env.txt` 看清楚。这个差距非常大。

6.4.2 `scripts/run_in_tmux.sh`：它为什么值得你反复讲给学弟听

如果只能挑一个最能体现当前 `wrapper` 层价值的脚本，我会很愿意选 `run_in_tmux.sh`。原因不复杂：它把长任务管理、`run` 目录规范和日志沉淀三件事一次性绑在一起了。你给它一个窗口名、一个 `topic` 和一条命令，它会先帮你创建 `run` 目录，把命令记下来，生成 `job` 脚本，再把任务放到 `splatnav_jobs` 这个 `tmux` 会话的新窗口里执行，并通过 `tee` 把终端输出同步写进 `stdout.log`。

你稍微体会一下，就会发现这不是普通的“后台运行脚本”。它是在告诉你一套完整的工程态度：长任务应该进入 `tmux`，`tmux` 任务应该有统一 `session`，`session` 里的窗口应该和 `run` 目录绑定，`run` 目录里应该能反查命令和环境，日志应该自动落盘而不是指望手工复制。你以后如果进入大厂，会发现这种“把工程习惯封进工具”的思路无处不在。

6.4.3 `scripts/watch_overnight_job.sh`：它在教你“工程要考虑失效恢复”

这个脚本很像守夜员，但它真正教会人的不是“晚上也有人盯着”，而是“成熟工程会显式考虑任务突然消失怎么办”。它会根据特定模式去找 `overnight` 进程，如果发现进程没了，就尝试重启。这里的关键，不是它一定能修好所有问题，而是它把“失效恢复”当成系统设计的一部分，而不是全靠人运气守着。

学弟第一次接触这种脚本时，最好顺势理解一个概念：自动恢复不是替代分析，而是降低脆弱性。它不能替你查清为什么挂，但它能避免一次中断直接让一整夜作废。这种思想，本身就是工程经验的一部分。

6.5 执行链脚本：从最小 `smoke` 到正式批跑，工作区是怎样一步步放大规模的

这组脚本是学弟最应该建立顺序感的一组。因为它们几乎直接对应你的学习和实验路径：先 `smoke`，再 `batch`，再 `overnight`，再报告。你一旦把这个顺序走顺，项目推进会非常自然。你如果跳着来，反而更容易焦虑。

6.5.1 `scripts/smoke_splatplan.py`：为什么说它是理解规划主链最好的第一入口

在所有执行链脚本里，`smoke_splatplan.py` 是最适合新人先啃的一份。因为它足够真实，又足够短。它已经具备一条完整规划链该有的关键结构：从命令行拿到 `--config`、`--scene`、`--method`、`--output`，根据场景预设组装机器人配置和体素配置，通过 `GSplatLoader` 读取模型，再根据方法名实例化 `SplatPlan` 或 `SafeFlightCorridor`，最后用一组固定起终点调用 `generate_path()`，把结果写进 JSON。

你只要真的把这份脚本看懂一次，项目里很多抽象概念都会落地。你会知道场景名不是空词，知道 `config.yml` 真的是模型入口，知道规划器构造时需要机器人参数和体素边界，知道输出并不只是一个轨迹数组，而是包含 `meta`、`device`、时间和规划结果的结构化对象。对新人来说，这种“缩微版的完整链路”价值极高。

6.5.2 `scripts/run_splatplan_smoke.sh`：它在做什么规范化动作

如果说 `smoke_splatplan.py` 是主逻辑，那么 `run_splatplan_smoke.sh` 更像一个规范化启动器。它负责调用 `create_run_dir.sh`，生成输出 `JSON` 路径，把命令落进 `cmd.txt`，激活 `splatnav` 环境，清理 `LD_LIBRARY_PATH`，最后再真正调用 Python `smoke` 入口。它没有改变规划算法，但它把“如何正确地跑一次 `smoke`”固定成了统一步骤。

这类 shell 包装脚本非常值得新人认真看。因为它恰好处在“工程规范”和“算法入口”之间，是理解整个工作区为什么不乱的重要桥梁。

6.5.3 scripts/run_splatplan_smoke_tmux.sh：为什么 smoke 也值得有 tmux 版本

有些同学会觉得 smoke 任务不是很短吗，为什么还要出个 tmux 版。这个问题问得很好。答案是：不是每次 smoke 都短到你愿意盯着，也不是每次网络都稳定到你敢把实验绑定在当前会话上。tmux 版的 smoke，其实是在延续同一种工程原则：只要任务值得保留日志和上下文，就尽量交给统一的 run + tmux 机制。

从教学角度看，这个脚本还很有帮助，因为它让你看见 wrapper 层是如何“层层复用”的。它并没有自己重写一套 tmux 管理，而是把命令进一步交给 run_in_tmux.sh。这种复用关系，就是工程结构感的体现。

6.5.4 scripts/run_official_style_batch.py：为什么它是当前工作区最关键的正式实验入口

如果说 smoke 是最小验证，那么 run_official_style_batch.py 就是正式分析的起点。它会围绕场景预设生成一批起终点对，支持多个方法名，比如 splatplan 和 sfc-1 到 sfc-4，并在多次 trial 上重复调用同一规划器接口。更重要的是，它会把结果整理成适合后续汇总和可视化消费的 batch_results.json：里面不仅有方法级统计，还有部分保留的轨迹样本、每个 trial 的压缩输出、可行率、平均耗时和 P95 等信息。

这一份脚本特别值得你带着学弟慢慢读。因为它恰好体现了“工程化地尊重上游算法本体”。它没有重写 SplatPlan 的数学，而是在调用层面把输入、输出、统计和保存方式做了标准化。对未来想练大厂工程感的人来说，这种写法非常有代表性。

6.5.5 scripts/run_overnight_12h.sh：为什么夜间任务本质上是在组织很多轮 batch，而不是在造另一套算法

新人很容易把 overnight 想得很神秘，仿佛一到了晚上，项目就切到了另一种模式。其实不是。run_overnight_12h.sh 的本质，是在给定时间窗口内重复发起正式 batch，每一轮都建自己的 run 目录、保存自己的 batch_results.json，并尝试顺手生成图表。它做的事情更像实验生产线调度，而不是算法创新。

理解这一点很重要。因为你以后会逐渐知道，很多“复杂实验系统”的本质，其实只是把基础单元组织得更稳了。overnight 之所以有价值，不是因为它玄乎，而是因为它让“多轮实验 + 自动落盘 + 次日可读”这件事变得低摩擦。

6.5.6 scripts/post_overnight_report.sh：为什么第二天早上应该看到的是报告，不是一堆零散 run 目录

夜间任务最怕的是什么？不是某一轮失败，而是第二天早上你打开 runs/，只看到几十个目录，不知道该先看哪个、该看什么。post_overnight_report.sh 的价值就是避免这种局面。它会等 overnight 任务结束，再调用聚合脚本，产出 CSV、Markdown 和稳定软链接。这样第二天的人不会面对“目录山”，而是先面对一份总报告。

这背后是一种很贴近实际工作节奏的思路：结果不只是为了存在，还要为了被人消费。你以后做周报、做组会、带学弟，都会很需要这种思维。

6.6 结果消费组脚本：为什么“会看结果”本身就是能力

很多新人以为项目能力主要体现在会不会把程序跑起来。其实再往前走一步你就会发现，真正稀缺的能力之一是“会看结果”。当前 `wrapper` 里负责这个部分的脚本，虽然不直接参与规划，但对项目推进同样重要。

6.6.1 `scripts/visualize_batch_results.py`：它在把 JSON 翻译成更容易汇报的语言

这份脚本会把 `batch_results.json` 里的内容进一步做成可视化图和简要 Markdown 摘要。比如不同方法的可行率柱状图、规划时间箱线图、示例轨迹的 XY 叠图。它存在的意义，不只是好看，而是把原本需要你盯着 JSON 逐项解释的内容，翻译成了更适合人眼和汇报的形式。

学弟越早理解这点越好。因为实验的终点从来不是一个 JSON 文件，而是“你能不能把 JSON 里的信息讲给别人听”。可视化脚本其实就是在帮助你完成这种翻译。

6.6.2 `scripts/aggregate_batch_runs.py`：它在告诉你，跨 run 的比较才是正式实验真正的视角

单次 `batch` 很重要，但如果你的实验要持续很多轮，真正能支撑决策的往往是跨轮比较。`aggregate_batch_runs.py` 就是在做这件事。它会去收集匹配场景名的多个 `batch_results.json`，按方法和 iteration 汇总成 CSV、Markdown 和趋势图。这让你不再只盯着某一轮结果，而开始看整体稳定性、方法排序和夜间任务的收敛走势。

这对培养工程和研究判断都非常关键。因为真正靠谱的结论，通常不是建立在一次“看起来挺好”的 `run` 上，而是建立在多次记录一致指向某种趋势之上。

6.7 同步维护脚本：它提醒你上游不是空气，`wrapper` 也不是孤岛

`sync_upstream.sh` 这类脚本，日常使用频率可能没有 `smoke` 和 `batch` 高，但它在概念上非常关键。它会帮你抓上游仓库的当前提交和分支状态，提醒你 `wrapper` 是建立在一个具体版本的上游之上的，而不是漂浮在空中。你如果以后开始做更多工程整理，就会越来越感受到这种版本感知的重要性。

因为一旦你不知道自己封装的是哪一版上游，后面别人问你“这个行为到底是原作者这样写的，还是你们外层后来改的”，你就很难说清楚。同步脚本存在的意义，就是帮你减少这种版本失忆。

7、从一条 `smoke` 命令出发，把调用链完整走一遍

讲到这里，如果还只是停留在“每个脚本大概干什么”，你对项目的掌握还不够稳。真正能让你脑子开始成型的，是把一条最小调用链完整走一遍。也就是说，不只是知道会调用哪些名字，而是知道命令从哪里进入、在哪里切层、什么时候从 `wrapper` 进入上游、什么时候再回到 `run` 目录落盘。

7.1 先从 shell 层开始：命令不是凭空跑的

假设你用的是最常见的 `smoke` 入口之一，也就是 `scripts/run_splatplan_smoke.sh`。当你从 shell 层调用它时，第一件发生的事情不是规划，而是工程规范动作。脚本会检查参数够不够，会用 `create_run_dir.sh` 先建立当前 `run` 目录，再在里面准备输出 JSON 的路径，同时把你即将运行的命令记录到 `cmd.txt`。这意味着在真正执行规划之前，当前任务已经有了一个归档容器。

这一点非常值得你反复体会。因为很多项目出问题，不是因为算法错，而是因为命令和结果没有被绑定。你日后再找这次运行时，连它用的参数是什么都说不清。当前 `wrapper` 把这件事提前做了，所以它一开始就在为未来的复盘负责。

7.2 环境切换发生在什么时候，为什么这一步必须显式写出来

继续往下走，shell 脚本会 `source /data/howard/miniconda3/etc/profile.d/conda.sh`，再 `conda activate splatnav`，同时显式清掉 `LD_LIBRARY_PATH`。这一步对新人非常关键，因为它意味着“我要用哪一个 Python、哪一套依赖、哪一套动态库解析规则”并不是空气，而是任务的一部分。工程上真正稳的做法，就是把这些前提写出来，而不是默默认定它们已经是对的。

你以后如果在日志里看到有些运行会故意 `env -u LD_LIBRARY_PATH python ...`，也不要觉得奇怪。这恰恰是在防止系统层的库解析污染当前 Conda 环境。对于共享机器，这是一种很常见也很成熟的克制做法。

7.3 进入 Python 入口后，第一件事不是规划，而是找上游源码

进入 `scripts/smoke_splatplan.py` 后，很值得学弟注意的第一件事，是它先做了 `resolve_upstream_root()`。也就是说，它先去确认上游官方仓库实际在哪里，再通过 `sys.path.insert(0, ...)` 把上游路径插进来。这个细节特别值得理解，因为它充分说明：wrapper 工作区并不是把上游代码复制进自己脚本里，而是显式把上游当依赖导入。

这就是当前工作区“官方仓库保留原样 + 外层工程化封装”的设计哲学在代码层的直接体现。你未来做更复杂的工程时，也会很常遇到这种结构：自己的脚本层负责组织，核心实现仍然从依赖仓库导入。

7.4 参数解析之后，scene preset 开始真正发挥作用

`smoke_splatplan.py` 里的 `SCENE_PRESETS` 是一个极其值得学弟重视的结构。因为它把论文实验里那些散落的场景特定参数，收拢成了一种可以复用的、本机可解释的配置表。里面不仅有 `radius`、`vmax`、`amax` 这些机器人相关参数，还有 `lower_bound`、`upper_bound`、`resolution` 这些体素空间边界，以及 `radius_config`、`mean_config` 这种用来构造起终点位置的参数。

你可以把 `scene preset` 理解成“论文场景在本机上的实验入口说明书”。它不是场景模型本身，却决定了你如何围绕这个场景发起实验。如果这一层你理解得很浅，后面就会经常困惑：为什么同样是 `old_union`，还要再有一堆边界、半径和分辨率参数。其实它们不是多余，而是把抽象场景落成可运行实验的必需条件。

7.5 GSplatLoader 介入的那一刻，项目才真正接上“场景”

在 planner 构造前，脚本会先实例化 `GSplatLoader(config_path, device)`。这一步非常关键，因为它是整个规划链第一次真正接触具体场景表示的地方。你在文档里、论文里、README 里听过很多遍 Gaussian Splatting，可直到这里，它才从概念变成当前 Python 进程里的一组真实张量：均值、旋转、尺度、协方差、颜色、不透明度。

所以以后你只要问“这个项目到底是怎么把场景读进来的”，答案就该自然落到这里。它不是从某张地图图片读进来的，也不是从一个抽象接口突然蹦出来的，而是通过 `GSplatLoader` 把 Nerfstudio 训练产物接进了规划系统。

7.6 planner 构造阶段：为什么同一个入口既能跑 splatplan，也能跑 sfc-*

接下来脚本会构造 `SplinePlanner`，再根据方法名决定实例化 `SplatPlan` 还是 `SafeFlightCorridor`。这一段对新人非常有启发，因为它让你看到一个好用的实验入口应该是什么样：同样的场景输入、同样的机器人参数和体素配置，通过不同 planner 类去跑，就能形成方法对比。

也正因为这一层结构清楚，wrapper 的 batch 脚本才能很自然地扩展到多方法比较。你未来做实验时，也应该逐渐培养这种思维：尽量让不同方法在统一的输入和落盘规范下被调用，这样比较才公平、结果才好看、后处理才不乱。

7.7 真正进入 generate_path() 后，规划链开始像论文那样展开

当 `planner.generate_path(x, g)` 被调用时，当前任务才正式进入论文主链。以 `SplatPlan` 为例，`generate_path()` 的第一步是让 `GSplatVoxel` 生成可导航体素网格，并在其上做 A* 初始化。然后，它把初始路径切成一段一段的 `line segment`，对每一段计算碰撞集合，再逐段构造多面体约束，最后把多面体序列交给 `SplinePlanner.optimize_b_spline()` 做平滑轨迹优化。

如果你把这条链真走明白一次，后面论文里很多抽象说法就不会再让你发虚。你会知道所谓“safety corridor”不是修辞，所谓“Bezier trajectory”也不是停留在图里，而是真的在 `spline_utils.py` 里通过 QP 去解的。代码一旦跟论文主张对应上，理解就会稳定很多。

7.8 JSON 落盘阶段：算法输出为什么要被重新整理

`generate_path()` 返回的结果虽然已经有很多信息，但 wrapper 并不会就地停下。`smoke_splatplan.py` 还会把 `meta`、`device`、`plan_time_sec`、`start`、`goal` 和 `output` 一起写成结构化 JSON。这个动作很值得重视，因为它体现了工程层一个非常成熟的判断：原始算法输出是一回事，适合长期保存和后续消费的实验记录是另一回事。

也就是说，wrapper 在做的是“结果包装”。它没有篡改算法意义，但它让结果更适合被以后的人理解、检索和比较。对学弟来说，这一点非常值得学习。因为真正好的工程，从来不是“只要能 print 出来就行”，而是会认真设计结果对象该长什么样。

7.9 如果你把这条链能口头讲出来，说明你已经真正开始会了

很多人喜欢用“我看懂了”来描述自己的进度，但更可靠的标准其实是：你能不能把一条最小调用链不用看代码、用自己的话讲出来。如果你现在已经能说清楚“shell 包装先建 `run` 目录，再激活环境，再调 Python 入口；Python 入口先找上游仓库，再根据 `scene preset` 和 `config.yml` 构造 `planner`；`planner` 再调 `generate_path()` 走 A*、碰撞集合、多面体和样条优化；最后结果被写成 JSON 并落进 `run` 目录”，那你就已经不是停留在文件浏览层了。

而一旦你过了这一关，后面的 `batch`、`overnight`、汇总和可视化，基本都是在这条最小链路上做规模放大和结果管理。项目会突然变得没那么神秘，这是一种非常明显的成长体验。

8、把上游源码树真正读出层次：不是所有目录都同等重要，但每个目录都在回答一个问题

到了这里，我们要正式进入 `splatnav-official/` 这一层。很多新人一到上游源码树就容易出现两种反应。一种是被目录多吓住，直接放弃，退回去只看 wrapper。另一种是过度兴奋，想一口气逐文件读完，结果越读越散。更合理的办法，是先把上游源码树按职责分层，再决定先读哪一层、后读哪一层。

你可以先接受一个现实：上游源码不是一本线性教材，它更像一个实验室里逐渐长出来的实现集合。里面有非常核心的主链文件，也有辅助可视化、基线比较、结果处理和定位实验工具。你的任务不是把它们平均用力，而是学会判断谁是主轴、谁是侧翼、谁是现在先知道存在就够了。

8.1 先把最核心的一句话记住：上游源码树至少同时承载了五种东西

第一种，是规划核心实现，也就是 `Splat-Plan` 真正在哪里算路径、造多面体、解样条。第二种，是场景表示与几何工具，也就是 `GSplat` 怎么被加载、协方差怎么被处理、碰撞集合怎么被抽出来。第三种，是定位相关实现，也就是 `Splat-Loc` 所依赖的视觉与位姿估计链。第四种，是对比方法和实验入口，比如 `run_rrt.py`、`run_nerfnave.py`。第五种，是可视化、绘图和结果处理工具。

你如果先把这五类分开，再看目录名，就不容易被吓住。因为目录多并不代表结构乱，很多时候只是因为把不同问题拆开了。

8.2 `run_splatplan.py`：为什么说它像作者自己的实验工作台

我们前面已经简要提过这份文件，这里要把它讲得更细一点。`run_splatplan.py` 非常像作者自己的实验工作台。它里面有一整套围绕场景名切换的实验设定，包括 `old_union`、`stonehenge`、`statues`、`flight` 这些论文场景；也有稠密和稀疏 `GSplat` 模型的切换；还有机器人半径、速度、体素边界和分辨率等参数。接着，它会加载 `GSplat`、构造 `SplinePlanner`，再根据方法名实例化 `SplatPlan` 或 `SafeFlightCorridor`，最后围绕圆形轨迹上的多个起终点对循环做规划，并把结果保存到 `trajs/` 目录。

为什么我说它像实验工作台？因为它暴露得很直接，很少做“用户友好封装”。你能从里面明显感受到作者自己在调实验、对比方法、改场景参数时的使用方式。对新人来说，这种文件一开始看起来不如 `wrapper` 漂亮，但它反而非常有帮助，因为它让你看见论文作者原始的实验心智模型是什么。

8.3 `run_splatloc.py`：它为什么不像规划入口那样短平快

`run_splatloc.py` 一眼看上去就比 `run_splatplan.py` 更厚重。原因很简单，定位问题在实现上本来就涉及更多模块：图像数据集、相机内参、RGB 观测、点云、特征匹配、全局配准、局部精配准、PnP、可能的递推估计等。它不是一条“给起点终点就生成轨迹”的单一链路，而是一条以感知和估计为核心的流水线。

这也是为什么我不建议零基础学弟第一周就扎进去硬啃。不是说它不重要，而是说它的认知压力会更大。更适合的节奏，是先知道它在总系统里负责什么，再随着项目推进逐步把 `pose_estimator/utls.py` 和 `ns_utils/nerfstudio_utils.py` 这些支撑文件拆开读。

8.4 `SFC/` 目录：它不只是一个 `baseline` 文件夹，而是一整个“安全走廊家族”

很多人第一次看到 `SFC/`，会很自然地以为它只是放一个 `baseline` 实现。实际上，它比“单个 `baseline`”更像一组围绕安全飞行走廊思想组织起来的工具模块。`corridor_utils.py` 是核心，它定义了 `Corridor` 抽象基类和 `SafeFlightCorridor` 具体实现，负责不同 `mode` 下的初始化、碰撞集合和多面体生成逻辑。`astar_utils.py`、`helper_utils.py`、`viz_utils.py` 则分别承担初始化搜索、几何辅助和可视化支持。

这一组代码非常适合学弟去理解“`baseline` 为什么不是一句名字”。论文里写 `sfc-1` 到 `sfc-4`，你如果只是把它们当成四个标签，理解会很浅。读过 `SFC/corridor_utils.py` 以后，你会知道这些 `mode` 在初始化网格、碰撞集合类型和安全缓冲策略上都真的有区别。这样你以后看实验结果时，就不会只会说“某个 `baseline` 好像更强”，而能说清它强弱背后的结构原因。

8.5 splat/ 目录：为什么说这里是“场景入口层”

Splat-Plan 再核心，也不能脱离场景表示自己工作。splat/ 目录的意义就在这里。它主要回答一个基础但关键的问题：当前项目如何把一个 Gaussian Splatting 场景读成后续模块能用的数据结构。splat_utils.py 里的 GSplatLoader 就是这层的主角。

当你真正去看它的实现时，会发现它做的事情其实很朴素但很关键：根据输入是 json 还是 Nerfstudio config.yml，走不同加载流程；把场景中的均值、旋转、尺度、协方差、颜色和不透明度整理成统一属性；必要时还能把 GSplat 转成 mesh 供可视化使用。你以后只要问“规划器到底在处理什么几何对象”，都绕不过这一层。

8.6 splatplan/ 目录：这是真正该反复读、反复讲、反复对照论文的一层

如果让我给学弟在上游源码里只挑一个最值得深耕的目录，那一定是 splatplan/。因为这里就是论文核心主张的主要落点。splatplan.py 负责从初始化路径到碰撞集合、多面体约束再到最终轨迹的整体流程组织；spline_utils.py 负责把多段多面体约束转换为样条优化问题，并调用 clarabel 或 cvxpy 相关求解流程求出轨迹系数。

一个很实用的建议是：你第一次读 splatplan.py 时，不要试图逐行证明，而要先把函数里的阶段划分看清。第二次再回来看 get_polytope_from_outputs()，理解它是怎么从碰撞集合和交点信息生成半空间约束的。第三次再去读 spline_utils.py，理解为什么这些多面体最终会变成一个带等式、带不等式、带二次目标项的 QP。这样读法会比一开始就死抠矩阵舒服很多。

8.7 initialization/ 目录：它在处理“先给我一条能走的种子路径”

规划里很容易被忽视的一层，是初始化。很多人张口就是优化、约束、多面体，好像一切都从目标函数开始。其实不是。initialization/ 目录提醒你，真正的规划往往先要有一条粗路径，再围绕它做后续几何与优化处理。grid_utils.py 里定义了 Voxel、PointCloudVoxel 和 GSplatVoxel，它们负责把场景离散成可导航网格，处理占据情况，并在需要时把起点或终点投影到最近可导航位置。astar_utils.py 则负责底层的 A* 搜索。

这一层对新人特别有启发，因为它让你看到：规划不是从连续优化凭空起步的，而是先通过离散网格建立可行的骨架路线。你以后再听到“hybrid planning”之类的词，也会更容易理解，因为你已经在当前项目里亲眼见过离散初始化和连续优化是怎么接上的。

8.8 polytopes/ 目录：它为什么是理解安全性的真正几何后院

如果说 splatplan/ 是主舞台，那么 polytopes/ 更像后台真正控制几何约束生成的地方。这里有 collision_set.py、decomposition.py、polytopes_utils.py、connectivity_utils.py 等文件，负责从路径段出发，抽取可能碰撞的几何基元，形成边界约束，并做半空间表示和相关辅助计算。

对新人来说，最值得先读的是 collision_set.py。因为它比较直接地把“碰撞集合”这件事展开了：根据路径段先构造一个包围盒，然后判断哪些点云点或哪些高斯椭球与这个包围盒相关，把这些相关对象打包返回。你只要把这一步理解清楚，后面再去看多面体生成，就不会觉得它像在凭空造约束。

8.9 ellipsoids/ 目录：为什么高斯不是抽象云团，而是会变成可算几何对象

GSplat 听起来像图形表示，但在规划里它要能变成可计算几何对象。ellipsoids/ 目录做的正是这种翻译工作。这里有协方差相关工具、相交测试、mesh 构造、椭球采样等函数。比如 covariance_utils.py 会处理

旋转和尺度到协方差的关系，`intersection_utils.py` 会参与线性运动与椭球相交的计算，`mesh_utils.py` 能把 GSplat 场景生成可视化 mesh。

你如果以后要向别人解释“为什么论文说高斯约束是数学严格的”，这层会非常重要。因为它在代码里把高斯从视觉对象变成了碰撞检测和半空间构造的几何基础。

8.10 ns_utils/ 目录：它是项目和 Nerfstudio 世界连接的接口层

上游 README 已经明确说这个仓库建立在 Nerfstudio 之上，所以你在源码里看到 `ns_utils/` 一点也不奇怪。这里的关键文件是 `nerfstudio_utils.py`，其中的 `GaussianSplat` 类和数据集加载函数承担了“把 Nerfstudio 训练输出和数据解析结果转成当前项目可用对象”的角色。你可以把它想成一座桥：一头连着 Nerfstudio 的训练与数据解析世界，一头连着当前项目的规划与定位世界。

这一层对学弟非常关键，因为它解释了为什么 `config.yml` 会如此重要。`config.yml` 不是某个普通参数文件，而是把你带回 Nerfstudio 训练结果上下文的入口。没有这层桥梁，规划器和定位器都不知道该去哪里拿模型和数据。

8.11 pose_estimator/ 目录：它让你看见 Splat-Loc 不是一句名字，而是一整套视觉与几何流水线

如果说 Splat-Loc 在宏观上代表定位模块，那么 `pose_estimator/` 就是在微观上把定位真正做出来的地方。`utils.py` 里面你会看到一大串看起来跨度很大的工具：SIFT、ORB、LightGlue、RANSAC、ICP、颜色 ICP、PnP、SE(3) 和李代数相关操作、甚至一些递推估计和 sigma points 相关函数。这种密度很容易把新人吓住，但你换个角度就会好很多。

更顺的理解是：定位本来就要同时面对视觉特征、几何配准和位姿表示这几类问题，所以工具会多是正常的。你不需要第一周就把这里的每一个函数都吃透，但至少应该知道它们是定位链路的工具箱，而不是和主项目无关的杂项。

8.12 nerfnav/ 目录与 run_nerfnav.py：它们在提醒你论文作者认真对比了另一条技术路线

很多新人一读当前项目，很容易把视线完全集中在 GSplat 世界。这样虽然自然，但也会漏掉论文很重要的一部分背景：作者不是只证明自己的方法能跑，还拿它和一类 NeRF 导航方法做了比较。`nerfnav/` 目录和 `run_nerfnav.py` 就是在源码层保留了这种对比路线。

你以后做汇报时，这一层很有帮助。因为它提醒你 Splat-Nav 的价值不只是“能规划”，而是“在 GSplat 表示上做规划与定位，有机会比 NeRF-based 导航更实时”。至于第一阶段要不要深入这里，答案是不急。但你要知道它存在，而且它不是装饰。

8.13 run_rrt.py：它在让经典采样法也进入实验比较视野

和 `run_nerfnav.py` 一样，`run_rrt.py` 也在提醒你：好的论文不会只和自己说话，而是会把经典方法拉进来对比。RRT 这类方法对机器人规划读者来说并不陌生，上游把它单独保留为实验入口，本质上是在支持横向比较。

对学弟来说，这有两个好处。第一，它帮助你建立一点研究感，知道方法评价从来不是自说自话。第二，它也帮你理解当前 wrapper 为什么优先聚焦 `splatplan` 和 `sfc-*`。因为这两类方法与当前离线仿真链的结构更近，更适合作为第一阶段的对比主线。

8.14 visualize.py、blender/、plotting/、results/：它们在负责“怎么把实验讲出去”

上游源码里并不只有“怎么算”，还有一整层是“怎么算完以后怎么展示”。visualize.py 会用 viser 加载 GSplat、mesh、体素和轨迹，支持交互式查看。blender/ 更偏离线渲染和视频相关的辅助可视化。plotting/ 与 results/ 则更偏统计结果的处理和图表生成。

这类目录在新人眼里很容易被当成可有可无，但它们其实很重要。因为一个项目如果只会产出数字，不会把结果讲给别人听，它的传播能力和协作效率都会很受限。你以后做周报、组会、专利图、论文图，都会逐渐感受到这些“展示层工具”的价值。

9、把几个核心类讲成人话：你以后读代码时要知道自己在看谁扮演什么角色

目录知道了，还不够。很多人真正卡住的，是一打开代码，满眼都是类和函数名，根本不知道自己该把谁当主角。这里我们把最关键的几个类和函数，用一种更接近“老师在白板前讲角色分工”的方式讲明白。

9.1 GSplatLoader：它像把场景搬进房间的人

如果把整个规划问题想成在一个房间里工作，那么 GSplatLoader 就像把房间本身搬进来的人。没有它，后面的 SplatPlan、SafeFlightCorridor、GSplatVoxel 都无从谈起。它把 config.yml 背后的模型状态读出来，整理成统一属性，让其他模块能够直接使用高斯的均值、旋转、尺度和协方差。

你以后读到哪里一旦看见 GSplatLoader(config_path, device)，脑子里就应该很自然地浮现一句话：好，场景现在要进内存了，后面的规划和几何模块终于有对象可以处理了。这个角色感一旦建立，代码会好懂很多。

9.2 GSplatVoxel：它像把连续场景压成“先走得起来”的离散骨架

GSplatVoxel 的价值，在于它让连续、复杂、密度很高的 GSplat 场景，变成了一个可以做网格搜索的离散工作空间。它会根据边界和分辨率建立网格中心，利用高斯包围盒等信息去判断哪些体素不可通行，并在需要时把起点和终点投影到最近可导航位置。最后，A* 就能在这个离散空间里跑起来。

所以你以后如果觉得论文里“先初始化一条路径再做精细规划”很抽象，就回到这里想。GSplatVoxel 就是在提供那条粗骨架。它不负责最终优美轨迹，但它保证你不是从一片完全连续的未知空间直接跳进优化器里。

9.3 GSplatCollisionSet 和 PointCloudCollisionSet：它们像两种不同口味的“相关障碍提取器”

这两个类很适合放在一起理解。它们都继承自 CollisionSet，都在回答一个共同问题：对于当前路径的一段，哪些障碍物最相关，值得拿来构造约束。不同之处在于，PointCloudCollisionSet 更像在点云或点采样层面工作，而 GSplatCollisionSet 更像直接在高斯椭球层面工作。

这就是为什么你在 sfc-* 不同 mode 里会看到它们以不同方式出现。某些 baseline 更依赖点云式近似，某些则更直接围绕 GSplat 椭球做处理。你把这层理解清楚，以后看不同方法的安全性和可行率差异，就会更有感觉。

9.4 SafeFlightCorridor：它像“先搭走廊，再从走廊中取轨迹”的家族总管

SafeFlightCorridor 不只是单个 baseline 类，它更像一整个家族总管。通过 mode 切换，它把几种安全走廊策略收在同一接口下。它会先做初始化路径，再针对每一段提碰撞集合，再生成对应多面体，最后同样调用样条规划器生成轨迹。你可以把它看成“多种 SFC 变体的统一外壳”。

这层统一外壳很重要，因为它让方法比较变得整齐。你以后不必每次都在脑子里重新记一套 `completely different` 的接口，而只需记住：`sfc-1` 到 `sfc-4` 是同一家族里不同的几何与安全缓冲策略。

9.5 SplatPlan：它是当前阶段你最该反复结交的主角

如果你未来一个月只能和一个类建立真正熟悉的关系，我会建议你选 `SplatPlan`。原因很简单：它是当前离线/仿真复现主线里的算法核心主角。它串起了 `GSplatVoxel`、`GSplatCollisionSet`、多面体生成和 `SplinePlanner`。你只要把它的主流程掌握扎实，项目最重要的“输入怎么变成输出”就已经抓住了大半。

更重要的是，这个类的结构很适合教学。`__init__` 里主要是把场景、机器人参数、环境参数和样条规划器绑在一起；`generate_path()` 里则显式地走完初始化、碰撞集合、多面体和优化四步。对新人来说，这种“骨架清楚”的类特别适合作为主学习对象。

9.6 SplinePlanner：它像一个把“安全区域”翻译成“平滑轨迹”的数学翻译官

很多新人一看到 `SplinePlanner` 会本能地害怕，因为这里会出现 `cvxpy`、`clarabel`、矩阵块拼接、等式约束和不等式约束。其实你先别急着怕。更好的理解是：前面多面体阶段给你的，是“每一段轨迹大概必须待在什么安全区域里”；`SplinePlanner` 做的，是把这些区域约束和首尾点约束组织起来，求出一条真正连续、平滑、满足连接条件的样条轨迹。

也就是说，它不是另起炉灶，而是在给前面几何阶段“翻译成成人能用的轨迹”。你一旦把这个角色感抓住，读 `spline_utils.py` 时心态会好很多。你不是在看一份无缘无故的优化作业，而是在看项目最后一公里如何被数学化。

10、把论文主张和代码落点——配对：以后你做汇报时就不会空讲

很多同学读论文时容易出现一种状态：内容大概懂了，可一旦要回到代码或做汇报，就会变空。嘴里都是论文里的漂亮词，但一说到“你本机上哪里体现了这件事”，就停住了。这一节的目标，就是帮你把几组最关键的论文主张和代码落点配对起来。以后你讲项目时，能做到“说一句论文主张，就能顺手指出代码在哪”。这会让你的表达质量立刻提升。

10.1 “GSplat 作为环境表示”对应哪里

这条主张最直接的代码落点，就是 `ns_utils/nerfstudio_utils.py` 和 `splat/splat_utils.py`。前者把 Nerfstudio 世界的模型和数据接口接进来，后者通过 `GSplatLoader` 把它们整理成当前规划和定位能直接使用的张量属性。你以后如果要用大白话解释，就可以说：论文里说用 Gaussian Splatting 表示环境，落到代码上就是先把 Nerfstudio 训练出来的 GSplat 模型通过 `config.yml` 读进来，再把高斯属性变成可供后续规划和定位使用的数据结构。

这个说法很重要，因为它把“环境表示”从抽象概念翻译成了实际输入链路。你不会再只会说“这是 3DGS”，而会知道当前项目到底是怎么用它的。

10.2 “安全走廊”对应哪里

这条主张最核心的落点，是 `SFC/corridor_utils.py` 和 `splatplan/splatplan.py` 里对多面体约束的生成流程。尤其是 `generate_path()` 和 `get_polytope_from_outputs()` 这一类函数。你如果要讲得更白一点，可以说：所谓安全走廊，不是把障碍物画出来那么简单，而是把沿路径段相关的障碍抽出来，逐段构造成一组半空间约束，从而得到机器人中心可以安全通过的多面体走廊。

一旦你能这样讲，听的人就会明显感觉你不是只背了个术语，而是真知道它在代码里怎么活着。

10.3 “Bezier 或样条轨迹” 对应哪里

这个落点主要在 `splatplan/spline_utils.py`。那里有时间基函数、导数矩阵、等式约束、不等式约束和二次目标项的构建，然后再用求解器算出轨迹系数。这里你不必执着于“论文里写 Bezier，我代码里看到的是 B-spline 命名，到底怎么说才完全精确”这种细枝末节。对入门阶段更重要的是抓住本质：前面几何阶段给出安全区域，后面样条优化阶段把安全区域转成一条真正平滑、可执行的轨迹。

你以后做汇报时完全可以这样讲，而且这比只说“这里用了优化器”要强得多。

10.4 “实时性” 对应哪里

这条主张最容易被讲空。因为一提实时性，很多人就只会复述论文或 README 里的数字。更好的做法，是把它拆成两层。算法层面，实时性对应规划与定位链路的求解组织方式，以及 CPU/GPU 负载分工；工程层面，当前工作区对实时性的落地则体现在 `wrapper` 把多轮实验、结果保存和可视化整理得足够顺，使你能快速重复调用和快速回看结果。

所以在当前阶段，你完全可以把“实时性”解释成：上游算法设计追求实时级导航，而本地 `wrapper` 负责把这套算法以低摩擦、可复盘的方式组织成可重复实验。这样讲，比空背一个“快”字要成熟得多。

10.5 “定位模块只靠 RGB 图像” 对应哪里

这条主张的主要落点，是 `run_splatloc.py` 配合 `pose_estimator/utils.py` 和 `ns_utils/nerfstudio_utils.py`。其中前者组织定位实验流程，后者提供视觉特征、PnP、配准和位姿表示相关工具，再结合 GSplat 场景和数据集图像完成定位任务。当前阶段你不一定要把这条链读透，但你至少应该知道，论文不是只说了一个定位名字，而是已经在上游代码里给出了完整入口和工具支撑。

这会让你以后讲项目全景时不至于只剩规划半边。

10.6 “对比 NeRF 导航和经典规划方法” 对应哪里

这条主张落在 `run_nerfnav.py` 和 `run_rrt.py`。它们提醒你：论文不是把自己的方法孤零零摆出来，而是和至少两条不同技术路线进行比较，一条偏 NeRF-based 导航，一条偏经典采样式规划。你在做全景介绍时，只要轻轻点出这两个入口，就已经足够让听众意识到这篇工作在研究比较上的完整度。

对学弟来说，这还会自然地培养一个很有价值的习惯：以后读论文时，不要只看自己方法的主图，也要留意它到底和谁比、为什么这么比、代码里有没有留下这些对比入口。

11、把输入资产和场景配置讲透：为什么一个 `config.yml` 背后站着一整棵目录树

学弟第一次接触这个项目时，常常会被一个现象困住：为什么命令里常常只传一个 `config.yml`，可大家又总在谈 `data/`、`outputs/`、`transforms.json`、`images/`、`sparse_pc.ply`、`dataparser_transforms.json` 这些东西。看起来好像有很多入口，实际又似乎都绕到同一个地方。这里最需要建立的，是“单个文件入口”和“背后整棵资产树”之间的关系意识。

11.1 config.yml 像门把手，不像整间屋子

如果你以后只能记住一句关于 `config.yml` 的比喻，我希望你记住这个：它像门把手，不像整间屋子。你在命令里传它，是因为脚本要通过它进入某个训练输出目录，并进一步找到模型、数据解析信息和相关场景上下文。也就是说，`config.yml` 自己不是全部资产，但它是打开资产世界最方便的手柄。

这也是为什么很多文档和脚本都把它当成最核心输入。因为从执行入口设计来看，传一个稳定的配置入口，远比让用户同时传十个文件路径要清晰得多。但你千万不要因此误以为“有了 `config` 就万事大吉”。它背后真正站着的，是一整套数据、模型和变换关系。

11.2 data/ 和 outputs/ 为什么要同时存在

上游 README 其实已经给过一个非常重要的暗示：训练数据在 `data/`，训练输出在 `outputs/`。这两个目录经常被新人混成一个“大场景目录”，但它们站在完全不同的阶段。`data/` 更像原料区，里面放图像、`transforms.json` 以及数据解析所需的原始内容；`outputs/` 更像生产结果区，里面放的是训练出来的模型及其配置，比如 `splatfacto/.../config.yml` 和 `nerfstudio_models`。

对当前项目来说，这个区分特别重要。因为规划和定位真正直接接触的往往是训练输出口，而不是原始图像文件本身。可训练输出之所以有意义，又离不开 `data/` 提供的数据来源和解析上下文。你一旦把这两者分清，后面看资产软链接和目录组织就不会糊。

11.3 transforms.json、dataparser_transforms.json、config.yml：三者为什么都重要，但职责不同

这三个名字很容易把人绕晕，尤其是对第一次接触 Nerfstudio 生态的同学。更顺的记法是：`transforms.json` 更接近数据解析输入，告诉系统图像与相机位姿等基础信息；`dataparser_transforms.json` 更接近训练过程中对数据坐标关系的进一步整理；`config.yml` 则是训练输出口，能把你带回整套训练产物和模型状态。

换句话说，它们不是互相替代的三个版本，而是处在链路不同位置的三个文件。你以后只要带着“一个偏原始数据解析，一个偏解析后的变换结果，一个偏训练产物入口”这个思路去看，就会顺很多。

11.4 images/、images2、images_2 这些名字为什么总让人紧张

这类名字的混乱感，几乎是所有新手都会有的。因为它们看起来只差一个数字或下划线，可背后可能对应完全不同的数据子集或场景版本。面对这种情况，最好的习惯不是靠猜，而是回到两个锚点：一是目录实际包含多少帧、哪些关键文件；二是当前 `config.yml` 到底指向哪一套资产。

所以当你看到 `images2` 相关脚本时，不要只把它当成“另一组图片”。更成熟的理解是：这往往代表作者或当前工作区在处理特定场景变体时的一套数据入口。你必须把它和具体配置路径绑定起来看，才能避免混乱。

11.5 sparse_pc.ply 为什么会反复出现在补数据脚本里

对新人来说，`sparse_pc.ply` 很容易被当成一个不太重要的辅助文件。但从当前工作区的数据补齐脚本设计来看，它恰恰是完整性判断的重要条件之一。这说明什么？说明这份文件在上游链路里并不是装饰，而是很多场景资产完整度的一部分。你如果把它漏掉，很可能会导致某些流程虽然表面还能跑，实际上上下文已经不完整。

所以你以后看数据准备脚本时，不妨换个角度：不是在看“作者怎么这么麻烦”，而是在看“项目真正依赖哪些最小资产集合”。这是非常有价值的工程敏感度。

12、再往前一步：一次规划从输入到输出，文件夹之间到底怎么传递信息

前面我们已经讲过数据流的大骨架，但这里要再落细一点。因为很多新人虽然能复述“`config.yml` 进来，结果落到 `runs/`”，可一旦被追问“中间哪一层起什么作用”，就又发虚了。现在我们把这件事说得更细一点，而且尽量贴近你日后真实会看到的文件。

12.1 输入层：命令行参数把哪几类信息带进系统

在当前工作区最典型的执行入口里，命令行参数至少会携带四类信息。第一类是场景入口信息，比如 `--config` 和 `--scene`。第二类是方法选择信息，比如 `--method splatplan` 或 `--method sfc-3`。第三类是实验规模信息，比如 `--trials`、`--seed`、`--keep_traj_limit`。第四类是输出落盘信息，比如 `--output` 或 `--output_dir`。

这四类信息加在一起，才构成一次真正完整的实验调用。你如果只盯着其中一类，比如只记场景路径，不记方法名和输出路径，理解就会偏。学弟越早形成这种“输入是分层的”意识，后面排错越稳。

12.2 配置层：scene preset 和 config.yml 是怎样互补的

上面说的四类输入里，最容易被误会的一对就是 scene preset 和 `config.yml`。它们不是重复输入，而是互补输入。`config.yml` 决定你进的是哪一个训练场景模型，scene preset 决定你围绕这个场景怎么摆实验：机器人半径多少、边界多大、体素分辨率多少、起终点怎样生成。

你可以把它们想成“模型入口”和“实验摆法”。少任何一边都不完整。只有把这两者拼在一起，当前规划链才真正具备可执行含义。

12.3 构造层：为什么 planner 初始化阶段已经做了很多重活

很多新人会把真正的“计算”只理解成 `generate_path()` 那一步。其实在 planner 初始化阶段，就已经做了相当多的重活。比如 `GSplatLoader` 会把场景模型读成张量属性；`SplatPlan` 初始化时会构建 `GSplatCollisionSet` 和 `GSplatVoxel`；`SafeFlightCorridor` 在不同 mode 下也会构建不同形式的 voxel 和碰撞集合；`SplinePlanner` 还会准备时间基函数和后续优化会用到的结构。

这意味着什么？意味着你以后看运行耗时，不能只盯着单次 `generate_path()`。有些准备工作在 planner 构造阶段就已经发生了。当前 batch 脚本之所以会单独记录 `load_time_sec`，就是因为它在明确区分“场景和 planner 构造成本”与“多次 trial 的纯规划成本”。这其实是很成熟的统计意识。

12.4 执行层：一次 generate_path() 到底交换了哪些对象

当 `generate_path(x, g)` 被调用时，系统手里至少已经有了这些东西：当前场景的高斯表示、机器人动力学近似参数、体素边界、离散网格工具、碰撞集合工具和样条优化器。然后它会以 `x` 和 `g` 为新的任务输入，产生一组路径段、多面体集合、轨迹点列、可行性标志和时间统计。换句话说，它不是单纯输入两个点、输出一条线，而是在调用一整套已经配置好的规划器系统。

这个理解很关键，因为它能让你以后在解释代码时避免一种常见错误：把 planner 当成一个轻飘飘的函数，而忽略它其实是带着大量初始化上下文运行的对象。

12.5 结果层：为什么当前工作区总在强调结构化输出

一次规划结果落盘后，当前工作区特别强调结构化输出。比如 `smoke` 会保存 `meta`、`device`、`plan_time_sec` 和 `output`；`batch` 会保存方法级统计和 trial 记录；可视化脚本再把这些结构化信息翻成图

表；聚合脚本则继续把多轮 batch 翻成 CSV 和总报告。看起来好像多绕了几层，但这恰恰是在把“程序输出”升级成“项目记忆”。

你以后做周报时会很感谢这种结构化。因为它让你不是在原始终端截图里找信息，而是在已经有层次的结果文件中查信息。

13、学弟第一次读源码，最容易在哪里迷路

一个文档如果只讲正确路线，不讲容易迷路的地方，往往是不够实用的。这里我想故意讲一讲，学弟第一次进源码最容易在哪些地方失去方向。不是为了吓人，而是为了让你提前有点免疫力。

13.1 迷路点一：把 wrapper 当上游，把上游当 wrapper

这是最常见的混淆。你明明在看 `scripts/run_official_style_batch.py` 这种 wrapper 文件，却以为自己已经进入论文算法核心；或者你明明在看 `splatplan/splatplan.py` 这种上游主逻辑，却又总是用“这个脚本会自动建 run 目录吗”之类的外层问题去要求它。出现这种错位，阅读就会非常别扭。

最好的补救方法很简单：每次打开文件前，先问自己一句，这个文件站在哪一层。只要层次先分出来，很多困惑会自然消失。

13.2 迷路点二：看到很多参数，就以为自己必须一次全懂

像 `SCENE_PRESETS` 这样的结构，第一次看确实会让人有点压力，因为里面同时出现半径、速度、边界、分辨率、均值偏移等参数。很多新人一看到就想：完了，我是不是得先把每个参数的物理意义和数值来源全搞懂，才能继续看。其实不用。

更合理的做法，是先把参数按角色分组：哪些是机器人参数，哪些是空间边界，哪些是起终点生成参数。只要角色先分清，数值细节可以后面再慢慢压。读工程代码最怕的不是暂时有些数值没背下来，而是因为怕自己没全懂，就不敢继续往前走。

13.3 迷路点三：一看见矩阵和优化就立刻失去耐心

这点尤其容易发生在 `spline_utils.py`。很多人前面都还看得挺顺，一到矩阵块、约束矩阵和求解器部分，立刻觉得太数学，整个人就抽离了。其实更稳的办法是，把这一层先看成“把前面的几何要求翻译成求解器语言”。只要这个角色先成立，后面每多看懂一块矩阵，都是在给这个翻译官加细节，而不是在硬吞陌生数学。

对数学稍微好一点、代码稍弱一点的同学来说，这里反而可能是你的优势区。你不一定一开始就写得出这些代码，但你是有机会把它背后的逻辑理解得很扎实的。

13.4 迷路点四：误把一次成功运行当成“我已经完全会了”

这其实是另一个方向的风险。很多同学第一次 smoke 成功以后，会短暂产生一种幻觉：好像自己已经把项目拿下了。实际上，一次成功运行只证明一条最小链路在当前条件下能通。它距离“真正理解项目结构、会排常见问题、会读 batch 结果、会讲清调用链”还有很长距离。

所以学弟如果第一次跑通了，不要过度膨胀，也不要反过来觉得“我还是不够会，干脆继续瞎翻资料”。更好的心态是：很好，至少链路通了。现在我终于有资格开始认真读它、问它、比较它了。

14、如果我是带学弟的老师，我会怎样安排他用一周把“术语附录 + 项目全景”啃下来

你说希望把第六编和第四编扩成能让学弟连续读一周的重型版本，这个目标非常好。但“能读一周”并不意味着“第一天就从头刷到尾”。更好的节奏，是让他每天带着一个清晰问题去读，而不是像背书一样硬压。这里我给出一种很适合零基础同学的阅读组织方式。

14.1 第一天：只做地图建立，不碰深层数学

第一天最重要的不是理解所有细节，而是建立“这到底是个什么项目”的轮廓。所以最适合的读法是：先把第一编重新过一遍，再看第四编前半部分，也就是论文总系统、三层结构、根目录和 wrapper 总览。读到这里，你的目标不是“我会复述所有脚本”，而是“我终于知道工作区、上游和结果层分别是谁”。

只要第一天能把这张图立住，后面所有深入都不会太虚。

14.2 第二天：把术语词典和目录结构互相对照

第二天很适合在第六编和第四编之间来回跳。比如你在第四编里看到 SSH、tmux、Conda、config.yml、GSplatLoader、SafeFlightCorridor，就回第六编查；查完再回第四编看这些词在项目里的位置。这样做比单独背词典舒服很多，因为术语会自动带着语境进入脑子。

对零基础学弟来说，这一步特别重要。因为它能让“词”和“路径”第一次绑在一起。

14.3 第三天：把 wrapper 主线啃下来，形成工程 workflow 感觉

第三天最合适的任务，是把第四编里关于 scripts/ 的部分认真读一遍。重点不是每个脚本都背出参数，而是知道它们在工程主线上的顺序：环境、数据、smoke、batch、overnight、汇总。到这一步为止，学弟应该开始形成一种感觉：原来当前工作区不是文件堆，而是一条可以顺着走的工程流水线。

这一步做成了，后面再碰任何脚本，都不太容易慌。

14.4 第四天：开始跟一条 smoke 调用链走到底

第四天最适合做的事，是把 run_splatplan_smoke.sh -> smoke_splatplan.py -> GSplatLoader -> planner -> generate_path 这条链路跟着读一遍。如果有精力，甚至可以边读边在终端里打开对应文件。此时的目标不是读懂所有张量运算，而是把调用关系压平。

只要学弟到第四天能口头讲清这条最小链路，他对项目的掌握就已经超过很多“只会翻文件”的状态了。

14.5 第五天到第七天：开始把上游主轴和论文主张配对起来

最后几天最合适的工作，是把第四编后半部分的上游目录讲解、核心类角色和论文主张映射认真过一遍。这个阶段不要贪图速度，而要有一种“终于开始把概念和代码焊在一起”的耐心。你会发现，到了这个阶段，第六编里的术语也会重新变得更有味道，因为它们不再是孤立名词，而成了你读代码时随时会用到的语言。

如果一周读下来，学弟还不能自己写算法，完全没关系。只要他已经能稳定回答“项目做什么、三层结构是什么、smoke 调用链怎么走、config.yml 和 scene preset 各干什么、SplatPlan 大概在哪几步完成规划”，这周就已经非常值了。

15、你以后给别人讲这个项目时，最值得保留的三种讲法

一个项目讲得清不清，不只体现在你自己有没有看懂，还体现在你能不能根据对象切换讲法。对这个项目来说，我特别建议你保留三种版本：一句话版、两分钟版和十分钟版。因为你以后一定会遇到不同场景：有人只是随口问，有人要听你做组会，有人可能要听你做完整汇报。

15.1 一句话版：给第一次听说这个项目的人

一句话版其实很简单：Splat-Nav 是把 Gaussian Splatting 场景表示进一步拿来和安全规划和视觉定位的导航系统；当前我们在本机主要复现的是它的离线/仿真规划链路，并用 wrapper 工程层把环境、运行和结果归档整理清楚。

这句话的好处在于，既讲了论文系统，又交代了当前工作区的现实边界。不会让人误以为你已经在做实车闭环，也不会把项目缩小成“只是几个脚本”。

15.2 两分钟版：给准备真正接手项目的学弟

两分钟版可以稍微展开一点：Splat-Nav 总体由 Splat-Plan 和 Splat-Loc 两部分组成。前者负责在 GSplat 场景里构造安全走廊并生成平滑轨迹，后者负责只根据 RGB 图像做位姿估计。当前工作区 `/home/howard/Splat-Nav` 不直接等于官方论文仓库，而是一个 wrapper 工作区。这里的 `scripts/` 负责环境、tmux、smoke、batch、overnight 和汇总；`splatnav-official/` 是官方源码入口；`runs/` 用来沉淀每次实验结果。第一阶段我们主要做离线/仿真，不碰实车，重点把 Splat-Plan 跑顺、读顺、讲顺。

如果一个学弟听完这段话还不觉得迷糊，说明项目交接已经成功了一半。

15.3 十分钟版：给组会、周报或阶段汇报

十分钟版就可以把论文结构、工作区三层结构、最小调用链和当前阶段进展一起讲出来。先说论文总系统，再说本机为什么要有 wrapper，接着说从 `config.yml` 和 `scene preset` 到 `GSplatLoader`、`SplatPlan.generate_path()`、`run` 目录落盘的完整链路，最后再说目前已经完成哪些 run、可行率大概怎样、接下来准备往哪里深化。这个版本是你未来最常用的主力版本。

而且你会发现，只要第四编真的读透了，这个十分钟版其实并不需要你背稿，它会很自然地长出来。

七、当前边界、典型场景与后续整理方向

最后这部分负责收边界、压重点，也顺带把几个关键场景和后续工程整理思路放进来。这样读到尾声时，读者不会只留下很多信息，而会留下一个比较稳的判断框架。

1、当前边界一定要再强调一遍：什么是现在该做的，什么不是

一份全景文档如果不反复强调边界，很容易把新人带到“什么都想碰”的状态。这里我想把当前阶段的边界再明确说一次，而且说得更像做项目而不是写概念文章。

第一，当前主任务是离线/仿真复现与工程整理，不是实车部署。第二，当前主重点是 Splat-Plan 链路，不是要求学弟第一周吃透 Splat-Loc。第三，wrapper 层可以适度重构、补注释、做大厂风格整理，但上游官方源码要保持边界意识。第四，任何涉及共享机器系统层的操作都要克制，尤其不能为了局部问题去动全局 CUDA、驱动或别人的账户环境。第五，结果要优先做成可汇报、可复盘、可交接，而不是只追求“我本地看起来跑了”。

这些边界不是在限制你，而是在保护推进节奏。尤其是带新人时，边界越清楚，项目越容易形成稳定预期。

2、这一部分读到最后，你脑子里最好留下哪几张图

一份好文档最后不应该只留下一堆文字，而应该留下几张很稳的图。第一张图，是论文系统图：Splat-Nav = Splat-Plan + Splat-Loc。第二张图，是当前工作区三层图：上游算法层、wrapper 工程层、文档与结果层。第三张图，是一次 smoke 的最小调用链：shell 包装、环境激活、上游导入、GSplat 加载、planner 构造、generate_path()、结果 JSON、run 目录落盘。第四张图，是上游源码树的角色图：splatplan/ 是主轴，splat/ 和 ns_utils/ 处理场景入口，polytopes/ 和 ellipsoids/ 处理几何，pose_estimator/ 处理定位，run_rrt.py 和 run_nerfnave.py 提供对比方法。

如果这四张图你已经开始能在脑中调出来，那第四编的任务就基本完成了。你再去读第三编时，会明显感觉轻松很多。因为现在你不再是在盲读一个项目，而是在沿着一张已经成形的地图走。

3、第一次真正读源码时，别平均用力：给学弟的一条顺序化读码路线

很多同学一听“要读源码”，脑子里会默认一个非常累的策略：从仓库第一层开始，一个文件不落地看下去。这种策略在课本上也许显得认真，在真实项目里却常常效率很低。因为真实工程从来不是平均展开的。它总有主轴、有侧翼、有暂时先知道名字就够的部分。所以真正高效的读码，不是平均用力，而是有意地先抓骨架，再补肌肉，最后才去看更细的血管和神经。

3.1 第一遍：只读“主轴文件”，目标是把链路拉直

第一遍读码，最适合看的其实只有少数几个文件。对当前项目来说，我最建议的顺序是：先看 scripts/run_splatplan_smoke.sh，再看 scripts/smoke_splatplan.py，然后看 splatnav-official/splat/splat_utils.py、splatnav-official/splatplan/splatplan.py、splatnav-official/splatplan/spline_utils.py。为什么是这几份？因为它们刚好构成最小规划主轴。

你要带着的目标也非常单纯：不是证明每一行都对，而是回答“命令从哪进、场景从哪读、planner 怎么构、generate_path() 里大概怎么走、最后结果从哪出”。这一遍只要能把主轴拉直，后面再读什么都不会太飘。

3.2 第二遍：补“支撑文件”，目标是知道主角背后站着谁

主轴拉直以后，第二遍就适合读支撑文件。也就是 SFC/corridor_utils.py、initialization/grid_utils.py、polytopes/collision_set.py、ns_utils/nerfstudio_utils.py 这类文件。它们不是你第一次最该盯住的主角，但它们在给主角输血。比如 grid_utils.py 解释了离散初始化从哪来，collision_set.py 解释了约束不是凭空造的，nerfstudio_utils.py 则说明 config.yml 背后到底连接着什么世界。

这一遍的目标也不是全懂，而是让你以后再回头看主轴时，知道主轴每一步到底借了哪些支撑模块。也就是说，第一遍像在看故事线，第二遍像在认识配角和幕后团队。

3.3 第三遍：再去看“侧翼入口”，目标是把项目全景彻底焊牢

到了第三遍，你就可以去读 run_splatloc.py、pose_estimator/utils.py、run_rrt.py、run_nerfnave.py、visualize.py 这些文件了。此时你已经不再是一个第一次进门的人，而是一个主轴比较稳的人。你再看这些

侧翼入口，会自然地把它放回整个项目全景里。你会知道哪些是定位链、哪些是对比基线、哪些是结果展示。

这时候阅读体验会轻松很多。因为你已经不需要靠它们来建立主线，而是在用它们丰富主线。

3.4 第一次看代码时，最不值得做的事情是什么

最不值得做的，是一上来就跟自己较劲，非要把每个类、每个矩阵、每个文件全吃透。这样做只会让阅读变成惩罚。更好的方法永远是：先问这份文件在大图里站哪一层，再问它回答了哪个问题，再决定这轮要不要细抠。工程源码和教科书不一样，它不是按你的认知顺序写出来的，所以你也没必要假装必须按文件自然顺序全懂。

学弟只要把这个观念立住，以后读任何项目都会顺很多。

3.5 一个很实用的读码笔记法：每个文件只先回答四个问题

很多新人读代码容易越看越散，是因为没有笔记抓手。我特别建议你前两遍读码时，每个文件只先回答四个问题。第一，这个文件在项目哪一层。第二，它主要回答什么问题。第三，它最关键的输入和输出是什么。第四，它和上一个或下一个文件怎么接。只要这四个问题写下来，你会发现很多之前“看着看着就丢了”的文件，突然变得有位置了。

这套方法看起来朴素，但非常适合零基础同学。因为它逼你先建立结构感，而不是一头扎进细节里出不来。

4、四个典型场景不只是名字：它们在当前工作区里各自意味着什么

很多人第一次看 `old_union`、`stonehenge`、`statues`、`flight` 时，会把它们简单理解成“论文用了几个场景”。这个理解当然不算错，但太浅了。对当前项目来说，这几个场景不是抽象标签，而是会直接影响配置路径、机器人半径、体素边界、起终点摆法、结果分布乃至后续你做汇报时的叙事重点。换句话说，它们不是背景板，而是实验个性的一部分。

4.1 `old_union`：为什么它在当前工作区里几乎成了默认主场景

从当前 `wrapper` 脚本、已有 `run` 目录和实际推进节奏看，`old_union` 在工作区里几乎扮演了“默认主场景”的角色。它的好处很现实：路径和资产相对清楚，已有结果较多，相关脚本默认值也常围绕它展开。所以对新人来说，这个场景非常适合做第一阶段主入口。你不是因为它“理论上最重要”才从它开始，而是因为当前机器和当前工程整理节奏下，它最适合作为稳定起点。

你以后带学弟时，也完全可以很坦然地这么说：先别追求平均地把四个场景都摸一遍，先把 `old_union` 跑顺、讲顺，它会给你最多的正反馈。

4.2 `old_union` 的参数为什么值得被认真看一次

在 `scene preset` 里，`old_union` 的 `radius_config`、`mean_config`、`lower_bound`、`upper_bound` 和 `resolution` 并不是随手写的杂项，它们共同定义了这个场景里实验是怎样被“摆出来”的。起终点大致在什么区域对穿、机器人半径按什么尺度估、体素网格覆盖哪块空间，都是这些参数在共同决定。

新人最容易忽视的一点是：这些值虽然看起来只是数字，但它们实际上代表着作者对场景结构和实验尺度的理解。你不需要第一天就推敲它们为何恰好取这个数，但你至少要知道，它们是实验设置的一部分，不是无关紧要的填空。

4.3 stonehenge：为什么它特别适合训练“别把场景名当空标签”的能力

`stonehenge` 这个场景很适合拿来帮助新人建立一个意识：不同场景并不是只换了个名字，很多实验设定会跟着变。你在 `preset` 里能看到它的 `radius_config`、边界和分辨率都和 `old_union` 不一样。这意味着什么？意味着规划器面对的搜索空间、路径形态和几何尺度都可能不同。

对学弟来说，读这种差异不是为了背参数，而是为了真正学会“场景个性化”。以后只要换了场景，你就不再下意识地以为所有设置都应该照抄。

4.4 statues：为什么它很适合提醒你“机器人尺度和场景尺度是耦合的”

在 `statues` 的 `preset` 里，你会明显看到机器人半径、边界和均值配置有自己的特点。这特别适合帮助新人理解一个重要观念：机器人尺度不是脱离场景独立存在的。你不能抽象地说“机器人半径是 0.01 或 0.03”，而不去问这是在哪个场景、以什么坐标尺度、围绕什么空间范围说的。

这种尺度感非常关键。因为很多代码读起来只是数字，一旦你忘了它们和场景的耦合关系，就会觉得所有参数都像魔法常数。

4.5 flight：为什么它会自然地让你想到“未来更接近真实飞行的味道”

虽然当前阶段不碰实车，但 `flight` 这个场景会天然更容易让人联想到飞行器导航、空间更开阔的运动和更接近动态应用的语境。它在边界设置和 `radius_z` 这些配置上，也能明显让你感受到和其他场景不完全相同的空间特征。

这并不意味着学弟必须一上来就优先跑它。更稳的理解是：当你在 `old_union` 上把流程压稳以后，切到 `flight`，你会明显感觉自己开始接触更丰富的场景风味。它很适合作为后续扩展时的第二梯队目标。

4.6 四个场景一起看时，你应该得到什么能力

如果你把这四个场景一起看，最该得到的能力不是背住四套参数，而是建立一种自然直觉：场景名背后站着一整组实验设定，场景切换意味着路径、边界、尺度和可能的结果分布都会一起变化。只要这个直觉长出来，你以后就不会再把 `scene` 当纯标签，也更容易理解为什么 `wrapper` 脚本和上游入口都如此重视 `scene_name`。

5、如果今天只给你一个下午，我希望你优先打开哪些文件

这一节我想写得非常实用一点。因为真实世界里，学弟不一定总有整块时间。他有时就只有一个下午，或者一晚上，想先把项目摸出点感觉。这种情况下，最怕的不是时间少，而是打开顺序错了。顺序一错，两个小时就会全花在迷路上。

5.1 第一份：本书开头的第一编

原因很简单，它能先给你定调。你至少会知道当前主线怎么分工，自己现在更适合看术语、看全景、看输入输出还是看快速上手。很多人一上来就跳进最厚的部分，结果看着看着不知道自己为什么要看。先把第一编读稳，就是在避免这个问题。

5.2 第二份：第四编本身

如果你只有一个下午，我会非常建议把大头时间留给第四编。因为它最能帮你解决“项目到底是什么”这个根问题。项目一旦有了地图，后面的术语和脚本才有位置。没有地图，任何局部解释都容易飘。

5.3 第三份： `scripts/run_splatplan_smoke.sh`

看完地图以后，不要急着看最数学的文件。先看这份最外层 `smoke` 入口，会让你非常踏实。因为你会第一次把“当前工作区怎么启动一次最小规划任务”看成一条现实链路，而不是抽象概念。

5.4 第四份： `scripts/smoke_splatplan.py`

这是最适合把“工程外层”和“上游内核”接起来的一份文件。你只要把它看懂，项目主轴就已经开始连起来了。尤其是 `scene preset`、`GSplatLoader`、`planner` 构造和输出 JSON 这几块，特别值得慢慢看。

5.5 第五份： `splatnav-official/splatplan/splatplan.py`

当你已经从 `wrapper` 层进入 Python 入口，再来读 `splatplan.py`，感受会很好。因为你不再是在无上下文地看一个上游算法文件，而是在接着前面的调用链往里走。这样它看起来会友好很多。

5.6 第六份： `splatnav-official/splat/splat_utils.py`

很多新人容易把它放在后面，其实它应该更靠前。因为一旦你把 `GSplatLoader` 看懂，项目的“场景到底怎么进系统”这件事就真正落地了。你会少掉很多飘着的不安感。

5.7 第七份： `splatnav-official/splatplan/spline_utils.py`

如果下午还有精力，再来碰它最合适。因为这时你已经知道自己为什么要看它。你不是在突然面对一份数学文件，而是在看主链最后那位“把安全区域翻译成轨迹”的求解器角色。

6、从 `wrapper` 到 `upstream` 的文件映射关系，最好在脑子里形成一张对照表

很多人项目做到后面会有一种奇怪的割裂感：`wrapper` 我也看了，上游我也看了，可两边始终像两套语言。更好的状态应该是，你脑子里逐渐形成一个自然对照表。也就是说，一看到某个 `wrapper` 入口，你就知道大概率会落到哪类上游文件；一看到某个上游主角，你也知道平时是通过哪些 `wrapper` 脚本接触到它。

6.1 `run_splatplan_smoke.sh` 对应 `smoke_splatplan.py`，而后者继续对应 `GSplatLoader + planner`

这是最基础、也最值得背熟的一条映射。你以后只要听到有人说“先 `smoke` 一下”，脑子里就应该自动想到：外层 `shell` 启动器先负责 `run` 目录和环境，再进入 Python `smoke` 入口，而 Python 入口又会把上游的 `GSplatLoader`、`SplatPlan` 或 `SafeFlightCorridor` 拉进来。

这种自动联想特别值钱，因为它说明你已经不是在看单个文件，而是在看文件之间的分工关系。

6.2 `run_official_style_batch.py` 对应的是“同一套 `planner` 被系统化重复调用”

很多新人会把 `batch` 文件误当成另一种算法实现。其实更成熟的理解是：它主要是在调用层面把相同 `planner` 放到多 `trial`、多方法、多统计环境里运行。所以你一看到它，就应该想到下面还是那些核心 `planner` 类，只不过外面多包了一层实验组织壳。

一旦这个映射关系稳了，你读 `batch` 文件时就不会误以为自己掉进了另一套世界。

6.3 `visualize_batch_results.py` 和 `aggregate_batch_runs.py` 对应的是“结果消费层”，不是“算法层”

这也是很值得压稳的一组映射。很多新人一看到 Python 文件就自然以为“这也是核心计算代码”。其实不是。它们主要服务的是结果消费：前者把单次 `batch` 的 JSON 翻成图和摘要，后者把多轮 `batch` 汇成总报告。你以后只要想到它们，就应该知道自己已经走到了实验后处理层。

这种层次感对做汇报尤其重要。因为你会更自然地知道某个图到底来自哪里，是算法直接产出的，还是后处理脚本进一步整理出来的。

6.4 sync_upstream.sh 对应的不是某个算法文件，而是版本感知本身

并不是每个 wrapper 都会落到一个具体上游算法文件。有些 wrapper 在回答的是“我们和上游现在是什么关系”。sync_upstream.sh 就属于这一类。它对应的是版本和提交状态，而不是某个数学模块。这种映射也值得学弟有感觉。因为它提醒你，工程里不是只有算法映射，还有版本映射、结果映射、环境映射。

6.5 真正成熟以后，这张对照表会变成你的本能

当你在项目里待上一段时间以后，这种对照关系最好会慢慢变成本能。你不再需要刻意背“这个 shell 对应哪个 Python，这个 Python 又对哪个 planner”，而是自然能顺着走。那时候你就会发现，自己已经从“文件浏览者”变成了“结构理解者”。这是一个很明显、也很值得开心的成长节点。

7、如果导师或师兄现在当面追问你几个问题，你应该怎样答才像真的理解了项目

很多同学自己看资料时，总觉得好像懂了一些；可一旦有人当面一问，就发现脑子还是散的。这很正常。项目理解真正成形，往往就是从“我自己觉得看过了”变成“别人一问，我也能稳稳讲出来”。所以这一节我故意按导师追问的口吻来写。不是为了制造压力，而是为了帮你提前练一种能力：把项目的地图、链路和边界讲成别人能听懂的话。

7.1 问：这个项目到底是在做三维重建，还是在做导航

一个很成熟、又不容易把自己说绕的话法是：这个项目的核心不是重新发明三维重建，而是站在 Gaussian Splatting 这种已经建立好的三维场景表示之上，进一步做安全规划和视觉定位，也就是导航系统里的两大核心问题。换句话说，三维重建在这里更像基础设施，而导航才是它真正要完成的系统目标。

你一旦这样回答，听的人就会感觉你已经知道“场景表示”和“导航任务”在项目里的上下位关系。你不会再把它讲成“这是一个 3DGS 项目顺便能导航”，也不会讲成“这是个传统导航项目碰巧用了点 3DGS”。而是会更准确地说：它是在新型场景表示上重做导航这件事。

7.2 问：为什么当前工作区里还有这么多 wrapper 脚本，不能直接跑官方仓库吗

这个问题问得非常好，而且很常见。更成熟的回答不是说“可以直接跑啊”，也不是说“wrapper 比官方好”，而是应该分层回答。官方仓库更像论文作者自己的实现与实验入口，保留它原样非常重要，因为它是事实来源。wrapper 工作区存在的意义，是在不破坏上游实现的前提下，把当前这台共享机器上的环境边界、数据落盘、tmux 长任务、run 目录、汇总图表和交接文档整理出来，让复现和后续带人更稳。

也就是说，wrapper 不是为了取代上游，而是为了让上游更容易在当前现实环境中被重复、被管理、被解释。你如果能这样回答，别人会很容易相信你知道为什么工程化外壳是必要的。

7.3 问：那你们现在到底复现到了什么程度

这时候最怕的是只回答“能跑了”。更好的回答应该是：当前离线/仿真主链已经围绕 Splat-Plan 建立起了稳定执行路径，能在作者场景上做 smoke、batch 和 overnight 形式的运行，并把结果按 runs 规范沉淀成 JSON、图表和汇总报告。与此同时，工作区结构、文档体系和 wrapper 层已经能支持后来者按更短路径入门。至于实车、ROS 节点部署和 Splat-Loc 深入链路，目前被明确放在第一阶段之外。

这样的回答之所以好，是因为它既不虚夸，也不把自己说得太弱。它能准确传达“我们已经把离线/仿真链路和工程组织做出了成果，但边界依然清楚”。

7.4 问：为什么你们现在不直接碰 ROS 和实车

很多新人一听这个问题，容易心虚，好像不碰实车就不算真的做项目。其实更成熟的答法是：当前阶段的目标不是追求表面上覆盖所有模块，而是优先把离线/仿真复现链和项目认知链打牢。因为只要上游场景加载、规划主链、结果验收、文档交接和工程边界都还没稳住，就直接去碰 ROS 和实车，只会把问题层次叠得更高，推进反而会变慢。

你完全可以非常坦然地说：现在先把离线链路做成一个后来者也能跑、也能讲、也能继续改的系统，这是对项目最负责的推进顺序。等这一层稳定后，再切到 ROS/实车会更合理，也更节省时间。

7.5 问：那学弟到底应该先看什么，为什么不是先看论文公式

这个问题其实正好能检验你是不是真的理解“项目入门”和“学术阅读”之间的关系。更成熟的回答是：对零代码基础、机械转来的学弟，先要建立的是工程地图和术语地图，而不是一上来硬啃公式。因为如果他连工作区三层结构都没分清、`config.yml` 和 `scene preset` 都还在混、`wrapper` 和上游还没分层，那么先去看公式只会把压力放大，并不会更快进项目。

所以更合理的顺序是：先看主线文档，形成路径感；再跟 `smoke` 主链读代码；随后再回到论文，把论文主张和代码落点焊在一起。这样公式才不会飘在空中。

8、如果要把这个项目进一步整理成更像大厂工程的样子，哪些地方先动，哪些地方不要乱动

你前面已经明确说过，希望这个项目在复现的同时，也能逐步整理成更符合大厂规范的样子。这件事完全可以做，但它特别需要顺序感。因为“工程化重构”这四个字非常容易让人兴奋，一兴奋就什么都想动。真正成熟的做法不是全仓开刀，而是先判断哪些地方动了能立刻降低认知成本、提高稳定性，哪些地方现在不应该碰，至少不应该重手碰。

8.1 第一优先级：文档入口、运行规范和结果结构

最值得先动的，其实不是最底层数学代码，而是认知与运行成本最高、但改动风险相对可控的外层。比如我们现在正在做的 `docs/Codex` 主线文档重建，比如 `runs` 目录命名规范，比如 `create_run_dir.sh`、`run_in_tmux.sh` 这种直接影响长期可追踪性的工程入口。这些地方一旦清楚，后来者不需要先懂所有算法，也能先有稳定工作流。

从大厂工程视角看，这一层非常像“把入口体验和可维护性先拉正”。它未必最炫，但收益通常非常大。

8.2 第二优先级：wrapper 脚本之间的职责边界和日志质量

再往下一层，非常值得整理的是 `wrapper` 脚本本身的职责边界。哪些脚本只负责环境，哪些只负责 `run` 目录，哪些只负责单次执行，哪些只负责汇总，哪些只负责可视化，这种边界最好尽量清晰。同时，日志内容也值得系统性整理。因为长期看，工程脚本最容易变乱的不是功能本身，而是边界混起来以后，谁都做一点，最后谁都说不清。

这种整理特别适合练工程感。因为它逼你思考“一个模块到底该负责什么”，而这正是大厂代码能力里很核心的一部分。

8.3 第三优先级：对上游核心类做“解释性增强”，不是先做算法性改写

很多新人一说工程化，就想先去改 `splatplan.py` 或 `spline_utils.py` 的内部逻辑。对当前阶段我并不建议优先这么做。更稳的方式，是先围绕上游核心类做解释性增强，比如在外层文档和注释里把输入输出、调用位置、结果结构讲清楚；再比如在 `wrapper` 层把参数命名和结果保存方式整理清楚。也就是说，先提升“别人读得懂、用得对”的程度，再决定是否真的要动算法实现。

这不是保守，而是成熟。因为算法核心一旦动，验证和责任链都会变重。先把解释层做厚，是更稳的推进顺序。

8.4 当前不建议重手动的地方：上游数学主链和论文对比基线内部实现

为什么不建议现在重手动？原因很简单。第一，上游核心实现是论文事实来源，动得太早会让“这是作者实现还是你们改过的实现”越来越难分清。第二，学弟当前最缺的不是“新的算法版本”，而是“能稳稳读懂并复现现有链路”的能力。第三，一旦重写或大改核心实现，验证负担会立刻变大，可能反而拖慢整个项目节奏。

所以当前最成熟的姿态，其实是：上游保持尊重和边界，外层做工程组织和认知降噪，局部小修小补以解释性和稳定性为主。

8.5 如果未来真的要重构上游，最成熟的前提是什么

不是“我觉得这里写得乱”，也不是“我想练练代码”，而是你已经对这条链有足够强的理解，能说清原实现做了什么、为什么这样写、你的改动不改变哪些行为、又希望改进哪些可维护性问题，并且你有足够明确的回归验证方式。换句话说，真正成熟的重构不是情绪驱动，而是理解驱动和验证驱动。

这也是为什么我一直建议现在先把 第六编和第四编压厚。没有理解，就没有好重构。重构不是为了看起来忙，而是为了让系统更稳、更清楚。

9、如果你第二天早上只看 `runs/`，也应该能大致反推出项目正在以什么方式工作

一个工程好不好，不只是看代码和脚本层，还可以从结果目录反推出来。当前项目尤其如此。因为 `runs/` 不是简单输出箱，而是整个工作区行为方式在文件系统上的投影。你如果真的理解这套工作区，哪怕第二天早上不先看代码，只看 `runs/` 里的目录命名、文件结构和产物类型，也应该能猜出昨晚大概干了什么。

9.1 时间戳目录为什么这么重要

当你看到 `YYYY-MM-DD_HHMM_topic` 这种目录时，不要把它只当成“方便排序”。它其实在告诉你：实验任务被显式当成一个个时间事件在管理。也就是说，每一次运行都不仅有主题，还有发生时刻。这个设计对后来者非常友好，因为它天然支持时间回放。你以后想回看哪一夜跑了什么、哪一次改动前后结果有什么差异，就会很依赖这个时间感。

工程上很多“说不清”问题，其实最终都是因为时间感缺失。`run` 目录把这件事补上了。

9.2 `env.txt`、`cmd.txt`、`stdout.log` 三件套，为什么像一次运行的身份证

如果你在一个 `run` 目录里看到这三样东西齐全，基本就能判断这次运行是按当前工程规范组织的。`env.txt` 像身份证背景面，告诉你这次任务运行时的用户、主机、GPU、Python 和上游提交信息。`cmd.txt` 像动作记录，告诉你它到底跑的是什么命令。`stdout.log` 则像过程录像，告诉你运行时说了什么、卡在哪、停在哪。

这三样东西之所以值钱，是因为它们共同让一次运行从“曾经发生过”升级成了“可以被追踪和解释”。

9.3 为什么图、JSON、Markdown 会一起出现

很多新人第一次看到一个 `run` 目录里同时有 JSON、PNG、Markdown，会下意识觉得是不是有点冗余。其实这是很成熟的结果分层。JSON 更适合程序继续处理和精细追踪；图更适合快速肉眼判断；Markdown 摘要更适合你第二天快速扫一眼就知道大概情况。这三者各自服务不同消费方式，不是重复，而是互补。

只要你有了一层理解，就不会再把结果目录看成“文件好多”。你会开始把它看成“结果被做成了多种阅读层次”。

9.4 从 run 目录反推执行类型：你能看出这是一条 smoke 还是一轮 batch 吗

这件事其实非常适合学弟训练自己。比如目录 `topic` 里如果带 `smoke`，结果文件里更可能只有单次 JSON 和最基础日志；如果 `topic` 带 `formal_batch`，则很可能出现 `batch_results.json` 和一个图目录；如果是 `overnight_report`，那你多半能看到 CSV、Markdown 汇总和跨轮趋势图。也就是说，文件结构本身就在泄露任务类型。

你以后越能从结果目录反推出执行类型，越说明你对整个工程链是有结构感的。

9.5 一个真正成熟的项目，结果目录本身就应该会“说话”

这句话非常值得记住。真正成熟的项目，不应该只有代码作者自己知道每个目录是什么意思。结果目录本身也应该能“说话”。当前工作区之所以强调 `topic`、时间戳、日志、摘要和图表，其实就是在让结果自己具备最小可读性。这样第二天来的人，不必把昨晚的人叫醒，也能迅速接手现场。

这就是工程组织能力最迷人的地方之一：它能让协作变轻，而不是变重。

10、为什么当前阶段明确不碰 ROS、语义分支和实车，不是因为它们不重要，而是因为排序更重要

在很多实验室环境里，新人最容易被一种氛围影响：仿佛越快碰到真机、越快碰到 ROS、越快碰到更复杂分支，就越显得自己在做“真的项目”。这种心理很能理解，但从推进效率看，常常不划算。当前阶段明确不碰这些东西，不是低估它们，而是在做更稳的排序。

10.1 ros 分支存在，说明它重要；当前不先碰，说明我们在做节奏管理

上游 README 明确提到 ROS 节点在 `ros` 分支。这件事本身就说明作者认真考虑过系统部署和机器人运行。但当前工作区主线没有让学弟先去碰它，理由非常实际：只要离线链路和工程入口还没稳住，就直接切 ROS，只会叠加更多系统层、依赖层和运行时复杂度。

所以更成熟的表达方式应该是：我们知道 ROS 分支的重要性，也尊重它在总系统里的位置；但对当前阶段目标来说，先不碰它，是在保护主线推进。

10.2 语义分支和语义目标查询为什么也不应抢第一阶段风头

同样地，上游 README 还提到了 `semantics` 分支和语义目标查询能力。这听起来很酷，也很容易激发兴趣。但如果学弟现在连 `config.yml` 背后资产树、`SplatPlan` 主链和 `run` 目录规范都还没完全压稳，就直接去看语义分支，只会把认知层次再往上摞一层。那不是积极，而是过载。

先把基础链路做成可解释、可复现、可汇报的状态，再去碰语义增强，会自然很多。

10.3 实车为什么尤其不能靠热情先冲

实车和离线/仿真最大的差别，不只是“真东西更难”，而是它会把安全、实时、部署、感知误差、系统耦合和硬件资源这些问题一次性压过来。你在离线阶段没稳住的那些小问题，到了实车阶段往往都会被放大。所以真正对项目负责的顺序，绝不是“赶紧上实车看看”，而是“先让离线链路足够稳，再决定要不要让更多复杂性进来”。

这点尤其适合讲给学弟听。因为它能帮他建立一种很重要的职业素质：真正稳的人，不是最敢冲的人，而是知道什么时候该先把地基打厚的人。

11、如果未来想从这个项目里长出创新点，第一反应最好不是“发明一个全新名词”，而是观察哪里最痛

很多学生一听到“最终目标是投会议论文”，马上就会被创新焦虑压住，好像必须立刻想出一个听起来全新的方法名字。其实对当前阶段来说，更成熟的创新起点常常不是命名，而是观察。也就是说，你先把现有链路跑顺、读顺、讲顺，再认真看哪里最痛、哪里最不稳定、哪里最费人、哪里最容易被误解、哪里对后来者最不友好。这些地方里，往往就藏着未来的改进点。

11.1 工程层痛点，也可能长成研究问题

很多新人会把工程和研究完全切开，觉得工程问题只能靠整理，研究问题才叫创新。其实不是。某些工程痛点如果背后反复对应同一种系统瓶颈，就很可能是研究问题的入口。比如某类场景切换时参数耦合太强、某类 baseline 可行率总是掉、某种初始化方式对某些场景特别脆、结果解释层成本太高，这些都可能往方法改进或系统论文写法上长。

所以你别轻易低估工程观察。很多很好的创新，就是从“为什么这里总这么痛”开始的。

11.2 方法层痛点，通常藏在“你已经能稳定复现”之后

如果链路还没稳定，方法层痛点很容易和环境问题、数据问题混在一起，根本看不清。只有当你已经比较稳定地知道什么是正常运行、什么是结构化结果、什么是典型场景表现，你才能开始认真分辨：这到底是系统边界导致的，还是方法本身的弱点。也就是说，真正高质量的方法观察，通常都发生在复现稳定之后。

这也是为什么现在先把 第六编、第四编、第三编、第二编 这类材料做厚，是非常值的。它们不是和创新无关，而是在给未来创新铺路。

11.3 创新点的最早形态，往往不是结论，而是一串好问题

很多人误以为创新点应该直接以答案出现。其实在训练阶段，更真实的情况常常是：你先长出一串质量很高的问题。比如“为什么某类场景对某种安全缓冲特别敏感”“为什么 wrapper 层统计出的趋势和某些单次可视化不完全一致”“为什么 GSplat 场景在某种离散边界设置下更容易出现初始化问题”。这些问题如果被追得够深，就很可能长出方法、实验设计或系统改进。

所以学弟如果暂时还提不出完整创新方案，完全不用焦虑。先长出好问题，已经非常有价值。

12、最后再送你一个更像老师带学生的建议：别急着把“全懂”当目标，先把“能稳定往前走”当目标

写到这里，其实我最想说的一句心里话是：很多项目不是被难度击败的，而是被不必要的完美主义拖住的。尤其是零基础学弟，很容易一上来就给自己设一个很重的目标，仿佛必须先把所有术语、所有脚本、所有数学和所有上游文件都搞懂，才配开始动手。这个标准实在太高，也不符合真实项目学习规律。

更成熟的目标应该是：我今天比昨天更不迷路一点，我已经能说清一个新概念，我已经把一条新链路看顺，我已经知道一个以前混淆的词其实该怎么区分，我已经知道明天该接着看哪里。只要这种“稳定往前走”的节奏形成，你后面会发现自己进步得非常快，而且不那么痛苦。

这也是为什么我希望 第六编和第四编不是写成冰冷资料，而是写成一种能陪你走路的东西。因为真正能把人带进项目的，从来不是知识点数量本身，而是知识点之间有没有路感、有没有层次、有没有一口气能继续往下看的动力。

13、从这里往下，你该自然切到哪几份文档

如果你把 第六编和第四编真读到这里，最自然的下一步不该是继续盲翻别的资料，而是顺着当前主线往下走。首先应该去读 第三编，因为现在你已经有了项目地图，正适合把“一次运行”拆开看清。接着应该去读第二编，因为你已经不再是完全陌生的人了，开始可以按任务顺序真正动手。最后，再去看 第五编，它会帮你把现在的理解往未来节奏里接。

如果你愿意，我甚至希望你把这整个阅读过程想成一条很正常的训练路线，而不是一次考试。第六编帮你认词，第四编帮你认图，第三编帮你认一次运行，第二编帮你认顺序，第五编帮你认长期节奏。到了这里，项目其实已经不再是那种“文件很多、看着吓人、我不知道从哪下手”的状态了。它开始变成一张真的可以走的地图。只要这件事发生了，这一轮文档重建就已经非常值了。

14、如果把整个项目讲成一个完整故事：从“我要从 A 点去 B 点”到“第二天我拿到报告”

到这里，我们已经从很多角度拆过项目：从论文、从目录、从脚本、从类、从数据流、从工程边界、从阅读顺序、从场景差异。现在我想把这些碎片重新收回来，讲成一个完整故事。因为对新人来说，真正让理解稳定下来的，往往不是知道了更多知识点，而是突然能把这些知识点串成一条活的叙事线。一旦叙事线出来，项目就会从“很多局部”变成“一个系统”。

14.1 故事的起点，不是脚本，而是一个很朴素的导航需求

想象一下，我们并不先从代码开始，而是先从任务开始。一个机器人现在处在某个三维场景里，它需要从当前位置去目标位置，还希望整个过程尽量安全、轨迹平滑、在复杂环境表示下也能实时运行。你如果从这个角度出发，会发现 Splat-Nav 论文其实是在回答一个很朴素但很难的问题：当环境不再只是二维栅格或传统点云，而是 Gaussian Splatting 这种更丰富的三维表示时，导航系统该怎样重新设计。

一旦你从任务出发，项目很多层次就会自然对位。场景表示层负责把“环境是什么样”这件事讲清楚，规划层负责把“从哪走到哪”这件事做出来，定位层负责把“我现在到底在哪”这件事实时估出来，工程层则负责让整套链路在现实机器上能稳定运行、能留下结果、能被后来者接手。

14.2 故事的第二步，是先得有一个可供导航系统使用的场景

这个场景在当前项目里不是凭空生成的，它通常来自 Nerfstudio 训练出来的 GSplat 模型。也就是说，在导航之前，世界已经先被某种方式表示出来了。这个表示不是一张照片，也不是一个简单 occupancy grid，

而是一套能够保留高斯均值、尺度、旋转、颜色和不透明度等信息的三维场景模型。上游 README 已经很明确地告诉你，当前仓库建立在 Nerfstudio 之上，而 `config.yml` 正是把你带回这个训练场景入口的关键门把手。

所以从故事角度看，`config.yml` 并不是某个不起眼的小配置，而像是“我要导航的这个世界”的准入证。没有它，后面的规划和定位就没有具体对象可言。

14.3 场景进入系统之后，当前项目会先把它翻译成不同模块都听得懂的话

一旦场景入口确定，`GSplatLoader` 这类模块就开始发挥作用。它会把训练产物整理成后续规划和几何模块都能直接使用的属性。这一步特别像一个翻译过程。因为原始训练产物属于 Nerfstudio 语境，而规划和定位模块需要的是一种更接近“我能拿来算”的内部表示。于是高斯的均值、协方差、颜色、不透明度等就被抽出来，放进统一对象里。

你以后只要把这一步想成“场景语言转换”，很多细节都会顺。比如你就不会再问为什么 `config.yml` 既像模型入口，又好像跟场景几何有关。因为它确实是在把训练世界和导航世界接到一起。

14.4 对规划来说，真正的难点不是一句“从起点到终点”，而是安全和可执行性要一起成立

如果我们只要求“给我一条连起来的路径”，那问题会简单很多。可导航项目真正难的地方在于，你不能只找一条视觉上连通的线，而要同时考虑碰撞、机器人尺度、轨迹平滑、连续性以及后续执行的合理性。这正是为什么论文里会强调安全走廊、多面体、样条和实时性这些词。因为系统不是在玩几何拼图，而是在为真实导航过程准备一条可靠路径。

对当前项目来说，这层难点主要落在 `Splat-Plan` 链上。它先通过网格初始化找到一条粗路径，再围绕这条粗路径逐段建立安全约束，最后才让样条规划器给出一条更平滑、更可执行的轨迹。你越把这件事想成“逐层收紧问题”，代码就越容易读懂。

14.5 初始化路径为什么像故事里的“草稿线”

很多新人一开始读 `A*` 这部分，会误以为那就是最后答案。其实更好的理解是：初始化路径更像草稿线。它先在离散体素空间里给你一条大致可行的路线，让系统知道应该从哪块区域穿过、哪块区域先避开。它的价值在于提供结构，不在于直接当最终轨迹。

这就像你画图时先打草稿一样。草稿不是最终画面，但没有草稿，你后面的细化就很容易失控。把这一层想明白，`GSplatVoxel` 和 `astar3D` 这些名字就不再只是一串工具，而开始像故事中的“起笔阶段”。

14.6 碰撞集合和多面体阶段，像是在给草稿线外面加一层又一层安全缓冲

有了草稿线以后，系统还不能宣布完成。因为草稿线只说明“有可能这么走”，并没有把安全性真正写进轨迹本身。于是接下来的碰撞集合和多面体阶段就登场了。它会沿着路径段去看，附近有哪些高斯椭球或点云近似对象是相关障碍，哪些半空间约束可以把安全区域刻画出来，哪些段已经被之前的多面体包住、哪些还需要新建约束。

这一步很像给一条细线逐段套上安全走廊。你走得不再是抽象线条，而是一个在几何上被明确界定出来的可通过区域。只有到这里，论文里那种“safe-by-construction”的味道才真正进入代码。

14.7 样条优化阶段不是另开新坑，而是在给故事收尾

很多新人一进入 `spline_utils.py` 就觉得故事突然变了，从几何切成了数学。其实更好的理解是：这不是新故事，而是老故事的收尾阶段。前面你已经知道大概要走哪、哪些区域安全、首尾点在哪，剩下的问题其实是：怎样把这些要求整成一条平滑、连续、看起来像真能执行的轨迹。SplinePlanner 做的正是这个工作。

一旦你把它当成“给前面所有几何约束收尾”的角色，心理负担会小很多。你不再是突然被扔进优化作业，而是在看故事最后一公里怎样被严谨地落成轨迹。

14.8 一次规划结束以后，工程故事才真正开始有“可交接性”

很多学生项目的问题就在这里：算法跑完就结束了，结果留在终端里，第二天谁也不知道昨天到底做了什么。当前工作区刻意避免了这种情况。通过 wrapper 和 run 目录规范，一次规划不仅结束在内存里，还会被写成 JSON、摘要、图表和日志，进入一个带时间戳的 run 目录。这个动作的价值不是“形式主义记录”，而是让工作具备可交接性。

你以后带人时会非常深刻地感受到这一点。真正让团队能往前走的，不只是“有人做出了结果”，而是“结果被整理成别人第二天也能接手的状态”。

14.9 如果任务不是一条 smoke，而是一整夜 batch，故事会怎样扩展

一旦任务变成 batch 或 overnight，这个故事并没有换主题，只是把同一套主角重复调用了很多次。场景还是那个场景，planner 还是那套 planner，区别在于起终点对更多、方法对比更系统、结果汇总更正式。换句话说，overnight 不是另一种算法，而是同一故事被讲了很多遍，然后被整理成更有统计意义的总报告。

这件事非常适合讲给学弟听。因为它能帮他理解：复杂实验系统往往不是“多一套神秘模块”，而是“把基础单元做成了可重复、可追踪、可统计的流水线”。

14.10 第二天早上拿到报告时，故事应该进入“解释”而不是“猜测”阶段

理想状态下，第二天你不该再猜昨晚发生了什么。你应该能打开汇总 CSV、Markdown 和趋势图，快速知道哪种方法更稳、哪种更慢、这一夜总共跑了多少轮、是否存在明显异常。也就是说，工程组织的目标是把第二天早上的工作，从“猜测式排查”变成“解释式阅读”。

这就是为什么当前工作区会有 `post_overnight_report.sh`、`aggregate_batch_runs.py` 和 `visualize_batch_results.py`。它们不是锦上添花，而是在确保故事的最后一幕不是混乱，而是清晰。

15、如果你拿着论文一页一页对照本地工作区，最值得怎样对照

很多人说“要配合论文理解代码”，这句话当然对，但还不够具体。真正的问题是：怎么配，配哪里，先配什么。很多新人拿着论文和代码坐下以后，最大的困惑不是态度不认真，而是不知道应该拿论文里的哪部分去对本地哪些文件。如果这一层不讲清，配合阅读就会变成漫无目的的来回翻。

15.1 先用论文的题目和摘要，对准当前项目的大边界

论文标题和摘要其实特别值得认真看，因为它们通常已经把系统边界讲得很清楚了。你在当前项目里最值得先对准的几个词是：Safe、Real-Time、Robot Navigation、Gaussian Splatting Maps。然后立刻回到本地工作区间：这些词分别落在哪。

Gaussian Splatting Maps 对应的是 `config.yml`、`Nerfstudio` 和 `GSplatLoader` 这条场景入口链；Robot Navigation 对应的是 `Splat-Plan` 和 `Splat-Loc` 两条系统主链；Safe 更偏多面体与碰撞约束；Real-Time

则既是算法主张，也是工程上强调低摩擦实验组织的原因。只要摘要这一层先对准，后面你读论文正文时就不会飘在天上。

15.2 论文总系统图出现时，立刻回到工作区三层结构图

论文里一旦出现系统总览图，你最该做的不是盯着每个箭头看半天，而是马上回想当前工作区的三层结构：上游算法层、wrapper 工程层、文档与结果层。这样会让你意识到，论文图主要在讲算法和系统设计，而当前工作区则额外承担了机器环境、运行规范和交接材料这些落地层。

这个对照特别重要。因为它能帮助你同时尊重论文原意和当前工作区现实，而不会把二者混成一个层面。

15.3 读到 Splat-Plan 段落时，该直接去看哪些本地文件

这一部分最适合对照 `run_splatplan.py`、`splatplan/splatplan.py`、`splatplan/spline_utils.py`、`initialization/grid_utils.py`、`polytopes/collision_set.py`。你不需要按这个顺序每份都读透，但至少要知道：论文里一旦开始讲规划链，你在本地最有价值的落点就是这几个文件。

一个很好用的读法是：先从 `splatplan.py` 看整体骨架，再回头用 `grid_utils.py` 看初始化，用 `collision_set.py` 看约束从哪来，用 `spline_utils.py` 看轨迹如何求出来。这样论文里的逻辑会和代码结构自然贴合。

15.4 读到 Splat-Loc 段落时，别急着陷进全部细节，先找入口和角色

对当前阶段来说，Splat-Loc 更适合作为“认位置”的阅读对象，而不是“马上打透”的主线对象。论文这部分你最应该先抓住的是：它解决的是基于 RGB 图像的位姿估计问题，并且与 GSplat 场景表示紧密相关。对应到本地，就是 `run_splatloc.py` 作为入口，`pose_estimator/utils.py` 作为工具箱，`ns_utils/nerfstudio_utils.py` 作为场景和数据接口桥梁。

只要先把这三件事对准，你就已经赢过很多一头扎进细节然后迷路的人了。

15.5 论文实验部分怎么和本地场景、方法、结果目录对应

一旦论文进入实验部分，你就应该自然地把它和本地场景名、方法名、run 目录结构联系起来。比如论文里提到的 `old union` 就对应当前工作区常见的 `old_union` 场景；`flightgate` 在上游 README 里对应 `flight`；SFC 相关对比在本地则会落成 `sfc-1.1.4` 这些方法名。你以后越能把论文实验和本地方法/场景一对上，越说明你不是在“看两份材料”，而是在看同一个系统的两个表述面。

15.6 当论文里给出速度或性能数字时，你在本地该怎么保持清醒

这是一个特别值得讲给学弟听的点。很多新人一看到论文或 README 里的性能数字，就会下意识把它当成本机必然能复现的一模一样数字。更成熟的态度应该是：这些数字首先告诉你方法定位和量级，说明作者宣称它具备什么级别的实时性和效率；但你在本地复现时，仍然要区分论文环境、作者硬件、当前机器、当前 wrapper、当前数据版本和当前阶段目标。

也就是说，论文数字要被当成参照，而不是当成机械验收线。你真正要先确保的是链路打通、结果解释稳定、趋势合理，而不是一上来就被数字牵着跑。

16、十种特别常见、但非常值得提前拆掉的误解

到了最后，我想故意把一些很常见的误解再集中拆一次。因为很多同学并不是不努力，而是脑子里有几处小误解一直没被点破，导致整个理解都偏着走。你把这些误解拆掉，项目会轻很多。

16.1 误解一：我只要把官方论文看明白，就自然会做当前工作区

不完全对。论文解决的是系统设计和方法主张，当前工作区还额外承载了本机环境、数据落盘、tmux 长任务、共享机器边界和交接路径这些问题。你当然要看论文，但不能期待论文替你回答所有工作区问题。

16.2 误解二：我只要把 wrapper 脚本跑通，就等于会项目了

也不对。wrapper 脚本让你更稳地运行项目，但真正的算法理解仍然在上游源码和论文主链里。只会跑 wrapper 而不知道它调到哪一层，项目理解会很空。

16.3 误解三：scene 名只是为了方便命令短一点

绝对不是。scene 名背后牵着一整套实验设定、场景资产和结果解读方式。你越早把 scene 看成“实验语义入口”，越不容易把场景切换当成换个字符串。

16.4 误解四：config.yml 就是一个普通小配置文件

它远比“普通小配置”重要。它更像进入训练产物世界的门把手。你对它越尊重，越容易把场景入口、Nerfstudio 背景和规划器加载链讲清楚。

16.5 误解五：结果目录只是存东西的地方

不对。当前工作区的 runs/ 既是结果存放层，也是工程规范和项目记忆层。它承载的是“这次运行发生过”的证据，而不是“东西先放这里”。

16.6 误解六：不碰 ROS/实车，是因为我们做得还不够真

这也是错位理解。当前阶段不碰它们，是因为我们在做排序管理和风险管理。把离线/仿真主链和工程组织做稳，本身就是很真也很值的工作。

16.7 误解七：创新一定要等我先把所有东西都学完

现实里很少这样。更常见的是，你先把链路做顺，途中慢慢长出问题意识，再从问题里长出创新点。要是非等“全懂”才敢想创新，往往会一直等不来那个时刻。

16.8 误解八：学弟如果一周后还不会改算法，就说明文档没用

这也太苛刻了。对零基础同学来说，一周真正有价值的目标是：能讲清项目做什么、知道从哪里跑起、知道结果去哪里看、能分清 wrapper 和上游、能跟着一条主链读到核心文件。做到这些，就已经非常值了。

16.9 误解九：工程整理不算真正的项目推进

如果没有工程整理，你后面很多算法结果都无法稳定沉淀、无法被别人接手、无法形成可比较记录。从长期看，工程整理不仅算推进，而且常常是让推进真正成立的条件。

16.10 误解十：看不懂的地方越多，说明我越不适合这个项目

这大概是最需要被拆掉的一条。一个中型项目本来就不可能第一次全懂。真正重要的是，你有没有一张地图，能不能知道自己现在不懂的是哪一层，能不能把不懂的问题逐步压小。只要这三件事在发生，你就是在很正常地进入项目，而不是被项目拒之门外。

17、这两份文档最终想帮学弟建立的，不是“知道很多名词”，而是三种底层能力

写到最后，我想把这轮重写的真正目标再说透一点。第六编和第四编如果只是让学弟知道很多词、看过很多文件名，其实还远远不够。它们真正想建立的，是三种底层能力。

第一种能力，是定位能力。也就是碰到一个词、一个脚本、一个目录、一个报错时，你能迅速判断它站在哪一层、跟谁有关、接下来该往哪走。第二种能力，是讲述能力。也就是别人问你项目是什么、主链怎么走、当前边界是什么时，你能把它讲成一条别人听得懂的线。第三种能力，是推进能力。也就是你不需要再靠运气乱翻文件，而是知道下一步该读哪份文档、看哪个脚本、跑哪条最小链路。

如果这三种能力开始长出来，那么哪怕你现在还不是很会写算法，也已经真正进入项目状态了。对一个零基础转进来的学弟来说，这恰恰是最宝贵的起点。

18、把第四编放下之前，你最值得记住的最后一句话

如果一定要把整个第四编压成一句最值得带走的话，我希望是这一句：当前 `/home/howard/Splat-Nav` 工作区，不是论文本体的替身，也不是一堆零散脚本的堆积，而是一套把上游算法、共享机器现实、运行规范、结果沉淀和后来者入门路径接在一起的工程化工作区。你只要把这句话真正装进脑子里，很多之前让人焦虑的分岔路都会慢慢变清楚。

因为从这句话往下，你自然会知道：该去哪里看算法，该去哪里看运行入口，该去哪里找结果，该去哪里看解释材料，该在哪里保持边界。项目一旦开始以这种方式在你脑中成形，它就不再是吓人的陌生对象，而会变成一张确实可以走、可以讲、可以继续扩展的地图。

19、如果你真的把论文 PDF 摊开来读，我建议你按怎样的节奏读，才能和当前工作区真正对上

前面我们已经讲过“论文应该和本地工作区对照着读”，但如果再往实操推进一步，你可能还会问：那我真的把 PDF 打开以后，应该按什么节奏读？我不想一上来就淹死在论文里，也不想读完了却和本机一点没对上。这个问题问得非常好，因为很多新人卡的恰恰不是态度，而是节奏。节奏一错，就会出现要么看得很痛苦、要么看完也落不回项目的情况。

19.1 第一轮阅读：只抓“题目、摘要、方法图、实验结论”，不急着碰推导

第一轮看论文时，我非常建议你先把姿态放轻。只读题目、摘要、方法总图、实验总览和结论性段落。为什么这么安排？因为这几块最能帮助你建立“作者到底想解决什么问题、系统有哪些部分、论文想主打什么价值”的宏观轮廓。对当前项目来说，这一轮最关键的输出不是笔记很多，而是你开始会说：这篇工作是在 GSplat 场景表示上做安全规划和视觉定位，规划和定位分别对应 `Splat-Plan` 与 `Splat-Loc`，作者强调的是安全、实时和系统级导航能力。

这种第一轮读法一点都不偷懒。相反，它特别成熟。因为它在帮你先建立问题框架，而不是一上来被细节拖住。

19.2 第二轮阅读：把论文里所有“名词大图”对到本地文件和目录

第二轮看论文时，最值得做的事情是对名词和本地落点。也就是说，当你看到 Gaussian Splatting map，你要想到 `config.yml`、Nerfstudio、GSplatLoader；看到 Splat-Plan，要想到 `run_splatplan.py`、`splatplan/splatplan.py`、`spline_utils.py`；看到 Splat-Loc，要想到 `run_splatloc.py`、`pose_estimator/utils.py` 和 `ns_utils/nerfstudio_utils.py`；看到 baselines，要想到 SFC/、`run_rrt.py`、`run_nerfnave.py`。

你只要真的做过一轮这样的配对，论文就会从“学术叙事”变成“本地可落地系统”。这是一个很大的认知跨越。

19.3 第三轮阅读：只对 Splat-Plan 深挖，因为它是当前阶段最值的主线

对当前机器、当前阶段和当前学弟能力背景来说，第三轮最值得深挖的其实不是整篇论文平均用力，而是把 Splat-Plan 这一块重点看深。因为这部分和当前工作区的主执行链最直接相关，也是目前最可能在离线/仿真阶段产出稳定结果、稳定汇报和稳定教学价值的部分。你去看论文里关于安全走廊、碰撞约束、平滑轨迹的描述时，就应该自然地回想本地 `splatplan/splatplan.py` 里 `generate_path()` 的几个阶段。

换句话说，第三轮最值得追的不是论文最难的部分，而是论文里当前项目最活的部分。

19.4 第四轮阅读：带着本地结果回看论文，而不是永远停留在“先懂论文再跑代码”

很多人读论文会形成一种单向想象：好像只有“先全懂论文，再跑代码”这一条路。真实项目里更常见、也更有效的节奏其实是来回走。也就是说，你先有了一点本地结果，比如一条 smoke 输出、一个 batch JSON、一张可行率图，再回头看论文。这个时候论文里的很多词会一下子不一样。你不再只是读到 `feasible`、速度、基线、场景，而是能把这些词和你昨晚真正跑出来的东西连在一起。

这种“结果反哺论文理解”的节奏非常值得鼓励。因为项目理解从来就不是只靠输入长出来的，它很大一部分来自于你真正做过事情以后，阅读重新变得有抓手。

19.5 第五轮阅读：当你开始想创新时，再去看论文没展开的边缘地带

真正到了开始想创新、想专利或想后续会议论文方向时，再去看论文里那些你第一轮完全不会关注的边缘地带才最有价值。比如作者没展开讲透的局限、某些对比没做全的部分、某些假设默认成立但在你当前工作区不一定稳的部分。这个阶段的论文阅读已经不是“帮我入门”，而是“帮我发现边界和缺口”。

所以别着急一上来就追这一层。它更像后劲，不像起跑线。

20、如果你现在要在白板前给学弟讲 20 分钟项目全景，我建议你就讲四张图

很多时候，一个人是不是真的把项目吃进去，最直接的检验不是他看了多少材料，而是他能不能站在白板前，在不依赖幻灯片的情况下，把事情讲成一条有骨架的线。如果你现在要带一个零基础学弟，用 20 分钟把整个项目讲清楚，我特别建议你只讲四张图。因为图一多，人就容易散；图一少，你反而更容易讲出骨架。

20.1 第一张图：论文系统图

第一张图最简单，也最关键。你就在白板上写： $\text{Splat-Nav} = \text{Splat-Plan} + \text{Splat-Loc}$ 。然后用最朴素的话解释：前者负责规划一条安全轨迹，后者负责根据图像估计当前位姿。接着再补一句：它们共同建立的是“在

GSplat 场景表示上做导航”的系统。你会发现，很多学弟一开始对项目焦虑，并不是因为他不聪明，而是因为没有人先把系统拆成这两个大盒子给他看。

这张图一旦画出来，整个项目至少不会再像一团雾。

20.2 第二张图：当前工作区三层图

第二张图就画当前 `/home/howard/Splat-Nav` 的三层结构：上游算法层、wrapper 工程层、文档与结果层。上游层写 `splatnav-official/`，wrapper 层写 `scripts/`，文档与结果层写 `docs/` 和 `runs/`。然后告诉学弟：以后你所有看到的文件，基本都能归到这三层之一。层次一清，很多问题自然就不乱了。

这张图的价值极大。因为它能立刻拆掉很多初学者最痛苦的混淆：到底哪些是论文算法，哪些是机器上的工程壳，哪些是我们后续要看的材料和结果。

20.3 第三张图：一次 smoke 的最小调用链图

第三张图建议你画成箭头：`run_splatplan_smoke.sh -> smoke_splatplan.py -> GSplatLoader -> SplatPlan / SFC -> generate_path() -> JSON -> run_dir`。这张图特别适合作为学弟第一次真正理解“项目怎么跑”的桥。因为它既没复杂到像 overnight 那么长，也足够完整，能把 shell、Python、上游依赖、规划核心和结果落盘串起来。

只要学弟能把这张图复述出来，他对项目的把握就会比只会翻文件强很多。

20.4 第四张图：上游主轴图

第四张图就画上游主轴。中心写 `splatplan/splatplan.py`，左边连 `splat/splat_utils.py` 和 `ns_utils/nerfstudio_utils.py` 表示场景入口，下面连 `initialization/grid_utils.py` 表示离散初始化，右边连 `polytopes/collision_set.py` 表示约束生成，最后再连 `spline_utils.py` 表示轨迹优化。你不用把每个函数都写出来，只要把这种“主轴与支撑”的结构讲出来，就已经非常有用。

因为学弟最怕的不是文件多，而是不知道这些文件之间有没有一条主线。你把主线画出来，剩下的细节都可以慢慢补。

20.5 为什么我只建议画四张图，而不是十张

因为新人认知最怕信息太平。图太多，看起来好像很全面，实际最容易让人失去主次。四张图刚好够：系统图告诉他项目做什么，三层图告诉他工作区怎么看，最小调用链告诉他任务怎么跑，上游主轴图告诉他代码怎么读。再多的内容都应该先依附在这四张图上，而不是另起炉灶。

这其实也是一种非常大厂的表达习惯：先给对方一张稳定主图，再把细节挂上去。不是不讲细，而是先让别人有地方接细节。

21、如果把当前项目当成你练代码能力和项目架构能力的训练场，它到底在训练你什么

你前面很明确地说过，希望这个项目不仅仅是复现论文，还能帮你练代码能力和项目架构能力，往大厂风格靠。这个想法非常对，而且当前项目其实非常适合作为这样的训练场。因为它既有真实论文算法，又有上游与外层包装的边界，还有共享机器的现实限制、长任务管理、结果归档和带新人文档需求。换句话说，它天然不是一个只写几百行 demo 就结束的小玩具。

21.1 它在训练你的分层思维

当前项目最先训练你的，其实不是某个单点算法技巧，而是分层思维。你得分清论文层和工程层，分清上游和 wrapper，分清单次运行和多轮实验，分清环境问题和算法问题，分清结果生成层和结果消费层。很多人做项目越做越乱，本质上就是分层能力不够。一旦这项能力长出来，你以后进任何稍复杂的工程都会轻松很多。

21.2 它在训练你的边界感

共享机器、上游保留原样、只在自己 Conda 环境里修问题、用软链接整理重资产、结果统一落盘，这些东西都在训练同一件事：边界感。真正成熟的工程师不只是会写，还知道什么该动、什么不该动、什么要留证据、什么要保持可回滚。这个能力在学校里很容易被忽视，但在真实协作里极其重要。

21.3 它在训练你的结果意识

当前工作区特别强调 runs/、JSON、图表、Markdown 摘要和跨 run 汇总，这其实是在训练结果意识。也就是说，你做任何事情都不应只停留在“程序跑了”，而要问：结果怎么保存、别人怎么看、下次怎么比较、周报怎么讲、出了问题怎么追溯。很多大厂工程能力，说到底就是这种结果意识特别强。

21.4 它在训练你的讲述能力

别小看文档和汇报。一个能把项目讲清楚的人，通常比一个只会闷头写的人更适合长期推进复杂项目。当前 docs/Codex 的重建，本质上就在训练你这件事：把复杂系统讲成后来者听得懂、跟得上、愿意继续往前走材料。这种能力以后无论写设计文档、周报、答辩还是技术面试，都会很值钱。

21.5 它还在训练你一种更隐性的能力：面对复杂系统时不慌

这可能是最宝贵的一项能力。复杂项目不会因为你希望它简单，就变得简单。真正优秀的人，往往不是一开始什么都懂，而是面对复杂系统时不慌，能一点一点分层、定位、拆开、记录、推进。当前项目如果你真的陪着它整理下来，它非常有机会帮你把这种气质练出来。

22、把这轮文档重建的意图说得更直一点：我们不是在写“资料堆”，而是在写一条可交接的路径

最后这一节，我想把整个 第六编和第四编重写背后的意图说得更直白一点。我们不是在比谁能收集更多资料，也不是在把 archive 里的内容简单扩写成更厚的文件。真正想做的事，是把当前项目整理成一条对零基础学弟也可交接的路径。所谓路径，意思就是：他能从门口进来，先认词，再认图，再认一次运行，再认上游主轴，再认长期节奏。一路上每一步都不是孤立的，而是有承接的。

这件事之所以重要，是因为很多项目并不是死在技术上，而是死在交接上。前人知道很多，但后人接不住；资料很多，但顺序感没有；脚本不少，但入口全是暗门。当前这轮整理如果能把这些问题至少压下一大半，它本身就已经是项目推进的一部分，而不是项目之外的额外工作。

所以你完全可以很坦然地把这轮文档重建看成真正的工程产出。它不是陪衬，不是装饰，而是在给项目未来的持续推进清路。

23、二十个最值得反复问自己的关键问题：把项目真正问熟，而不是只看熟

到最后，我特别想留下这一节。因为很多项目理解其实不是“看”出来的，而是“问”出来的。你问得好，项目就会一层一层开；你如果从来不问，很多内容即使看过，也仍然停留在表面。所以这里我故意整理出二

十个最值得反复问自己的问题。它们不是考试题，而是帮你把项目真正问熟的一组抓手。以后你带学弟、做周报、做汇报，甚至自己卡住时，都可以回到这里重新问一遍。

23.1 问题一：当前 `/home/howard/Splat-Nav` 到底是不是官方论文仓库本体

最稳的回答是：不是完全等同，但它把官方论文仓库稳定地接进来了。当前工作区是一个 `wrapper` 工程工作区，主要负责环境边界、运行规范、结果落盘和文档交接；真正的上游核心实现通过 `splatnav-official/` 入口被接入，并最终指向数据盘上的官方源码仓库。把这件事说清，是你理解整个项目的第一关。

23.2 问题二：既然官方仓库已经在，为什么还要保留 `wrapper` 这一层

因为现实机器和论文作者自己的实验环境从来不会完全一样。共享工控机、数据盘与系统盘分工、`tmux` 长任务、结果规范、后来者接手路径，这些都不是官方论文仓库天然负责的事情。`wrapper` 的价值就在于把这些现实问题吸收进一个稳定外壳里，让上游源码既保持事实来源，又能在当前环境中更可靠地被运行和解释。

23.3 问题三：为什么 `heavy assets` 要尽量放在 `/data` 而不是系统盘

这不仅是空间问题，也是边界问题。系统盘紧张时，把大体量数据、缓存、结果和环境资产集中到数据盘，能显著降低系统层压力，也更适合共享机器的长期维护。对当前项目来说，这一点特别重要，因为 `run` 目录、数据集、训练输出和可视化结果都会持续增长。如果没有这层纪律，项目很容易被磁盘问题拖垮。

23.4 问题四：为什么当前工作区里要大量使用软链接

因为软链接允许你同时满足两件事：一是用户面对的是清楚、短、好讲的工作区入口；二是真实大体量资产仍然可以放在更合适的位置，比如数据盘。你既不用复制两大份资产，也不用为了适配路径去大改上游逻辑。对当前项目来说，`splatnav-official` 和 `runs` 这两个入口的软链接设计，都是非常典型的成熟工程取舍。

23.5 问题五：为什么 `config.yml` 这么重要，它到底是不是普通配置文件

它当然是配置文件，但绝不是“普通得可以忽略”的那种。它更像进入某个训练产物世界的门把手。它把你带回 `Nerfstudio` 训练输出、数据解析结果和场景模型上下文，让 `GSplatLoader` 有地方去加载真正的场景表示。如果你把 `config.yml` 看轻，项目的整个输入链就会变得模糊。

23.6 问题六：为什么当前阶段先围绕 `old_union` 组织，而不是把所有场景平均推进

因为工程推进最怕平均用力。`old_union` 在当前工作区已经具备比较清楚的路径、结果和默认脚本支持，更适合作为稳定主场景。先把一个场景跑顺、讲顺、文档顺，再切到其他场景，会比四个场景一起浅推进更有效。这种选择不是偷懒，而是节奏管理。

23.7 问题七：为什么每次运行都要建 `run_dir`，感觉会不会太啰嗦

如果你只看一次运行，当然会觉得多建目录有点繁琐。可一旦你开始做 `batch`、`overnight`、周报和多人交接，就会立刻明白 `run` 目录有多值钱。它让每次运行都有自己的时间戳、命令记录、环境摘要、日志和产物，避免“昨天那个结果到底怎么来的”变成只能靠回忆猜的事情。对长期项目来说，这一点非常重要。

23.8 问题八：为什么日志不能只看终端，非要落到 `stdout.log`

因为人会走，终端会断，记忆会糊。日志落盘的价值就在于它把运行过程从瞬时现象变成可追踪证据。尤其在 `tmux` 长任务和第二天复盘场景里，没有日志文件几乎等于没有历史。你以后只要真的查过一两次问题，就会非常感谢这个习惯。

23.9 问题九：为什么 `smoke` 一定要先于 `batch` 和 `overnight`

因为 `smoke` 的价值不是“规模小”，而是“排雷快”。它让你用最小代价检查主链是否通、输入是否合理、输出是否结构化可读。如果一上来就跳 `batch`，甚至 `overnight`，任何小问题都会被更长执行时间和更复杂结果掩盖。成熟的工程从来不是一上来就大跑，而是先做最小有效验证。

23.10 问题十：为什么 `run_in_tmux.sh` 会被看得这么重

因为它不是简单后台运行工具，而是把长任务管理、日志落盘和 `run` 规范绑定在一起的工程入口。很多项目之所以夜里任务常常“跑没了也说不清”，根源就在于缺这种统一入口。当前工作区能比较稳定地组织长任务，很大程度上靠的就是这种工程细节被封装好了。

23.11 问题十一：为什么 `batch` 结果不止一个 JSON，还要做图和 Markdown 摘要

因为结果有不同消费层。JSON 适合程序继续处理和精细追踪；图适合快速肉眼判断；Markdown 摘要适合第二天快速扫读和做汇报。这不是重复劳动，而是在降低不同阅读场景下的理解成本。真正成熟的结果管理，都会做这种分层。

23.12 问题十二：为什么 `SplatPlan` 是当前阶段最值得优先吃透的类

因为它正处在论文主张、上游实现和当前工作区复现主线的交汇点上。你吃透它，等于同时吃透了“场景怎么进系统”“规划主链怎么组织”“结果为什么长这样”这三件最关键的事。相比之下，很多其他文件虽然也重要，但不具备这么强的主轴性。

23.13 问题十三：为什么现在不优先让学弟钻 `Splat-Loc`

不是因为 `Splat-Loc` 不重要，而是因为它在当前阶段对认知负担更重。定位链同时牵涉图像、特征、配准、位姿表示和递推估计。对零基础学弟来说，先把 `Splat-Plan` 和工作区三层结构稳住，再回来看 `Splat-Loc`，推进顺序会更合理。排序清楚，比一上来贪多更重要。

23.14 问题十四：为什么当前阶段明确不碰 ROS、语义分支和实车

因为它们每一样都会显著提高系统复杂度，而当前第一阶段的核心目标是离线/仿真复现与工程化整理。只要主链、结果验收、文档交接和工程边界还没稳住，就直接把 ROS、语义增强和实车全压进来，项目几乎肯定会失焦。真正稳的推进，不是能碰的都碰，而是知道什么时候先不碰。

23.15 问题十五：为什么工程整理本身也算项目成果，而不只是陪衬

因为没有工程整理，很多结果只是一次性现象；没有文档和规范，很多理解无法被后来者接住；没有 `run` 目录和汇总，很多实验无法被稳定比较。也就是说，工程整理不是和项目分开的附属工作，而是在把项目从“个人尝试”升级成“团队可接手系统”。对长期推进来说，这非常值钱。

23.16 问题十六：如果未来要重构代码，为什么不能凭感觉乱动上游

因为上游仓库是论文事实来源，动它就等于在动“作者原实现”这层事实。重构当然可以做，但前提应该是你已经非常清楚原逻辑做了什么、你的改动边界在哪、验证方式是什么，以及哪些行为绝不能被改坏。否则所谓重构很容易变成把项目重新搅乱一次。

23.17 问题十七：学弟读完 第六编和第四编以后，最起码应该具备什么状态

并不是“会发明新算法”，也不是“能把所有代码默写出来”。更现实、也更有价值的状态是：他能用大白话说清项目做什么、能区分上游和 wrapper、知道该从哪条 smoke 链路起步、知道结果先去哪看、知道 config.yml 和 scene preset 各干什么、知道 SplatPlan 是当前主轴、知道第一阶段为什么不碰实车。这些状态一旦成立，后面真正动手就会顺很多。

23.18 问题十八：如果学弟一周后还觉得有些地方没完全懂，是不是文档失败了

完全不是。只要他已经不再迷路，知道问题站在哪一层、知道接下来该读哪、敢开始问具体问题、能把项目讲出骨架，那这本手册就已经非常成功了。复杂项目本来就不是一周背完，而是靠一周建立稳定地图，再逐步压细节。

23.19 问题十九：如果未来要从这里长创新点，最健康的起点是什么

不是强行发明新名词，而是基于已经稳定的复现和理解，去观察哪里最痛、最脆、最费人、最容易出错、最值得对比再做深一点。很多创新最早都只是好问题，而不是完整答案。学弟越早接受这一点，越不容易被“我现在还提不出大创新”这种焦虑困住。

23.20 问题二十：什么叫“这套工作区已经真正可交接了”

最稳的标准不是“你自己觉得差不多”，而是后来者能按主线文档找到入口，能区分三层结构，能在不破坏机器边界的前提下跑起一条最小 smoke，能读懂结果目录，能继续往 batch 和更长周期推进，同时你还讲得清每一层在干什么。如果这些条件基本都成立，那这套工作区就已经不只是“能用”，而是“能传下去”。

这其实也是我们这轮重建 第六编和第四编最根本的目标。不是把资料变厚而已，而是让项目真的长出可交接性、可讲述性和可持续推进性。

24、把当前工作区和上游仓库的一、二级目录再做一轮“索引式人话说明”

前面的内容已经从故事、脚本、主轴和论文对照等角度把项目讲开了。这里我想再补一层更像“索引册”的说明，但仍然尽量保持人话和路感。这样做的价值在于，当学弟以后真的开始点目录时，不会只看到文件名，而能很快在脑子里冒出一句“这个地方大概是干什么的”。这层说明不追求对每个文件都深讲，但追求目录覆盖更完整、入口感更清楚。

24.1 工作区一级目录再索引：当你站在 /home/howard/Splat-Nav 门口时，到底能往哪几条路走

先从最上层说起。当前工作区里最值得你脑子里形成稳定感的一级入口，一共就这么几类。configs/ 是机器约定和工程配置入口。docs/ 是知识和交接入口。runs/ 是结果证据入口。scripts/ 是工程执行入口。splatnav-official/ 是上游源码阅读入口。third_party/ 是第三方依赖关系声明入口。README.md 是门口说明牌。Makefile 则是常用动作的快捷入口。

你以后只要一进项目就先意识到“我现在站在一级入口层”，很多不必要的乱翻就会减少。因为你会先问自己：我现在是来读材料、跑任务、看结果，还是读上游代码。问题一旦先分层，动作就不乱。

24.2 docs/ 的二级目录：三种文档人格，三种使用方式

`docs/archive/` 更像旧讲义和历史资料库，信息广，但不适合零基础直接主线阅读。`docs/guidang/` 更像执行规范和阶段归档区，适合你开始有节奏地做项目以后回来看“做到什么算完成”。`docs/Codex/` 则是当前真正的主线阅读区，专门围绕当前机器、当前工作区和零基础学弟的最短路径重写。

很多人读文档会吃亏，就是因为把这三种人格完全混着用。你以后只要记住：`archive` 偏资料、`guidang` 偏规范、`Codex` 偏主线，就已经很稳了。

24.3 scripts/ 的二级视角：虽然都在一个目录里，但其实分成六种工作性格

你以后不要把 `scripts/` 仅仅看成一个装满可执行文件的目录。更有帮助的做法，是在脑子里把它再切成六种性格。环境脚本负责“让机器准备好”；数据脚本负责“让资产完整并对上路径”；`run` 和 `tmux` 脚本负责“让任务稳定发生”；`smoke/batch/overnight` 脚本负责“让实验真正执行”；可视化和聚合脚本负责“让结果会说话”；同步脚本负责“让上游关系不失忆”。

一旦你有了这种二级视角，哪怕以后再往 `scripts/` 里加脚本，你也会更自然地知道它应该属于哪一类，而不是随手堆进去。

24.4 runs/ 的二级结构：目录名、日志文件、结构化结果、图和摘要分别承担什么角色

`runs/` 里的二级世界也很值得形成直觉。最外层的时间戳目录是在讲“这是哪次任务”；目录里的 `env.txt`、`cmd.txt`、`stdout.log` 是在讲“任务如何发生”；JSON 是在讲“程序级结构化结果”；图和 Markdown 摘要是在讲“人如何快速消费这些结果”；如果是聚合或报告目录，CSV 和跨轮趋势图又是在讲“多次实验之间如何比较”。

所以 `runs/` 不是简单“文件越来越多”的地方，而是一个结果层次不断往外展开的地方。你越有这种层次感，越会觉得当前工程是清楚的。

24.5 third_party/ 的二级存在感：它虽然目录不大，但概念很重要

当前 `third_party/` 最显眼的就是 `splatnav-official` 这个入口。它的物理内容不复杂，但概念价值很大。因为它在反复提醒你：上游官方仓库不是和 `wrapper` 混写的一堆文件，而是一份被接入进来的第三方依赖。对于长期维护和后续带人，这个提醒特别有用。它会让你一直记得“哪些是事实来源，哪些是我们自己的工程组织”。

25、上游仓库的一、二级目录索引：以后你点开时，脑子里最好先冒出什么话

现在把目光转到 `splatnav-official/`。这里目录看上去会更“研究仓库”一点，也就是既有核心实现，也有实验入口、可视化工具、历史结果和辅助脚本。对新人来说，最有用的不是把这些目录当成平铺世界，而是先给它们分配一句“角色句”。下面这一轮索引，目的就是帮你建立这种角色句。

25.1 SFC/：这里在讲 baseline 家族，而不只是某一个方法文件

点开它时，你脑子里最好先冒出一句：这里是安全飞行走廊家族的实现区。`corridor_utils.py` 是主轴，`astar_utils.py` 和 `helper_utils.py` 是支撑，`viz_utils.py` 是展示辅助。只要这句角色句在，你后面看 `sfc-1..4` 时就不会再像看四个凭空来的标签。

25.2 splat/：这里在讲“场景怎样被带进系统”

点开 `splat/` 时，你最好立刻想到：这是场景入口层。核心关键词就是 `GSplatLoader`。也就是说，这里不是在做路径规划，而是在做“让后续系统真正能拿到 `GSplat` 场景对象”。只要这句话在脑中稳住，`splat_utils.py` 的位置就非常清楚。

25.3 splatplan/：这里就是规划主轴，不用客气，优先级最高

这一层你几乎可以直接把它当成“当前阶段最重要目录”。点开它时脑子里就应该是：这里是当前离线/仿真复现主轴。`splatplan.py` 负责主流程，`spline_utils.py` 负责轨迹求解。以后你只要对项目主链有任何疑问，都很适合优先回到这里。

25.4 initialization/：这里在讲“先给我一条能走的骨架线”

你点开它时，最值得先想起的就是这句话。这里负责的是离散初始化，不是最后答案。`grid_utils.py` 会让你看到 `Voxel`、`PointCloudVoxel` 和 `GSplatVoxel` 这些角色，帮助你理解连续场景为何要先压成网格骨架。它的角色感一旦成立，后面很多 A^* 和投影到最近可导航点的行为就很好理解。

25.5 polytopes/：这里在讲“安全约束从哪里长出来”

点开它时，你应该先想到：这里是安全走廊和约束真正被构造出来的几何后台。`collision_set.py` 很适合作为第一入口，因为它直接把“相关障碍提取”和“包围盒 + 半空间”这种概念展开了。以后你只要问“这些约束到底怎么来的”，很自然就该回到这里。

25.6 ellipsoids/：这里在讲“高斯怎样变成可算几何对象”

这个目录一开始可能不那么亲切，但它特别关键。因为高斯如果只停留在视觉效果层，就很难真正进入导航约束。这个目录承担的，就是把高斯变成协方差、椭球、相交测试、`mesh` 等可计算几何对象的工具层。它不总是第一个读，但你越往后越会感谢它的存在。

25.7 ns_utils/：这里是和 Nerfstudio 世界握手的地方

点开它时，最好先想到“桥梁”两个字。这里不是在做规划本身，也不是在做结果展示，而是在让 `Nerfstudio` 的训练世界和当前项目的规划/定位世界能够接上。只要这层角色稳住，`config.yml` 和 `GaussianSplat` 这些东西的关系就不容易乱。

25.8 pose_estimator/：这里是定位工具箱，而不是项目的杂物间

新手很容易被这里函数多、概念杂的样子吓住。其实更好的角色句是：这是定位工具箱。`SIFT`、`LightGlue`、`PnP`、`ICP`、位姿表示和一些递推估计相关工具都在这里，是为了支持 `Splat-Loc` 这条链。只要知道它是“为定位服务的工具箱”，它就不会显得那么杂乱无章。

25.9 nerfnav/：这里在提醒你作者不是只和自己比

这个目录的角色句可以很简单：这里对应一条 NeRF-based 导航对比路线。它提醒你论文研究不是只展示自己方法，还保留了与另一类技术路线比较的实现痕迹。对做全景理解和后续可能的研究感培养都很有帮助。

25.10 results/、plotting/、blender/、visualize.py：这里统称“结果会说话的一层”

如果把这几部分放在一起记，会很轻松。results/ 偏已有结果和处理脚本，plotting/ 偏图表辅助，blender/ 偏离线可视化或渲染辅助，visualize.py 偏交互式查看。你不用第一次就把它们全分得很细，但至少要知道：这里属于展示与结果处理层，而不是规划主轴层。

25.11 run_splatplan.py、run_splatloc.py、run_rrt.py、run_nerfnav.py：这些文件名本身就在泄露项目组织方式

看到这些入口文件时，你最好形成一个自然习惯：文件名前半通常就在告诉你它对应哪条方法链，run_ 则说明它更像实验入口而不是底层工具。只要有这个习惯，你以后点开任何 run_*.py 都会先问：这是在跑哪条链，入口之外它会调到哪些目录去。这种提问方式非常值钱。

26、如果以后你一段时间没碰这个项目，又回来继续看，最推荐的复位方式是什么

项目不是总能连续推进。有时你会忙几天、忙一周，或者切到别的事上去。等你再回来时，最怕的就是又陷入“我上次看到哪了来着”的失重感。所以这里我想再留一个非常实用的小建议：如果以后你有一阵没碰这个项目，再回来时不要试图从记忆硬接，而是主动做一次复位。

最推荐的复位顺序其实很简单。先回到 /home/howard/Splat-Nav 根目录，看一眼一级结构；再回到本书开头的第一编，用几分钟把主线重新校准；然后扫一眼第四编和第六编的目录，把自己拉回地图；再根据当前任务是“读代码”还是“看结果”，去 scripts/、splatnav-official/ 或 runs/ 继续走。只要你接受“项目复位也是工作的一部分”，就不会再因为短暂停顿而把自己搞得特别焦虑。

这其实也是这轮文档重建很想做到的一件事：不仅让第一次进门的人有路，也让中途暂停后再回来的人能迅速找回路。真正好的项目材料，不该只服务第一次阅读，而应该能支持很多次重新进入。

27、当论文、项目页、README 和当前工作区说法不完全一致时，你该相信谁、怎么判断

真正做项目的人很快就会发现，一个系统在不同材料里出现时，口径并不总是完全一致。论文会更关注方法主张和实验结论，项目页会更关注总览和展示，README 会更关注依赖、数据和入口，当前工作区文档则更关注本机边界、工程组织和交接路径。对新人来说，这种不完全一致非常容易造成心理波动，好像自己是不是看错了。其实大可不必。真正成熟的做法，不是期待所有材料句句一模一样，而是学会判断“这份材料在回答什么问题”。

27.1 第一原则：先分语境，再谈对错

只要你发现两份材料的说法不完全一样，最先应该做的不是判断谁错，而是先问：它们各自在回答什么问题。论文回答的是系统设计和方法主张；项目页回答的是如何快速理解这项工作；README 回答的是仓库怎么开始；当前工作区文档回答的是在这台机器上怎么稳当地复现、整理和带人。很多所谓不一致，真正原因只是语境不同。

你一旦有了这个第一原则，心态会稳很多。因为你不会再一看到差异就觉得自己看不懂，而是会先去找它们的关注点差异。

27.2 第二原则：涉及“方法做什么”，优先看论文和上游核心代码

如果冲突发生在系统目标、模块作用、方法步骤这种层面，那么最值得优先参考的，通常还是论文和上游核心代码。因为它们更接近作者原始主张和实现事实。比如 `Splat-Nav = Splat-Plan + Splat-Loc`，比如安全走廊和样条轨迹的大骨架，这类东西的事实锚点就更应该落在论文和上游实现上。

当前工作区文档在这一层做的，是翻译和解释，而不是另起一套方法定义。

27.3 第三原则：涉及“在这台机器上怎么跑”，优先看当前工作区 README 和 docs/Codex

如果问题变成“这台机器上路径怎么组织”“为什么结果放在 `/data`”“为什么长任务进 `tmux`”“为什么这里有 `wrapper` 脚本”，那这时最优先的参考就不再是论文，而是当前工作区自己的 README 和文档。因为这些问题本来就不属于论文要回答的范围。它们是当前机器、当前协作方式和当前工程化组织的产物。

你以后越早接受“不同问题对应不同事实锚点”，越不会在材料差异前焦虑。

27.4 第四原则：涉及“具体命令和目录”，优先看本机实际文件系统

这条尤其重要。无论论文、README 还是文档怎么写，真要落到这台机器上，最终的最高优先级事实仍然是当前文件系统里真实存在的目录、软链接和脚本。也就是说，`readlink -f`、`find`、`ls`、`sed`、打开真实文件本身，很多时候比争论文字口径更可靠。因为路径有没有连对、文件是不是在、脚本实际怎么写，这些都属于机器上最直接的事实。

成熟的项目习惯里，最终永远有一层“回到现场看真实文件”。

27.5 第五原则：一旦发现差异，最好的动作是“补解释”，不是“先焦虑”

当前这轮文档重建本质上就在做这件事。不是假装所有口径天然一致，而是把差异的原因、层次和桥接关系讲出来。比如为什么论文讲系统层，工作区文档讲工程层；为什么 README 提数据结构，而 `wrapper` 文档提 `run` 目录和 `tmux`；为什么上游入口像作者实验台，而本机脚本像可交接外壳。

所以以后只要你发现新差异，不要先焦虑，先想：我能不能把这层差异解释清楚。很多看似冲突的问题，一旦被解释开，反而会让你的理解更深。

28、以后无论新场景、新脚本、新结果还是新需求进来，你都应该怎么把它重新挂回这张地图

真正有生命力的项目，不会永远停在今天这批文件和这套结果上。未来很可能还会有新场景、新脚本、新实验方法、新结果格式，甚至新的带人需求。如果你真的把这份全景文档吃进去了，那么面对新增内容时，你最不该做的是慌着把它当成“新宇宙”。更好的做法，是先问：它应该挂回当前地图的哪一层。

28.1 新场景进来时，先把它挂到“场景入口 + 实验设定”这两层

不要一看到新场景就立刻陷进路径细节。先问两个最根的问题。第一，它的 `config.yml` 和相关资产入口在哪里。第二，它对应的 `scene preset` 或实验摆法怎么定。这两个问题一旦明确，新场景就不再是陌生物，而是被挂回了“场景入口层”和“实验设定层”。

只要这个动作先做，后面无论是 `smoke` 还是 `batch`，都比较容易接上当前工作区的主流程。

28.2 新脚本进来时，先问它属于环境、数据、执行、汇总还是同步哪一类

这条特别适合你未来自己继续工程化整理时用。不要让新脚本直接以“又多了个脚本”的形式进入项目。你应该先问它在六种 wrapper 性格里属于哪一种：环境、数据、run/tmux、执行链、结果消费还是同步维护。只有把这个类先分出来，脚本才不会越来越像杂物堆。

这其实就是很实用的架构思维训练。

28.3 新结果进来时，先问它服务的是程序消费、人工快读还是跨轮比较

同样地，新结果文件也不该只是“多了个产物”。你先问它是服务程序继续读取的结构化结果，还是服务人工快读的图和摘要，还是服务跨 run 比较的聚合结果。只要这个功能定位先稳住，结果目录就不容易越长越乱。

很多项目后期难维护，不是结果太多，而是没人再说得清每类结果到底给谁用。

28.4 新需求进来时，先问它是在扩展主线，还是在引入另一条复杂性

未来导师、师兄或者项目阶段推进时，很可能会带来新需求。比如换场景、加一类 baseline、补更多图、切 ROS、碰语义、做专利、做论文初稿。面对这些需求时，最值钱的问题永远是：它是在扩展当前主线，还是在引入另一条复杂性。如果是扩展主线，可以挂到现有地图上继续推进；如果是引入新复杂性，就应该先明确边界和时机，而不是被热情裹着走。

这条判断能力，几乎就是项目推进质量的分水岭。

28.5 真正成熟以后，你会发现地图不是束缚，而是让复杂性变得可接住的工具

很多人一开始会担心，把项目讲得这么分层、这么有地图感，会不会反而限制灵活性。其实恰恰相反。地图越稳，你接复杂性的能力越强。因为新增内容再多，也只是挂回已有层次，而不是每来一个东西都把你打回迷路状态。

这也是第四编 最后真正想送给你的东西：不是一份静态说明书，而是一张能够继续接住未来变化的地图。只要这张地图在，你就不太容易被项目新的复杂性吓住。你会知道，先分层，先定位，再推进。很多看起来很大的事情，其实就是这样一点一点被接住的。

29、十个“一分钟复述模板”：以后无论口头汇报、和师兄沟通还是给学弟讲解，都能直接拿来用

有些知识你明明已经懂了七八分，但一张嘴还是容易乱。这不是理解没到，而是表达模板还没长出来。所以最后我再补一个很实用的小附录：十个一分钟复述模板。它们不是标准答案，而是帮助你快速把项目讲清楚的“起手架”。以后你不管是在组会里被点到、和师兄临时沟通、还是带学弟过项目，都可以从这里借一个骨架开始说。

29.1 模板一：一分钟讲清“这个项目到底是干什么的”

你可以这样说：这个项目的核心不是重新做三维重建，而是站在 Gaussian Splatting 场景表示上，进一步完成导航系统里的两件核心事。第一件是 Splat-Plan，也就是在场景里规划一条安全、平滑的轨迹；第二件是 Splat-Loc，也就是根据 RGB 图像估计机器人当前位姿。当前我们在这台机器上主要先把离线/仿真的 Splat-Plan 链路复现和整理清楚，同时用 wrapper 工程层把环境、运行和结果归档规范化。

这段话的好处在于，既讲了论文系统，也交代了当前阶段边界。

29.2 模板二：一分钟讲清“为什么工作区里会有 wrapper 和官方源码两层”

你可以这样说：官方源码是论文算法和作者实验入口的事实来源，必须尽量原样保留；wrapper 工作区则负责解决当前机器上的现实问题，比如 Conda 环境、数据盘落盘、tmux 长任务、run 目录和交接文档。也就是说，wrapper 不是替代上游，而是让上游在本机环境里更稳、更可追踪、更适合后来者接手。

这段话特别适合用来回答“为什么不直接只看官方仓库”。

29.3 模板三：一分钟讲清“从命令到结果，主链路是怎么走的”

你可以这样说：当前最小主链一般是从 shell 包装脚本进入，比如 `run_splatplan_smoke.sh`。它先建 run 目录、记录命令、激活环境，再调用 `smoke_splatplan.py`。Python 入口会根据 `config.yml` 和 scene preset 构造 planner，通过 `GSplatLoader` 读入场景，再调用 `generate_path()` 完成初始化路径、碰撞集合、多面体约束和样条优化，最后把结果写成 JSON，并把日志和相关产物沉淀到 `runs/` 目录。

只要你能把这条链顺着说下来，对项目的骨架理解就已经很稳了。

29.4 模板四：一分钟讲清“为什么 config.yml 这么关键”

你可以这样说：`config.yml` 虽然看起来只是一个配置文件，但它在当前项目里更像进入训练场景模型的门把手。它背后连接的是 Nerfstudio 训练输出、数据解析结果和 `GSplat` 场景上下文。没有它，`GSplatLoader` 就不知道去哪里拿场景，后面的规划和定位链也就失去了具体对象。

这段话很适合讲给第一次接触 Nerfstudio 生态的人听。

29.5 模板五：一分钟讲清“为什么要先 smoke，再 batch，再 overnight”

你可以这样说：smoke 的目的不是跑得少，而是先用最小代价验证主链是否通、输入是否合理、输出是否结构化。batch 的重点则是形成统计意义上的方法比较，比如可行率、平均耗时和 P95。overnight 不是另一套算法，而是在更长时间窗口里反复组织 batch，把多轮实验沉淀成第二天可直接查看的总报告。也就是说，这三层的区别不只是规模大小，更是任务目标不同。

这种讲法会让你的实验理解显得很清楚。

29.6 模板六：一分钟讲清“为什么当前阶段不碰 ROS 和实车”

你可以这样说：当前阶段先不碰 ROS 和实车，不是因为它们不重要，而是因为我们在做更稳的排序。只要离线/仿真链路、结果验收、工程边界和交接路径还没稳住，直接去碰更高复杂度系统只会让问题层次叠加。现在先把 Splat-Plan 的离线主链做成可跑、可讲、可交接的状态，后面再切 ROS 和实车会更合理，也更节省时间。

这段话既能保护节奏，也能显得你对系统复杂性有判断。

29.7 模板七：一分钟讲清“学弟第一周最应该学会什么”

你可以这样说：第一周最重要的不是会改算法，而是建立项目地图和运行地图。也就是说，学弟至少要能分清上游和 wrapper，知道 `config.yml` 和 scene preset 各干什么，能顺着 smoke 主链讲出一次运行怎么走，知道结果先去哪看，知道 `runs/` 里的日志、JSON、图和摘要各是什么角色。只要这些骨架立住，后面真正动手就不会那么慌。

这段话特别适合你以后和学弟对齐预期时用。

29.8 模板八：一分钟讲清“这套项目为什么也很适合练工程能力”

你可以这样说：这个项目不只是论文复现，它还天然是一个很好的工程训练场。因为它同时要求你处理上游与外层包装边界、共享机器环境约束、长任务调度、结果沉淀、文档交接和后来者入门路径。也就是说，你在这里练到的不只是算法阅读，还包括分层思维、边界感、结果意识和讲述能力，这些都很符合大厂工程能力的训练方向。

这样讲，会让项目的价值不再只剩“跑论文”。

29.9 模板九：一分钟讲清“如果未来要想创新点，该从哪观察”

你可以这样说：创新点不一定从发明新名词开始，更常见的起点是先把现有链路跑顺、读顺、讲顺，然后认真看哪里最痛、最脆、最费人、最容易解释不清。工程层痛点如果反复对应同一类瓶颈，很可能就是未来研究问题的入口。也就是说，现在先把系统做成可稳定观察的状态，本身就在给未来创新铺路。

这段话能很好地帮新人从“创新焦虑”里缓一口气。

29.10 模板十：一分钟讲清“什么叫这套工作区已经可交接”

你可以这样说：所谓可交接，不是我自己觉得差不多，而是后来者能按主线文档找到入口，能区分工作区三层结构，能在不破坏机器边界的前提下跑起一条最小 smoke，能读懂 `runs/` 目录和结果文件，并知道下一步该往哪条主线推进。如果这些条件基本成立，那这套工作区就已经不是“只有原作者会用”，而是真正长出了后续生命力。

这其实就是我们现在重建 第六编和第四编最想做到的事。不是让资料更厚而已，而是让项目真的可以被接住、被讲清、被继续往前推。

30、每次接手、复位或开工前，都值得做的五个 sanity check

写到这里，我再补一个很实用的小附录。因为再厚的文档，如果不能转成一些具体动作，很多时候也容易停留在“看过”。所谓 sanity check，你可以先把它理解成“开工前的理智检查”。它不是复杂流程，而是帮你在项目开始前快速确认自己没站歪、没想歪、没把层次弄混。对共享机器、上游依赖、长任务和多文档并存的当前工作区来说，这种小检查特别值钱。

30.1 第一个 sanity check：我现在到底站在哪台机器、哪个目录、哪个入口层

很多混乱都是从这里开始的。你以为自己已经进入项目，其实还在本地终端；你以为自己站在主工作区，实际上停在某个数据目录或旧 run 目录里；你以为自己在看官方源码，实际上点开的是 wrapper 文档。这种错位一旦发生，后面的判断都容易跟着歪。所以每次开工前，先确认 SSH 上的是哪台机器，当前路径是不是 `/home/howard/Splat-Nav`，自己现在准备进入的是 `docs/`、`scripts/`、`runs/` 还是 `splatnav-official/`。

这个动作听起来很基础，但它特别像开车前先看自己是不是坐对了车。基础，却不能省。

30.2 第二个 sanity check：我现在解决的是哪一层问题

这是整个项目里最值钱的小问句之一。你现在卡的是环境、路径、数据完整性、wrapper 调度、上游算法还是结果解释？只要这层分不清，排错和阅读就会非常乱。比如你明明碰到的是 `config.yml` 找不到、软链接没对好，却把自己拖去研究 `spline_utils.py`；或者你明明只是 `batch` 汇总图没出来，却开始怀疑 SplatPlan 核心逻辑。这种误配层次的消耗非常大。

所以每次卡住时，先别急着乱翻资料。先问自己：这到底是哪一层的问题。这个问题一旦答出来，路就会短很多。

30.3 第三个 sanity check：我现在看的材料，回答的是我眼前这个问题吗

有时候你会发现自己看了很多东西，但问题并没有更清楚。多数情况下，不是你不努力，而是你拿错了材料。你现在想知道的是“这台机器上该怎么跑”，却去翻论文；你现在想知道“为什么这个 scene preset 这样设”，却一直停在 README；你现在想知道“这个实验目录到底算成功没”，却又回头去看术语词典。材料并没有错，只是它们不一定在回答你此刻的问题。

所以 sanity check 的第三步，是提醒自己别让材料和问题错位。材料选对，项目理解会快很多。

30.4 第四个 sanity check：我现在需要的是“继续深入”，还是“先做最小验证”

很多新人一陷入复杂感，就会本能地继续读、继续看、继续搜，仿佛多收集一些信息就会自然解决问题。其实有时候更好的动作恰恰是停下来，做一个最小验证。比如先跑一个 `smoke`；先打开一个真实 `batch_results.json`；先 `tail` 一下 `stdout.log`；先 `readlink -f` 看看软链接真实落点。你会发现，很多抽象困惑一旦遇到具体证据，就会立刻缩小。

所以每次被复杂感压住时，也许你不是真的需要更多理论，而是需要一个小而真的验证动作。

30.5 第五个 sanity check：我这一步有没有留下后来能接住的痕迹

这是最像工程师的小检查。你这一步做完以后，是不是别人能接住？有没有 `run` 目录、命令记录、日志、结果文件、简短说明？还是说这件事只存在于你脑子和终端滚动文本里？如果是后者，那它对长期推进的价值会大打折扣。当前工作区之所以不断强调 `run` 规范、文档、结果摘要，说到底都是在训练这个习惯。

真正能持续推进的项目，不是每一步都巨大，而是每一步都尽量留下一点后来者接得住的痕迹。

31、第一次给导师或师兄做阶段汇报时，最值得避开的五个表达错误

最后我再补一个特别像带学生时会忍不住讲的东西。很多同学不是没做事，而是一汇报就把自己做过的努力讲散了。对当前项目来说，尤其容易犯几类表达错误。你早点知道，后面会轻松很多。

31.1 错误一：只说“我跑了很多”，不说“我现在把哪条链路讲清了”

“跑了很多”听起来很勤奋，但信息密度很低。更成熟的讲法应该是：我现在已经把哪条链路跑通、讲顺、结构化沉淀下来了。比如“我已经把 Splat-Plan 的 `smoke` 到 `batch` 主链和 `run` 目录结果结构讲清楚了”。这会比“我跑了不少实验”显得更扎实。

31.2 错误二：把 wrapper 成果讲成算法创新，或者把算法理解讲成工程整理

这两种错位都很常见。你如果在讲 `wrapper` 层成果，就坦率讲工程组织、可复现性、日志规范和交接价值；如果在讲算法层理解，就讲上游主轴、场景入口和规划主链。不要把两层混着邀功，也不要把其中一层的价值说轻。层次清楚，表达就会非常稳。

31.3 错误三：只展示一个好看的结果，不交代边界和下一步

一个好看的图当然很重要，但如果你不说明它是在什么边界下做出的、当前还没做到哪、下一步准备怎么推进，汇报就容易显得虚。对当前项目更好的说法通常是：我们现在已把离线/仿真链路做到什么程度，边界仍然在哪，比如还没碰 ROS/实车，下一步准备补哪层材料或哪条执行链。这样汇报会显得非常成熟。

31.4 错误四：一紧张就开始堆术语，不再说人话

这几乎是所有新人都会犯的错。其实导师和师兄大多数时候并不需要你一口气堆十几个术语，他们更想知道你是不是真的知道这些术语在当前项目里各指什么。所以你越紧张，越要提醒自己把话说成人话。比如不是只说“GSplat 和 polytope corridor”，而是补一句“也就是场景怎么进来、安全区域怎么被逐段构造出来”。这会立刻让你的表达变得可信很多。

31.5 错误五：把“还没完全懂”误表达成“我现在什么都不太行”

这也是特别可惜的一种表达。复杂项目里，“还没全懂”是常态，不等于“我什么都不行”。更成熟的说法是：当前我已经把哪些层讲清了，哪些层还在补，卡点目前在哪一层。这样的表达既诚实，又不会把自己说得很虚。它会让对方更容易看见你的真实进展，也更容易给你有针对性的帮助。

如果你把这五个表达错误提前避开，后面无论是和师兄沟通，还是正式做阶段汇报，都会稳很多。这也是为什么我一直希望第六编和第四编不只是资料，而是能真正帮学弟长出“会看、会讲、会接”的底子。

32、这两份文档不是一次性材料，而是会随着项目推进反复回来的“底图”

最后我想再强调一件事。第六编和第四编并不是那种“入门时看一遍，以后就再也不碰”的一次性材料。更好的理解是：它们是当前项目的两张底图。第六编更像词的底图，帮你在术语海里不迷路；第四编更像结构的底图，帮你在论文、代码、目录、结果和未来需求之间始终看见主线。只要项目还在推进，这两张底图就会不断被回看、被重新理解、被补上新经验。

32.1 当你第一次带学弟时，它们是引路材料

第一次带学弟时，它们最主要的作用是引路。也就是说，让后来者别一上来就被术语和目录打散，而是先有一张能站稳脚跟的地图。这个阶段，文档价值主要体现在“降低陌生感”和“减少乱试错”。

32.2 当你开始稳定做实验时，它们会变成复位材料

一旦你开始连续做实验、隔几天回来继续推进，这两份文档的价值会变成复位。你会重新看 scene、看 wrapper、看 run 目录、看上游主轴，把自己从短暂停顿里拉回主线。这个时候你会发现，好的文档并不是只在第一次打开时有用，而是在项目一遍遍重启时都能把人接住。

32.3 当你开始思考创新、专利和论文时，它们又会变成边界材料

到了更后面，这两份文档会帮助你守住边界。你会知道哪些是现有系统已稳的部分，哪些是仍然薄弱的部分，哪些观察更像工程整理，哪些问题有机会长成研究问题。换句话说，它们会开始帮你判断“从哪往外长”才更自然，而不是让创新冲动把项目主线冲散。

32.4 当你以后真的进入更复杂的工程环境时，这套思路也不会白费

就算未来你离开这个具体项目，这两份文档里真正值钱的也不只是 Splat-Nav 的局部知识，而是那种面对复杂系统时的处理方式：先分层、先定边界、先认入口、先抓主轴、先建立最小可跑链、再让结果会说话、最后让后来者能接手。你如果把这套思路练熟，它会远远超出当前项目本身。

32.5 所以这轮重写真正想做的，不是“再写厚一点”，而是“把路修出来”

如果一定要给这轮工作下一个最准确的定义，我会愿意把它叫作“修路”。不是简单扩字数，不是机械堆知识点，而是把原本零散、跳跃、容易让新手读不进去的内容，重新铺成一条能让人一步一步走下去的路。只

要这条路开始存在，这轮文档重写就已经非常值了。因为复杂项目里，真正稀缺的从来不只是知识本身，而是通向知识的路。

33、读完整个第四编以后，最值得带走的五句话

最后的最后，我不想再继续铺新信息了。我更想帮你把前面这么长的一份文档，压成五句真正值得带走的话。因为很多时候，一个人不是缺更多材料，而是缺几句在关键时刻能把自己重新拉回主线的话。下面这五句话，如果你真的记住，以后无论是读代码、跑任务、带学弟，还是和导师沟通，都会很有用。

33.1 第一句：这个项目首先是“在 GSplat 场景表示上做导航”，不是单纯的三维重建项目

只要这句话还在，你就不会一上来把项目理解歪。你会始终知道，三维场景表示在这里是基础设施，真正要完成的系统目标是规划和定位两条导航主线。这个认知会帮你在读论文、看 README、看上游源码和看当前工作区时都保持方向感。

33.2 第二句：当前 `/home/howard/Splat-Nav` 是工程化工作区，不是上游论文本体的替身

只要这句话还在，你就不会再把 wrapper 和上游混成一层。你会自然知道：想看算法事实，去上游；想看当前机器怎么稳稳跑、怎么留结果、怎么带人，回到 wrapper 工作区。层次一清，很多焦虑都会自己掉下来。

33.3 第三句：任何复杂感上来时，先分层，先定位，先做最小验证

这句几乎可以当成整个项目推进的心法。复杂不是问题本身，混层才是。你只要先问“这是哪一层的问题”，再做一个小而真的验证动作，比如看真实路径、看真实日志、看真实 JSON，事情往往就会开始变清楚。你以后越早把这句话变成习惯，越不容易被复杂项目吓住。

33.4 第四句：结果不是跑出来就完了，而是要做成后来者接得住的形状

这句话会直接决定你是不是在做长期可推进的项目。run 目录、日志、图、摘要、汇总，不是形式主义，而是在把一次运行变成可回看、可比较、可交接的证据。只要你还记得这句话，项目就不太容易重新滑回“只有跑的人自己知道发生了什么”的状态。

33.5 第五句：真正的目标不是一下子全懂，而是让自己能稳定地继续往前走

这大概是最该送给学弟、也最该送给你自己的最后一句话。复杂系统本来就不可能第一次全懂。真正成熟的状态，是今天比昨天更不迷路一点，知道自己还没懂的是哪一层，知道下一步该接着读哪、看哪、跑哪。只要这种稳定往前走的节奏长出来，项目就已经真正开始被你接住了。

如果你把这五句话带走，那第四编 就不仅仅是一份长文档了。它会开始像一张在你脑子里活起来的地图，在你每次重新进入项目时，重新把你带回主线。

第五编 把长期节奏放稳

做项目不能只盯今天。这里把一周、一个月、三个月分别该做到什么、怎么看自己是不是在正路上，完整收拢到一起。

这一份文档不是拿来把人绑死在日程表上的，而是拿来帮助你判断：我现在处在什么阶段，这个阶段最重要的目标是什么，哪些事还不该急，哪些事再拖就会空。很多新人最容易陷入的状态，不是偷懒，而是把完全

不同层次的目标同时背在身上。今天想把环境配完，明天想读懂论文，后天想改代码，再后天又想出创新点。最后看起来每天都很忙，但一个稳定台阶都没踩实。

所以这里不用“每天几点到几点做什么”的方式来写，而用阶段目标来写。你可以在每个阶段内自行安排节奏，但必须知道这一阶段最基本的验收口径是什么。因为自由不等于没有边界。对零基础学弟来说，真正有效的自由，是在明确的阶段目标内部自由推进，而不是完全凭感觉飘。

一、先把阶段目标的总原则说清楚

阶段目标最怕的是一上来就混层。所以这一部分先总原则立住：当前阶段先复现、先汇报、先形成闭环，而不是一开始就把创新焦虑压到新人身上。

1、先说总原则：当前阶段先复现与汇报，不先卷创新

你未来当然要思考专利、创新点和会议论文，但这些目标必须建立在“主链路已经真的看懂、跑通、讲清楚”的基础上。一个对输入输出还说不清楚、对 run 目录和证据还不敏感的人，太早进入创新讨论，往往会很虚。不是能力不够，而是地基不稳。

所以当前最合理的路线，是先把环境、工作区、smoke、batch、结果阅读和汇报能力扎实做起来。只有当这些动作形成稳定闭环后，你再进入专利、创新点和论文预研，才会更像真实推进，而不是概念上的兴奋。

二、第一层台阶，一周内先做到不再发懵

一周目标的核心不是“全懂”，而是从完全陌生走到开始能接手。这一部分会把一周阶段的状态、证据、假进展和保底路径都讲细。

1、第一阶段：一周内要达到什么状态

一周目标不是“完全掌握 Splat-Nav”，这句话太大，也太空。一周的合理目标，是完成从陌生到能接手的最小跃迁。也就是说，你应该从“看到工作区发懵”进化到“知道项目是什么、该从哪看起、能做最小检查、能解释一轮最小运行结果”。

1.1 一周阶段的目标状态

到一周左右，你应该能稳定通过 SSH 和 Cursor 进入工作区，知道 `/home/howard/Splat-Nav` 的三层结构是什么，知道 `scripts/` 是 wrapper 层、`splatnav-official/` 是上游源码入口、`runs/` 是结果沉淀层。你应该至少读过前四编，并对术语附录和项目全景这两块有基本印象，对 `config.yml`、`Splat-Plan`、`run_dir`、`feasible`、`smoke` 这些高频词建立起初步认知。

更进一步，你应该已经做过一次最小环境检查，最好还能完成一次 smoke，至少能读懂一份已有的 smoke run 目录。你不一定已经熟练，但你不能再停留在“只知道目录里有很多文件”的状态。你要能用自己的话说出：这个项目现在主要在复现什么，我第一次应该跑哪条链路，跑完以后先看哪些文件。

1.2 一周阶段的证据要求

这一周真正的证据，不是你说自己“差不多看懂了”，而是你手里至少有一份你自己能解释的 run 目录，或者你能对着现有 run 目录做出可信的口头说明。你还应该能拿出一份简短但结构清楚的汇报：项目做什么、当前工作区由哪三层组成、最小运行输入是什么、最小输出在哪里看、自己目前卡在哪。

如果你能做到这些，就说明你已经不是门外人了。你可能还不会改算法，但你已经有能力进入项目内部继续推进。

1.3 一周阶段最容易出现的假进展

第一种假进展，是文档看了很多，但一条最小命令都没真正跑过，或者连 `run` 目录都没认真看过。第二种假进展，是命令跑过了，但自己并不知道跑出来的 JSON 和日志是什么意思。第三种假进展，是花很多时间在查各种外部知识，却没有回到当前工作区这条主线。第四种假进展，是已经被论文里的高级词吓住，于是一直在“准备准备再开始”，最后迟迟不进入真正的最小动作。

学会识别这些假进展，你就知道自己是不是在真正前进。

1.4 一周阶段如果进度慢，怎么保底

如果一周内你还没跑通 `smoke`，也不要慌。最稳的保底策略是先确保三件事：SSH 和工具链已打通；新文档主线已经读明白；至少能跟着师兄现有结果目录解释一遍输入输出和验收逻辑。也就是说，即使自己暂时还没独立跑成，也要先把“看懂”和“复述”做扎实。这个保底状态，依然比漫无目的地乱试要有价值。

1.5 一周阶段如果进度快，可以自然往哪延展

如果你一周内不仅看懂了主线，还能独立跑出一次 `smoke`，并且会读 JSON 和日志，那么下一步最自然的延展不是立刻造新算法，而是进入 `batch` 的最小版本，开始形成统计视角。也就是说，你可以开始对比不同方法的可行率和耗时，而不是只停留在“某一条路径跑通了”。

三、第二层台阶，一个月内形成标准执行链

到了一个月，目标就不能只停留在认识项目，而要开始稳定地执行、稳定地读结果、稳定地对外表达。这里会把这个阶段最该盯的能力形态单独讲开。

1、第二阶段：一月内要达到什么状态

一个月阶段，目标就不能只停留在“知道是什么”。这个阶段更合理的定位，是“能够稳定接项目、能独立完成一条标准执行链、能围绕结果做正式汇报”。换句话说，你应该从“新人入门”进入“可以被放心分配具体任务”的状态。

1.1 一月阶段的目标状态

到一个月左右，你应该已经比较稳地掌握当前工作区的主线文档，并能独立在离线/仿真范围内完成环境检查、一次 `smoke`、一次小规模 `batch`，以及对结果目录的基本复盘。你对 `scripts/` 中的核心 wrapper 脚本已经有清晰认识，知道 `run_splatplan_smoke.sh`、`run_official_style_batch.py`、`run_in_tmux.sh`、`visualize_batch_results.py` 分别负责什么。

更重要的是，你应该开始能把项目讲给别人听。这里的“别人”，不是指你讲得多华丽，而是指你能用大白话给组里没那么熟悉这套代码的人解释：这个项目为什么要用 `GSplat`，为什么当前先做 `Splat-Plan` 离线复现，为什么 wrapper 层和上游层要分开，为什么这次结果可以算成功或不算成功。

1.2 一月阶段的证据要求

一个月阶段的证据，至少应该包括几类。第一，你应该有自己独立完成的标准 `run` 目录，不止一次最小 `smoke`，而是至少一轮小规模 `batch`。第二，你应该能拿出一份正式一点的汇报材料，哪怕只是

Markdown 或 PPT 草稿，也要能讲清主链路、实验设置、结果概览和当前问题。第三，你应该能对一到两个核心上游文件做口头走读，比如 `run_splatplan.py` 和 `splatplan/splatplan.py`，说明它们在全局链路里的位置。

1.3 一月阶段最容易出现的假进展

一个月阶段最典型的假进展，是“实验跑了不少，但没有形成可讲述的结构”。比如你可能有很多图、很多目录，但别人一问“哪个脚本做了什么”“为什么这次方法 A 比方法 B 好”，你答不上来。另一种假进展，是开始急着动大重构、动核心算法，但对原链路的输入输出和验收还没有稳定控制。这样做往往会让自己既失去基线，又增加排错难度。

1.4 一月阶段如果进度慢，怎么保底

如果一个月下来你还没有稳定 batch，不要盲目加压。更稳的做法是先确保：至少有一条标准 smoke 链路完全掌握，至少能读懂并复述一份已有 batch 结果，至少能拿出现有工作区的结构化介绍和汇报提纲。也就是说，哪怕执行深度还不够，表达和理解深度也要先补上。因为在科研和工程里，会讲清楚本身就是非常重要的能力。

1.5 一月阶段如果进度快，可以自然往哪延展

如果一个月内你已经能稳定跑小规模 batch、读懂结果并做汇报，那么最自然的下一步是两条线并行推进。一条线是更深入的上游源码阅读，尤其是 `SplatPlan`、`SafeFlightCorridor`、`GSplatLoader` 和样条优化器相关代码。另一条线是围绕五一前后准备可视化、可汇报材料，开始为专利草案和创新观察积累素材。注意，这里说的是“积累素材”，不是立刻硬写创新点。

四、第三层台阶，三个月内开始形成自己的判断

三个月阶段不再只是“我会做”，而要逐渐走向“我会判断、会组织、会提出问题”。这一部分负责把这种能力升级说清楚。

1、第三阶段：三个月内要达到什么状态

三个月阶段，才适合谈“逐渐进入研究生式推进”。这个时候，如果前面两级台阶都踩稳了，你就不应再只是一个会跑命令的人，而应该开始具备“理解、执行、表达、改进”四件事并行的能力。

1.1 三个月阶段的目标状态

到三个月左右，你理想中的状态应该是：对当前工作区非常熟，能够独立完成离线/仿真主线的标准执行和复盘；能读懂并讲解核心链路代码；能把一段时间内的实验结果组织成结构清楚的汇报；并且开始对项目可能的工程改进点、实验补强点或研究切入点形成自己的判断。

这里的“自己的判断”不要求你立刻写出顶会论文，但至少应该体现在：你不再只是照着做，而是开始思考哪些地方的实验设计可以更稳，哪些代码入口更值得重构，哪些结果呈现更有说服力，哪些限制条件未来需要补实验或补系统。

1.2 三个月阶段的证据要求

三个月阶段的证据，应该比前两个阶段更成体系。你不仅要有 run 目录和图表，还要有能支撑阶段汇报或专利预研的材料，比如结构化周报、阶段总结、对某个核心模块的代码解读、对某类方法差异的实验观察，甚

至一份初步的创新问题清单。也就是说，证据不再只是“我跑了”，还包括“我观察到了什么、我如何解释、我认为哪里值得继续做”。

1.3 三个月阶段最容易出现的假进展

这个阶段最大的假进展，是开始频繁谈创新、谈发论文、谈大改架构，但没有稳定可复现的基线结果，也没有严谨的实验和代码证据支持。另一种假进展，是陷入无休止的读论文和搜资料，却迟迟不把自己的判断落回当前工作区。说得更直接一点，三个月阶段的风险，不再是“什么都不会”，而是“容易开始飘”。

1.4 三个月阶段如果进度慢，怎么保底

如果三个月下来你还没真正进入创新探索，也不必自责。更稳的保底是：把当前工作区的离线/仿真主线做成可长期复用的稳定资产。也就是说，环境稳、脚本稳、结果稳、汇报稳、文档稳。一个项目真正的研究推进，从来都不只靠新点子，也靠稳定基线和清晰证据。你把这些做扎实，本身就是很有价值的工作。

五、现实节点、带教策略与长期节奏

除了抽象的阶段划分，项目推进还会遇到现实节点和带教问题。这一部分把五一节点、学弟进度慢时的处理方式，以及带教时该看什么放到一起。

1、五一前后的现实节点应该怎么看

你们当前谈到的一个现实节点，是 2026 年 5 月 1 日前后。这种时间节点很有用，但不能理解成“到那天必须把所有东西做完”。更合理的理解是：把它当成一个阶段汇报节点。到那个时间附近，最理想的状态不是“已经投出论文”，而是你们手里已经有比较像样的可视化、可汇报结果，以及对当前工作区和主链路的稳定理解。

如果能在五一前后做到这一点，就已经很不错了。后续围绕专利、创新点和会议论文的推进，才会有真实地基。

2、师兄带学弟时，最值得盯住的不是进度，而是能力形态

如果你未来要带这个学弟，你最值得观察的，未必是他某一天是不是看了多少页文档，而是他是不是慢慢形成了几种能力形态。第一，遇到问题会不会先确认上下文，而不是乱试。第二，跑完任务会不会主动找 run 目录证据，而不是只看终端最后一行。第三，说不懂的时候，能不能明确说出自己到底卡在术语、路径、环境、代码还是结果解释。第四，汇报时能不能用自己的话讲清楚，而不是只复述文档原句。

这些能力形态一旦长出来，后面的成长会越来越快。反过来，如果这些习惯一直没形成，就算偶尔跑通一轮实验，也很难真正接住项目。

3、最后给学弟的一句话

你不需要在第一周就成为算法高手，也不需要一开始就把整个 Splat-Nav 论文和代码全部吃透。你真正需要做的，是一层一层把台阶踩实。先把门打开，先把地图看清，先做最小动作，先留下证据，先讲明白眼前这一段。把这几件事做好，你就已经在一条非常稳的路上了。

当你愿意这样推进时，这个项目不会再是“看不懂的大工程”，而会变成一个你可以逐步接管、逐步复现、逐步讲清楚、最后逐步做出自己工作的真实项目。

4、阶段目标真正要防的，不是“慢”，而是“混”

很多新人一听到阶段目标，会本能地把注意力放在时间上，担心自己是不是太慢、是不是跟不上、是不是别人一个星期做到的自己要两个星期。这个焦虑可以理解，但如果只盯着快慢，很容易把真正的风险看偏。对当前项目来说，阶段目标更应该防的，其实不是慢，而是混。也就是说，你可能看起来动作很多，但阶段边界混了，目标层次混了，评价标准也混了。

比如一周目标本来应该是先把项目地图、最小动作和结果证据站稳，可你却把自己逼进“我是不是应该马上想创新”；或者一个月目标本来应该是能稳定 batch、能看懂结果、能做汇报，可你却还停留在只会看终端最后一行；又或者三个月本来该开始形成研究判断，你却还在反复重新适应环境和路径。这种混乱比慢更伤。因为慢往往还能积累，混则很容易把力气白白耗掉。

所以学弟以后判断自己状态时，先别急着给自己贴“快”或“慢”的标签。更值得问的是：我现在做的事，和我所在阶段真正该做的事，对得上吗？只要这件事对得上，慢一点其实也还是在稳步前进；但如果对不上，再快也可能是在空转。

5、一周目标再细化一点：真正该长出来的是哪几种基础能力

前面已经说了一周目标的总体状态，但如果要把它再讲得更落地，其实可以把它拆成几种基础能力，而不是只看你碰到了哪些文件。第一种能力，是上下文定位能力。也就是你能不能快速确认自己现在在哪台机器、哪个账户、哪个工作区、哪套环境里。第二种能力，是最小阅读能力。也就是你能不能顺着“先读第一编，再借助第六编查词，随后进入第四编”把项目总貌和最常见术语先站稳。第三种能力，是最小动作能力。也就是你能不能在第二编的引导下做出环境检查或一次最小 smoke。第四种能力，是最小解释能力。也就是你能不能对着一个 run 目录，把输入、输出和验收讲出个大概。

你会发现，这四种能力都不夸张，但非常硬。因为它们共同构成了“这个人开始能接项目了”的最小形状。如果一周结束时，这四种能力至少已经有了雏形，那就说明这周很值。反过来，如果表面上看了很多内容，可这四种能力都还是空的，那就说明这一周很可能只是热闹。

6、第一周给学弟最合理的心理预期，不是“我要全懂”，而是“我要先不再发懵”

这句话我特别想单独拎出来说。因为零基础学弟最容易给自己一个过重的心理包袱，仿佛自己第一周就应该把整个 Splat-Nav 论文、工作区、脚本、结果和代码逻辑全啃透。这个目标听上去认真，其实不现实，也不必要。第一周更合理、也更健康的目标，是先不再发懵。也就是说，你看到工作区时不再完全没有方向，看到 run 目录时不再完全无从下手，听到 config.yml、feasible、batch、GSplatLoader 这些词时不再完全发怵。

不要小看这种“先不发懵”的状态。它其实是很关键的一道坎。因为一旦人从“完全混沌”进入“虽然还不熟，但开始能分出层次”，学习速度会明显加快。很多人真正的突破，不是突然聪明了，而是从这道坎跨过去了。

7、一月目标再细化一点：为什么这个阶段最重要的是“标准执行链”和“稳定表达”

一个月阶段之所以比一周阶段更难，不是因为内容单纯变多了，而是因为要求开始从“我知道一点”转向“我能稳定地做出来，并让别人听懂”。这就是为什么我会特别强调标准执行链和稳定表达。标准执行链的意思是，你不再只靠偶然状态跑出一轮结果，而是对环境、脚本、run 目录、结果阅读和基础复盘已经形

成相对稳定的动作顺序。稳定表达的意思则是，你可以比较平静地把项目主线、当前工作区三层结构、一次 batch 的输入输出和自己当前卡点讲给别人听，而不是每次一开口就散。

这两件事看起来一个偏执行，一个偏表达，但其实是同一件事的两面。执行不稳，表达就会虚；表达不稳，执行往往也说明还没真正吃透。所以一个月阶段最值得盯的，不是今天多跑了一轮还是少跑了一轮，而是你的标准执行链是不是开始成形，你的表达是不是开始不那么飘了。

8、一月阶段最适合开始补的，不是“全部源码”，而是“核心链路源码”

到了一个月左右，很多人会开始认真问：那我是不是现在该系统读源码了？答案是应该开始读，但读法很关键。更适合当前阶段的方式，不是立刻把整个 `splatnav-official/` 每个目录都一口气吃掉，而是围绕最常用、最贴近当前执行链的部分去读。比如 `run_splatplan.py`、`splatplan/splatplan.py`、`splat/splat_utils.py`、`SFC/corridor_utils.py` 这些核心链路文件，往往就比一上来横扫所有目录更有效。

原因很简单。你现在不是在做纯源码欣赏，而是在让“我刚刚做过的动作”和“上游实现到底在哪里”对上号。只要这条对号线开始形成，源码阅读就会很有成就感。否则你很容易读了很多文件，却始终找不到自己亲手运行过的那些结果到底从哪长出来。

9、一月阶段最值得开始训练的一种能力，是“拿结果做正式汇报”

很多新人会把汇报理解成最后阶段才需要的东西，仿佛等项目做得差不多了，再去想怎么讲。其实对当前项目来说，一个月左右恰恰是开始训练正式汇报能力的好时候。因为到了这个阶段，你通常已经不再是完全空手，已经有了一些 smoke 或 batch 结果、一些 run 目录、一些日志和图表。也就是说，你手里已经开始有材料，而汇报能力本质上就是把这些材料组织成别人听得懂、能接得住的结构。

这类训练越早开始越好。因为它会逼着你把很多含糊的理解说实。比如你如果讲不清 `wrapper` 和上游的关系，那说明这层你还没真稳；如果你讲不清为什么某一轮 batch 不能算完全成功，那说明你对验收还不够敏感。换句话说，汇报不是展示之外的额外工作，它本身就是一种非常强的理解训练。

10、三个月目标真正开始值钱的地方，在于“你开始形成自己的判断，而不是只沿着别人的句子走”

三个月阶段和前面最大的不同，不只是内容更多，而是你和项目的关系开始变了。前面两个阶段，你更多还是在学习已有路线、已有结构、已有证据。到了三个月，真正开始值钱的，是你能不能在已有材料上形成自己的判断。这个判断不一定立刻是创新点，但它至少应该体现为：你开始知道哪些实验值得补，哪些代码值得整理，哪些结果值得重点讲，哪些现象看起来可疑、值得再验证。

这里的关键不在于你是不是一下子提出了多厉害的新方法，而在于你不再只是沿着别人的句子走。你开始能说“我观察到”“我怀疑”“我认为这里更值得继续补”。这种判断力是研究训练真正开始发力的地方。

11、三个月阶段最自然的两条延展线：工程化沉淀和研究化观察

如果三个月前面的台阶都踩稳了，后面最自然的延展通常有两条。第一条是工程化沉淀，也就是把当前已经跑顺的离线/仿真主线继续做得更可交接、更可复用、更符合规范。比如继续整理脚本、规范结果、压实文档、补注释、统一入口。第二条则是研究化观察，也就是开始围绕现有结果产生问题意识：不同方法为什么差异是这样，哪些场景更难，哪类约束更敏感，哪些地方值得做更系统的对比。

这两条线并不是冲突关系。恰恰相反，它们互相支撑。工程化沉淀会让你的研究观察站在更稳的基线上，而研究化观察会让你的工程整理不至于变成只讲规范、不碰问题的空工程。对你未来想往大厂规范和科研能力两边一起练，这种双线推进其实非常合适。

12、如果师弟一个月后还比较慢，带教的人最该做的不是催，而是帮他重新找到“最小闭环”

现实里完全可能发生这样一种情况：一个月过去了，学弟并没有像理想中那样顺滑地进入 batch 和汇报，而是还在一些基本问题上来回打转。这个时候，带教的人最不该做的往往是单纯催进度。因为很多慢并不是态度问题，而是最小闭环还没真正长出来。你催他往前冲，他只会更容易乱。

更有效的帮助方式，通常是带他重新找到最小闭环。也就是重新确认工具链、工作区、主线文档、最小 smoke、run 目录和解释能力这几个点里，到底哪一个还没站稳。只要把那个最松的环重新扣上，很多后续问题会自己缓解。对零基础转码的人来说，这种“帮他重新找回闭环”的支持，比一味催更有用。

13、五一节点最值得产出的，不一定是“结题”，而是“可汇报的阶段成果包”

前面提到了 2026 年 5 月 1 日前后的现实时间节点。这里我想再把它说得更具体一点。对当前项目更现实、更有价值的目标，不一定是到那时“所有东西都做完”，而是能拿出一个像样的阶段成果包。所谓阶段成果包，至少应该包括：一套能讲得清的项目介绍、一份对当前工作区与主链路的稳定说明、一到几轮可复盘的结果目录、几张关键图或汇总表、以及一份能在组会或内部沟通里说得过去的汇报材料。

为什么我特别强调“成果包”这个概念？因为它比“我做了很多事”更有形，也更利于后续接着推进。你们到五一前后如果手里真的有这样一包东西，无论是继续打磨专利、继续补实验，还是让学弟接着做，都会容易很多。

14、专利、创新点和会议论文，应该在哪个阶段开始以什么强度进入

这件事非常容易让人焦虑，所以值得说细一点。专利、创新点和会议论文当然是最终方向，但它们进入主线的强度，不应该从第一周就拉满。更合理的节奏通常是这样的：第一周到一个月，主要还是把主链路、输入输出、结果证据和汇报能力做稳。到了一个月后，如果基础已经开始成形，就可以低强度地开始“观察式进入”专利和创新，也就是先积累问题、积累现象、积累能主张的工程与系统价值。到了三个月，如果前面基线和表达都已经稳了，再逐渐把这种观察压成更明确的创新方向或专利表述。

这样做的好处是，不会让创新讨论变成空中楼阁。很多新人一开始一谈创新就发虚，本质上就是因为手里没有足够稳定的事实和结果支撑。你让创新晚一点、但稳一点进入，反而更容易长出真东西。

15、第五编 最适合怎么用：不是偶尔翻一眼，而是每隔几天回来重新定位一次

如果说第二编更像行动路线，那么第五编更像阶段罗盘。罗盘的用法不是一直盯着它不走，而是走一段回来校准一次。所以这一部分最推荐的使用方式，不是第一次看完就扔一边，而是每隔几天、每完成一轮稍微像样的动作之后，再回来重新定位一下：我现在其实更像一周阶段，还是已经摸到一个月阶段的门口了？我现在最缺的是动作闭环、结果阅读、表达能力，还是开始形成自己的判断？

你只要真的这么用两三次，就会明显感觉第五编的价值。它不会替你工作，但它会防止你在不同阶段之间乱跳。很多项目推进的痛苦，都不是因为事情太多，而是因为人太容易在不同层次目标之间来回摇摆。

六、最后把阶段判断习惯真正留下来

最后这部分不再继续铺任务，而是把整份文档压回一种习惯：会判断自己当前在哪个阶段、该补哪一层、怎样避免看起来很忙却没有沉淀。

1、读完整个第五编后，最值得留下来的，不是一句“我要努力”，而是一个更稳的阶段判断习惯

最后我想把这一部分压回一句最实在的话。它真正想留下来的，不是鸡血，也不是口号，而是一种更稳的阶段判断习惯。你开始知道，不同时间尺度下，什么叫合理目标，什么又只是看起来很忙；你开始知道，遇到慢的时候该怎么保底，进度快的时候又该怎么自然延展；你也开始知道，带一个零基础学弟，真正该盯的不是表面速度，而是他是不是慢慢长出了能接住项目的的能力形态。

只要这种阶段判断习惯开始长出来，第五编 就已经值了。因为复杂项目从来不是靠一天冲刺做出来的，而是靠一段一段踩稳台阶走出来的。台阶不怕高，怕的是踩乱。只要你们一直在帮学弟把台阶踩稳，这个项目就会越做越顺。

第六编 术语附录与查词索引

这一编放在最后，不是因为它不重要，而是因为它更像一本随时回查的词典。前面正文看不懂的词，回到这里查，最省力。

如果你是第一次认真接触一个像 Splat-Nav 这样的项目，很容易在第一天就被术语打懵。不是因为这些词每个都特别难，而是因为它们来自不同世界：有的是 Linux 世界的词，有的是 Python 环境世界的词，有的是三维重建世界的词，有的是导航规划世界的词，还有的是实验记录和工程协作世界的词。它们像几股水一下子冲进来，如果没有人帮你拢一下，你脑子里就会出现一种很典型的感受：每个词都好像听过一点，但一放在项目里就不知道到底在说什么。

这一份文档就是来做这件事的。我们不追求把你训练成“术语答题机器”，也不打算上来就拿定义压人。更实际的目标是：把你在当前工作区里一定会碰到、一定会反复看到、如果不懂就会卡住的词，一个一个放回真实语境里讲明白。你会看到我反复问同一个问题：这个词在当前项目里到底指什么，它为什么重要，它和哪些词最容易混，它一般会出现在什么路径、什么脚本、什么日志、什么结果里。

你完全不需要在第一次阅读时把所有词死记硬背。真正有效的用法，是把它当成一本贴着项目现场的“词典型陪跑文档”。你在读 第四编 时，读到不熟的词就回来查；你在跑 `smoke`、看 `stdout.log`、打开 `batch_results.json` 的时候，碰到拿不准的概念，也回来查。这个节奏是对的。没有必要把自己逼成一天内背完几十页名词的人。

一、先知道这本词典应该怎么用

这一部分先不急着想术语本身，而是先把使用方法讲明白。因为真正有效的词典，从来不是背完的，而是在项目推进中反复回查、反复加深的。

1、你该怎么用这份词典，而不是怎么“背完”它

先把一个误区拆掉。很多同学拿到这类文档，会很自然地想按顺序一页一页往下刷，像背单词书一样把它背完。这种想法听起来认真，其实不太适合当前项目。因为项目不是单向输入，而是边看、边跑、边回查、边

纠正理解的过程。你今天可能先碰到 `SSH`、`tmux`、`Conda`，明天再碰到 `config.yml`、`GSplatLoader`、`feasible`，下周才会认真面对 `polytope`、`Bezier`、`P95` 这些词。理解的顺序，本来就不是绝对线性的。

所以更有效的读法，是把这一部分看成一张“术语地图”。地图不是为了让你把每条街一次背熟，而是为了让你不迷路。你今天在某一块区域活动，就重点看那一块。你明天进入下一块，再回过头把前面的词和后面的词连起来。慢一点完全没关系，只要每次回来看时，你能比上一次多懂一点，就已经在前进。

你还会发现，这份词典虽然叫“术语全解”，但其实不是纯词条堆砌。我会尽量让相邻词之间有上下文关系。比如讲完 `SSH`，我会接着讲 `Shell`、`Terminal`、绝对路径、相对路径，因为它们本来就经常一起出现；讲完 `Conda`，我会接着讲 `pip`、`Python` 解释器、环境变量、`LD_LIBRARY_PATH`，因为这些词在排错时几乎是连着来的。这样读起来会轻松不少，也更像有人在旁边带你，而不是让你一个人在词海里扑腾。

二、进入工作区前必须站稳的系统词汇

这一部分聚焦你一上机器就会碰到的词：`Linux`、路径、权限、日志、终端、搜索。这些词听起来基础，但它们决定了你后面是不是会一直脚下发虚。

1、你坐到工位上，第一批最先会撞到的词

1.1 Linux 到底是什么，为什么实验老爱在它上面做

不少跨专业同学第一反应会觉得，自己以前也写过点代码，为什么一到实验室项目就突然都是 `Linux`。这里先别把它想得太高深。你完全可以先把 `Linux` 理解成当前这台机器的工作语言。它不是为了显得专业才出现，而是因为很多科研环境、GPU 工位机、远程服务器、开源工具链、自动化脚本和工程部署流程都在这个生态里更顺手、更稳定。

你眼前这台机器上，项目工作区在 `/home/howard/Splat-Nav`，数据和结果很多又放在 `/data/howard/...`。这种目录结构、权限方式、软链接机制、环境变量和远程登录方式，本来就是 `Linux` 世界的日常。你只要真在这个项目上待一周，就会发现自己并不是在“学一门陌生操作系统”，而是在适应一个默认规则不同的工作环境。

真正会让新手焦虑的，不是 `Linux` 本身，而是刚上来时看到很多陌生命令。这个焦虑很正常。你要做的不是假装自己全懂，而是先接受一个事实：你现在不是来当系统运维的，你只是需要掌握一套足够让自己稳定工作、稳定排错、稳定读结果的基本动作。只要把这件事想清楚，压力会小很多。

1.2 SSH：你和这台机器建立关系的第一根线

`SSH` 可以先很朴素地理解成“远程上机器”的标准方式。你人不在工控机前，但项目和显卡在那台机器上，于是你需要通过网络把自己接进去。很多新同学第一次看到 `ssh howard@<ip>` 会莫名紧张，仿佛自己在做一件很危险的事。其实大可不必。它本质上只是告诉系统：我现在想以这个用户身份，进入这台远程主机。

对当前项目来说，`SSH` 的重要性不是抽象的，而是具体得不能再具体：如果它没通，你连工作区都进不来；如果它通了但你没意识到自己现在已经在远端，你后面看路径、跑脚本、查日志时就会一直混淆“这是我电脑上的东西，还是服务器上的东西”。所以你以后只要一开始工作，脑子里最好先确认一句：我现在的这个终端，到底连到了哪里。

一个非常小、但非常值得反复做的确认动作是：连接上去之后，先打 `pwd` 和 `hostname`。前者告诉你当前目录，后者告诉你当前机器名。听起来土，但极其有效。很多“我明明跑了怎么结果不见了”的事故，追到最后都只是因为人其实跑在了错误的地方。

1.3 Terminal 和 Shell：别把窗口和语言混成一个东西

很多人嘴里会把“终端”“shell”“命令行”混着说，日常聊天没问题，但一到排错就容易乱。一个相对清楚的记法是：`Terminal` 更像窗口或门面，`Shell` 更像你在这个窗口里真正用来和系统对话的语言解释器。你打开一个终端窗口，里面通常跑着一个 shell，在这台机器上通常就是 `bash`。

为什么这个区分值得记一下？因为你后面会遇到好几种上下文。你可能有本地电脑上的终端，有 SSH 进去后的远程终端，还有 `tmux` 里的远程终端。表面看都是黑框，实则上下文完全可能不同。你如果只记“我刚刚在终端里敲过这个命令”，是远远不够的。你真正应该记的是：我是在本地终端敲的，还是在远程终端敲的；我是在普通 shell 里敲的，还是在 `tmux` 会话里敲的；当时是不是已经激活了 `splatnav` 环境。

这种区分一开始看起来细碎，后面却会救你很多次。因为工程问题最怕的，从来不是某个命令太复杂，而是你连自己站在哪里都没看清。

1.4 `pwd`、`cd`、`ls`：你在项目里走路时最像“脚感”的三个动作

一说 Linux 命令，很多新手最容易小看这三个：`pwd`、`cd`、`ls`。觉得这不就是看路径、切目录、列文件吗？的确是，但它们的重要性恰恰在于太基础了。项目里的很多错误，本质上都不是算法问题，而是“你人站错地方了”。

`pwd` 是“我现在站在哪”。`cd` 是“我现在往哪走”。`ls` 是“我站在这儿能看到什么”。把这三个动作作用熟，不是为了炫命令行，而是为了让自己在目录结构里有脚感。你以后只要一感到混乱，就先别急着点文件、别急着敲长命令，先问自己：我现在在哪？我该在哪？我看到了什么？这三个问题，往往比多搜十篇帖子更快。

比如说，你今天第一天接手项目，连上机器以后，最值得做的不是立刻去找 `splatplan.py`，而是先这样确认：

```
pwd
cd /home/howard/Splat-Nav
ls
```

这三行的意义，不是你不会写长命令，而是你先把“主工作区门口”找准。你看到了 `docs`、`scripts`、`splatnav-official`、`runs` 这些目录，你脑子里的地图才真正开始长出来。

1.5 `mkdir`、`cp`、`mv`、`rm`：会用，但不要轻易地用

这些命令看起来都很常见，但对共享机器来说，态度要稍微稳一点。`mkdir` 是建目录，`cp` 是复制，`mv` 是移动或重命名，`rm` 是删除。前三个本质上是在“增添或搬运东西”，`rm` 则是让东西直接消失。对于当前工作区这种共用机器、共用资源、又明确划了边界的环境，`rm` 尤其要慢一点。

不是说 `rm` 不能用，而是你要先形成一个反射：每次准备删东西前，先确认自己删的是哪一层的东西。你删的是自己新建的 `run` 目录，还是师兄也在看的共享结果？你删的是 `home` 目录里的文档草稿，还是某个被

软链接出去的重要位置？尤其是当你看到带通配符的删除命令时，更应该停半秒。新手最怕的不是不会删，而是删得太爽。

对学弟来说，一开始你甚至可以把一个习惯记死：凡是涉及 `rm`、涉及密码、涉及 `sudo` 的动作，先想明白“它到底改谁”。你会发现，这个习惯能帮你规避很多本来完全没必要付出的代价。

1.6 `cat`、`less`、`head`、`tail`：看文件不是只能靠编辑器

很多同学一开始觉得，看文件当然就是打开编辑器。可是在远程和日志场景里，这种习惯并不总是最高效。尤其是当你只是想快速确认一个文本文件里有没有某一段内容、最后几十行是不是报错、开头几行是不是头信息时，命令行查看工具反而更轻巧。

`cat` 适合很短的文件，一口气全看完。`less` 更适合长文件，可以翻页、搜索。`head` 看前几行，`tail` 看后几行。到了项目里，这几个动作会非常频繁。你跑完一次任务，不会第一时间把整个 `stdout.log` 拖进编辑器慢慢滚，而是很可能先 `tail -n 40 stdout.log` 看看结尾发生了什么。你怀疑 `cmd.txt` 记录的命令不对，可能就 `cat cmd.txt` 快速确认。

你别小看这些“好像很初级”的动作。真正成熟的工作习惯，往往就是这样一点一点长出来的。你越早接受“远程项目里看文件不只靠编辑器”，后面会越来越顺。

1.7 `grep`、`find`、`rg`：当你开始不知道某个词在哪儿时，它们会救你

你刚接项目时很容易出现一种情况：我知道这个词出现过，我知道这个脚本大概在哪层，但就是不知道具体在哪个文件、哪一行。这个时候，搜索工具的价值就出来了。传统上很多人用 `grep` 和 `find`，现在更常用、更快的是 `rg`，也就是 `ripgrep`。

如果你要找文件名，`find` 很有用；如果你要搜内容，`rg` 往往更舒服。比如你想知道工作区里哪里提到过 `old_union`，或者哪个脚本里写了 `LD_LIBRARY_PATH`，与其靠肉眼翻，不如搜索。对新人来说，真正值得练的是一种工作心态：不知道位置不是耻辱，不会利用搜索工具才会让自己一直卡住。

我非常建议学弟尽早接受一个现实：一个中型项目里，没人会靠记忆把所有文件和函数位置背下来。会搜、会定位、会顺藤摸瓜，本身就是能力。

1.8 `df` 与 `du`：磁盘空间是项目现实，不是旁枝末节

你们这台机器有一个非常现实的背景：系统盘剩余空间紧，数据盘更宽裕。所以像 `df -h`、`du -sh` 这样的命令，不是系统管理员才会用的工具，而是你做项目时需要有的现实感。`df -h` 看的是整体分区剩余空间，`du -sh <目录>` 更像是在问某个目录已经长到多大。

为什么这件事在当前项目里尤其重要？因为你后面会生成很多 `runs`，会下载场景数据，会产生 JSON、图、日志，甚至可能做更多可视化。如果你对磁盘空间完全没感觉，最后很容易把本来是实验问题，拖成空间问题。

你不需要天天盯着磁盘，但至少养成一个常识：系统盘紧张时，别往 `/` 附近乱堆重资产；结果和缓存尽量走数据盘；看到异常大的 `run` 目录要有感知。工程感有时候就是这么朴素。

2、路径、权限、软链接：为什么你总觉得“好像有两份代码”

2.1 绝对路径和相对路径：不是谁高级，而是谁更不容易让你迷路

这两个词是所有新人都会遇到的，但真正理解它们的差别，通常要到第一次踩坑以后。绝对路径，就是从系统根目录 `/` 开始一路写死的位置。比如 `/home/howard/Splat-Nav/scripts/run_in_tmux.sh`。相对路径，则依赖你当前所站的位置。比如你已经在 `/home/howard/Splat-Nav` 根目录下，那 `scripts/run_in_tmux.sh` 就能工作。

听起来只是写法长短不同，实际上它们影响的是“歧义大小”。在你还没对项目目录很熟的时候，绝对路径像路牌，长一点但不容易走丢。相对路径像熟人之间的口头简称，前提是双方都知道你现在站在小区哪栋楼下。对于刚接项目的学弟，我会很明确地建议：前期多用绝对路径，把地图压实；后期熟了，再自然切到相对路径提高效率。

2.2 权限：先不用学得太系统，但要知道“谁能读、谁能写、谁能执行”

Linux 的权限讲深了会很多，但你第一阶段只需要抓住最朴素的一层：文件和目录不是天然想干嘛就能干嘛的，系统会区分谁能读、谁能写、谁能执行。你以后看到一个脚本能不能直接运行，和它有没有执行权限有关；你能不能修改某个目录里的东西，和它的写权限有关。

对于当前项目最有价值的，不是背权限位细节，而是先建立一种边界意识：你用的是自己的账户，在自己的工作区里做事情；不要轻易假定自己对某个系统位置天然有写权限；也不要一遇到权限问题就想 `sudo`。很多时候，正确做法反而是先问：我为什么会碰到这个位置，我本来是不是就不该动它。

2.3 软链接：理解了它，你就不会再问“为什么 home 和 data 都有一份仓库”

软链接 是当前项目里必须真正讲透的词。你可以把它先粗略理解成“快捷入口”。它看起来像一个目录或文件，但真正内容不在这里，而是在别的真实位置。对当前工作区，最典型的例子就是 `/home/howard/Splat-Nav/splatnav-official`。你看上去像是在 home 目录里打开了官方源码，其实它通过 `third_party/splatnav-official` 又继续指向 `/data/howard/repos/splatnav-official`。

这就是为什么很多新人第一次会困惑：“怎么 home 下面有一份，data 下面也有一份？”真正答案往往是：不是两份，而是一个入口加一个真实位置。如果你不把软链接理解清楚，后面读代码、解释项目结构、甚至汇报时都会绕进死胡同。你会一直怀疑自己是不是在看错仓库。

一个很适合记住的确认命令是：

```
readlink -f /home/howard/Splat-Nav/splatnav-official
```

这条命令的价值不在于炫技巧，而在于“追根问底”。你想知道这个入口最后落到哪里，就让系统替你把链路摊平。

2.4 双跳链：为什么当前工作区要这么设计

当前工作区最值得新人尽早建立的路径直觉之一，就是所谓“双跳链”。也就是 `splatnav-official -> third_party/splatnav-official -> /data/howard/repos/splatnav-official`。为什么不直接只留一个？原因其实很工程化。这样做能同时满足几件事：工作区内部有一个好找、好讲、好点击的源码入口；第三方依赖的概念被显式表达出来；真实仓库存放在数据盘，更符合空间和协作边界。

你可以把它理解成办公室里常见的一种安排：门口贴了个牌子写“会议室在这边”，但真正的会议室在走廊尽头。那个牌子的价值，不是它自己能开会，而是它让所有人都更容易找到真正的房间。软链接很多时候就是这个牌子。

2.5 readlink -f 值得你变成熟练动作

有些命令不是天天用，但一旦会了，就特别值。`readlink -f` 就属于这一类。你以后只要看到一个目录“感觉像软链接”、一个路径“看上去不太对劲”、或者一个仓库入口“怎么像是转了一道弯”，都可以直接用它来追真实位置。

对于当前项目，尤其推荐学弟在刚开始就对 `runs`、`splatnav-official` 这些关键入口跑一下 `readlink -f`。不是为了完成某个任务，而是为了把“当前工作区里有些东西只是入口，有些东西才是真实位置”这个观念真正装进脑子。观念一旦稳了，后面很多目录问题就不再吓人。

2.6 为什么短路径入口很有价值，不是为了偷懒

有的同学一看到软链接，会觉得是不是只是为了少打一段长路径。这个理解不全错，但太浅了。短路径入口真正的价值，不只是省字符，而是省认知负担。比如你让学弟以后都去记 `/data/howard/repos/splatnav-official/splatplan/splatplan.py`，当然也能记，但负担会更大。相反，如果工作区里统一从 `/home/howard/Splat-Nav/splatnav-official/...` 进入，路径叙事会清晰很多。

你未来带人的时候会更明显感受到这一点。新人最怕的不是字多，而是路径没有规律。一个路径入口如果既稳定又容易讲，它就不仅是工具，也是教学材料。

2.7 相对路径什么时候会突然坑你

很多新手第一次真踩坑，就是因为相对路径在某个目录下能跑，在另一个目录下突然不行。比如你在工作区根目录下运行 `scripts/run_splatplan_smoke.sh` 没问题，结果进了别的目录以后照着敲，系统就找不到脚本了。这时候千万别急着怀疑脚本坏了。先回到最基础的问题：你现在到底站在哪。

这类坑的核心不是“相对路径不好”，而是“相对路径需要上下文”。当你对上下文不敏感时，最好先稳一点，用绝对路径。等你在工作区里来回走得已经很熟了，再逐步切回相对路径就好。

3、日志、环境变量、进程：项目跑起来以后你真正会用到的词

3.1 环境变量：不是神秘黑魔法，而是“当前上下文的一张说明卡”

环境变量 这个词听起来特别像考试内容，所以很多人一听就犯怵。其实你先完全可以把它理解成“当前 shell 上下文里附带的一组说明信息”。系统和程序在运行时，会参考这些说明信息决定很多行为，比如去哪找命令、去哪找动态库、临时目录放哪、缓存放哪。

在当前项目里，环境变量最常见的价值不是让你自己发明一堆变量，而是帮你理解“为什么同样一条命令，在不同会话里行为不一样”。因为变量不同，上下文就不同。等你后面看到 `PATH`、`LD_LIBRARY_PATH`、`TMPDIR`、`PIP_CACHE_DIR`，你就知道，它们其实都只是环境变量家族里的成员。

3.2 管道 |：把一个命令的输出递给下一个命令

管道的图像化理解很简单：前一个命令吐出来的文本，不落地，直接顺着管子流给后一个命令。比如你列了一堆文件，接着想搜某个关键词，就可以把前者的输出直接喂给后者。它特别适合日志查看、过滤结果和快速检查。

你以后看一些脚本时，会发现很多 shell 技巧其实没有想象中复杂，管道就是其中最值得尽早熟悉的一个。它帮你建立的是一种“把简单动作串起来”的思维，而这和项目工程化的逻辑其实很一致。

3.3 重定向和 tee：程序说的话，怎么同时留在屏幕和文件里

`stdout.log` 为什么会有内容？很多时候就是靠重定向或 `tee` 这类机制。你可以先粗略理解成：普通重定向是把输出写到文件里，`tee` 则像一个分流器，一边继续显示在屏幕上，一边同时写进文件。对于当前工作区，这特别重要，因为长任务不可能都靠你盯着屏幕看，输出必须落盘。

当你看到脚本里有 `|& tee "$RUN_DIR/stdout.log"` 这种写法时，不要把它当魔法。它只是很认真地在做一件事：把标准输出和错误输出都留下来。后面你做复盘时，会特别感谢它。

3.4 进程：程序不是“运行中”或“没运行”那么简单

在 Linux 里，程序运行起来以后会变成进程。你现在不用对进程模型理解得很底层，但至少要知道：一个长任务在跑、一个 tmux window 里挂着一个 Python 脚本、一个 watchdog 在后台盯着 overnight，本质上都对某些进程。你以后看到 `pgrep -f ...`，其实就是在问“某类进程现在还活着吗”。

这个概念的重要性，在于它把“任务状态”从一种模糊感觉，变成一种可观察事实。不是你觉得它应该还在跑，它就真的在跑。你可以查。工程感有时候就是从这种地方长出来的。

3.5 退出码：为什么“没输出”不一定等于“成功”

程序结束以后，系统通常会带一个退出码。最常见的习惯是 `0` 表示成功，非 `0` 表示异常。很多脚本开头会写 `set -euo pipefail`，其中一个核心目的，就是当命令失败时不要继续装作没事，直接停下来。对共享机器和实验复现来说，这种“失败就承认失败”的态度比什么都重要。

新人很容易被一种情况骗到：终端上没看到特别明显的报错，就以为没问题。可实际上，命令可能早就失败了。你如果能逐渐对退出码和 shell 的失败传播机制有感觉，后面读脚本时会更踏实。

3.6 pgrep：不是高级命令，是“我到底有没有在跑那个任务”的实话器

当你开始接触 overnight、watchdog、tmux 里的长任务时，`pgrep` 会变得很实用。它的朴素意义就是：按名字或匹配模式找进程。你以后看到 wrapper 脚本里用 `pgrep -f` 去查某个任务是否存在，不要觉得神秘。它只是把“我感觉任务还活着”变成“我真的查到了对应进程”。

这种把感觉变事实的能力，在实验项目里特别值钱。

4、在 Cursor 里读代码，和在项目里真的“会读代码”，差别在哪

4.1 工作区文件夹：你打开的，不只是一个目录

当你在 Cursor 或 VS Code 里打开 `/home/howard/Splat-Nav` 时，你打开的不只是一个目录，而是一个上下文。这个上下文里既有文档，又有 `wrapper`，又有上游源码入口，还有结果目录和配置目录。也正因为如此，读代码不能只靠“看到文件就点”，而要先知道自己现在是在读哪一层。

当前工作区里，你至少要有个简单分层意识：`docs` 是解释层，`scripts` 是 wrapper 执行层，`splatnav-official` 是上游算法层，`runs` 是结果证据层。你只要先把这个层次感带进编辑器，阅读成本就会下降很多。

4.2 全局搜索：不是偷懒，是中型项目里的基本生存技能

很多新人一开始读代码的方式，是从一个文件顶部开始硬往下刷。这种方式在小作业里可以，在中型工作区里效率很低。因为你真正的问题常常是“这个名字在哪里被定义”“这个方法到底是谁调用的”“这个字段在结果里哪里出现过”。这时候，全局搜索不是捷径，而是正路。

比如你在文档里看到 `feasible`，想知道它在代码里从哪来；或者你在某个 JSON 里看到 `num_polytopes`，想追到上游实现里谁在写它；这时候搜索比死翻靠谱得多。很多人误以为“会读代码”就是能一眼把所有文件串起来，实际上会搜、会跳、会定位，才是真正的读码能力。

4.3 Traceback：一段报错不是羞辱你，而是在给你路线图

Python 报错最容易让新人情绪受挫。看着一长串 `traceback`，会本能地觉得“完了，我什么都不会”。其实反过来想，`traceback` 正是在给你一条线索链：最后一行通常告诉你错误类型和直接症状，上面的栈则告诉你是从哪些函数一路走到这里的。它不是故意吓你，而是在交代案发现场。

你一开始不需要把每层栈都读懂，最有效的方法是先看最后一句，再往上找第一段明显属于你当前工作区的路径。比如你看到报错里出现 `/home/howard/Splat-Nav/scripts/...` 或 `splatnav-official/...`，这就已经是很有价值的锚点。慢慢地，你会发现自己对报错不再只剩恐惧，而是开始能从里面抽出结构。

4.4 求助时最有价值的，不是“我报错了”，而是你能带来上下文

新人最怕求助，担心自己显得太基础。其实真正让人难接球的，不是基础问题，而是没有上下文的问题。如果你来一句“师兄我报错了”，但不说你跑了什么命令、在哪个目录、哪个环境、哪份日志、最后一行是什么，别人很难帮你。反过来，如果你能把 `cmd.txt`、`stdout.log` 末尾、当前 `run` 目录和自己的判断一起说出来，哪怕问题很基础，推进也会快很多。

这其实也是为什么当前工作区这么强调 `run` 目录和日志落盘。不是为了形式，而是为了让问题能够被复盘、被协作、被继续推进。

三、环境、GPU 与运行上下文

很多新人真正的第一道坎不是算法，而是环境。这里把 Conda、Python、CUDA、Torch、tmux 和 Git 这些会反复出现的运行词汇集中整理在一起。

1、Conda、pip、解释器：环境到底在管什么

1.1 为什么当前项目必须认真对待环境，而不是“装上就行”

很多同学刚开始写代码时，对环境没什么概念。觉得能跑就行，装在哪儿都差不多。可一旦项目开始涉及 GPU、Torch、Nerfstudio、Open3D、可视化和多个依赖源，环境立刻变成一件非常现实的事。当前项目又是共享机器、共享硬件、明确不能乱动系统 CUDA 的场景，所以环境更不可能靠“随手装一装”解决。

环境在这里的价值，可以先理解成一个受控的工作小屋。你在小屋里装包、修包、调解释器，不去砸整栋楼的总线路。这个比喻虽然朴素，但很适合当前工作区。你们之所以专门在 `/data/howard/miniconda3` 下装 Miniconda、再建 `splatnav` 环境，就是在做这件事。

1.2 Conda: 给项目一间相对独立的屋子

Conda 不是一句“装 Python 的工具”就能讲完的。对当前项目，最有价值的理解是：它帮你为这个项目准备一个相对独立、相对可控的软件空间。你可以在里面固定 Python 版本、安装一组依赖、做一些局部修复，而不用把系统环境和别人的环境一起拖下水。

你后面看到 `conda activate splatnav` 时，不要把它当礼仪性动作。它真正的含义是：我现在准备在 `splatnav` 这间小屋里做事，而不是在系统大厅里乱摆工具。这件事在共享机器上特别重要，因为边界一旦模糊，排错和协作都会变得很难。

1.3 Miniconda: 为什么这里不是 Anaconda 全家桶

Miniconda 可以理解成更轻量的安装方式。对于当前机器来说，这种选择很合理。原因很简单：系统盘紧、数据盘相对宽裕、项目强调边界和可控性，不需要一上来就拉一个巨大生态包。把 Miniconda 安装在 `/data/howard/miniconda3`，既避免压迫系统盘，又方便把环境、缓存和工具链集中管理在相对独立的位置。

你以后在脚本里看到 `source /data/howard/miniconda3/etc/profile.d/conda.sh`，应该意识到这是在显式接入当前机器上这套 Miniconda 环境，而不是假设用户的 shell 已经自动配置好了所有事情。

1.4 conda activate 真正在改变什么

很多新手会觉得，激活环境就是“形式上切换一下”。实际上，它通常至少在改几件关键事情：当前 `python` 指向哪一个解释器，当前 `pip` 会把包装到哪里，以及一部分依赖和路径的查找顺序。也就是说，激活环境不是给你一种心理安慰，而是在真正改变你接下来命令运行的上下文。

这也是为什么“我明明装了包但 `import` 不进来”这种问题，很多时候先别怀疑包本身，而是先问：我刚才到底激活了哪个环境？

1.5 pip: 在环境内部继续装具体 Python 包

当前项目里，环境管理的常见路线是：用 Conda 建立壳，再用 `pip` 装很多具体 Python 依赖，比如 `nerfstudio`、`clarabel`、`cvxpy`、`open3d`、`viser`、`lightglue` 等。你可以把它理解成：Conda 先帮你准备房间和基本框架，`pip` 再把房间里的家具一件件搬进来。

这里最需要避免的，不是用 `pip`，而是脱离上下文乱用 `pip`。同样一条 `pip install`，在不同解释器和不同环境里装进去的位置可能完全不一样。你以后看到任何与 `pip` 相关的问题，第一反应都应该回到上下文：我在什么环境里装的？我现在又在什么环境里跑的？

1.6 Python 解释器：不是只有“有 Python”这一个维度

很多新手心里对 Python 的图像是：机器里有个 Python，就能跑脚本。真正项目里，事情通常没那么简单。同一台机器上可能有系统 Python、Conda 环境里的 Python、别的工具链附带的 Python。当前项目最值得你养成的不是“我会写 Python”，而是“我会先确认当前到底是哪一个 Python 在跑”。

一个很朴素但很好用的动作，是在关键环境里看一下：

```
which python
python --version
```

这不是形式主义，而是在防止你后面调半天包，结果其实调错了解释器。

1.7 渠道、缓存和临时目录：为什么脚本里会主动设置 TMPDIR 和 PIP_CACHE_DIR

你在 `scripts/bootstrap_env.sh` 和 `scripts/run_in_tmux.sh` 里会看到对 `TMPDIR`、`PIP_CACHE_DIR` 的设置。这件事对新人很容易显得细碎，但它背后其实很现实：当前机器系统盘紧，数据盘更合适放缓存和临时文件。把这些位置显式指向 `/data/howard/splatnav/tmp` 和 `/data/howard/splatnav/cache/pip`，是在避免安装和运行把系统盘悄悄塞满。

你不用把它理解得太工程黑话。先把它看作“给垃圾桶和中转区找对地方”。找对地方，后面很多看似奇怪的空间问题都会少很多。

2、GPU、CUDA、Torch、LD_LIBRARY_PATH：最像黑魔法的一组词，其实也能讲明白

2.1 GPU：不是越满越好，也不是不满就说明程序没跑

很多新人第一次接触 GPU 项目，会把 `nvidia-smi` 当心电图看。显存高一点就兴奋，利用率低一点就怀疑程序没跑。这个反应很真实，但还不够成熟。对当前项目更准确的理解是：GPU 只是某些计算阶段的重要资源，不是所有阶段都必须把它打满。官方 README 也明确提到，某些轨迹和优化步骤主要跑在 CPU，这样能把 GPU 资源留给更需要它的地方。

所以你以后看到 GPU 利用率不高，不要立刻惊慌。先看任务到底处在哪个阶段，再结合日志和结果判断。工程判断最怕只盯一个数字。

2.2 驱动：它决定系统能不能和显卡好好说话

驱动 是一个特别容易被新人抽象化的词。你可以先把它理解成“操作系统和显卡沟通的底层桥梁”。当前工作区明确的一条边界就是：不去升级系统驱动，不去升级系统 CUDA，不去碰师兄已有环境。这个边界不是保守，而是非常现实。共享机器上，乱动驱动往往是牵一发动全身。

这意味着当前项目的很多修复动作，思路都不是“把楼拆了重盖”，而是“在自己这间屋里把兼容性问题解决掉”。只要把这个原则记住，很多后续决策都会清晰很多。

2.3 CUDA：你先把它理解成“GPU 运行时家族”就够了

CUDA 这个词第一次出现时，很多人会自动紧张，仿佛背后藏着一整个专门学科。对当前项目第一阶段，完全没必要把它想那么大。你现在真正需要的，是知道 Torch 和一些依赖会和某个 CUDA 运行时版本绑定，而当前系统驱动又要能支持这套绑定。也就是说，它是兼容链里的一环。

你在脚本里会看到 `cu124` 这类字样，也会看到 `torch.version.cuda`。这些都在回答一个问题：当前这个 Torch 是按哪套 CUDA 运行时构建的。你暂时不需要会配开发环境，只要会读这层信息，就已经很有用了。

2.4 `torch.cuda.is_available()`：它不是人生判断题，但它是很好的岔路口

当前项目里很常见的一个检查，就是 `torch.cuda.is_available()`。这个值如果是 `True`，说明 Torch 至少认为自己能用 CUDA。它如果是 `False`，你后面很多依赖 GPU 的步骤都会受影响。对新人来说，这个布尔值最大的价值不是“神奇”，而是它帮你把排查路线切成了两条。一个 `True`，说明你可以继续往别的地方查；一个 `False`，说明你先别急着碰算法，要先回到环境和兼容链。

很多工程排错之所以显得神秘，就是因为大家没有先找一个像这样的路口值。你只要记住这一个检查，后面排错会稳很多。

2.5 Torch：当前项目不是“深度学习项目”，但 Torch 仍然是主力数值引擎

有些跨专业同学会以为，既然当前重点在规划，不在训练，那 Torch 会不会没那么关键。恰恰相反。即使当前重点不是端到端训练，Torch 依然是很多几何和张量计算的主力。GSplatLoader 会读张量，SplatPlan 和 SafeFlightCorridor 会在张量层面处理场景和路径，设备管理也靠 Torch。

所以你对 Torch 的最初理解，不一定要从“神经网络框架”开始。你完全可以先把它理解成当前项目在 CPU/GPU 上搬运和计算多维数组的主要工具。这样会更贴近你眼前的代码现实。

2.6 LD_LIBRARY_PATH：为什么它老和奇怪的导入错误绑在一起

到了这里，我们要认真讲一下 LD_LIBRARY_PATH。如果只给定义，很多人读完还是糊。更有效的讲法是：程序启动时，除了要找到主程序，还要找到它依赖的各种动态库文件。LD_LIBRARY_PATH 就是在告诉系统，优先去哪几类位置找这些库。问题在于，如果这条路径里混进了和当前环境不匹配的库，程序就可能被“错误地帮助”，最终表现为导入失败、符号不匹配或更隐蔽的问题。

当前工作区采取了很明确的策略：很多关键 Python 命令运行前，先 unset LD_LIBRARY_PATH，或者直接用 env -u LD_LIBRARY_PATH python ...。你要理解，这不是土办法，而是一种很务实的隔离方式。因为系统里可能存在别的 CUDA 相关库、别的用户环境遗留的库，而你们又明确不想升级系统层。那最直接的办法，就是尽量让当前 Conda 环境自己解决自己的库问题。

2.7 为什么 fix_torch_cuda.sh 的思路是“环境内修”，不是“系统级修”

fix_torch_cuda.sh 的价值非常能体现当前项目的工程边界。它不是去碰系统驱动，也不是去碰全局 CUDA，而是在 splatnav 环境里强制重装一套与当前驱动兼容的 Torch。这个思路对学弟尤其值得学，因为它背后其实是一种成熟判断：当机器是共享的、系统不能乱动时，你应该优先在自己控制范围内修复问题。

这种思路以后会迁移到很多地方。你不只是修 Torch 时这么想，未来改路径、改缓存位置、改日志输出，也应该先从自己这层入手。边界感，不是保守，而是合作环境里的基本素养。

3、tmux、Git、版本与协作：项目不是只做给当下这一分钟看的

3.1 tmux：当你不想让任务随着网络一起掉线时，它就是救命绳

对很多新同学来说，tmux 是第一个看起来“有点像高手工具”的东西。其实完全不用把它神化。你可以先把它理解成远程机器上的持久终端会话。你本地断线了，窗口关了，Cursor 重载了，tmux 里的任务还可以继续跑。对于当前这种会有 smoke、batch、overnight、watchdog 之类长任务的项目，它非常实用。

当前工作区甚至已经帮你把 tmux 用法包了一层，也就是 scripts/run_in_tmux.sh。这更说明一件事：tmux 在这里不是锦上添花，而是主流程的一部分。学弟最值得先形成的，不是手动记所有 tmux 子命令，而是一个判断力：这个任务会跑很久，那我就别赌当前窗口稳定，尽量送进 tmux。

3.2 会话、窗口：tmux 里真正值得你先懂的两个词

session 可以先理解成一组长期存在的工作空间，window 则像这个空间里的标签页。当前工作区统一使用 splatnav_jobs 作为一个会话名，然后不同任务对应不同窗口。这种组织方式的好处很明显：你第二天回来看，不会完全失去线索。

新手经常一听到 `tmux` 就想把所有快捷键都记住。其实没必要。先把“会话”和“窗口”这两个概念装进去，就已经够你看懂 `wrapper` 脚本在干什么了。因为你至少知道，它不是在凭空变戏法，而是在替你管理远程任务的生命周期。

3.3 Git：不是为了装专业，而是为了让修改有来有回

`Git` 在项目里经常被两种极端理解。一种是神话它，觉得不会 `Git` 就不配做工程；另一种是轻视它，觉得不就是保存历史吗。都不太对。对当前工作区更准确的理解是：`Git` 让修改有来有回，让你知道什么是原来的、什么是你改的、什么时候变的、能不能回头看。

尤其当你们现在已经有 `wrapper` 层、上游层、文档层时，`Git` 的意义更大。因为你不是一个人在纸上写草稿，而是在真实项目里积累可交接资产。没有版本意识，很多工作都会越来越难讲。

3.4 commit：不是“任务做完了才配提交”，而是“某个小阶段有了稳定证据”

很多新人会把提交想成一件很隆重的事，觉得必须等到“做了很大事情”才能 `commit`。其实一个更成熟的理解是：当你完成了一个稳定的小阶段，比如修了一份文档结构、补了一条脚本说明、让一条 `smoke` 路线变得可复现，就已经可以形成一个提交节点。提交不是庆功，而是把某个状态钉住。

你现在不一定马上就要提交，但这种“把阶段钉住”的意识，越早越好。它会让你做事情的时候更自觉地组织证据和边界。

3.5 上游仓库和 `wrapper` 仓库：为什么脑子里必须有“双仓库模型”

当前工作区最值得新人尽早建立的协作心智之一，就是“双仓库模型”。一个是上游官方仓库，也就是论文作者的实现；另一个是你们本地为了复现、文档、日志、长任务管理而加的 `wrapper` 工作区。它们不是竞争关系，而是分工关系。上游负责“算法本体”，`wrapper` 负责“工程护栏”。

你以后一旦读代码、改文档、写汇报，就会不断用到这个模型。因为很多话只有分层说才说得清。比如“这个 `bug` 在 `wrapper` 层还是上游层”“这个改动属于工程封装还是算法逻辑”“这个路径规范为什么不写进上游代码里”。如果你脑子里没有双仓库模型，很多讨论会一直混成一锅。

四、项目本体与核心技术词汇

从这一部分开始，术语不再只是系统和环境层，而开始真正进入项目本体。你会在这里建立对 `Splat-Nav`、`Splat-Plan`、`GSplat`、`SFC` 和轨迹优化的基本语言感觉。

1、进入项目本体：`Splat-Nav`、`Splat-Plan`、`Splat-Loc` 到底各是谁

1.1 `Splat-Nav`：先别把它想成一个脚本，它是一整套系统的名字

`Splat-Nav` 是论文和项目的总名字。很多新人一开始会下意识地吧项目名和某个主脚本等同起来，仿佛只要找到一个叫 `splatnav.py` 的文件就万事大吉。其实不是。`Splat-Nav` 更像总品牌，下面分着不同模块和链路。官方 `README` 和项目页都明确写了，它由 `Splat-Plan` 和 `Splat-Loc` 两个关键组件组成。

你以后在汇报时特别需要分清这个层次。说“我现在在复现 `Splat-Nav`”时，最好能继续说清“具体聚焦的是哪一条链路”。对当前工作区第一阶段，最主要的是 `Splat-Plan` 离线/仿真链路，而不是整套硬件闭环都已经完成。

1.2 Splat-Plan: 当前第一阶段最该真正理解的主角

如果说当前项目第一阶段有一个必须讲明白的核心词，那一定是 `Splat-Plan`。你可以先用最通俗的话理解它：给定一个用 `GSplat` 表示的场景，再给定起点和终点，它的任务是先找出一条安全走廊，再在这条走廊里生成一条平滑可执行的轨迹。项目页里说的是“safe-by-construction polytope corridor”，代码层面则最终会落到 `splatplan/splatplan.py` 和相关样条优化器上。

为什么这个词如此重要？因为你当前绝大多数真正能跑、能看、能讲的主线工作，都是围绕它展开的。

`smoke_splatplan.py` 是在调用它，`run_official_style_batch.py` 是在批量比较它和基线，`runs` 里的很多结果最终也都是围绕它的可行性和耗时在组织。

1.3 Splat-Loc: 现在先知道它是谁，不着急立刻啃透

`Splat-Loc` 是论文的另一个关键模块，作用是用 RGB 图像做位姿估计。它和 `Splat-Plan` 合起来，才构成完整导航闭环。对当前学弟第一阶段来说，它不是最优先的执行主线，但绝对不能完全不知道它的存在。因为如果你只知道 `Splat-Plan`，读论文时会误以为整套系统就只是规划。

一个很稳的态度是：现在先知道它负责“我现在在哪”的问题，并且知道上游有 `run_splatloc.py` 这条入口；等你把当前离线规划主线站稳以后，再回来认真啃它。这种节奏是合理的，不是逃避。

1.4 Gaussian Splatting / GSplat / 3DGS: 别让三种叫法把你绕晕

这几个词经常在项目里交替出现，所以很多新人会担心它们是不是三个完全不同的技术。其实你先完全可以把它们理解成同一片语义区域里的不同说法。核心意思都是：场景不是主要用传统稠密网格或纯点云来表示，而是用大量带有位置、方向、尺度、颜色、不透明度等属性的高斯元素来近似表示。

当前项目的关键，不是“会背这个表示的历史”，而是知道：`Splat-Nav` 不是把场景只是看一眼，而是把这种表示本身拿来服务规划和定位。这个点很重要。因为它解释了为什么当前项目和单纯“训练一个漂亮场景模型”不是一回事。

1.5 Nerfstudio: 为什么 config.yml 会成为很多脚本的入口

`Nerfstudio` 是当前上游依赖生态里的一个重要名字。你不用一开始就把它研究成独立课题，但至少要知道：`Splat-Nav` 在读取和使用 `GSplat` 模型时，是建立在 `Nerfstudio` 风格的训练产物和目录结构之上的。这就是为什么你会不断看到 `outputs/<scene>/splatfacto/.../config.yml` 这样的路径。

理解了这一点，你对 `config.yml` 的直觉就会更准确。它不是一个神秘黑箱，也不是随便起个文件名，而是连接训练产物和下游规划使用的重要入口。

1.6 config.yml: 它不是“参数堆”，而是场景模型的门把手

`config.yml` 是项目里最值得早理解的文件类型之一。对学弟来说，你可以先把它粗略看成“进入某个场景模型的门把手”。你告诉脚本这个门把手在哪，它就知道去读对应的模型、数据解析信息和场景上下文。你以后看到 `smoke`、`batch`、上游 `run_splatplan.py` 都围绕它打转，就不会奇怪了。

很多新人第一次看实验命令，只盯着 `--scene old_union`，觉得场景就是 `old_union` 这个字符串。其实这只是一半。真正决定“你到底读的是哪套场景模型”的，往往还是 `config.yml` 路径。字符串是标签，配置才是入口。

1.7 Wrapper: 别小看这个词, 它决定你怎么理解整个工作区

`wrapper` 在很多人耳朵里像是“套壳”, 好像不够高级。对当前工作区, 我反而很希望学弟尽早摆脱这种偏见。`wrapper` 层不是多余, 而是在共享机器、复杂依赖、长任务和结果沉淀环境里, 保证复现稳定性的关键护栏。你们现在能用统一 `run` 目录、统一 `tmux` 启动方式、统一结果汇总、统一可视化输出, 很大程度上都是 `wrapper` 层的功劳。

所以当你看 `scripts/` 目录时, 不要用“这不是论文创新, 所以不重要”的态度看。更准确的想法应该是: 它不是论文算法本体, 但它是让论文算法在这台机器上变得可管理、可交接、可复盘的那一层。

1.8 Upstream: 为什么这个词一懂, 项目结构就顺了很多

`upstream` 指的是上游官方实现, 也就是论文作者原始仓库那一层。你们当前工作区通过软链接把它接进来, 是为了既保留它的原貌, 又让本地有一个统一入口读它。学弟最值得形成的一个习惯, 是以后谈问题时尽量区分清楚“这是 `upstream` 的行为, 还是 `wrapper` 的行为”。

这个区分一开始看起来像措辞细节, 后面会变成真正的工程判断力。比如某个结果字段是谁定义的, 某个路径规范是谁加的, 某个脚本为什么和官方 `README` 不一样, 这些问题都要靠“`upstream vs wrapper`”这个分层来解释。

2、规划、几何和结果: 那些一开始最像天书的词, 其实也能慢慢讲顺

2.1 A*: 不要把它想成“另一个项目”, 它只是初始化路径的一种办法

很多新人听到 `A*` 会立刻切换到“路径搜索算法课本”模式, 觉得又要学一大块新东西。对当前项目, 你先用不用走那么深。一个足够有用的理解是: 在 `Splat-Plan` 和 `SafeFlightCorridor` 这类方法里, 往往先需要一条粗路径做初始化。`A*` 很多时候就在扮演这个角色。它不是最终轨迹, 也不是整个算法的全部, 而是一条“先把方向找出来”的路径骨架。

当你带着这个理解去看 `generate_path` 里的处理流程, 就会舒服很多。你会知道: 这不是从零直接魔法变轨迹, 而是先有粗路径, 再有安全几何, 再有平滑优化。结构一旦清楚, 数学压力会小不少。

2.2 Voxel: 体素就是把空间切成很多小格子

`voxel` 的图像其实很好记。你可以把它想成三维版像素。二维图像里一个个小格子叫像素, 三维空间里规则划分的小方格就可以叫体素。为什么这个词在项目里频繁出现? 因为把连续空间离散成体素后, 很多搜索和占据判断会更好组织。上游里有 `GSplatVoxel`、`PointCloudVoxel` 这些类, 正说明它在不同方法里承担的是空间离散化和初始化路径相关的职责。

新手最容易混淆的是: 体素和点云不是一回事。点云是散点集合, 体素是规则格子。它们都能表达空间信息, 但侧重点不同。你只要先抓住这个区别, 后面阅读就会顺很多。

2.3 Point Cloud: 不是所有三维表示都一样, 项目恰恰在比较这种差异

点云对很多人来说更熟一些, 因为它比较直观: 空间里一堆三维点。当前项目里, 一些 `sfc-*` baseline 就会围绕点云相关表示组织碰撞集合和初始化路径。论文和代码的一个关键价值恰恰在于, 它不是只会用传统点云思路, 而是把 `GSplat` 这种表示真正带进规划逻辑里。

所以你以后如果看到 `PointCloudCollisionSet`、`PointCloudVoxel`, 不要觉得这是旁枝末节。它们是在告诉你: 项目不是只有一个方法, 而是在拿不同表示或不同安全策略做对比。

2.4 Collision Set: 为什么不需要把整个场景都拿来碰撞

`collision set` 可以先理解成“当前这段路径附近真正值得关注的障碍子集”。它不是整个场景所有元素的无差别全集。这个直觉非常重要。因为如果每走一小段路径都把整个场景所有高斯或所有点全拿进来算，不仅慢，也不必要。项目里很多几何处理，都是围绕“局部相关元素”展开的。

你一旦把这个词理解成“为当前这小段路径挑出最相关的一批对象”，后面看到 `compute_set_one_step` 或类似逻辑时，脑子里就不会只有一团黑箱。

2.5 Polytope: 先把它想成“安全多面体房间”

`polytope` 初看会很唬人，因为它背后连着线性约束、半空间、凸几何这些内容。对当前第一阶段，你完全可以先用一个直观比喻：它像一个由很多平面围出来的安全房间，只不过这个房间不是拍脑袋画的，而是由一组不等式严格定义出来的。路径不直接在整个场景里乱飞，而是先穿过一串这样的安全房间。

一旦你接受了这个房间比喻，`num_polytopes` 这种结果字段就好懂了。它在说的是：这条规划结果一共用了多少个安全房间。你现在不需要马上会写 $Ax \leq b$ ，先把图像感建立起来就够了。

2.6 Safe Flight Corridor / SFC: 思想和方法名，这两层不要混

`Safe Flight Corridor` 一方面是一种经典规划思路，也就是先构造连续的安全区域走廊，再在走廊里找轨迹；另一方面，在当前项目里，它又具体化成了 `sfc-1` 到 `sfc-4` 这些 `baseline` 名称。这两个层次很容易被新人混成一层。

更稳的理解是：SFC 先是一种思想，然后项目里把不同实现细节和安全保守程度做成了几个模式。你看到 `SafeFlightCorridor` 类时，更多是在看实现；你看到 `sfc-1`、`sfc-4` 时，更多是在看实验方法标签。这个区分一旦有了，你后面就不会一直把“思想”和“具体实验方法名”搅在一起。

2.7 Ellipsoid: 你可以先把它看成高斯几何的“方便外形”

椭球这个词一出来，很多人脑子里立刻跳到复杂数学。其实你先不必着急。对当前项目更贴近实际的理解是：高斯体和椭球类近似天然关系很近，所以很多碰撞和安全几何处理会借助椭球这一层来建立更可算的约束，再进一步转成多面体或半空间。

你现在不用一开始就掌握全部推导，只要知道它在链路里的职责：它常常不是最后结果，而是中间几何处理的一层桥梁。知道它在桥上，不在终点，阅读时就不容易被它吓住。

2.8 Bézier、B-spline、SplinePlanner: 它们都在回答“怎么让轨迹平滑下来”

论文里说的是 Bézier 轨迹，代码里核心类叫 `SplinePlanner`，新手常常因此犯迷糊，仿佛这是两套不同东西。其实你先抓功能就行：前面安全走廊已经出来了，但机器人真正执行时需要的是一条平滑、连续、满足约束的轨迹。样条、Bezier 这些词，都是围绕“怎么把走廊变成平滑轨迹”这个问题展开。

对新人最稳的方式，不是立刻去抠公式，而是先问：它在链路里处于哪一段？答案是：走廊之后、最终轨迹之前。这个定位一旦清楚，名字差异反而不会让你乱。

2.9 feasible: 这是项目里最值得你尽早敏感起来的结果词

`feasible` 是当前整个项目里，我最希望学弟尽早真正理解的词之一。因为它直接关系到你有没有能力判断“这次到底算不算成功”。它说的不是“程序是否退出”，而是“在当前约束下，有没有找到可行解”。

也就是说，脚本不报错，不等于这次规划就一定成功；同样，某次 `feasible` 为 `false`，也不自动等于程序坏了。两者完全不是一回事。

你以后在 smoke JSON、batch 统计、汇总 CSV、图表标题里会无数次看到这个词。只要你真正把它装进脑子，很多结果误判都会自动减少。因为你会开始问正确的问题：这次是链路没通，还是链路通了但方法不可行？

2.10 PrimalInfeasible 这种词听着可怕，其实是在说“约束打架了”

当你开始进入优化求解器相关日志时，可能会遇到一些像 `PrimalInfeasible` 的词。这类词一开始很吓人，但你可以先把它翻成人话：在当前这组约束下，求解器找不到满足所有条件的解。你不用一开始就背严谨定义。只要先知道它是在说“这组要求彼此冲突或太紧了”，不是在说“电脑坏了”，就已经很有用了。

很多新手一看到这种词就彻底不敢看日志，其实完全没必要。把它翻成人话以后，心理负担会小很多。

五、实验、结果与汇报常用词

当你开始真正动手时，最常见的困惑会转向运行规模、结果结构和汇报表达。这一部分把实验链路里最常反复出现的词集中放在一起。

1、一轮实验里你最该看懂的词：smoke、batch、overnight、run_dir

1.1 smoke：别嫌它小，它是你最值钱的第一步

`smoke` 在工程里通常就是冒烟测试。放到当前项目，就是用最小成本确认主链路能不能通。你不该把它理解成“简化版正式实验”，而应该理解成“先排雷”。当前工作区里，`smoke_splatplan.py` 和 `run_splatplan_smoke.sh` 就是在做这件事。

新手最容易犯的错，是觉得 `smoke` 太小，不如直接跑大实验。其实恰恰相反。越是刚接项目，越要先珍惜 `smoke`。因为一旦 `smoke` 都不稳，你直接冲 `batch` 或 `overnight`，排错成本会指数级上升。

1.2 batch：从“单条样例”走向“有统计意义的一批结果”

`batch` 在这里不是深度学习里那种 mini-batch 训练，而是指批量实验执行。它会在一组起终点上运行多种方法，并统计可行率、平均耗时、P95 耗时等指标。也就是说，它关注的是一批样本上的整体表现，而不是某一条轨迹偶然跑得好不好。

你从 `smoke` 进入 `batch`，相当于从“看个例”进入“看总体”。这个台阶非常重要。因为真正能支撑汇报和比较的，通常都不是单次样例，而是有统计意义的一组结果。

1.3 overnight：长时段批跑，不是另一个新项目

`overnight` 这个词听着像一个很高级的新阶段，其实你先把它理解成“把 `batch` 一轮轮串起来，在夜间窗口里持续跑”就够了。当前 `wrapper` 层用 `run_overnight_12h.sh`、`watch_overnight_job.sh`、`post_overnight_report.sh` 共同支撑这条链。

它的重要性不在于名字，而在于工程组织方式。你开始会体会到：项目推进不是只看一条命令，而是要考虑长时间运行、自动汇总、异常恢复和第二天快速消费结果。这就是为什么 `wrapper` 层如此重要。

1.4 run_dir：项目不是做给当前屏幕看的，而是做给未来复盘看的

`run_dir` 可以理解成“某一次任务的证据盒子”。一个规范的 `run` 目录里，不只是结果，还有 `cmd.txt`、`stdout.log`、`env.txt`、`notes.md` 这些上下文材料。它把“做了什么、怎么做的、当时环境怎样、结果长什么样”尽量放在一个地方。

新手如果能尽早尊重 `run` 目录，成长会非常快。因为你会逐渐从“我记得我昨天好像跑了什么”变成“我知道我昨天哪一轮任务的证据在哪儿”。这两种状态在科研和工程里差别很大。

1.5 stdout.log、cmd.txt、env.txt、notes.md：四个文件，四种不同价值

这四个文件是当前工作区特别值得学弟尽早重视的一组。`stdout.log` 是过程记录，告诉你程序运行时说了什么、报了什么。`cmd.txt` 是命令记录，告诉你当时到底是怎么启动的。`env.txt` 是上下文摘要，记录用户、时间、GPU、Python 和上游提交等环境信息。`notes.md` 则是人工补充位，适合写这轮做了什么、结果怎样、还卡什么。

这四个文件一起，才构成真正有复盘价值的一轮任务证据。只看其中一个，往往都不够。

1.6 JSON、CSV、Markdown 报告：不是重复劳动，而是不同层级的结果表达

新手经常会问：既然已经有 JSON，为什么还要 CSV、还要 Markdown 摘要、还要图？其实它们承担的是不同层级的表达。JSON 更适合结构化细节和程序继续读取，CSV 更适合横向表格比较，Markdown 和图则更适合人快速消费和汇报。

你慢慢会体会到，一个真正成熟的实验输出，不会只给一种形式。因为不同角色需要看的东西不同。你自己排错时可能先看 JSON 和日志，师兄看汇报时可能先看 Markdown 和图，做进一步分析时又会回到 CSV。

1.7 Seed：为什么结果里的“随机性”不能被当成空气

`seed` 是随机数种子。很多新人一开始会对它有两种极端态度：一种是完全忽略，觉得不重要；另一种是过度要求，觉得所有结果都应该一模一样。更成熟的理解是：`seed` 是在帮助你管理一部分随机性。你不需要把它消灭，但你需要记录它、解释它。

当前 `batch` 脚本里，`seed` 会影响样本顺序等行为。这就意味着，如果你完全不记录 `seed`，后续复现会变难；如果你把结果的每一点波动都当灾难，也会让自己很紧张。对新人来说，先接受“随机性是要被管理的，不是要被神化的”，就已经很够用。

2、汇报、协作与判断：让你从“会跑”走向“会讲”

2.1 复现：不是把命令抄下来就算结束

复现 这个词非常容易被说空。对当前项目，一个更扎实的理解至少有三层。第一，你真的在这台机器上把主链路跑起来了。第二，你保留了足够的上下文和证据，让别人能知道你是怎么跑的。第三，你能解释自己得到的结果，知道它和论文、上游实现、`wrapper` 封装之间是什么关系。

你只要缺一层，复现就还不够稳。抄命令不是复现，跑出一张图也不是复现。真正的复现，是能落地、能回看、能解释、能交接的。

2.2 验收：你要学会对自己讲“这一步到底算不算过”

验收是一个看起来很像管理词，但对新人非常重要。因为如果没有明确验收口径，人就会很容易自我感觉“差不多了”。而项目最怕的，恰恰就是这种模糊完成。比如一次 smoke 的验收，不该只是“终端退出了”，而应包括 run 目录完整、结果 JSON 可读、关键字段可解释。一次 batch 的验收，也不是“有几张图”，而是日志、JSON、图、摘要彼此对得上。

学会验收，本质上是在培养一种对事实负责的习惯。

2.3 周报：不是行政负担，而是训练你把事情讲清楚的场子

不少同学一提周报就烦，觉得是在额外写材料。可你把它放到当前项目里看，会发现它其实是在逼你练很重要的一项能力：把“这周做了什么、看到了什么、卡在哪里、下周准备干什么”讲清楚。一个好的周报，应该建立在 run 目录、日志、结果图和自己的判断之上，而不是靠印象编。

你越早把周报和证据绑定起来，越不容易在汇报时心虚。这个习惯以后不仅对科研有用，对工作也非常有用。

2.4 假成功：工程里最应该敏感起来的危险词之一

如果说当前整套词典里，有一个非正式但我非常建议你尽早敏感起来的词，那就是“假成功”。它指的是：看上去任务结束了，甚至还有图、有目录，但真正关键的东西没成立。比如 feasible 全是 false，比如 JSON 缺字段，图和统计对不上，或者你其实看错了 run 目录。

很多新手的问题，不是不会跑，而是太容易被假成功安慰。学会对假成功保持警惕，会让你比很多看起来更会敲命令的人更稳。

2.5 带人时最重要的，不是你讲得多，而是你有没有把上下文交出来

你未来如果要把项目交给学弟，这份词典背后的真正目的就更清楚了。带人不是把所有知识一股脑丢过去，而是把上下文交出去：这台机器怎么进，工作区怎么分层，哪个脚本是做什么的，为什么 run 目录重要，为什么这次不能直接去动系统 CUDA。你只要把上下文交清楚，学弟即使一开始不懂所有细节，也不至于在完全错误的方向上狂奔。

这其实也是文档为什么值钱。文档不是替代交流，而是在帮交流把底盘铺平。

3、读到这里，你脑子里应该已经长出来的那几根“主钉子”

如果这一份重型词典起作用了，你现在不一定已经掌握了所有概念，但脑子里至少应该有几根钉子开始稳定了。第一，你应该知道当前项目里哪些词属于系统和环境层，哪些词属于项目与论文层，哪些词属于规划几何层，哪些词属于实验和结果层。第二，你应该已经不再把 scripts/、splatnav-official/、runs/ 混成一团。第三，你应该开始能把 SSH -> 环境 -> config.yml -> planner -> run_dir -> feasible -> 汇报 这条词汇链串起来。

请你留意一下，这其实已经不是“记住了很多定义”，而是开始形成一个工作语言。真正的项目理解，往往就是这样慢慢长出来的。你会发现，某些词一开始很可怕，读到第三次时已经不再吓人；某些词一开始看起来很抽象，等你真的在代码、日志和结果里碰到它几次，就突然活了起来。

如果你现在已经有这种感觉，说明这一部分在起作用。下一步，最适合做的事情不是继续埋头背新词，而是带着这些词去读第四编。到了那一份文档里，这些词不会再像散点，而会开始进入真正的项目地图。

六、扩展系统、三维表示与工程边界词汇

这里收拢的是那些不会在第一天全用上，但很快就会在交流里频繁听到的词。提前把它们归到一处，能明显降低后面被陌生名词打断节奏的次数。

1、你现在还没真正用上，但很快会听到的那批系统词

1.1 ROS2：现在先知道它是“系统集成层的语言”，不是第一周的主战场

ROS2 这个词对很多机器人背景项目来说都绕不过去，所以新人一看论文或者听组会讨论，很容易立刻觉得“我是不是必须先把 ROS2 学透，才有资格看 Splat-Nav”。这种想法特别正常，但当前阶段不必这么卷。你先把 **ROS2** 理解成机器人系统里常用的通信与模块组织框架就够了。它更像整车里的总线和接口层，而不是你当前离线规划主线必须先啃透的对象。

为什么这个词仍然值得现在就讲？因为它会频繁出现在项目背景和未来方向里。上游官方 README 也提到 ROS 节点在另一个分支中。也就是说，完整系统最终确实会和 ROS2 发生关系，但对当前工作区第一阶段来说，你的主任务仍然是离线/仿真侧的 **Splat-Plan**。知道它存在，知道它大致负责“系统怎么串起来”，就够了。不要一听到 ROS2 就把自己吓得不敢往下走。

1.2 Gazebo：它更像仿真舞台，不是当前第一步的门槛

Gazebo 往往和仿真环境联系很紧。你可以把它先理解成“给机器人提供一个虚拟世界去测试”的仿真舞台。很多同学第一次听到机器人项目，会本能地把 Gazebo 和“我现在要做的一切”绑定起来。其实对当前项目第一阶段，并不是这样。你现在主要在看的是作者已有场景和离线/仿真规划实验，不是立刻去自己搭一个完整的 Gazebo 世界。

换句话说，Gazebo 是你未来可能会碰到的舞台，但不是当前最小主线的第一道门。你先把这个边界记住，会轻松很多。因为你不会再一看到它就觉得自己要同时学仿真引擎、学规划、学视觉、学系统集成。

1.3 Nav2：如果未来谈系统导航，它会出现，但现在先别抢戏

Nav2 经常被理解成 ROS2 生态里的导航栈。这个理解大方向没问题。它主要关心的是在系统层怎样把地图、定位、规划、控制这些模块串起来。对当前项目而言，Nav2 不是你这一周必须操作的工具，但它是你以后听到“系统怎么接起来”时很可能会出现的话。

所以我们现在给它一个不费劲的定位：先知道它属于机器人系统级导航框架的一部分，和当前工作区主线不是同一层；以后真的走到系统集成那一步，再回过头认真学。这个顺序没问题，不是偷懒。

1.4 RViz：先把它看成“系统可视化窗口”，别把它和项目结果图混掉

RViz 也是 ROS 相关讨论里很常见的词。你可以先理解成一个系统可视化窗口，用来显示机器人状态、地图、轨迹、坐标系等信息。很多新人听到“可视化”就会自动把 RViz 和当前项目里生成的 PNG 图、viser 结果、trajectory overlay 图混在一起。其实不是一回事。

当前工作区里你最常看到的可视化，更偏实验图和网页可视化，而不是 RViz 这种系统级调试窗口。先分清这一点很有必要。因为“可视化”这个词一旦被你混成一层，后面听别人说话时会一直串戏。

1.5 为什么要现在提这些词，但又不要你现在精通

原因其实很简单：新人最容易焦虑的，不是眼前这个词不会，而是感觉“后面还有好多东西等着我”。把这些词提前点出来，反而会让你安心。因为你知道：是的，它们存在；但不是现在这一周必须一起拿下的对象。你只要知道它们大致在机器人系统图谱中的位置，就已经够用了。

这种“先知道边界，再决定何时深入”的能力，本身就是很成熟的学习方式。

2、导航闭环里那些你迟早会碰到的词

2.1 定位、地图、规划、控制：很多大词，其实是四块朴素分工

当别人说“导航系统”时，新人最容易把它想成一个巨大黑盒。更好一点的理解，是把它拆成四块：定位回答“我在哪”，地图回答“世界长什么样”，规划回答“我该怎么走”，控制回答“我怎么真的动起来”。你现在之所以已经能对 Splat-Nav 有点轮廓，就是因为论文本身就在这四块里挑了两块重点做：Splat-Loc 更贴近定位，Splat-Plan 更贴近规划，而地图则是 GSplat 场景表示。

控制那一块在当前离线工作区里没有被展开到第一阶段，但这并不影响你先建立闭环视角。相反，一旦你脑子里有了这四格图，以后很多系统讨论都会变得更清楚。你会知道某个问题到底是在“我在哪”这层，还是“怎么走”这层，而不是所有东西都混成导航两个字。

2.2 坐标系：你先把它想成“大家说的位置是不是在同一本词典里”

坐标系 是机器人和三维项目里绕不过去的词。它第一次出现时，几乎所有新人都会紧张，因为觉得这玩意儿一听就数学很多。其实你先可以用很生活化的一句话理解：坐标系是在约定“位置 and 方向到底用哪套参照来表达”。如果两个人都说自己在“左边一点”，但参照方向不同，那就会产生混乱。坐标系就是在防止这种混乱。

当前项目里，`config.yml`、Nerfstudio、GSplat 模型、起终点、轨迹结果之间，其实都隐含着坐标表达问题。你现在不需要把每套变换矩阵都推到明明白白，但至少要知道：路径点不是孤立数字，它们总是在某个坐标系里成立。只要先建立这层敬畏，后面读代码和结果就不会太草率。

2.3 frame：为什么有时候大家说的是同一个位置，却数字不一样

frame 往往就是“某个具体坐标参照框架”的意思。这个词在视觉、重建、导航里都很常见。最容易让新人困惑的，是“明明都在说同一个物体，为什么数字不一样”。原因通常不是谁错了，而是表达所依据的 frame 不同。一个在相机 frame 里，一个在世界 frame 里，数字当然可能完全不一样。

你不用一上来就把 frame 关系全画出来，但至少要对这个现象敏感。只要你以后看到两个结果对不上，就别急着说哪个错了，先问一句：是不是根本不在同一个 frame 里。

2.4 世界坐标、相机坐标、数据坐标：它们不是故意为难你

很多跨专业同学会在这儿心态崩一下，觉得“怎么同一个项目里还要有这么多坐标说法”。其实这很正常。相机拍到的内容天然有一套相机视角表达，数据采集过程可能有自己的数据坐标，训练和重建过程中又可能映射到某种统一场景坐标。你先接受这是不同层的表达，而不是故意玩花活，压力就会小很多。

当前项目里，尤其值得你先记住的是一句：当你看到坐标相关问题，先别把它当成“数字没对上”的普通 bug，它很可能是表达语境的问题。这个判断一旦会了，很多疑惑都能少走弯路。

2.5 goal: 在这个项目里, “目标”并不总是一串随便写的数字

`goal` 这个词听起来太普通了, 所以新人反而容易忽略它的细节。在当前项目语境里, `goal` 有时是明确的终点坐标, 有时在论文完整系统层还可以是更高层语义目标。也就是说, 它不是永远都只意味着“你在代码里手填的三个浮点数”。

对第一阶段来说, 你先把 `goal` 理解成规划终点就够了。但你应该知道, 论文完整叙事中它可以延伸得更丰富。这层认识, 会帮助你以后读论文和看项目页时不觉得跳跃。

2.6 trajectory: 路径和轨迹, 别先把它们混成一个词

很多人口语里会把路径和轨迹混着用, 问题不大。但你在当前项目里, 最好还是有一个基本区分。`path` 更像几何上的通路, 强调“从哪里经过”; `trajectory` 更像准备给执行系统用的、更平滑、更具动态约束意味的表达。上游结果里经常会同时出现 `path` 和 `traj`, 这不是重复, 而是在区分不同层级的输出。

只要你把这个区别先记住, 后面看 JSON 结果时就不会疑惑“怎么两份结果都像路线”。

3、三维表示世界里那些很容易混在一起的词

3.1 点云、网格、NeRF、3DGS: 它们都在回答“场景怎么表示”, 但答案不同

这四个词一开始看上去像同类型名词堆在一起, 确实容易让新人发懵。更轻松一点的理解方式是: 它们都在回答“我拿什么来表示一个三维场景”, 只是回答方式不同。点云更像一堆离散点, 网格更像面片拼起来的表面, NeRF 更像连续隐式辐射场表示, 而 3DGS / GSplat 则是用很多高斯元素来近似场景。

在当前项目里, 你不需要把这四者的论文历史都啃一遍, 但你至少要知道: Splat-Nav 之所以叫这个名字, 就是因为它不是建立在纯点云或纯网格地图上的, 而是建立在 Gaussian Splatting 场景之上的。这层差异本身就是项目存在的意义之一。

3.2 为什么不是所有“看起来都能表示场景”的方法都适合做同样的导航

这是一个特别值得新人早一点理解的点。很多时候我们会本能觉得: 只要场景能表示出来, 后面导航不就这样做吗? 其实不是。场景表示方式会深刻影响规划和定位怎么做、速度怎样、安全约束怎么构造、计算成本落在哪。项目之所以强调 GSplat, 不是为了跟风, 而是在强调这种表示方式对实时导航有现实影响。

你把这个逻辑抓住了, 后面读论文时就不会只把 Gaussian Splatting 当作“画面好看”的噱头。

3.3 splatfacto: 它在路径里出现时, 意味着你已经站在训练产物门口了

在当前目录里, 你经常会看到 `outputs/<scene>/splatfacto/.../config.yml` 这种路径结构。这里的 `splatfacto` 不是随便起的文件夹名, 它通常代表 Nerfstudio 里某类 GSplat 模型训练产物的组织路径。对新人来说, 它最有价值的不是精确定义, 而是路径直觉: 看到它, 说明你正在接近场景模型产物, 而不是普通脚本目录。

这类路径词一开始看起来像细节, 但当你多看几次以后, 会慢慢形成一种“路径读图能力”。这很重要。

3.4 transforms.json: 很多时候它在帮助你把图像和空间关系对上

这个文件在数据和重建语境里很常见。你现在不需要精读其格式细节, 但要知道, 它通常在帮系统记录图像与空间变换、相机视角等相关信息。当前项目的数据下载和补齐脚本也会关心它是否存在, 这说明它在场景数据可用性上不是无关紧要的附件。

对新人来说，最重要的不是会手改它，而是知道它属于“场景数据几何关系的重要文件”，缺了它很多链路就不完整。

3.5 sparse_pc.ply：它为什么会出现在下载验收条件里

ply 文件是点云或网格类数据常见的格式之一。当前脚本里会把 sparse_pc.ply 是否存在当成场景下载完整度的一部分检查条件，这说明它至少在数据准备链上扮演重要角色。新人不一定要马上打开它，但要知道：它不是普通图片，而是和空间几何有关的资产。

理解这一点，能帮你更快看懂为什么下载脚本和补齐脚本会特别在意它。

3.6 images_2、images、frame 图像：别把“图片文件夹”看得太随意

很多新手看到数据目录里的图像，会觉得不就是图像吗，在哪个文件夹区别大吗？在真实项目里，区别往往很大。不同图像目录可能代表不同下采样、不同筛选策略、不同训练或评估版本。当前下载脚本里甚至会直接用帧数门槛来判断某些目录是不是足够完整。这就说明图像目录不是随便起的名字，而是数据链中的有意义节点。

你未来只要养成一个习惯：看到图像目录，别只把它当“图片堆”，先问它在数据流里是什么角色。

4、优化与求解器：先理解“它在解决什么”，再去想“它怎么算”

4.1 优化：为什么规划里常常绕不开它

很多同学一听到优化，就自动回到数学课的紧张情绪里。其实你先可以用非常朴素的话理解：当我们希望轨迹既平滑、又满足安全区域、又满足起终点要求时，这本身就是一类“在若干条件下找一个最合适解”的问题。优化就是在处理这类事。

对当前项目而言，你最重要的不是先会推导所有矩阵，而是知道优化在链路里的职责：前面有走廊，后面要变成平滑轨迹，这个转换不是拍脑袋完成的，而是通过求解一个带约束的问题来完成。

4.2 凸优化：你先把它理解成“相对更规整、相对更好求的一类问题”

凸优化 这个词不用上来就往严格定义上走。你先记一个工程直觉就行：它通常意味着问题结构相对规整，因此更有机会被稳定、高效地求解。当前项目里，当你看到 cvxpy、Clarabel 这些词，通常就是在和这种规整的优化问题打交道。

这层直觉很有价值。因为你会知道项目不是在随便乱试，而是在把轨迹生成问题落到一类可求解、可复现的结构里。

4.3 cvxpy：别把它看成“算法本身”，它更像建模语言

cvxpy 对新人来说最容易被误解成某种“核心创新算法”。更贴切一点的理解是：它更像是你用来描述优化问题的一套建模工具。你在里面写变量、写约束、写目标，然后调用后端求解器去解。也就是说，它更像问题表达层，不是最后真正动手算的那台机器。

一旦你理解了它更像“说清问题”的工具，而不是“就是它发明了算法”，很多读码和读日志时的紧张感会下降。

4.4 Clarabel: 它更像“真正负责求解的那个人”

和 `cvxpy` 对照看, `Clarabel` 更像真正负责把那个优化问题解出来的求解器。你在当前项目里看到它, 重点不是背它的论文, 而是知道它在链路里的职责: 问题已经建好了, 现在轮到它来算。这样你以后看到 `Clarabel` 日志时, 就不会误以为自己进入了一个完全无关的新世界。

4.5 QP: 你不必现在精通缩写, 但知道它在说“某类优化问题”就够了

QP 是 quadratic programming, 也就是二次规划。对新人来说, 你第一阶段并不需要背它的标准形式。知道它是在说某类有特定目标函数和约束结构的优化问题, 就够了。你以后在看样条优化器时, 只要知道这不是在瞎调参数, 而是在解某种结构明确的问题, 就已经很不错的了。

4.6 为什么看不懂求解器细节, 不妨碍你先把主链路讲明白

这是一个很重要的心理减压点。很多同学一看到优化器细节, 就觉得“完了, 这个项目我暂时讲不清了”。其实不是。你完全可以先把主链路讲清楚: 场景是怎么读进来的、路径怎么初始化、安全走廊怎么构出来、最后怎么通过样条优化变成轨迹。等这条线稳了, 再回头慢慢补求解器的细节。顺序没错, 也很正常。

5、数据、下载和实验室边界: 这些词不花哨, 但特别决定你能不能稳着干

5.1 gdown: 很多场景数据进入你电脑前, 会先经过它

`gdown` 本质上是一个帮助你从 Google Drive 下载数据的工具。放在当前项目里, 它的重要性在于: 官方数据、图像目录、某些场景资源, 很可能要靠它来批量拉取。你不一定非得自己手敲每次下载, 但至少要知道, 很多 wrapper 下载脚本其实都是在帮你更稳定地使用 `gdown`。

这个词的价值不是高深, 而是现实。知道数据是怎么来的, 后面你对路径和缺文件问题的敏感度就会高很多。

5.2 partial: 为什么有时候“先下够最小可跑资产”是合理策略

`partial` 这个词在项目执行里很有意思。它不是“敷衍做一半”, 而常常是在现实资源条件下, 先把最小能跑的部分准备齐。比如系统盘紧、网络不稳定、某些场景资产很大, 那就可能先拉最必要的一部分, 至少把最小链路走通。这种做法不是偷懒, 而是一种策略。

当然, 这里有边界。你要知道自己现在拿到的是 `partial` 资产, 哪些结论可以先做, 哪些结论暂时别说得太过。工程诚实也很重要。

5.3 transforms.json、sparse_pc.ply、帧数门槛: 下载成功不是“命令结束”, 而是“结构真的完整”

当前下载和补齐脚本之所以会去检查这些文件和帧数, 不是因为写脚本的人太较真, 而是因为“下载命令看起来结束了”并不自动等于“场景资产足够可用”。一个目录里少了关键 `JSON`, 或者图像少得离谱, 后面照样会出问题。

你如果早一点接受这种“结构验收”思维, 会比只盯退出码成熟很多。实验室里的很多事故, 本质上都不是程序有多坏, 而是数据准备阶段就已经不完整了。

5.4 共用机器、共用资源: 边界意识不是客套, 是工程礼貌

这一点在当前项目里尤其需要反复说。你们共用这台工控机, 师兄也有自己的账户和环境。当前所有 wrapper、环境和文档策略, 本质上都在围绕一个原则: 尽量把影响收敛在 `howard` 自己的工作区和

Conda 环境里，不碰系统级 CUDA，不碰别人的账户环境，不把系统盘再压爆。对新人来说，理解这个原则，比会多几条命令更重要。

因为这决定了你将来遇到问题时，是不是会本能地先在自己这一层找办法，而不是动不动就想“我去升个系统 CUDA 试试”。共享环境里，边界感就是职业素养。

七、把术语重新串回工作流与场景

前面是拆词，这一部分开始重新连词。目标不再是“我认识多少个词”，而是“这些词在真实工作里会怎样一起出现、怎样一起帮助我理解项目”。

1、最后，把这些词连成一个你可以带着走的故事

到这里，如果你还把这些词看成散点，那这份词典就还没真正起作用。我们最后试着把它们连成一个尽量顺的故事。你先通过 SSH 进入 Linux 工位机，在 Terminal 里通过 Shell 开始工作。你用 `pwd`、`cd`、`ls` 确认自己来到 `/home/howard/Splat-Nav` 这个工作区。你知道这里有 wrapper 层、有 upstream 层、有 run 目录层。你理解软链接，所以不会再被 `splatnav-official` 的双跳链吓到。

接着，你通过 Conda 进入 `splatnav` 环境，明白当前 `python` 和 `pip` 的上下文是什么，也知道为什么 `LD_LIBRARY_PATH` 需要小心对待。你跑环境检查，确认 `torch.cuda.is_available()` 正常。然后你拿着一个 `config.yml`，让脚本通过 `GSplatLoader` 读取某个 `GSplat` 场景。你知道当前主角是 `Splat-Plan`，而不是把完整系统所有模块一次性都扛起来。你知道它会先做初始化路径，再构造安全多面体，再做样条优化。你看到结果里的 `feasible`、`path`、`traj`、`num_polytopes`，不会再觉得这些都是凭空冒出来的词。

最后，你把一次任务的证据沉淀到 run 目录里，看 `stdout.log`、看 `cmd.txt`、看 JSON 和图，知道什么叫假成功，也知道什么样的结果才能拿去做汇报。这时候，这套词汇就不再是一堆分离名词，而是开始组成你自己的工作语言了。

如果你能读到这一步，说明你已经不是那个刚打开目录就头皮发麻的状态了。接下来再去看 第四编，你会明显觉得顺很多。因为你现在已经不是在看陌生字，而是在看一张能落回现实项目的地图。

2、最容易混掉的词，我把它们成对摆给你看

2.1 PATH 和 LD_LIBRARY_PATH：一个找程序，一个找库，别再混了

这是新人里最经典的一组混淆。因为名字都带 `PATH`，又都跟“找东西”有关，所以很多人脑子里会自动把它们混成一种东西。更轻松一点的记法是：`PATH` 更像在问“当我敲一个命令名时，去哪里找这个程序本体”；`LD_LIBRARY_PATH` 更像在问“当程序运行起来以后，去哪里找它依赖的动态库”。一个是在找主角，一个是在找配角。它们都重要，但作用层不一样。

这个区分对当前项目非常关键。因为环境排错时，你会同时遇到“python 解释器找错了”和“动态库解析错了”两类问题。如果你脑子里只有一个模糊的 `PATH` 概念，就很容易把两类问题混在一起。混在一起以后，排错就会变得特别痛苦。真正成熟的状态，不是会背定义，而是出问题时能先判断：这更像是程序入口问题，还是库入口问题。

2.2 path 和 traj：都像路线，但不是同一层东西

这一组词在结果文件里经常一起出现，所以特别容易混。前面已经讲过，`path` 更像几何通路，`traj` 更像给执行侧看的平滑轨迹。这里再换个角度说一次，因为它太重要了。你可以把 `path` 想成你在地图上先画出的走法骨架，把 `traj` 想成真正能开车、能飞、能平滑经过这些位置的轨迹曲线。

为什么要这么反复强调？因为学弟以后很容易拿着 `path` 去解释执行可行性，或者拿 `traj` 去讨论初始化搜索。两者如果一直混着说，汇报会显得非常飘。只要你肯把它们分清，表达质量会立刻提升一个台阶。

2.3 scene 和 config.yml：一个像标签，一个像门把手

`scene` 常常是 `old_union`、`flight`、`stonehenge` 这种名字，看起来很像“场景本体”。但实际项目里，它更像一个标签，用来帮脚本选取场景预设。真正决定读取哪套模型和相关配置的，通常还是 `config.yml`。所以这两个词的区别，不是有没有关联，而是层级不同。`scene` 帮你定位语义和预设，`config.yml` 帮你真正进入具体资产。

很多新手第一次看命令时，会下意识只盯着 `--scene`，觉得这个已经把场景说完了。其实不够。只要你脑子里建立了“`scene` 像标签，`config` 像门把手”这个图像，后面理解会轻松很多。

2.4 smoke 和 batch：一个是排雷，一个是统计，不是大小版关系而已

很多人以为 `batch` 就是“多跑几次的 `smoke`”，这话只对了一半。更准确一点说，`smoke` 的重点在于尽快验证主链路是否能通，`batch` 的重点则在于形成统计意义上的比较。一个的心态更像工程排雷，一个的心态更像实验分析。你如果只把它们看成大和小的关系，会漏掉它们各自的功能定位。

这也是为什么我一直强调，学弟不能把 `smoke` 当正式结论，也不能直接跳过 `smoke`。两者都重要，但重要得不一样。

2.5 wrapper 和 upstream：一个是护栏，一个是本体

这一对词会贯穿整个工作区。最容易犯的错是把它们都叫“项目代码”，然后讲半天谁也不知道你在说哪层。更顺一点的记法是：`upstream` 更像论文算法本体，`wrapper` 更像让这套本体在本机上更稳定、更好管理地跑起来的护栏层。一个解决“算法怎么做”，另一个解决“我们怎么稳当地跑和管”。

这两层都重要，但混起来就会特别乱。比如你看到 `scripts/run_overnight_12h.sh`，就应该知道这明显更偏 `wrapper`；你看到 `splatplan/splatplan.py`，则明显更偏 `upstream`。只要肯一直带着这个眼镜看项目，很多结构问题会自动变清楚。

2.6 success 和 feasible：程序成功结束，不等于规划可行

这也是特别需要反复敲的一个点。程序成功结束，说的是链路有没有正常走到头；`feasible` 说的是当前规划问题有没有找到可行解。它们可能同时成立，也可能只成立其一。比如程序跑完了，但 `feasible = false`；也可能程序自己就炸了，连 `feasible` 都没正常写出来。把这两种状态分清，是新人迈向成熟非常关键的一步。

真正会读结果的人，从来不会只看“脚本有没有退出”。他会同时问：退出状态怎样，结构化结果是否完整，`feasible` 如何，图和统计是否一致。这种多看一眼的习惯，会让你少踩很多坑。

2.7 run_dir 和 workdir：一个是结果落点，一个是命令执行位置

这两个词也很容易被轻忽。run_dir 是结果沉淀目录，用来保存当前任务的日志、命令、环境摘要和结构化产物。workdir 更偏执行上下文，也就是命令真正在哪个目录下运行。它们当然可能相关，但绝对不是一回事。很多任务会在某个工作目录下执行，然后把结果写到另一个 run 目录。

为什么要区分？因为你以后排错时会非常需要这层意识。你不能因为结果在 runs/...，就以为命令当时一定也是在那个目录里跑的。当前脚本链里，这两者往往是清楚分开的。

2.8 data、outputs、runs：都是目录，但职责完全不同

data 更偏原始数据和数据解析资产，outputs 更偏模型训练产物，runs 更偏你当前工作区的实验执行证据。很多新手一看到都是目录，就会把它们笼统看成“项目文件”。这样看太粗了。真正成熟一点的看法是：这三个目录分别站在数据流的不同阶段。一个偏输入原料，一个偏中间产物，一个偏实验证据。

你把它们分开以后，后面再讲“输入、模型、实验结果”就不会老是绕在一起。

3、用五个真实场景，把术语串成你真的会遇到的故事

3.1 场景一：学弟第一天连上机器，看着黑框发愣

这几乎是所有人都会经历的一幕。你第一次用 SSH 连上机器，终端一黑，提示符一出来，脑子里第一反应通常不是“我已经进入生产环境”，而是“我现在到底在哪”。这时真正有用的词，不是什么高大上的三维表示，而是 pwd、ls、绝对路径、相对路径、hostname。你先用这些词把自己站稳，先知道自己是不是已经到了 /home/howard/Splat-Nav，再谈后面的事情。

你会发现，越是第一天，这些看起来“基础得不好意思问”的词，越是决定你能不能平稳起步。只要第一步没站稳，后面所有高阶词都只能浮在空中。

3.2 场景二：学弟觉得环境应该差不多好了，但程序就是 import 失败

这时跳出来的词往往是 Conda、pip、Python 解释器、PATH、LD_LIBRARY_PATH、Torch、CUDA。很多新人此时的情绪会突然变糟，觉得自己是不是天生不适合弄环境。其实你只是在第一次面对“项目环境不是系统默认空气”这件事而已。成熟一点的排法，不是乱装包，而是回到上下文：我激活了哪个环境？当前 python 指向哪里？是不是库解析被污染了？

只要你脑子里已经有这几个词的基本定位，排错就不会完全失控。哪怕还不能自己一步到位修好，也至少能把问题说清楚。

3.3 场景三：学弟终于跑了一条命令，但不知道跑出来的是啥

这时真正重要的词会切到 run_dir、stdout.log、cmd.txt、env.txt、JSON、feasible、path、traj。也就是说，你的注意力开始从“程序能不能启动”转向“结果到底说了什么”。很多人走到这一步会短暂兴奋，觉得“终于有东西了”；但如果没有术语支撑，这个“东西”就仍然是黑箱。你看见 JSON，却不知道先看哪几个字段；你看见 feasible，却不知道它和程序是否成功结束是两回事；你看见图，却不知道它和 JSON 是否一致。

所以术语并不是读书人的乐趣，它在这里是在帮你形成结果判断能力。没有这层能力，结果再多也很难沉淀成真正的经验。

3.4 场景四：学弟开始读代码，结果被 GSplatLoader、SafeFlightCorridor、SplinePlanner 吓住

这时你会明显感觉到，术语已经从系统层切换到了项目与算法层。好消息是，如果前面的词你已经有了基本底子，这里不会再像第一天那样全是黑的。你会知道 GSplatLoader 更接近“把场景读进来”，SafeFlightCorridor 更像某类基线方法实现，SplinePlanner 更像最后把轨迹平滑出来的那个环节。你未必已经会推导它们，但你已经不会完全失去方向。

这就是术语体系带来的真正价值：它不替你阅读代码，但它让你不会一上来就被名字压垮。

3.5 场景五：学弟准备做周报，不知道怎么把这周做的事讲清楚

这个场景往往比前面几个更能检验术语是不是进脑子了。因为到了这一步，你不能只会识别词，还要会用词来组织叙述。比如你要说清：这周主要在 wrapper 层完成了环境检查与最小 smoke；输入是某个 config.yml 指向的 GSplat 场景；输出写进了 run 目录；当前主要问题不在 Splat-Plan 核心逻辑，而在环境和数据完整性；下一步准备进入小规模 batch 比较 splatplan 和 sfc-^{*}。

你看，到了周报这一步，术语已经不再是单独存在的知识点，而是你组织思路的骨架。如果骨架稳，你讲什么都会稳很多。

4、最后送你一套“术语肌肉记忆”

到了这里，我不想再加更多新词了。相反，我更想帮你把已经见过的词压成一套肌肉记忆。不是那种死板背诵，而是你以后碰到项目时，脑子里能自动冒出来的几句问话。

第一句是：我现在在哪里？这句话对应的是 SSH、终端、路径、软链接、工作区层次。第二句是：我现在在什么环境里？这句话对应的是 Conda、Python 解释器、Torch、CUDA、LD_LIBRARY_PATH。第三句是：我现在跑的是哪条链路？这句话对应的是 Splat-Plan、Splat-Loc、wrapper、upstream、config.yml、scene。第四句是：结果到底落在哪、我该先看什么？这句话对应的是 run 目录、日志、JSON、图、feasible。第五句是：我当前的问题属于哪一层？这句话对应的是系统、环境、数据、脚本、算法、结果解释这几层分工。

如果你把这五句问话带走，这份词典就已经非常值了。因为它不只是帮你认识了很多词，更帮你建立了一套遇事先分层、先定位、再判断的习惯。这个习惯一旦长出来，后面你学什么都会更快、更稳，也更不容易慌。

读到这里，请你先别急着继续往下刷资料。更推荐的动作是回到第四编，看看你现在是不是已经能更从容地把那些路径、脚本、目录和论文主线串起来了。如果答案是“是”，那这份词典就已经开始真正发挥作用了。

八、协作、工具与研究表达

等你开始真正接手项目，协作、工具和研究表达会同时出现。这一部分把 Git、Cursor、Codex、ChatGPT、文件格式、汇报指标和研究型术语收在一起。

1、当你开始真正接手项目时，一批“协作型术语”会集中出现

前面那二十一章，更多是在帮你把系统、环境、路径、规划和结果这些词压稳。到了这里，我们要进入另一批很实际、但很多跨专业同学也会发怵的词：代码协作词汇。它们乍看没有 GSplat、polytope 那么“学

术”，可如果你真的开始接手一个项目、开始跟别人协作、开始整理自己的修改、开始让师兄或导师看到你的进展，这一批词的重量一点也不低。

很多新人一开始会下意识把它们看成“程序员圈子里的黑话”。其实完全没必要。你可以把这一批词想成项目协作的基本礼仪和交通规则。懂了它们，你就知道自己今天做的改动在哪里、如何不把别人的工作弄乱、怎么把自己的进展说清楚。对当前项目这种明确有上游源码、wrapper 工作区、共享机器和后续交接目标的环境来说，这一层尤其重要。

1.1 repo、repository、仓库：别把它想得太神秘

repo 或 repository，说到底就是一套被版本控制组织起来的工程目录。你完全可以把它理解成“一个有历史、有规则、有边界的项目文件夹”。它和普通文件夹的区别，不在于长得更花哨，而在于它通常带着 .git/，能记录修改历史、分支关系和提交状态。

为什么这个词你必须尽早适应？因为你以后听到“官方仓库”“本地工作区仓库”“上游仓库”“第三方仓库”时，如果脑子里没有一个稳定概念，就会总以为大家在说完全不同的抽象东西。其实他们很多时候只是在区分不同层次、不同来源、不同责任边界的工程目录。

在当前项目里，/home/howard/Splat-Nav 是你现在最常接触的 wrapper 工作区仓库，而 splatnav-official 最终指向的是上游官方仓库。把这两个都叫 repo 没问题，但你必须知道它们不是同一层。

1.2 commit：一次提交不是“我改完了”，而是“我留下一块可回看的历史”

很多新手一听 commit，第一反应就是“提交代码”。这当然没错，但还不够。更好的理解是：一次 commit 是你把当前一组相对完整、相对可解释的修改，压成一个可回看、可比较、可恢复的历史节点。它不只是技术动作，也是表达动作。你在说：我在这个时间点，对项目做了这一组事情。

为什么这个词对学弟重要？因为一旦你开始真正整理项目，就会发现“我改了很多东西”和“我留下了一组可理解的历史”完全不是一回事。前者很混乱，后者才有工程价值。哪怕你现在还不急着正式提交代码，先把 commit 理解成“项目历史上的清晰刻度”，对以后也非常有帮助。

1.3 branch：它像平行工作线，不是另一个宇宙

branch 常被翻译成分支。很多人一听分支，就想象成项目被切成完全不同的几个世界。其实更好的理解是：它像一条并行工作线。主线继续往前走的时候，你也可以在另一条线上实验、开发、整理、对照，不必把所有试验都直接砸在主线上。

当前上游 README 就提到了 ros 分支和 semantics 分支。这其实特别值得学弟理解。因为它说明同一个仓库可以因为任务目标不同，在不同分支上承载不同形态的实现。你以后一看到分支名，就要习惯先问：这个分支是为了什么任务，它和当前主线是什么关系。

1.4 fetch、pull、push：这些词为什么听起来像搬运，但实际上是在同步历史

这三个词是很多新人最容易混的。你可以先用最朴素的方式记：fetch 更像把远端的新信息拿到本地，但先不自动并入当前工作线；pull 更像获取远端更新并尝试并入本地当前分支；push 则是把你本地已有的提交送到远端。它们表面像在搬东西，实际上是在同步项目历史。

对当前项目来说，你暂时未必每天都要执行这些动作，但你非常有必要把概念分清。因为一旦以后你要同步上游、保护自己的修改、和别人对齐版本，这几个词会很频繁地出现。概念一清楚，胆子就不会那么慌。

1.5 diff：它像“改动体检报告”，能告诉你到底变了什么

很多同学一开始写代码时，最容易陷入一种模糊感：我改了不少，可到底改了哪些地方、哪一行变了、是不是把别的东西也带坏了，不太说得清。diff 的价值就在这里。你可以把它理解成“改动体检报告”。它不是告诉你项目是什么，而是告诉你“和之前比，现在哪里不一样了”。

这个词特别值得学弟早一点习惯。因为真正成熟的协作，不是只会改，还得会检查自己到底改了些什么。你以后越早形成“每做一轮修改，就看看 diff”的习惯，越不容易写着写着把自己也绕糊。

1.6 README：不是每份 README 都是教材，但每份 README 都在回答“怎么从门口进来”

前面我们在项目全景里已经讲过 README 的角色，这里换个术语视角再说一遍。README 的本质，不是“所有知识都写在这里”，而是“后来的人第一次打开仓库时，至少不至于完全没有门”。所以你读 README 时，重点不应该是期待它把所有问题都讲透，而是先抓它回答了哪几个门口问题：这个仓库干什么、怎么开始、依赖是什么、路径大概怎样、典型命令从哪进。

你以后自己写项目，也最好逐渐形成这个意识。README 的职责不是替代整套文档，而是做项目门口的第一块说明牌。

2、Cursor、Codex、ChatGPT、SSH 这些“新工具词”，别让它们把你吓住

对于机械转代码的学弟来说，现代工具链往往比算法词更容易造成心理压力。因为这些工具的界面变化快、视频很多、说法又杂，很容易让人觉得“我是不是先要把所有工具研究明白，才能开始项目”。其实完全不用。更好的节奏是：把这些工具词一个个压成很朴素的用途理解。只要用途理解稳了，具体按钮长什么样、界面怎么变，都不会太影响你。

2.1 Cursor：先把它当成“带远程开发能力的代码编辑器”

你如果第一次接触 Cursor，最容易被网上各种标题带偏，好像它是一个“必须先学明白一大堆 AI workflow”的平台。其实对当前项目更朴素、也更够用的理解是：它首先是一款代码编辑器，而且能比较方便地配合 SSH 打开远程机器上的项目目录。你真正需要它做的第一件事，是稳定连上这台工控机，看到 /home/howard/Splat-Nav 里的文件，然后能比较舒服地读文档、读代码、改脚本。

所以你完全不用一开始就给自己加压，觉得必须把 Cursor 所有插件和面板全学明白。先把它作为“远程看代码和改文件的窗口”用稳，就已经很不错了。

2.2 SSH 在 Cursor 里再次出现时，不是新概念，只是新入口

很多新手会在这里多绕一层：命令行里学过一次 SSH，到了 Cursor 里看到 Remote SSH，又觉得自己得重新学一遍。其实不需要。概念还是同一个概念，只是入口换了。命令行 SSH 是你手打命令连远端，Cursor 里的 SSH 是编辑器帮你把连接过程做得更图形化、更适合直接打开远程目录。

所以你完全可以把它们看成“同一条路的两个门”。底层关系没变，你还是要知道自己连的是哪台机器、用的是哪个账户、路径是不是远端路径。

2.3 Codex：把它先理解成“能帮你读、写、改、查的协作型代理”

对于第一次接触 Codex 的学弟来说，最容易发生的事就是把它想得又神又玄。其实一个更稳的理解是：它像一个比较能干的协作型代理，可以帮你读代码、解释脚本、补文档、修改文件、跑命令，但前提是你给它足够清楚的上下文和任务边界。它不是神灯，也不是你可以完全不理解项目就放心交出去的黑盒。

你以后越早形成一个习惯越好：把 Codex 当成搭档，而不是当成替你承担全部理解责任的外包工。这样你会问出更好的问题，也更容易从它的回答里真正学到东西。

2.4 ChatGPT：别把它当搜索引擎，也别把它当老师傅的替身

很多新人一开始用 ChatGPT，容易走两个极端。一个极端是把它当搜索引擎，什么都问一句，得到零碎答案，结果脑子更乱。另一个极端是把它当老师傅，自己完全不判断，只盼它一步到位告诉自己“应该怎么做”。这两种用法都不太理想。

更成熟的用法是：当你已经有一部分上下文，知道自己卡在什么层，比如环境、路径、脚本链、术语解释、代码阅读顺序，这时再去问，它的帮助会很大。也就是说，你越能明确上下文和目标，它越能真的帮到你。

2.5 prompt：提示词不是咒语，它更像任务描述

很多关于 AI 工具的内容会把 prompt 讲得很玄，好像你必须掌握一套秘密咒语才能让模型工作。其实对当前项目来说，更好的理解是：prompt 本质上就是任务描述。你把上下文、目标、边界、输入路径、希望产出的形式讲得越清楚，对方越容易帮到你。

所以以后如果你要让工具帮你读某个脚本，不如直接说清：文件在哪、你现在看到哪一层、你不懂的是输入输出还是调用链、你希望它用大白话还是带代码引用来讲。你会发现，这比追求所谓“高级提示词技巧”更有用。

2.6 context：上下文不够，工具再强也容易答偏

context 这个词你以后会听很多次。可以先把它理解成“为了让对方不乱猜，必须先提供的背景信息”。对当前项目来说，上下文至少可能包含：你现在是在 `/home/howard/Splat-Nav` 这个 wrapper 工作区里，真正上游源码入口在 `splatnav-official/`，当前目标是离线/仿真复现，不碰实车，机器是共享工控机，不能乱动系统 CUDA。

你只要把这种习惯养起来，以后无论问人还是问工具，都会更高效。因为真正让沟通失控的，往往不是对方不够聪明，而是你默认了很多背景对方其实并不知道。

3、YAML、JSON、Markdown、CSV、日志：这些文件格式看着普通，却是项目语言的一部分

很多跨专业同学会下意识觉得，文件格式只是琐碎细节，不值得花脑力去理解。可一旦你真在项目里待上几天，就会发现这些格式简直无处不在。你打开的是 Markdown，传进去的是 YAML，跑出来的是 JSON，汇总后变成 CSV，排错时盯的是日志。如果这批词你一直停留在“听过”的层面，阅读和排错成本会一直很高。

3.1 YAML：先把它当成“更适合写配置的人类可读文本”

YAML 最大的特点，不是它语法多高级，而是它很适合承载层次化配置。当前项目里最典型的的就是 `config.yml`。你打开它时，会看到键值、层次和路径关系。这种文件的重点，不在于你要手写出一整份复杂 YAML，而在于你至少要能看懂：这里在描述什么，哪一层像模型配置，哪一层像数据解析入口，哪一层像训练产物引用。

你可以先把 YAML 理解成“适合人读的配置语言”。这个理解对入门已经够用了。

3.2 JSON：它更像“结构化结果记录本”

和 YAML 相比，JSON 在当前项目里更常出现在结果侧。比如 smoke 输出的 JSON、batch 结果的 JSON。它通常把信息组织成键值对、列表和嵌套对象，适合程序保存和后续脚本读取。你以后要培养一个特别实用的能力：看到 JSON 时，不要慌着从第一行读到最后一行，而是先找顶层键，再判断哪部分是 meta，哪部分是实验结果，哪部分是 trial 记录。

这种“先看结构”的习惯，会让你读 JSON 的速度快很多。

3.3 Markdown：它不是装饰性文本，而是工程里最常见的说明格式

Markdown 之所以在工程里这么常见，就是因为它比纯文本更有结构感，又比复杂文档格式更轻。README、手册、总结、汇总摘要、notes 都很爱用它。对学弟来说，Markdown 最重要的不是会不会写炫格式，而是你能不能用它把一件事讲清楚。标题、段落、列表、代码块，这几样基本元素已经足够你写出非常有用的项目说明。

你现在看到的这些 docs/Codex 文档，本质上就在示范这件事：Markdown 完全可以承载很重的项目知识，只要结构和叙述安排得合理。

3.4 CSV：它像“表格化结果的最朴素通用格式”

当你开始跨 run 汇总实验时，CSV 会变得很有价值。它不像 JSON 那样层次丰富，但非常适合存一张规整表，便于脚本处理、便于表格软件打开、也便于快速做后续分析。比如一行是一种方法在某轮 batch 的摘要，列里放可行率、平均耗时、P95、来源文件等信息，这就很适合 CSV。

你以后一看到 CSV，不要觉得它“没什么技术含量”。在结果管理层，它往往非常实用。

3.5 stdout 和 stderr：终端里出来的字，其实也有分工

很多新人第一次排错时，会把终端里所有输出都看成“一堆字”。其实更稳的理解是：标准输出 stdout 往往承载正常运行过程中的信息，标准错误 stderr 更常承载报错或警告。现实里这两者当然可能混着看，但一旦你知道它们的概念，像 `|& tee stdout.log` 这样的写法就不再神秘了。

你会明白，这类写法是在认真地把正常输出和错误输出都一起落盘，为以后复盘留证据。

3.6 log、日志：日志不是废话，而是程序留下的行为轨迹

log 这个词以后你会看得越来越顺眼。因为你会慢慢意识到，日志不是程序多嘴，而是程序留下的行为轨迹。它告诉你什么时候进入哪一步、读了什么、检查了什么、报了什么异常、最后停在了哪里。尤其在远程和长任务场景里，日志几乎就是你事后唯一能完整回看任务过程的材料。

所以以后别嫌日志烦。学会读日志，本身就是做项目的重要能力。

4、实验指标和汇报词汇：你以后做周报时最容易用到的一批词

到了项目中期，很多词的价值会从“帮助你读代码”转向“帮助你讲结果”。也就是说，你不只是自己看懂了，还得开始会汇报、会比较、会总结。这一批词看起来不如算法词高深，但在实际工作里使用频率极高。

4.1 trial：它更像“一次试验单元”，不是“程序开了一次”

在 batch 实验里，`trial` 往往对应一次起终点对上的单次规划尝试。它和“脚本启动一次”不是同一层。一个 batch 脚本启动一次，里面可能跑几十次、上百次 `trial`。把这个词分清很重要，因为你以后讲“这次实验跑了多少”时，必须说清你指的是脚本级、run 级，还是 `trial` 级。

这点看似细，但在汇报里特别影响清晰度。

4.2 seed：随机种子不是玄学，它是在帮你减少“每次都不一样”的混乱

很多新人会把 `seed` 看成一个神秘数字。其实你可以先把它理解成：它是让带随机性的过程尽量可重复的钥匙。在当前 batch 脚本里，`seed` 会影响起终点顺序打乱等行为。你以后做对比实验时，如果想让不同运行之间更可比，这个词就会很重要。

它的意义不在于让世界完全 deterministic，而在于减少没必要的漂移。

4.3 feasible 和 feasible_rate：一个看单次，一个看整体

`feasible` 是单次规划有没有找到可行解，`feasible_rate` 则是多次 `trial` 里可行比例有多高。你一定要把这两个词区分清。因为很多新人第一次汇报时，特别容易拿一两次成功例子去替代整体统计。这样很不稳。真正成熟的说法通常会说：单次案例看上去不错，但整体 `feasible_rate` 还需要结合更多 `trial` 判断。

一旦你开始这样讲话，别人就会明显感觉你对实验的理解更像做项目的人，而不是只会展示一个漂亮案例的人。

4.4 avg_plan_time、p95_plan_time：为什么“平均快”不等于“整体稳”

平均规划时间好懂，可它不够。因为平均值可能掩盖少量特别慢的尾部情况。`p95` 的价值就在这里，它更像在问：绝大多数情况下，规划时间大概卡在什么水平以下。你可以先不追求特别精确的统计学直觉，但至少要知道，`avg` 和 `p95` 讲的是两个不同侧面：一个偏整体中心，一个偏尾部稳不稳。

所以以后如果你看到一个方法平均时间还不错，但 `p95` 很难看，就要有感觉：它也许不够稳定。

4.5 summary、report、overview：都是总结，但力度不同

很多汇报词看起来像近义词，其实语气和用途是不同的。`summary` 更像摘要，压缩关键信息；`report` 更像完整汇报，有背景、有过程、有结果、有结论；`overview` 更像总览，先把大图给人看清。你以后写文档和周报时，越早对这些词有点感觉，表达会越自然。

当前工作区里 `summary.md`、`overnight_summary.md` 这类文件名，也正是在用这些词提醒你：这是一层偏快速消费的内容，不是完整长篇叙事。

4.6 visualization：可视化不是美化，它是在压缩理解成本

很多新人一提可视化，脑子里会先想到“让图更好看”。其实更重要的意义是压缩理解成本。比如同样的批跑结果，可行率柱状图能让你一眼看出方法排序，时间箱线图能让你直观看到分布，轨迹叠图能让你感受到几何差异。这些都不是“锦上添花”，而是在帮助你更快理解实验。

所以以后千万别把可视化看成最后有空再做的装饰工作。它本身就是结果理解的一部分。

5、共享机器、谨慎操作、边界意识：这是你做项目时最像“职业习惯”的一批词

最后这一章，我想把一些非常“现实”的词再压一遍。它们不浪漫，也不学术，但它们决定你在共享机器上做项目是不是让人放心。对机械转代码的学弟来说，这一章尤其值得带着一点敬畏去读。因为很多事故不是不会算法，而是基本边界感不够。

5.1 `sudo`：它不是“帮你解决问题的万能钥匙”

很多新人一遇到权限问题，第一反应就是要不要 `sudo`。这是非常危险的直觉。更稳的做法永远是先问：我为什么会碰到这个问题，这个位置是不是本来就不该由我来改，当前问题能不能在自己的 `Conda` 环境或工作区边界内解决。`sudo` 真正代表的，是更高权限，而更高权限从来不是默认应该用的。

对当前项目来说，这一点尤为重要。因为你们明确说了，这台机器是共享的，不能乱动系统层和别人环境。那就意味着，`sudo` 在这里几乎天然属于需要停下来重新判断的词。

5.2 `rm`：删除不是最可怕的，可怕的是你不知道自己删的是哪一层

前面我们已经讲过 `rm`，这里再从协作角度补一句。真正让人害怕的，从来不是删除这个动作本身，而是你对删除对象没有层次感。删的是自己的 `run` 缓存，还是共享结果目录？删的是 `wrapper` 草稿，还是上游关键资产入口？删的是软链接本身，还是它背后的真实目录？

只要这三四个问题你没想清，`rm` 就不该轻易敲。共享机器上尤其如此。

5.3 `shared machine`：共享机器不是一句背景，而是一整套行为约束

很多同学会把“这是共享机器”理解成一句背景说明。其实它更像一整套行为约束：不要乱动系统层；不要默认别人没在用某些目录；不要为了自己方便就升级全局组件；重资产尽量往约定的数据盘走；日志和结果尽量规范落盘；排错优先在自己的边界里解决。你以后越早把这些约束内化，越像一个让人放心的合作者。

真正成熟的人，不是技术多炫，而是别人愿意把机器和项目交给他而不担心。

5.4 `environment pollution`：环境污染听起来抽象，其实就是“你改了不该全局改的东西”

这个词在很多工程里都会出现。可以先把它理解成：你为了局部任务，改动了全局环境，导致别的任务或别的人也受到影响。比如乱动系统 `CUDA`、全局 `pip` 安装、把奇怪路径塞进全局环境变量、覆盖掉共享依赖。对当前项目，这显然都是要尽量避免的。

你以后每次排错时，心里不妨多问一句：我现在这个修法，是只影响我自己的小环境，还是会把整台机器都拖下水？这个问题非常值钱。

5.5 `ownership`：责任感不是“我全都懂”，而是“我知道自己该守住什么边界”

做项目时，`ownership` 常被翻译成拥有感、责任感。很多人会把它理解成“我是不是得什么都扛”。其实更成熟的理解是：我知道自己当前负责的是哪一层、哪组文件、哪段逻辑、哪类结果，也知道哪些不是我该随便碰的。真正的负责，不是越界，而是在边界内把事情做扎实。

这对学弟特别重要。因为入门阶段最怕两种极端：一种是什么都不敢碰，另一种是什么都想碰。真正能持续推进的人，往往是在这两者之间找到清楚边界的人。

5.6 这一章读完以后，你至少应该多出一种稳感

如果前面的术语更多是在帮你建立知识地图，那么这一章其实是在帮你建立项目气质。一个人是不是适合长期做工程，很多时候不体现在他说了多少大词，而体现在他对边界、风险、共享资源和结果规范有没有稳感。你现在未必要一下子全做到，但至少应该开始有一种判断力：哪些动作该快，哪些动作该慢，哪些地方该自己试，哪些地方该先停一下问清楚。

只要这种稳感开始长出来，你后面无论是读第四编这种全景部分，还是实际跑脚本、改代码、做汇报，都会越来越像一个真正进入状态的人。

6、当你开始从“复现”走向“汇报、专利、论文”时，会遇到的一批研究型术语

如果说前面的章节更多是在帮你把项目跑起来、看明白、讲顺，那么这一章开始，我们要面对另一批词。它们通常会在组会、导师交流、专利构思、论文阅读和阶段总结里频繁出现。很多新人到了这一步会突然又紧张起来，觉得自己是不是从“不会代码”升级到了“不会科研表达”。其实不用把这件事想得太玄。更好的理解是：这些词就是研究工作中的常用语言，你早点把它们压成大白话，后面会轻松很多。

6.1 paper、论文：别先把它当圣经，先把它当“作者交代自己做了什么、为什么值”的正式说明书

很多跨专业同学一提论文，心理压力会瞬间变大，好像那是某种只有专家才能进入的神圣文本。其实对于项目入门阶段，更合适的理解是：论文首先是一份正式说明书，它在系统地回答作者做了什么、为什么这么做、和谁比、结果怎样、贡献在哪里。它当然有学术语言，但你完全没必要一开始就把它当成一道考试。

对当前项目来说，论文最重要的价值，是帮你确认这套系统的总主线和方法定位。你不用第一次就把每个公式都吃透，但至少要知道总系统是 `Splat-Plan + Splat-Loc`，知道作者为什么要强调 `GSplat` 表示，知道他们和哪些基线比较，知道他们主打的是安全、实时和视觉定位。

6.2 arXiv：先把它当成“论文公开草稿或公开版本的入口”

`arXiv` 对新手来说往往听起来很高大上，其实它最朴素的作用，就是给你一个论文公开版本的入口。你完全可以把它理解成“论文页面和 PDF 的稳定入口”。对当前项目来说，<https://arxiv.org/abs/2403.02751> 就是 `Splat-Nav` 论文的公开页面。

为什么这个词值得学？因为以后你做项目时，别人让你“先看看论文”，很多时候说的就是先去 `arXiv` 页面和 PDF 看总结构，而不是必须立刻去查期刊版本、会议最终版的所有差异。

6.3 project page：项目页通常是“论文的更友好导览版”

和论文相比，`project page` 往往更像导览版。它通常会放摘要、方法图、视频、关键结果图和常用链接。对新人来说，项目页非常适合做第一眼总览，因为它的信息密度高，但阅读压力比论文正文低。当前项目页就很适合作为你理解 `Splat-Nav` 总系统和演示效果的入口。

所以以后只要看到项目页，不要觉得它只是宣传页面。对入门者来说，它常常是最好懂的总览材料之一。

6.4 baseline：基线不是“随便找个差的方法来衬托自己”

很多新人第一次听 `baseline`，会误以为那只是论文拿来陪跑的角色。实际上，一个严肃的 `baseline` 往往代表一条合理、公开、在相关任务上有一定代表性的对比路线。对当前项目来说，`SafeFlightCorridor` 不同变体、`RRT`、`NeRF` 导航方法都在扮演不同类型的 `baseline` 角色。

你以后做汇报时，如果把 `baseline` 讲得太轻，会显得对实验设计理解不够。更成熟的讲法应该是：`baseline` 是为了让别人判断你的方法到底比什么更好、好在什么维度、是否值得采用。

6.5 ablation：消融不是“删着玩”，而是为了回答“到底哪部分在起作用”

`ablation` 也就是消融实验。很多新人一听这个词，会觉得是不是把模型某块删掉试试看。这个理解只对了一半。更完整一点的说法是：消融实验是为了隔离不同设计因素，看看系统表现到底由哪些部分支撑。如果少了一块，性能怎样；换一种初始化，结果怎样；安全缓冲变了，轨迹和可行率怎样。

对当前项目来说，`sfc-1` 到 `sfc-4` 这种 `mode` 差异，其实就很适合被理解成某种结构化变体比较思路。学弟如果以后想做创新，一定会反复碰到这个词。

6.6 novelty、创新点：不是“必须从零发明世界上没有的东西”

很多新人一提创新，就会立刻进入一种非常重的心理状态，仿佛创新必须是完全颠覆性的发明。其实对研究训练阶段更务实的理解是：创新点通常是在已有方法链上，提出一个有根据的新观察、新组合、新约束方式、新系统连接或新的实验验证角度。它不一定要从无到有造一个宇宙。

所以你以后带学弟思考创新时，也尽量别一开始就问“你有什么全新大招”。更现实的问法是：当前链路哪里最痛，哪层最薄，哪种对比不够充分，哪个工程边界如果整理好了会自然形成方法贡献。这样会更容易长出真正靠谱的想法。

6.7 patent、专利：它更像“把一个有实用价值的技术方案说清并保护起来”

很多学生第一次听到专利，会误以为那就是“把论文换种格式写一遍”。其实不太一样。专利更强调技术方案、适用场景、系统流程和可实施性。它关心的是你这个方案到底解决了什么问题，流程是什么，模块怎么配合，和已有方案相比有什么实际价值。

对当前项目来说，如果后面要写专利，第一阶段最关键的准备工作不是急着造词，而是把系统链路、输入输出、关键环节和可以主张的改进点真正讲清楚。没有这个底子，专利写作会非常空。

6.8 contribution：贡献不是“我做了很多事”，而是“我做了哪些值得别人记住的事”

这是一个特别值得学弟反复体会的词。很多人写汇报时喜欢说“我这周做了很多工作”，但这不等于 `contribution`。更成熟的说法是：我完成了哪几件真正改变项目状态的事，比如把离线链路跑通了、把结果归档规范固定了、把某个难点解释清楚了、做出一组可信对比、把某个模块工程化整理到了可交接状态。

所以以后无论写周报还是想论文贡献点，都最好少说泛泛的“做了很多”，多说“项目状态因此具体发生了什么改变”。

6.9 reproduction、replication、复现：重点不在字面差别，而在“别人能不能跟着你走到相同结果”

在学术讨论里，这两个词有时会有更细的区分，但对当前项目入门阶段，你完全可以先把它们放在一起理解：核心关注点都是别人能不能根据公开材料和你的整理路径，走到相近的结果。也就是说，复现不只是“我这台机器上偶然跑通”，而是“方法、路径、输入、输出和结果解释足够清楚，后来者也能跟上”。

这也是为什么你现在花这么大力气整理 `docs/Codex` 是值得的。因为文档和工程规范本身，就是复现质量的一部分。

6.10 benchmark：基准不是名词摆设，而是你判断结果有没有位置的参照系

很多新人第一次出结果会非常开心，这是好事。但再往前走一步，你就必须学会问：这个结果放在什么基准上看才有意义。benchmark 更像一个参照系，告诉你这个任务、这个场景、这个指标通常是怎样比较的。没有 benchmark，你的结果就容易悬空，看着不错，但不知道到底站在什么位置。

对当前项目来说，论文已有的场景、基线和指标体系，本身就构成了一个很重要的 benchmark 语境。

6.11 robustness、鲁棒性：不是“偶尔成功”，而是“换些扰动还基本站得住”

很多同学初学时更容易关注“能不能成功一次”。真正做研究往前走时，robustness 会越来越重要。它关心的是：场景换一点、参数抖一点、输入条件变一点、任务多跑几次，系统表现是不是还大体稳定。这个词和 feasible_rate、p95 这些统计词其实很有缘分，因为它们都在帮助你从“单例漂亮”走向“整体可靠”。

以后你只要听到导师说“这个结果还不够 robust”，通常不是在否定你，而是在提醒你别太早把单次成功当结论。

6.12 regression：回归不总是统计课里的回归，它也常指“本来好的东西被你改坏了”

这是一个很容易误会的词。在机器学习或统计语境里，regression 可能指回归任务；但在工程语境里，它还常常表示“功能退化”或“行为回退”。比如你改了一段脚本，本来能跑的 smoke 现在不跑了，这就可能叫 regression。对学弟来说，早点知道这个双重含义很重要，不然以后听到别人说“这次改动引入了 regression”，你可能会一头雾水。

九、收束与使用方法回归

最后这部分不再继续铺新名词，而是帮助你把整份词典真正变成工作中的陪跑工具。它关注的是使用姿势，而不是继续增加记忆负担。

1、这一份词典最好的使用方式，不是从头背到尾，而是跟着项目回查

写到这里，我其实特别想再提醒你一次：这份词典不是拿来背诵通关的。你越把它当成陪跑工具，越会用得顺。比如你今天在第四编里读到 GSplatLoader、run_dir、config.yml、baseline，就回来看；明天 batch 跑起来，碰到 trial、seed、feasible_rate、p95，再回来看；下周开始想专利、想创新点，碰到 contribution、novelty、ablation，再回来看。

这种回查式学习其实很健康。因为项目里的很多词，本来就不是一次看懂、一次记住就结束的。你会发现，同一个词在第一周、第二周、一个月后读起来，味道都不一样。第一次你只是知道它大概什么意思，第二次你开始知道它在项目里出现在哪，第三次你才会真正知道它对推进工作意味着什么。

如果你能接受这种节奏，第六编 就会越来越像一本真的“工作中的词典”，而不是一次性资料。那它的价值就算真正出来了。

2、最后送你一个“查词三层法”：以后遇到新词，不要只问“它是什么意思”

这份词典写到最后，我还想再送你一个很实用的小方法。因为项目不可能永远只出现我们已经写进来的这些词。以后一定还会有新词冒出来，尤其是当你开始看更多论文、更多分支、更多脚本时。如果每次遇到新词，你都只问“它是什么意思”，理解常常会停在很浅的层面。更好的做法，是用一个三层法去问它。

第一层，当然还是“它是什么意思”。也就是最基本的大白话解释。这一层解决的是陌生感，让你不至于看到词就发怵。第二层，要追问“它在当前项目里出现在哪儿”。也就是它会不会出现在某个脚本名、某个日志字段、某个结果目录、某个上游文件或某个报错里。第三层，最重要，要问“如果我现在不懂它，接下来会卡在哪一步”。这一步会让你一下子知道，这个词对你来说到底是眼下必须掌握的，还是先知道名字就够。

你会发现，一旦这么问，词就不再是孤零零的字典条目，而会变成真正有位置、有后果、有行动意义的东西。比如你今天遇到 `dataparser_transforms.json`，第一层是知道它和数据解析变换有关；第二层是知道它常和 Nerfstudio 输出、`config.yml` 背后的训练产物联系在一起；第三层则会提醒你，如果这层不懂，你后面很难真正讲清场景入口链。再比如你遇到 `p95_plan_time`，第一层是知道它表示尾部时延表现；第二层是知道它会出现 batch 汇总和图表里；第三层则是在提醒你，如果忽略它，你很可能把“平均值看着不错”误讲成“整体都很稳”。

所以以后你完全可以把这套三层法当成自己的默认查词姿势。先问意思，再问位置，再问后果。只要这个习惯长出来，即便未来再冒出很多新术语，你也不会像最开始那样容易被压住。因为你已经知道，词不是用来吓人的，而是用来帮助你在项目里继续往前走的。

3、把前面这些术语真正串成一天工作，而不是一堆孤立名词

如果一份词典写到最后，读者脑子里还是只有很多零散词条，那这份词典其实还没有真正完成任务。因为真实的项目推进，从来不是一个术语排队出现的，它更像是一整天的工作被很多词共同支撑起来。你早上接入机器，上午确认环境，中午开始读脚本，下午跑 smoke，傍晚看结果，晚上写一段简短汇报。在这个过程中，你不会先单独遇到 `SSH`，然后把它放一边；再单独遇到 `Conda`，又放一边；再单独遇到 `feasible_rate`，又放一边。它们是混在一起、互相咬合着出现的。

所以这一章，我们不再按“词目录”的方式往前推，而是换一种更像真实工作日的讲法。你可以把它当成一条从早到晚的轻量 workflow，也可以把它当成一个小型复盘。目的只有一个：让你感觉前面那些看似分散的术语，开始慢慢长成一张网，而不是继续一颗一颗漂在水面上。

3.1 早上第一件事，不是着急跑命令，而是先确认你现在到底连到了哪里

很多新手一坐下，最容易出现的状态就是“赶紧做点什么”，结果一着急，命令是敲了，事情却并没有真正开始。更稳的做法，其实是先用两分钟做定位。你通过 `SSH` 连上机器，打开 `Terminal`，进入当前用户的 `Shell`，然后先看 `hostname`、`pwd`、`whoami`。这几个动作看起来很朴素，但它们会直接帮你回答三个问题：我现在在哪台机器上，我当前站在什么目录里，我现在是谁的账户。

这一步为什么值得反复强调？因为你正在用的是公用工控机，而且你已经非常明确地说过，不要碰你师兄的环境，不要乱动 `CUDA`，不要把公共机器搞脏。这些要求并不是“附加提醒”，而是项目边界的一部分。也就是说，`SSH`、`hostname`、`pwd`、用户目录、绝对路径这些看起来像系统基础的词，在当前项目里并不只是“基础知识”，它们直接关系到你能不能安全地开始工作。

当你意识到这一点之后，很多动作会自然变稳。你不会再一上来就把任何路径都当成“自己当然知道”；你也不会轻易把 `/home/howard/Splat-Nav` 和 `/data/howard/...` 混起来。术语一旦和风险绑定，它就不再抽象。

3.2 早上的第二件事，是确认你今天说话的“Python 方言”到底是哪一个环境

你可以把 Conda 环境想成“今天工作时我们约定使用哪一套 Python 世界”。项目是不是能正常 import 某个包，torch 能不能认出 CUDA，脚本里依赖能不能对得上，很多时候都不是代码逻辑先出问题，而是你说话的这套环境根本没切对。

所以在真实工作里，Conda、python、pip、环境变量、LD_LIBRARY_PATH、which python 这些词，往往是一串出现的。你如果把它们拆开背，会觉得琐碎；可一旦放到“今天我要不要开始跑任务”这个动作前面，它们就会变得非常有秩序。你会知道：现在我不是在抽象地学环境管理，我是在确认自己到底有没有资格往下运行。

这也是为什么很多老练的人，每次开始之前都会先做一个很轻的小检查。不是因为他们记性不好，而是因为他们知道，环境类问题的本质并不在于“你是不是记住了上次怎么配”，而在于“你有没有在今天、在此刻、在这个终端里重新确认”。这其实是一种工程习惯，而不只是知识点。

3.3 当你打开工作区时，目录就不再只是文件夹，而是项目结构本身在说话

接下来你会进入 /home/howard/Splat-Nav。这时候，README.md、docs/、scripts/、runs/、splatnav-official/、third_party/ 这些名字就会一起冒出来。很多新手在这一刻会觉得乱，原因并不是目录太多，而是心里还没有“目录会说话”的意识。其实目录从来不是中性的。一个项目把什么放在顶层、什么拆进子目录、什么专门留出 docs 和 runs，本身就在告诉你：这里在重视什么，这里的工作流大概怎么走。

比如你看到 scripts/，不要立刻想“这里全是代码”。更好的第一反应是：这里很可能是把常见动作脚本化的地方。你看到 runs/，也不要只把它当输出垃圾桶，而要意识到这是项目为了留存证据专门安排的空间。你看到 docs/，更不应该只把它当“以后有空再看”的资料区，而要知道，这里其实是在补算法代码天然不擅长表达的那一层解释。

一旦你这样看目录，项目就开始从“很多文件夹”变成“很多有分工的房间”。这时，path 这个词也不再是单纯的斜杠字符串，而会慢慢带上结构含义。

3.4 中午开始读脚本时，文件名、入口函数和参数，三者要一起看

等你真正开始往 scripts/ 或 splatnav-official/ 里看，术语又会换一批出现。你会看到 argparse、入口脚本、参数、sys.path、配置文件、输出目录、日志重定向、JSON 汇总、可视化脚本。这个阶段特别容易犯的错，是只盯着代码正文看，却不先看“这份脚本到底是谁触发的、收什么输入、把结果放哪里”。

对新人来说，更稳的姿势其实是三件事一起看。第一，看文件名，因为文件名经常已经在提示这份脚本负责哪个动作。第二，看入口，因为入口能帮你知道它是被人直接运行，还是被其他脚本间接调用。第三，看参数，因为参数会直接告诉你作者认为“可变”的东西是什么。很多时候，一份脚本的核心意图，不在几十行实现细节里，而在入口和参数层面已经说得七七八八了。

所以以后你看到 smoke_splatplan.py、run_official_style_batch.py、aggregate_batch_runs.py 这类名字时，第一反应不要是“我要先把代码每行都看懂”，而应该是“它分别处在工作流哪一段、吃进去什么、吐出来什么”。这是读脚本时非常省力的思路。

3.5 下午真正开始跑 smoke 时，你第一次会感觉很多词突然活了

到了运行阶段，前面看起来有点散的术语会开始集体落地。你会遇到 `tmux`、`stdout.log`、`run_dir`、时间戳、主题后缀、`feasible`、规划耗时、统计 JSON、图表文件。这个时刻特别宝贵，因为很多词第一次从“我大概知道它是啥”变成“我亲眼看见它出现在自己的结果里”。

比如你以前知道 `tmux` 是长任务会话工具，但只要你亲手把任务放进 `tmux`，它就不再只是一个工具名，而会变成“我离开终端以后任务还能继续跑”的安全感。你以前知道 `stdout.log` 是日志文件，但只要你真的在里面追过一次错误，它就不再是平淡的名词，而会变成“事情到底发生了什么”的证词。你以前知道 `feasible` 大概和可行性有关，但只要你真的在 batch 汇总里看到不同方法、不同迭代的 `feasible rate`，它就不再只是概念，而会变成你理解结果好坏的抓手。

这也是为什么真实运行特别重要。因为有些词，光靠解释只能懂到六成，必须亲手见过一次，才会真正进入你的工作语言。

3.6 傍晚看结果时，指标不该被当成成绩单，而该被当成问题定位器

新人在第一次看到结果汇总时，常常会本能地去找一个最像分数的数字。比如平均可行率高不高，计划耗时快不快，最后一轮图看着漂不漂亮。这种直觉可以理解，但如果一直停留在“找分数”的阶段，后面很难真正长出研究和工程判断。

更成熟的看法是：指标首先是问题定位器。`feasible_rate` 不是为了让你喊一句“挺高”，而是为了帮你知道方法整体是否稳定。`plan_time` 不只是快慢比较，它还在告诉你实时性边界到底在哪。`p95` 之类的尾部统计，不是为了显得专业，而是为了提醒你平均值掩盖不了那些拖后腿的极端情况。

一旦这么看，指标就会从“成绩单”变成“手电筒”。它会把你照向更具体的问题：到底是哪类 `scene` 更难，哪种模式更不稳，哪一步耗时最突出，哪一次运行其实只是看上去成功。术语一旦能帮你提问题，它就开始真正服务于推进项目，而不是只服务于阅读材料。

3.7 晚上写汇报时，词汇量已经不再重要，重要的是你能不能把一天说清楚

到了晚上，如果你要给师兄、导师或者未来的学弟留一点记录，这一天其实就进入了另一个阶段：从“做事”切换到“讲事”。这时候你会突然发现，前面学过那么多词，真正值钱的不是你能说出多少个，而是你能不能把今天这条链讲顺。

比如更成熟的表达不会是“我今天看了很多资料、跑了很多命令”，而会更像：我今天先确认了远程环境和路径边界，然后定位了当前 `wrapper` 层和上游层的关系，接着用 `smoke` 跑通了一次最小链路，最后检查了 `runs` 目录下的日志与汇总输出。你看，这里面虽然自然会出现 `SSH`、`wrapper`、`smoke`、`run_dir`、`log`、`summary` 这些词，但它们已经不是炫耀术语的装饰品，而是在帮助你把一天说清楚。

这其实也解释了为什么你会觉得以前那种“满篇分点、满篇术语”读起来成本高。因为人的脑子不是靠词堆起来的，而是靠“事情是怎么发生的”来理解的。现在我们把术语重新放回一天的动作链里，它们就开始顺了。

3.8 真正的学习感，不是“今天新认识了多少词”，而是“今天少迷糊了哪一段”

很多人评价自己学得好不好，喜欢用一种很硬的方式，比如今天背了多少概念、看了多少页材料、记了多少命令。可在复杂项目里，更真实也更健康的衡量方式其实是：今天有没有哪一段比昨天少迷糊一点。也许你

还是没有完全理解 `Bezier` 为什么放在这里，也还不能把 `polytope` 的几何推导完整讲出来，但你已经不再把 `wrapper` 和上游混成一团，也知道 `runs` 里的文件应该先看谁后看谁。那这就是很实在的进步。

所以如果你以后带学弟，也尽量别老问那种会把人问虚的问题，比如“你都学会了吗”。更好的问题是：你今天最不迷糊的是哪一段，最迷糊的又是哪一段。前者能帮助他确认进展，后者能帮助你判断下一步怎么带。很多时候，学习动力不是来自“我已经全会”，而是来自“我终于有一块地方开始看明白了”。

4、十组最容易混的词，最好现在就把边界拉开

前面我们把很多术语各自讲开了，但真到了实际使用里，新手最容易卡住的，往往不是完全陌生的词，而是“看起来差不多”的词。也就是说，不是所有难点都来自不认识，而有不少混乱来自“仿佛都认识，但边界总是糊”。这一章我们就专门处理这种情况。你不需要把每一组都背成标准答案，只要把它们之间最重要的那条分界线抓住，后面很多理解成本就会降下来。

4.1 工作区 和 上游源码

这两个词最容易因为都长得像“项目目录”而被混成一个东西。更清楚的说法是，`/home/howard/Splat-Nav` 是当前机器上的工作区，它承载的不只是算法代码，还有脚本、文档、结果规范和交接结构；而 `splatnav-official` 则更偏向论文本体实现，是算法事实主要落地的地方。前者更像工程组织层，后者更像方法实现层。

只要这层关系稳住，你后面无论是解释目录、读脚本还是讲成果，都会自然很多。否则一切都会绕在一起，最后谁都说不清自己在整理什么。

4.2 脚本 和 模块

脚本 更像一个被拿来直接执行的动作入口，而 模块 更像被其他地方引用、复用的功能单元。举个更直白的说法，你通常会“跑一个脚本”，但你更常“调用一个模块”或“`import` 一个模块”。对当前项目来说，`scripts/` 里的很多东西偏脚本，而 `splatplan/`、`splat/`、`SFC/` 下的很多实现则更偏模块。

新手如果不分这两者，就容易读脚本像读模块，或者反过来，结果总觉得信息组织得很散。其实不是项目乱，而是阅读姿势还没切对。

4.3 配置 和 参数

这两个词看起来都像“控制程序行为的东西”，但在工作流里的感觉不一样。`参数` 往往是你在运行命令时临时带上的，它更即时、更贴近这一次执行；而 `配置` 更像预先写在文件里的约定，通常更稳定，也更成体系。对学弟来说，早点形成这个感觉很重要，因为你后面做复盘时就会自然知道：这次结果的变化，到底是某个命令行参数改了，还是底层配置变了。

4.4 日志 和 结果

很多人会把运行后留下来的东西一股脑都叫“结果”，这在聊天时可以，但在分析时会很粗。更细一点说，`日志` 主要回答“运行过程中发生了什么”，而 `结果` 更偏“运行完成后沉淀出了什么”。比如 `stdout.log` 更像日志；某张可视化图、某个 `summary.json`、某个汇总 CSV，则更像结果。两者都重要，但它们回答的问题不同。

4.5 成功运行 和 可信运行

这组词非常关键。很多任务“跑完了”，并不代表“跑对了”。前者只说明程序没有明显中断；后者还要求输入合理、输出齐全、指标能解释、目录可追溯。你以后越早把这两者分开，后面越不容易被假成功骗到。真正成熟的项目习惯，永远更接近“可信运行”，而不是“反正出东西了”。

4.6 复现 和 创新

这两个词不是对立关系，但确实是不同阶段的问题。复现 更像把已有链路真正走通、讲清、沉淀下来；创新 则是在这个基础上提出有意义的新东西。很多新人痛苦就在于还没把复现站稳，就已经在逼自己想创新。更好的顺序永远是先复现出稳定底图，再从底图上长出创新方向。

4.7 看懂 和 会讲

有时候你自己心里觉得“好像懂了”，但一张嘴就讲散了。这个差异非常正常。看懂 更偏你和材料之间的关系，而 会讲 则要求你把脑子里的结构重新整理出来，让别人也能跟上。对当前项目来说，真正标志你开始掌握一段内容的，往往不是你默读时觉得顺，而是你能不能把它讲给学弟或者导师听。

4.8 平均值 和 尾部情况

这一组看似偏统计，其实非常贴日常判断。平均值会给你一个整体印象，但尾部情况决定你是不是会在关键时刻掉链子。比如 p95 这类词，就是在提醒你别只看平均表现。你后面如果要真正讲“稳不稳”，就不能只拿均值说话。

4.9 知道名字 和 建立工作感

有些词你一查就知道大概意思，但这和真正建立工作感还不是一回事。所谓工作感，是你看到它能立刻想到它大概出现在哪、有什么风险、影响哪一步、自己接下来该做什么。比如知道 tmux 是终端复用工具，只是知道名字；可如果你已经自然明白“今晚长任务一定要放 tmux 里，不然断线就白跑”，这才叫建立工作感。

4.10 会用工具 和 依赖工具

最后这组词，我特别想送给刚开始用 Cursor、ChatGPT、Codex 这些工具的同学。会用工具，是知道它们适合拿来做什么、什么时候帮得上忙；依赖工具，则是离开它们就完全失去判断。我们当然希望学弟尽快用上这些工具，但更希望他在使用过程中长出自己的结构感和判断力。工具应该帮你走得更快，而不是替你把方向感拿走。

5、读完第六编后，你脑子里最好留下来的，不是更多名词，而是一套更稳的读项目姿势

如果第六编 真起作用了，那么读到这里时，你最该带走的其实不是“我今天又记住了几十个词”，而是三种更稳的姿势。第一种姿势，是遇到新词别慌，先问它在人话里是什么意思，再问它在当前项目里出现在哪。第二种姿势，是别把术语当摆设，要始终追问它会不会影响你接下来的一步动作。第三种姿势，是允许自己反复回查，不把“一次看懂全部”当作合理目标。

你会慢慢发现，真正让人能走下去的，不是突然变得特别聪明，而是开始学会和复杂材料相处。你知道自己可以先看个大概，知道有问题就回来翻，知道今天懂六成也没关系，只要明天能到七成就已经很好。这种松弛但不散的状态，反而最适合带一个零基础学弟进项目。

所以这份词典最后想送给你的，并不是一场“术语考试”的准备，而是一种处理复杂项目的基本功。你以后换方向、换代码库、换团队，这个基本功也照样值钱。因为任何复杂系统，最终都绕不过同一件事：你得先学会不被那些看起来很多、很杂、很硬的词压住，然后才有余力去真正理解它、改造它、讲清它。只要这一点开始发生，第六编 就没有白写。